CFD Assignment 3 : Lid-Driven Cavity Flow

Chen Peng Chung

December 20, 2025

Contents

1 Introduction **3**

2 Problem Descriptions **4**

3 Introduction of Methodology Adopted **5**
3.1 Discretize the Governing Equations 5
3.1.1 Staggered Grid: Problem and Solving Method 5
3.1.2 Discrete Form of the Continuity Equation 6
3.1.3 General Discrete Form of the Momentum Integration Equations 7
3.1.4 High Order Scheme of the Momentum Integration Equations 9
3.2 SIMPLE Algorithm 12
3.2.1 Predicted-Correction Method 12
3.2.2 Algorithmic Steps and Under-Relaxation 13
3.2.3 Staggered Grid and Pressure-Velocity Coupling 14
3.2.4 Continuity Residual and Convergence Criterion 14
3.2.5 Boundary Conditions for Pressure Correction 14
3.2.6 SIMPLE vs. SIMPLEC 14
3.3 Patankar Omission in SIMPLE Algorithm 14
3.3.1 Derivatation of Pressure Correction Equation 15

4 Results and Discussion **17**
4.1 Vortex Structure Preview 17
4.2 Streamline Plots 18
4.2.1 Arrangement of Streamline Patterns 18
4.2.2 Central Difference Scheme (CDs) 19
4.2.3 QUICK Scheme 20
4.2.4 TVD-MUSCL Scheme 21
4.3 Velocity Magnitude Contours 22
4.3.1 Arrangement of Velocity Magnitude Contours 22
4.3.2 Central Difference Scheme (CDs) 24
4.3.3 QUICK Scheme 25
4.3.4 TVD-MUSCL Scheme 26
4.4 Velocity Centerline Comparison for Different Re Numbers 27
4.4.1 Arrangement of u-Velocity and v-Velocity Profiles 27
4.4.2 Centerline Velocity Profiles - Grid 81x81 28
4.4.3 Centerline Velocity Profiles - Grid 161x161 29
4.5 Velocity Centerline Comparison for Different Schemes 30
4.6 Velocity Centerline Comparison for Different Grid Sizes at High Re Number . . 31
4.7 Accuracy Comparison With Different Schemes 32

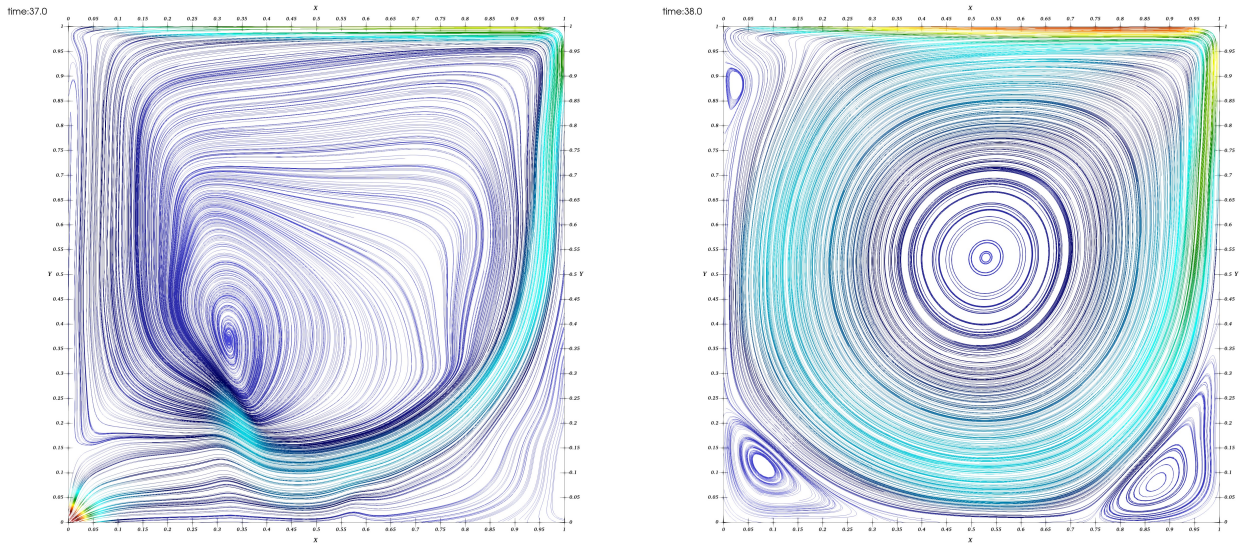
A Source Code **33**
A.1 Header Files (.h files) . . . 33
A.1.1 TDMA Solver . . . 33
A.1.2 Pressure Correction Equation . . . 34
A.1.3 Pressure Correction Equation with TDMA . . . 38
A.2 Main Programs (.cpp files) . . . 41
A.2.1 FDM Central Difference Scheme . . . 41
A.2.2 FVM Non-Deferred Correction . . . 51
A.2.3 FVM Deferred Correction . . . 72
A.2.4 FVM Deferred Correction with ADI+TDMA . . . 94

B SIMPLE Algebraic System **116**

1 Introduction

For Assignment 3, solving Lid-Driven Cavity Flow, I have used four types of methods to solve this classic problem. Corresponding to the difficulties I encountered during the Re number increasing or numerical oscillation happening for the high order schemes, I can make a fast review below:

1. A.2.1: FDM Central Difference Scheme
   This method is Finite Difference Method with SIMPLE algorithm to solve the unsteady Navier-Stokes Equations. The disadvantage of this code is the accuracy in space is only 2nd order, and time accuracy is only 1st order, because the discrete equation in this code is scheme of forward difference in time and central difference in space (FTCS).
2. A.2.3: FVM Deferred Correction
   I would like to describe this problem from the figure below **??**. When I tried to compute at Re = 5000 with non-deferred correction FVM code A.2.2, I found that whatever how I reduce the relaxation factor, I still can't get the appropriate iteration process, even only one converged solution at all. The solving method I took is that changed the structure of the discrete equation of momentum equations, i.e., moving the anti-diffusion term to the source term, and meaning is using old value to substitute those terms, leading to elements of coefficient matrix maintain the Convection Boundedness Criterion(CBC) condition.



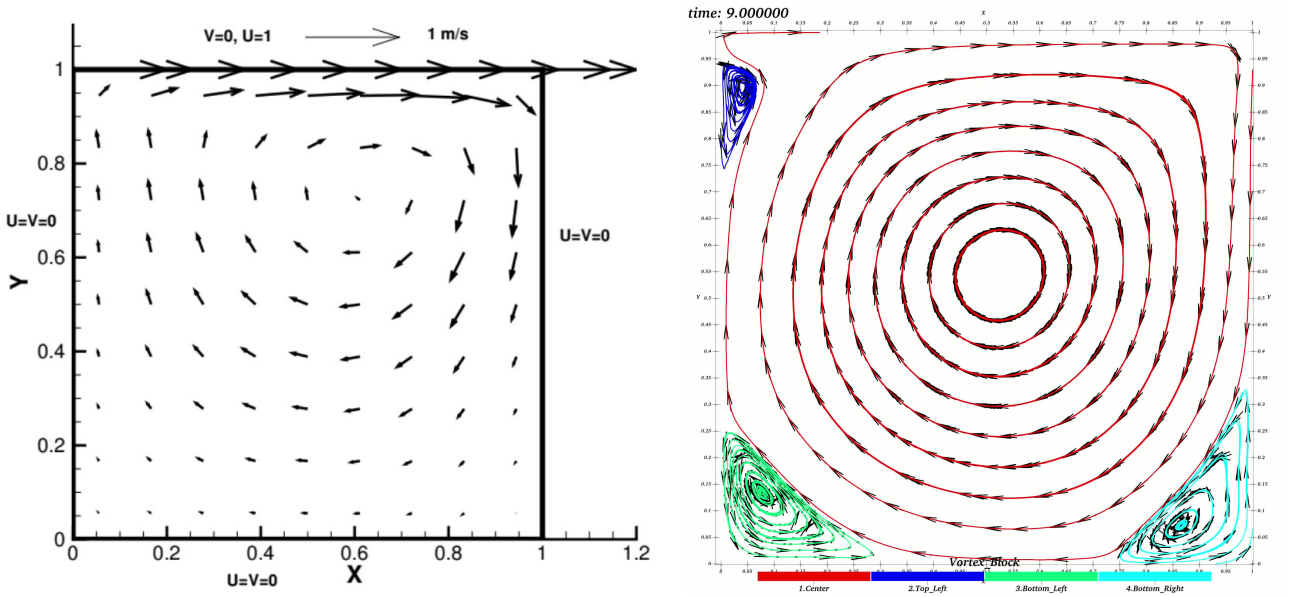
(a) Non-deferred correction failure (Re = 5000) **(b)** Deferred correction success (Re = 5000)

Figure 1.1: Comparison of correction methods for Lid-Driven Cavity Flow

3. A.2.4: FVM Deferred Correction with ADI+TDMA
   The last problem is about how can I reduce the computational time, furthermore, choosing the most appropriate relaxation factor set including factors of momentum equations and pressure factor in SIMPLE algorithm. When I was just using Gauss-Seidel method to solve the pressure correction equation, I had to wait for the converged iteration final for 3 days, and when I took the ADI (Alternating-Direction Implicit technique) to rearrange the discrete form of the momentum equation and pressure Poisson equation, the time I used reduced to 10 minutes.
4. Relaxation factor settings
   After I realized that I have to increase the computational efficiency, I also took the numerical experiment more and more again to find the most appropriate relaxation factor set. Finally, results I observed are summarized below:

- When pressure relaxation factor is smaller than velocity relaxation factor, the solution must be diverged despite how small the relaxation factors are.
- In some normal range (e.g., pressure relaxation factor = 0.3, velocity relaxation factor = 0.05), the difference of both is larger, the final error compared with the benchmark is smaller.
- The most appropriate relaxation factor set I found is: relaxation factor for pressure is 0.9, and for velocity is 0.01.

2 Problem Descriptions



(a) Problem Description (Re = 100) **(b)** Simulation Result (Re = 5000)

Figure 2.1: Lid-Driven Cavity Flow Problem and Simulation

Lid Driven Cavity Flow can be controlled by Navier-Stokes Equation, and continuity equation. Writing down the conservation form of Navier-Stokes Equations below:

$$\frac{\partial \rho U}{\partial x} + \frac{\partial \rho V}{\partial y} = 0$$

$$\frac{\partial \rho U}{\partial t} + \frac{\partial \rho UU}{\partial x} + \frac{\partial \rho VU}{\partial y} = -\frac{\partial P}{\partial x} + \mu \left[\frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2} \right] \tag{2.1}$$

$$\frac{\partial \rho V}{\partial t} + \frac{\partial \rho UV}{\partial x} + \frac{\partial \rho VV}{\partial y} = -\frac{\partial P}{\partial y} + \mu \left[\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} \right]$$

(a) Please compute the cavity flows with Reynolds number being 100, 1000 and 5000.
(b) Please use mesh size at least 81x81 and 161x161.
(c) Please also compare your results with the benchmark solutions from Ghia et al. (1982) for u(y) at x=0.5 and v(x) at y=0.5.
(d) Please compare the accuracy of the central difference, QUICK and MUSCL schemes.

and I will show the analysis below to answer these problems.

1. Streamline plots and velocity contours at Re = 100, 1000, 5000 with mesh size 81x81 and 161x161 for CDs, QUICK, MUSCL scheme.(36 figures)
2. centerline velocity profiles $u(y)$ and $v(x)$ comparison with different Reynolds number. (12 figures)
3. centerline velocity profiles comparison with different scheme. (6 figures)
4. centerline velocity profiles comparison with mesh size (Grid 81x81, 113x113, 161x161, 227x227). (4 figures)
5. Accuracy analysis and discussion.

3 Introduction of Methodology Adopted

3.1 Discretize the Governing Equations

3.1.1 Staggered Grid: Problem and Solving Method

Definition 3.1. Construct three types of mesh system :

1. primary point: The storage point of pressure or other scalar variables, and located at (I, J).
2. x-Velocity vice point: The storage point of x component of velocity, and located at (i, J).
3. y-Velocity vice point: The storage point of y component of velocity, and located at (I, j).

Definition 3.2. For solving flow field problem, the centers of Navier-Stokes equations were split into three parts :

1. control volume center of Continuity Equation : primary point (I, J)
2. control volume center of x-momentum equation : x-Velocity vice point (i, J)
3. control volume center of y-momentum equation : y-Velocity vice point (I, j)

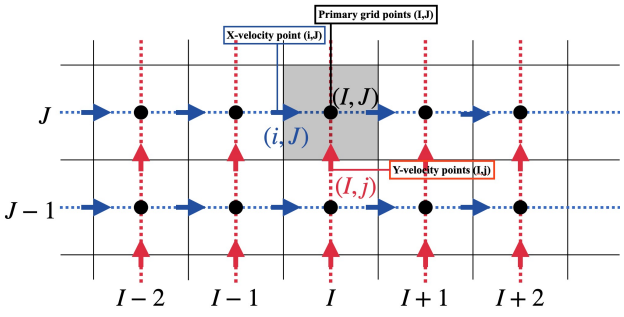


Figure 3.1: definition of primary point and vice point

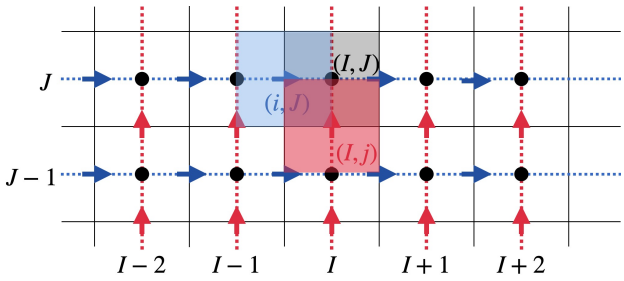


Figure 3.2: control volume center setting

Figure 3.3: staggered grid system

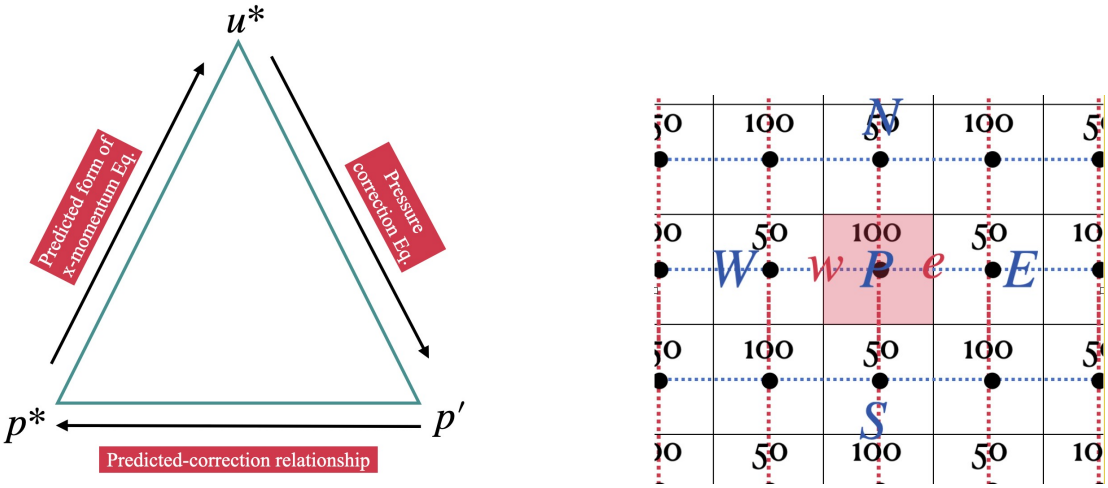
Why do we must use staggered grids to deal with flow field problem? The problem generated from difference scheme of the momentum equation, especially the discrete form of pressure gradient term.

$$-\frac{1}{\rho}\oint_{\partial\Omega} dy \cdot p \approx -\frac{1}{\rho}(p_e - p_w) \approx -\frac{1}{2\rho}(p_E - p_W) \tag{3.1}$$

so from equation (3.1), if we take the pressure gradient as central difference scheme, we will face the difficulty that difference distance of it is "2-δx". However, in SIMPLE algorithm, i.e.,

SIMPLE algorithm is a type of pressure and velocity coupling correction method.

we give an initial pressure predicted value at first, and solve the predicted form of momentum equation to get velocity predicted value. Furthermore, we use pressure correction equation to update pressure corrected value, and then update the next step pressure predicted value.



(a) coupling correction relationship

(b) checkerboard pressure field

Figure 3.4: discussion of reason why we adopted staggered grids setting

As figure 3.4 shows, we can find that the velocity and pressure fields are corrected in a coupled manner in the SIMPLE algorithm. If the pressure predicted value adds a sawtooth wave in the process of updating, the velocity generated from 2-δx pressure gradient will not correct the pressure correction value, and correct pressure predicted value at the next step. The result of the iteration is that the non-physical error will be maintained to the iteration final.

Velocity generated from 1-δx pressure gradient
can modify the non-physical phenomenon of the pressure predicted value updating.

So this is the reason why we must use staggered grid setting, because we have to create the velocity predicted value generated from 1-δx pressure gradient. If the velocity field is stored at its vice point, the corresponding pressure gradient can directly adopt the form: $\frac{A_e p_e - A_w p_w}{\rho}$, and then avoid the checkerboard pressure field generated at this moment.

3.1.2 Discrete Form of the Continuity Equation

The integration form of the Continuity Equation is :

$$\oint_{\partial\Omega} d\vec{a} \cdot \rho\vec{u} = 0 \tag{3.2}$$

- center of the control volume : primary point (I, J), as the definition 3.2 states.
- For incompressible flow and steady state flow, the density ρ is constant in space and time, so we can simplify the equation to :$\oint_{\partial\Omega} d\vec{a} \cdot \vec{u} = 0$

The discrete form of the Continuity Integration Equation is :

$$(u_e A_e - u_w A_w) + (v_n A_n - v_s A_s) = 0 \tag{3.3}$$

Furthermore, because we adopted the staggered grids setting, the variables v_n, v_s stored at the north and south side of the control volume, and variables u_e, u_w also stored at the east and west side.

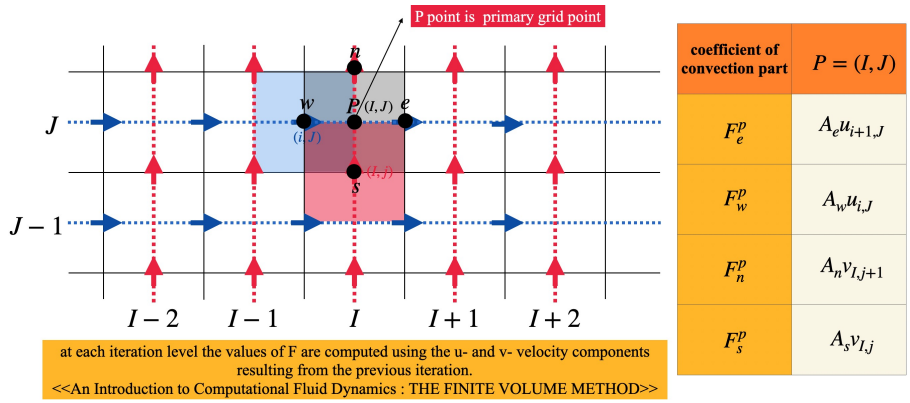


Figure 3.5: linear combination coefficients of the discrete Continuity Equation

General discrete form of the Continuity Equation:

$$F_e^P - F_w^P + F_n^P - F_s^P = 0 \tag{3.4}$$

where $F_e^P = u_e A_e$, $F_w^P = u_w A_w$, $F_n^P = v_n A_n$, $F_s^P = v_s A_s$ as the figure 3.5 shown.

3.1.3 General Discrete Form of the Momentum Integration Equations

For the title "General", meaning is that the center value of the each side of the control volume is substituted by the face variables first, and high order scheme [1] will discuss in the next section. The Derivation process below discusses the x-momentum integration equation as an example. For incompressible flow and steady state, **x-momentum integration equation** is :

$$\underbrace{\oint_{\partial\Omega} dy\, u \cdot u + dx\, v \cdot u}_{\text{Convection term}} = \underbrace{\int_{\Omega} d^3 r \left(-\frac{1}{\rho}\frac{\partial p}{\partial x}\right)}_{\text{Pressure gradient term}} + \underbrace{\frac{1}{R_e}\oint_{\partial\Omega}\left(dy\,\frac{\partial u}{\partial y} + dx\,\frac{\partial u}{\partial y}\right)}_{\text{Diffusion term}} \tag{3.5}$$

where Ω is the control volume, and $\partial\Omega$ is the close surface of the control volume.

- center of the control volume : x-Velocity face point (i, J)

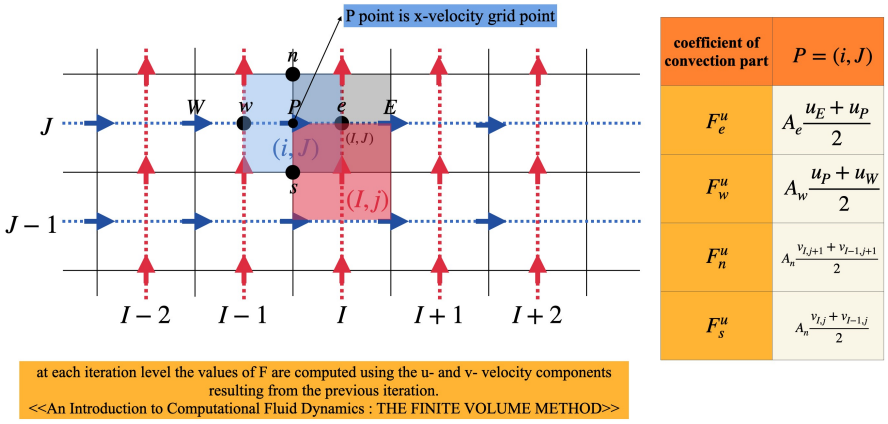


Figure 3.6: coefficients of the general discrete form of the x-momentum integration equation

For discretizing the equation (3.5), we start from discretizing each part of the equation :

1. pressure gradient term : the method is that I took the 2nd order Central Difference of the first order partial derivative of pressure below.

$$-\frac{1}{\rho}\oint_{\partial\Omega} dy \cdot p \approx -\frac{1}{\rho}(p_e - p_w) = p_{i,J} - p_{i-1,J} \tag{3.6}$$

[1] high order schemes in this assignment are Central Difference scheme, QUICK, and TVD-MUSCL scheme, talking about them at the next subsubsection.

2. momentum diffusion term : the key point about discretizing this part is using 2nd order Central Difference to approximate the first order partial derivative of the u-velocity located at the center of each side of the control volume.

$$\frac{1}{Re}\int_e dy \cdot \frac{\partial u}{\partial x} \overset{\text{Central Difference}}{=\!=\!=\!=\!=\!=\!=\!=\!=\!=\!=} \frac{A_e}{Re\delta x}\left(u_E - u_P\right) \tag{3.7}$$

where $\frac{A_e}{Re\delta x}$ is often defined as the symbol D_e, called it as "discrete diffusion coefficient".

3. momentum convection term : the difficulties of discretizing the convection term is how to decide the x-velocity values at the east and west of the control volume, i.e., u_e^{old}, u_w^{old}, and decide the y-velocity at the north and south face of the control volume, i.e., v_n^{old}, v_s^{old} ,as the figure 3.6 shown.

Using 2nd order Central Difference Scheme to approximate the variables at each face of the control volume of the x-momentum equation.[2]

(1) x-velocity at the east face of the control volume:

$$u_e \overset{\text{CDs}}{=} \frac{u_E + u_P}{2} = \frac{u_{i+1,J} + u_{i,J}}{2} \tag{3.8}$$

(2) x-velocity at the west face of the control volume:

$$u_w \overset{\text{CDs}}{=} \frac{u_P + u_W}{2} = \frac{u_{i,J} + u_{i-1,J}}{2} \tag{3.9}$$

(3) y-velocity at the north face of the control volume:

$$v_n \overset{\text{CDs}}{=} \frac{v_{i,J+1} + v_{i-1,J+1}}{2} \tag{3.10}$$

(4) y-velocity at the south face of the control volume:

$$v_s \overset{\text{CDs}}{=} \frac{v_{i,J} + v_{i-1,J}}{2} \tag{3.11}$$

$$\oint_{\partial\Omega} dy\, u \cdot u + dx\, v \cdot u \approx \left(u_e A_e u_e^{old} - u_w A_w u_w^{old}\right) + \left(v_n A_n u_n^{old} - v_s A_s u_s^{old}\right) \tag{3.12}$$

where $u_e^{old}, u_w^{old}, v_n^{old}, v_s^{old}$ are generated from last iteration step. This equation is the general discrete form of the momentum convection term.

Combining equations (3.6), (3.12), and diffusion term together, we can get **the general discrete form of the x-momentum integration equation below** :

$$\begin{aligned} F_e^u u_e - F_w^u u_w + F_n^u u_n - F_s^u u_s = \frac{1}{\rho}\frac{-(p_{I,J} - p_{I-1,J})}{\delta x}\delta x \delta y \\ - u_P(D_w + D_e + D_s + D_n) + u_W(D_w) + u_E(D_e) + u_S(D_s) + u_N(D_n) \end{aligned} \tag{3.13}$$

[2] The approximating method of the u_e, u_w, v_n, v_s is independent of the equation that the variables maintain, so I call the method as "Central Difference" for all variables.

3.1.4 High Order Scheme of the Momentum Integration Equations

"High order scheme" means that take high order approximation method to center value of each side of the control volume, so we can write down the general form of the approximation about them with flux limiter function :

$$\begin{aligned} u_f &\xlongequal{\text{general approximation}} D(r_e)u_D + U(r_f)u_U + UU(r_f)u_{UU} \\ &\xlongequal{\text{CDs approximation}} 0.5u_D + 0.5u_U + 0u_{UU} \\ &\xlongequal{\text{QUICK approximation}} \frac{3}{8}u_D + \frac{6}{8}u_U + \frac{-1}{8}u_{UU} \\ &\xlongequal{\text{TVD-MUSCL approximation}} d(r_e)u_D + c(r_f)u_U + uu(r_f)u_{UU} \end{aligned} \tag{3.14}$$

TVD-MUSCL	$D(r_f)$	$U(r_f)$	$UU(r_f)$
$r_f < 0$	0.0	1.0	0.0
$0 \le r_f < \frac{1}{3}$	0.0	2.0	−1.0
$\frac{1}{3} \le r_f < 3$	0.25	1.0	−0.25
$3 \le r_f$	1.0	0.0	0.0

Figure 3.7: combination coefficients of TVD-MUSCL scheme approximation

For incompressible flow and steady state flow, **high order scheme of the x-momentum integration equation** :

$$\begin{aligned} &a_P u_P + a_{EE}u_{EE} + a_E u_E + a_W u_W + a_{WW}u_{WW} \\ &\quad + a_{NN}u_{NN} + a_N u_N + a_S u_S + a_{SS}u_{SS} \\ &\quad = \frac{1}{\rho}\frac{-(p_P - p_w)}{\delta x}\delta x \delta y \end{aligned} \tag{3.15}$$

Below we will set the different linear combination coefficients of the equation (3.15) for different high order scheme,i.e., QUICK and TVD-MUSCL scheme.

Discussion : (1) For QUICK scheme, the linear combination coefficients are :

$$\begin{cases} a_P = -(a_{EE} + a_E + a_W + a_{WW} + a_{NN} + a_N + a_S + a_{SS}) + F_e^u - F_w^u + F_n^u - F_s^u \\ a_{WW} = 0 + (\alpha_w \frac{-1}{8})(-F_w^u) \\ a_W = (\alpha_e \frac{-1}{8})F_e^u + ((1-\alpha_w)\frac{3}{8} + \alpha_w \frac{6}{8})(-F_w^u) - D_e \\ a_E = ((1-\alpha_e)\frac{6}{8} + \alpha_e \frac{3}{8})F_e^u + ((1-\alpha_w)\frac{-1}{8})(-F_w^u) - D_w \\ a_{EE} = ((1-\alpha_e)\frac{-1}{8})F_e^u + 0 \\ a_{SS} = 0 + (\alpha_s \frac{-1}{8})(-F_s^u) \\ a_S = (\alpha_n \frac{-1}{8})F_n^u + ((1-\alpha_s)\frac{3}{8} + \alpha_s \frac{6}{8})(-F_s^u) - D_n \\ a_N = ((1-\alpha_n)\frac{6}{8} + \alpha_n \frac{3}{8})F_n^u + ((1-\alpha_s)\frac{-1}{8})(-F_s^u) - D_s \\ a_{NN} = ((1-\alpha_n)\frac{-1}{8})F_n^u + 0 \end{cases} \tag{3.16}$$

As the figure shown :

QUICK scheme

If $(F_e, F_w, F_n, F_s) > 0$, then $(\alpha_e, \alpha_w, \alpha_n, \alpha_s) = 1.0$, else $= (\alpha_e, \alpha_w, \alpha_n, \alpha_s) = 0.0$.

coefficients of linear combination	a_P	a_{WW}	a_W	a_E	a_{EE}	a_{SS}	a_S	a_N	a_{NN}	
Convection part	$((1-\alpha_e)\frac{3}{8}) + \alpha_e\frac{6}{8})F_e^u$	0	$(\alpha_e\frac{-1}{8})F_e^u$	$((1-\alpha_e)\frac{6}{8} + \alpha_e\frac{3}{8})F_e^u$	$((1-\alpha_e)\frac{-1}{8})F_e^u$		$*(-1)$			F_e^u
	$((1-\alpha_w)\frac{6}{8} + \alpha_w\frac{3}{8})(-F_w^u)$	$(\alpha_w\frac{-1}{8})(-F_w^u)$	$((1-\alpha_w)\frac{3}{8} + \alpha_w\frac{6}{8})(-F_w^u)$	$((1-\alpha_w)\frac{-1}{8})(-F_w^u)$	0		$*(-1)$			$(-F_w^u)$
	$((1-\alpha_n)\frac{3}{8} + \alpha_n\frac{6}{8})F_n^u$					0	$(\alpha_n\frac{-1}{8})F_n^u$	$((1-\alpha_n)\frac{6}{8} + \alpha_n\frac{3}{8})F_n^u$	$((1-\alpha_n)\frac{-1}{8})F_n^u$	F_n^u
	$((1-\alpha_s)\frac{6}{8} + \alpha_s\frac{3}{8})(-F_s^u)$					$(\alpha_s\frac{-1}{8})(-F_s^u)$	$((1-\alpha_s)\frac{3}{8} + \alpha_s\frac{6}{8})(-F_s^u)$	$((1-\alpha_s)\frac{-1}{8})(-F_s^u)$	0	$(-F_s^u)$
Diffusion part	$D_w + D_e + D_s + D_n$		$-D_w$	$-D_e$			$-D_n$	$-D_s$		

Figure 3.8: coefficients setting of QUICK scheme

(2) For TVD-MUSCL scheme, the linear combination coefficients are :

$$
\begin{cases}
a_P = -(a_{EE} + a_E + a_W + a_{WW} + a_{NN} + a_N + a_S + a_{SS}) \\
\qquad + (D(r_w) + U(r_w) + UU(r_w))\,(-F_w^u) \\
\qquad + (D(r_e) + U(r_e) + UU(r_e))\,F_e^u \\
\qquad + (D(r_s) + U(r_s) + UU(r_s))\,(-F_s^u) \\
\qquad + (D(r_n) + U(r_n) + UU(r_n))\,F_n^u \\
a_{WW} = 0 + (\alpha_w UU(r_w))(-F_w^u) \\
a_W = (\alpha_e UU(r_e))F_e^u + ((1-\alpha_w)D(r_w) + \alpha_w U(r_w))(-F_w^u) - D_w \\
a_E = ((1-\alpha_e)U(r_e) + \alpha_e D(r_e))F_e^u + ((1-\alpha_w)UU(r_w))(-F_w^u) - D_e \\
a_{EE} = ((1-\alpha_e)UU(r_e))F_e^u + 0 \\
a_{SS} = 0 + (\alpha_s UU(r_s))(-F_s^u) \\
a_S = (\alpha_n UU(r_n))F_n^u + ((1-\alpha_s)D(r_s) + \alpha_s U(r_s))(-F_s^u) - D_s \\
a_N = ((1-\alpha_n)U(r_n) + \alpha_n D(r_n))F_n^u + ((1-\alpha_s)UU(r_s))(-F_s^u) - D_s \\
a_{NN} = ((1-\alpha_n)UU(r_n))F_n^u + 0
\end{cases}
\tag{3.17}
$$

As the figure shown :

TVD-MUSCL scheme

If $(F_e, F_w, F_n, F_s) > 0$, then $(\alpha_e, \alpha_w, \alpha_n, \alpha_s) = 1.0$, else $= (\alpha_e, \alpha_w, \alpha_n, \alpha_s) = 0.0$.

coefficients of linear combination	a_P	a_{WW}	a_W	a_E	a_{EE}	a_{SS}	a_S	a_N	a_{NN}	
Convection part	$((1-\alpha_e)D(r_e) + \alpha_e U(r_e))F_e^u$	0	$(\alpha_e UU(r_e))F_e^u$	$((1-\alpha_e)U(r_e) + \alpha_e D(r_e))F_e^u$	$((1-\alpha_e)UU(r_e))F_e^u$		$*(-1)$			$(U(r_e) + D(r_e) + UU(r_e))F_e^u$
	$((1-\alpha_w)U(r_w) + \alpha_w D(r_w))(-F_w^u)$	$(\alpha_w UU(r_w)))(-F_w^u)$	$((1-\alpha_w)D(r_w) + \alpha_w U(r_w))(-F_w^u)$	$((1-\alpha_w)UU(r_w))(-F_w^u)$	0		$*(-1)$			$U(r_w) + D(r_w) + UU(r_w))(-F_w^u)$
	$((1-\alpha_n)D(r_n) + \alpha_n U(r_n))F_n^u$					0	$(\alpha_n UU(r_n))F_n^u$	$((1-\alpha_n)U(r_n) + \alpha_n D(r_n))F_n^u$	$((1-\alpha_n)UU(r_n))F_n^u$	$(U(r_n) + D(r_n) + UU(r_n))F_n^u$ $*(-1)$
	$((1-\alpha_s)U(r_s) + \alpha_s D(r_s))(-F_s^u)$					$(\alpha_s UU(r_s))(-F_s^u)$	$((1-\alpha_s)D(r_s) + \alpha_s U(r_s))(-F_s^u)$	$((1-\alpha_s)UU(r_s))(-F_s^u)$	0	$(U(r_s) + D(r_s) + UU(r_s))(-F_s^u)$ $*(-1)$
Diffusion part	$D_w + D_e + D_s + D_n$		$-D_w$	$-D_e$			$-D_n$	$-D_s$		

Figure 3.9: coefficients setting of TVD-MUSCL scheme

Remark: The corresponding C++ implementation of the TVD-MUSCL scheme coefficients in equation (3.17) can be found in the non-deferred correction code (see Appendix A.2.2). Below we present the key code snippets that demonstrate how these theoretical coefficients are computed in practice.

Remark 3.1 (Implementation of TVD-MUSCL Coefficients)**.** The implementation follows a systematic approach for each control volume face (west, east, south, north). For each face, we compute the contributions to neighboring coefficients $(a_{WW}, a_W, a_E, a_{EE}, a_{SS}, a_S, a_N, a_{NN})$ and the source term based on the flux limiter functions $D(r)$, $U(r)$, and $UU(r)$.

West Face Implementation (lines 405–409): The west face contributes to coefficients a_{WW}, a_W, a_E, and a_P according to equation (3.17):

```
1 a_WW[i-1][J-1] += (alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1]) ;
2 a_W[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*D(r_w_u[i-1][J-1])+alpha_w_u[i-1][J-1]*U(r_w_u[i-1][J
      -1]))*(-F_w_u[i-1][J-1]) - D_w ;
3 a_E[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1]);
4 a_P[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*U(r_w_u[i-1][J-1])+alpha_w_u[i-1][J-1]*D(r_w_u[i-1][J
      -1]))*(-F_w_u[i-1][J-1])+D_w ;
5 S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho ;
```

Listing 1: West face coefficient computation

East Face Implementation (lines 417–421): The east face contributes to coefficients a_{EE}, a_E, a_W, and a_P:

```
1 a_EE[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J-1]) ;
2 a_E[i-1][J-1] += (alpha_e_u[i-1][J-1]*D(r_e_u[i-1][J-1])+(1-alpha_e_u[i-1][J-1])*U(r_e_u[i-1][J
      -1]))*(F_e_u[i-1][J-1]) - D_e ;
3 a_W[i-1][J-1] += (alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J-1]) ;
4 a_P[i-1][J-1] += ((1-alpha_e_u[i-1][J-1])*D(r_e_u[i-1][J-1])+alpha_e_u[i-1][J-1]*U(r_e_u[i-1][J
      -1]))*(F_e_u[i-1][J-1])+D_e;
5 S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho ;
```

Listing 2: East face coefficient computation

South Face Implementation (lines 445–448): The south face contributes to coefficients a_{SS}, a_S, a_N, and a_P:

```
a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1]) ;
a_S[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(alpha_s_u[i-1][J-1])*U(r_s_u[i
    -1][J-1]))*(-F_s_u[i-1][J-1])-D_s ;
a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1]) ;
a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*U(r_s_u[i-1][J-1])+alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J
    -1]))*(-F_s_u[i-1][J-1])+D_s ;
```

Listing 3: South face coefficient computation

North Face Implementation (lines 473–476): The north face contributes to coefficients a_{NN}, a_N, a_S, and a_P:

```
a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J-1]) ;
a_N[i-1][J-1] += (alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i-1][J-1])*U(r_n_u[i-1][J
    -1]))*(F_n_u[i-1][J-1]) - D_n ;
a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J-1]) ;
a_P[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u[i-1][J-1]*U(r_n_u[i-1][J
    -1]))*(F_n_u[i-1][J-1])+D_n ;
```

Listing 4: North face coefficient computation

Let's go back to the equation (3.15), we only need to change the coefficients setting (i.e., equations (3.16) and (3.17)) to get the different high order discrete scheme of the x-momentum integration equation. Similarly, we can get the high order scheme of y-momentum integration equation with the same method, and the thing you have to watch out is that coefficients setting is very complicated, in the next section, I will simplify the form of the discrete equation using deferred correction.

3.2 SIMPLE Algorithm

3.2.1 Predicted-Correction Method

In predicted-correction method, split the variables, i.e., u-velocity, v-velocity, pressure field, into three parts, i.e., correct value, correction value and predicted value.

$$\begin{aligned} u_{i,J} &= u'_{i,J} + u^{\star}_{i,J} \\ v_{I,j} &= v'_{I,j} + v^{\star}_{I,j} \\ p_{I,J} &= p'_{I,J} + p^{\star}_{I,J} \end{aligned} \tag{3.18}$$

So in this method, we have increased the number of variables from 3 to 9.

3 variables: $\begin{cases} u_{i,J} \\ v_{I,j} \\ p_{I,J} \end{cases}$ $\xrightarrow{\text{Predicted-Correction Method}}$ **9 variables:** $\begin{bmatrix} u_{i,J} & u^{*}_{i,J} & u'_{i,J} \\ v_{I,j} & v^{*}_{I,j} & v'_{I,j} \\ p_{I,J} & p^{*}_{I,J} & p'_{I,J} \end{bmatrix}$

Problem Identification: In Navier-Stokes equations, pressure and velocity fields are coupled with each other; therefore, no variable can be solved by itself alone in the momentum equation system. The solving method for the problem above is that when I give a pressure field, I can get the corresponding velocity from the momentum equations. The implication is that the correct

pressure can yield the correct velocity. However, how can I define a correct pressure? We have another governing equation, the Continuity Equation,

$$A_e u_{i+1,J} - A_w u_{i,J} + A_n v_{I,j+1} - A_s v_{I,j} = 0$$

, so we hope that this equation can be equivalent to another form for pressure maintenance. Solving Method: We set the variables into two parts, i.e., predicted value and correction value. First, given the pressure predicted value $p^\star_{I,J}$, we can obtain the corresponding velocity predicted value. Second, use the velocity I get to go back solve the presswure corection and correct the pressure in the next step. so in this algorithm, we can find a thing is pressure and velocity are corrected in an iterative way using predicted value and correction value.

3.2.2 Algorithmic Steps and Under-Relaxation

This subsection presents a textbook-level SIMPLE description. At iteration N, the velocity and pressure fields are decomposed into predicted and correction parts. The predicted fields are obtained from the momentum equations using the predicted pressure $p^\star$, and the continuity residual of the predicted field drives the pressure-correction equation.

Under-relaxation in the predicted momentum equations. The momentum equations are linearized as $a_P\phi_P = \sum_{nb} a_{nb}\phi_{nb} + b$. To stabilize the nonlinear coupling, under-relaxation is applied to the predicted velocities. This is achieved by scaling the diagonal coefficient and adding a compensation term to the source. For the staggered grid used in this report, the predicted momentum equations are:

$$\begin{aligned}
\frac{(a_P)^{\star N}_{i,J}}{\alpha_u} u^{\star\, N+\frac{1}{2}}_{i,J} &= \sum_{nb} (a_{nb})^{\star N}_{i,J} u^{\star N}_{nb} - \frac{p^{\star N}_{I,J} - p^{\star N}_{I-1,J}}{\rho\,\delta x}\,\delta x\,\delta y + \frac{1-\alpha_u}{\alpha_u}(a_P)^{\star N}_{i,J} u^{N}_{i,J} \\
\frac{(b_P)^{\star N}_{I,j}}{\alpha_v} v^{\star\, N+\frac{1}{2}}_{I,j} &= \sum_{nb} (b_{nb})^{\star N}_{I,j} v^{\star N}_{nb} - \frac{p^{\star N}_{I,J} - p^{\star N}_{I,J-1}}{\rho\,\delta y}\,\delta x\,\delta y + \frac{1-\alpha_v}{\alpha_v}(b_P)^{\star N}_{I,j} v^{N}_{I,j}
\end{aligned} \tag{3.19}$$

where α_u and α_v are the relaxation factors for the predicted velocities. The added source terms ensure consistency with the original momentum equations while damping the update.

Continuity residual and pressure correction. The predicted velocity field generally violates continuity; therefore the discrete continuity residual

$$d^{\star N}_{I,J} = A_e u^{\star\, N+\frac{1}{2}}_{i+1,J} - A_w u^{\star\, N+\frac{1}{2}}_{i,J} + A_n v^{\star\, N+\frac{1}{2}}_{I,j+1} - A_s v^{\star\, N+\frac{1}{2}}_{I,j} \tag{3.20}$$

is used as the source term of the pressure-correction equation (derived in the next subsection). Once p' is solved, pressure is still under-relaxed, while velocities are corrected by the standard SIMPLE relations:

$$\begin{aligned}
p^{N+1}_{I,J} &= p^{N}_{I,J} + \alpha_p p'^{\,N+1}_{I,J} \\
u^{N+1}_{i,J} &= u^{\star\, N+\frac{1}{2}}_{i,J} + u'^{N+1}_{i,J} \\
v^{N+1}_{I,j} &= v^{\star\, N+\frac{1}{2}}_{I,j} + v'^{N+1}_{I,j}
\end{aligned} \tag{3.21}$$

where α_u and α_v are used in the predicted momentum equations (3.19), while α_p is applied to the pressure update in (3.21) to stabilize the iteration.

Algorithmic loop. The SIMPLE iteration used in this report can be summarized as:

1. Solve predicted momentum equations with $p^\star$ to obtain $u^\star$ and $v^\star$.
2. Assemble and solve the pressure-correction equation using the continuity residual of $u^\star, v^\star$ (equation (3.20)).
3. Correct pressure and velocities with equations (3.21).
4. Check continuity; if residual is above tolerance, set $p^\star \leftarrow p^{N+1}$, $u^\star \leftarrow u^{N+1}$, $v^\star \leftarrow v^{N+1}$ and repeat.

3.2.3 Staggered Grid and Pressure-Velocity Coupling

The SIMPLE algorithm is typically implemented on a staggered grid to avoid the checkerboard pressure field. Pressure is stored at cell centers, while velocity components are stored at the corresponding cell faces. This arrangement ensures that the pressure gradient in the momentum equations and the mass flux in the continuity equation are evaluated at consistent locations, providing a stable and physically meaningful pressure–velocity coupling.

3.2.4 Continuity Residual and Convergence Criterion

The pressure-correction equation is driven by the continuity residual computed from the predicted velocity field. A practical convergence criterion is to monitor the normalized residual:

$$R_m = \frac{\sum_{I,J} \left| d^{\star N}_{I,J} \right|}{\sum_{I,J} \left(A_e + A_w + A_n + A_s \right)} \tag{3.22}$$

The iteration is considered converged when R_m falls below a prescribed tolerance (e.g., 10^{-6}) and the corrections p', u', v' become negligibly small. This criterion ensures global mass conservation.

3.2.5 Boundary Conditions for Pressure Correction

The pressure correction p' must be assigned boundary conditions consistent with the velocity boundary conditions. For no-penetration walls, the normal velocity correction is zero, which leads to a homogeneous Neumann condition for pressure correction:

$$\frac{\partial p'}{\partial n} = 0 \quad \text{on solid walls.} \tag{3.23}$$

One reference pressure point is usually fixed to remove the null space of the Poisson operator.

3.2.6 SIMPLE vs. SIMPLEC

SIMPLEC modifies the velocity-correction relation by partially retaining the neighbor correction terms omitted in SIMPLE, which effectively increases the coupling between pressure and velocity. This leads to a stronger correction and typically faster convergence, but it may require smaller relaxation factors for stability. The present report follows the standard SIMPLE formulation for clarity and robustness.

3.3 Patankar s Omission in SIMPLE Algorithm

To derive the velocity correction equation from the momentum equation, we start with the predicted form and the corrected form of the discrete x-momentum equation. Subtracting the predicted form from the corrected form yields the momentum correction equation:

$$\begin{aligned} &a_P u'_P + a_{WW} u'_{WW} + a_W u'_W + a_E u'_E + a_{EE} u'_{EE} \\ &\quad + a_{SS} u'_{SS} + a_S u'_S + a_N u'_N + a_{NN} u'_{NN} \\ &\quad = -\frac{1}{\rho} \frac{(p'_P - p'_w)}{\delta x} \delta x \delta y \end{aligned} \tag{3.24}$$

where the superscript $'$ denotes the correction value. Equation (3.24) represents the exact correction relationship. However, solving this equation is computationally expensive because it couples the velocity correction at all neighboring nodes.

Patankar's Simplification: To reduce computational cost, Patankar proposed to *omit the neighboring velocity correction terms* on the left-hand side of equation (3.24). This simplification yields:

$$a_P u'_P \approx -\frac{1}{\rho} \frac{(p'_P - p'_w)}{\delta x} \delta x \delta y \tag{3.25}$$

Solving for the velocity correction u'_P:

$$\boxed{u'_P = -\frac{\delta y}{\rho\, a_P}(p'_P - p'_w)} \tag{3.26}$$

This simplified form has a clear physical interpretation: the velocity correction is proportional to the pressure correction gradient. The key advantages of Patankar's omission are:

1. **Computational efficiency**: The velocity correction can be calculated explicitly from the pressure correction without requiring iterative solution of coupled equations.
2. **Algorithm simplicity**: The decoupling allows for a sequential solution procedure in the SIMPLE algorithm.
3. **Convergence behavior**: Although the omission introduces approximation error, the iterative nature of SIMPLE algorithm compensates for this error through successive corrections.

Remark: The omitted terms $(a_{WW}u'_{WW} + a_W u'_W + \cdots)$ are recovered through the iterative process. As the solution converges, both $u' \to 0$ and $p' \to 0$, making the omission exact at convergence. This is why SIMPLE is called a *Semi-Implicit* method Xthe pressure-velocity coupling is only partially implicit.

3.3.1 Derivatation of Pressure Correction Equation

The pressure correction equation, also known as the pressure Poisson equation, is derived by enforcing the continuity equation on the corrected velocity field. Starting from the discrete continuity equation (3.4) at the primary control volume centered at (I, J):

$$A_e u_{i+1,J} - A_w u_{i,J} + A_n v_{I,j+1} - A_s v_{I,j} = 0 \tag{3.27}$$

Step 1: Decompose velocities into predicted and correction parts

Substitute the predicted-correction decomposition $u = u^\star + u'$ and $v = v^\star + v'$ into equation (3.27):

$$\begin{aligned} &A_e(u^\star_{i+1,J} + u'_{i+1,J}) - A_w(u^\star_{i,J} + u'_{i,J}) \\ &\quad + A_n(v^\star_{I,j+1} + v'_{I,j+1}) - A_s(v^\star_{I,j} + v'_{I,j}) = 0 \end{aligned} \tag{3.28}$$

Rearranging to separate predicted and correction terms:

$$\underbrace{(A_e u^\star_{i+1,J} - A_w u^\star_{i,J} + A_n v^\star_{I,j+1} - A_s v^\star_{I,j})}_{\text{mass flux imbalance from predicted velocity}} + \underbrace{(A_e u'_{i+1,J} - A_w u'_{i,J} + A_n v'_{I,j+1} - A_s v'_{I,j})}_{\text{correction to satisfy continuity}} = 0 \tag{3.29}$$

Step 2: Substitute Patankar's velocity correction formula

From equation (3.26), the velocity corrections are:

$$\begin{aligned} u'_{i,J} &= -\frac{\delta y}{\rho\,(a_P)_{i,J}}(p'_{I,J} - p'_{I-1,J}) \\ u'_{i+1,J} &= -\frac{\delta y}{\rho\,(a_P)_{i+1,J}}(p'_{I+1,J} - p'_{I,J}) \\ v'_{I,j} &= -\frac{\delta x}{\rho\,(b_P)_{I,j}}(p'_{I,J} - p'_{I,J-1}) \\ v'_{I,j+1} &= -\frac{\delta x}{\rho\,(b_P)_{I,j+1}}(p'_{I,J+1} - p'_{I,J}) \end{aligned} \tag{3.30}$$

where $(a_P)_{i,J}$ denotes the diagonal coefficient for x-momentum at location (i, J), and $(b_P)_{I,j}$ denotes the diagonal coefficient for y-momentum at location (I, j).

Step 3: Assemble the pressure correction equation

Substituting equations (3.30) into the correction part of equation (3.29):

$$\begin{aligned}
&\left(\frac{\delta y}{\rho(a_P)_{i+1,J}}+\frac{\delta y}{\rho(a_P)_{i,J}}+\frac{\delta x}{\rho(b_P)_{I,j+1}}+\frac{\delta x}{\rho(b_P)_{I,j}}\right)p'_{I,J}\\
&\quad-\frac{\delta y}{\rho(a_P)_{i,J}}p'_{I-1,J}-\frac{\delta y}{\rho(a_P)_{i+1,J}}p'_{I+1,J}\\
&\quad-\frac{\delta x}{\rho(b_P)_{I,j}}p'_{I,J-1}-\frac{\delta x}{\rho(b_P)_{I,j+1}}p'_{I,J+1}\\
&=\underbrace{-(A_e u^\star_{i+1,J}-A_w u^\star_{i,J}+A_n v^\star_{I,j+1}-A_s v^\star_{I,j})}_{\text{source term: negative of mass flux imbalance}}
\end{aligned} \tag{3.31}$$

This is a **Poisson equation** for the pressure correction p'. The source term on the right-hand side represents the mass flux imbalance computed from the predicted velocity field. The pressure correction field is solved to eliminate this imbalance and enforce mass conservation.

Physical Interpretation:

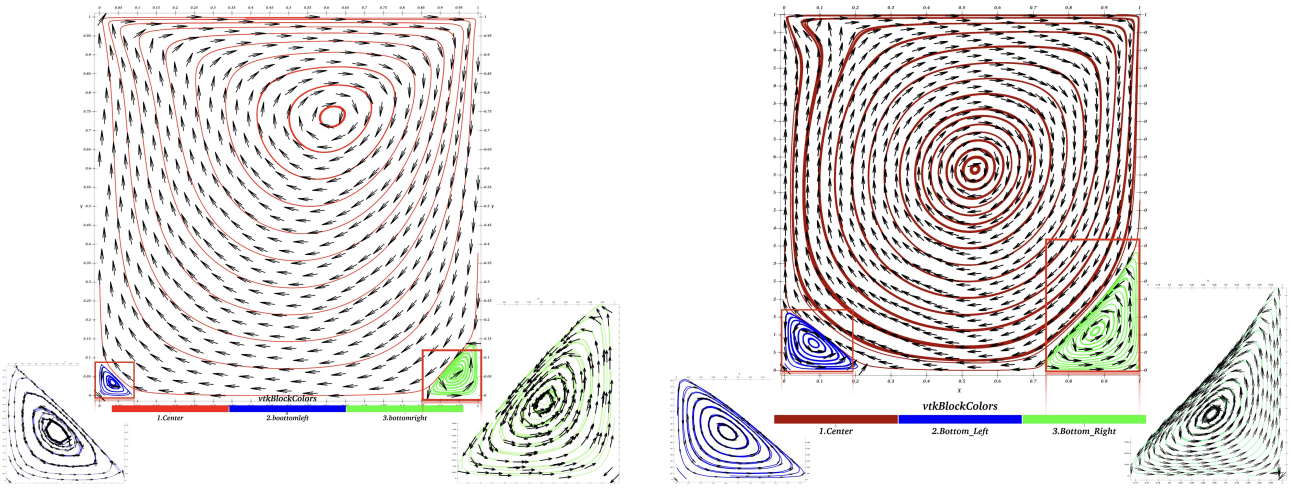
- When the predicted velocity satisfies continuity (i.e., source term = 0), the pressure correction p' becomes zero.
- A positive source term (net outflow) indicates that pressure needs to decrease, driving fluid into the control volume.
- The Laplacian structure of the left-hand side ensures that pressure adjustments propagate throughout the domain to satisfy global mass conservation.

The detailed algebraic form of this equation is presented in Appendix B, equation (B.2).

4 Results and Discussion

4.1 Vortex Structure Preview

Three figures below shows that the vortex structure of Lid-Driven Cavity Flow at different Re numbers(Re = 100, 1000, 5000)



(a) **(b)**

Figure 4.1: Vortex structures at Re = 100, 1000, (a) Re = 100, (b)Re = 1000

As the figure 4.1 shown, we can see the fact that when Re number increases form 100 to 1000, despite of moving to the center of the domain for primary vortex, two second vortex structure at bottom-left corner and botto-right corner is gradually growing.

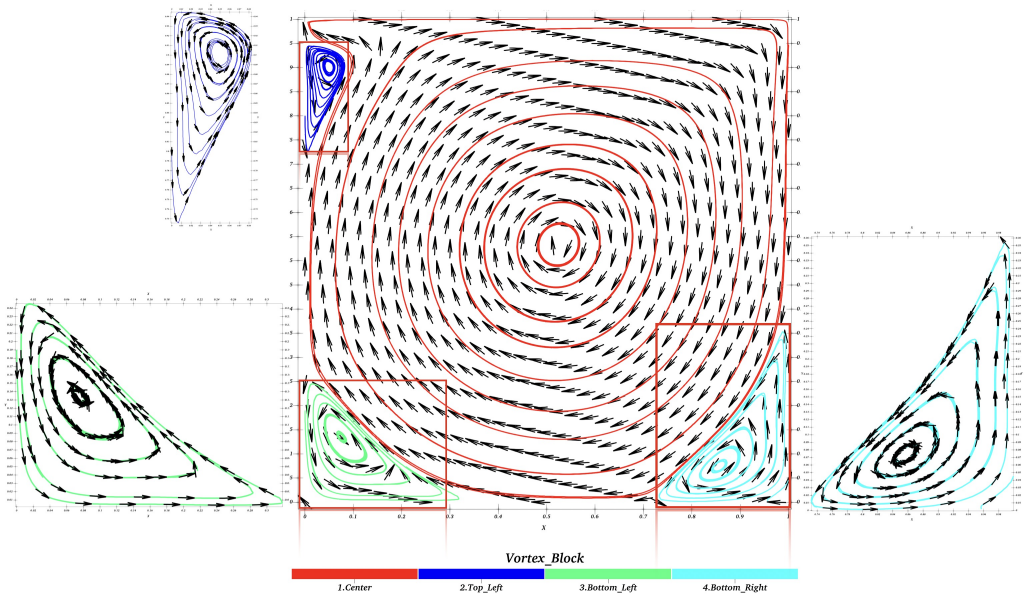


Figure 4.2: Vortex structure at Re = 5000

As shown in Figure 4.2, when the Reynolds number equals 5000, a third secondary vortex appears at the top-left of the flow field. This phenomenon indicates that the vortex structure becomes more complicated as the inertia of the flow field increases. (1)Red domain is the primary votex, (2)blue domain is the second vortex at top-left corner, (3)green domain is the second vortex at bottom-left corner of the flow field ,and(4)light-blue domain is the second vortex at bottom-right corner of the flow field.

4.2 Streamline Plots

This section presents streamline patterns for three numerical schemes (Central Difference Scheme, QUICK, and TVD-MUSCL) across different Reynolds numbers (Re = 100, 1000, 5000) and grid resolutions (81 81 and 161 161).

4.2.1 Arrangement of Streamline Patterns

The below figure shows the arrangement method of my streamline plots results, split them into 81 81 and 161 161 parts at first, and separate figures into three parts based on 3 types of Re numbers.

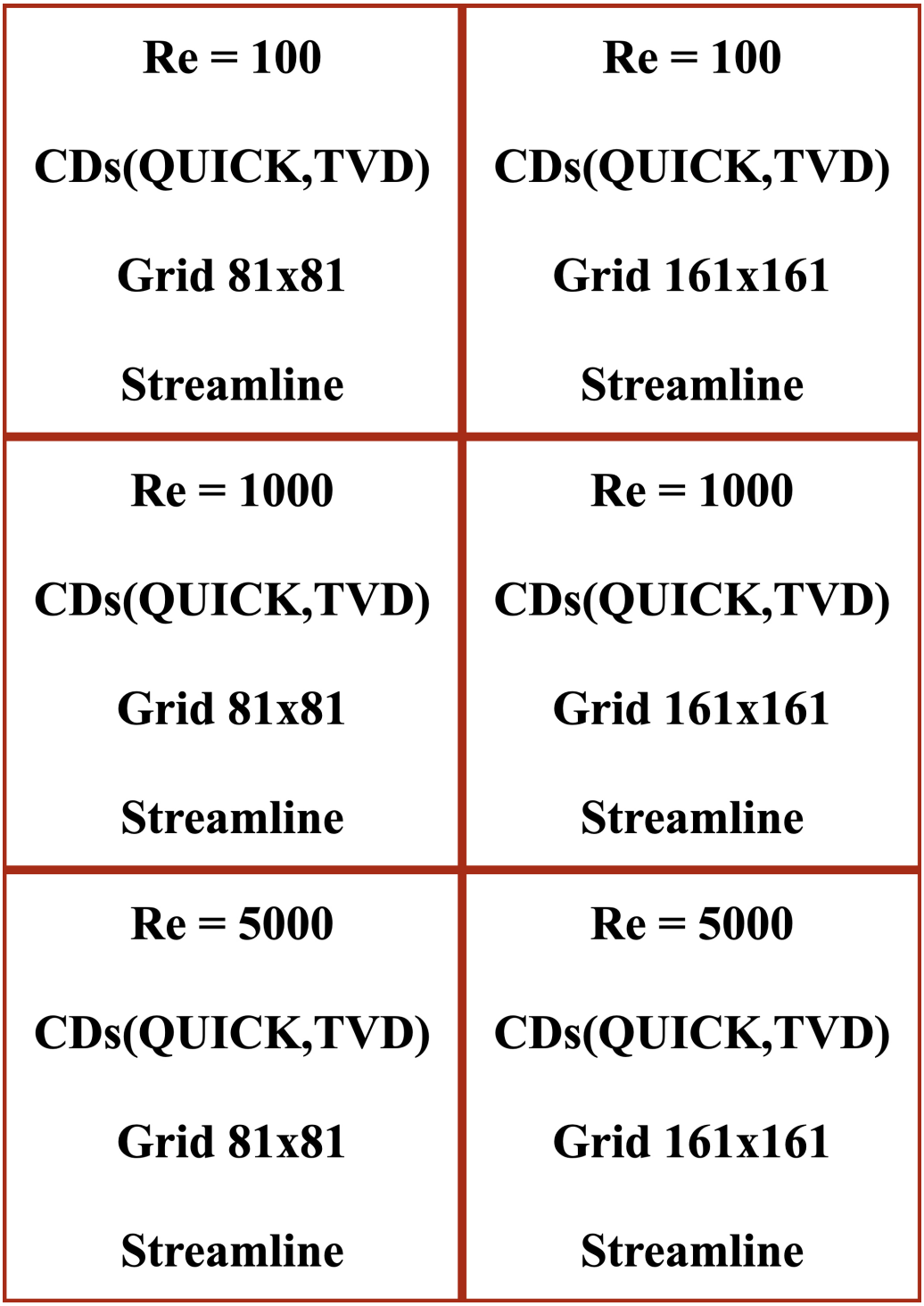
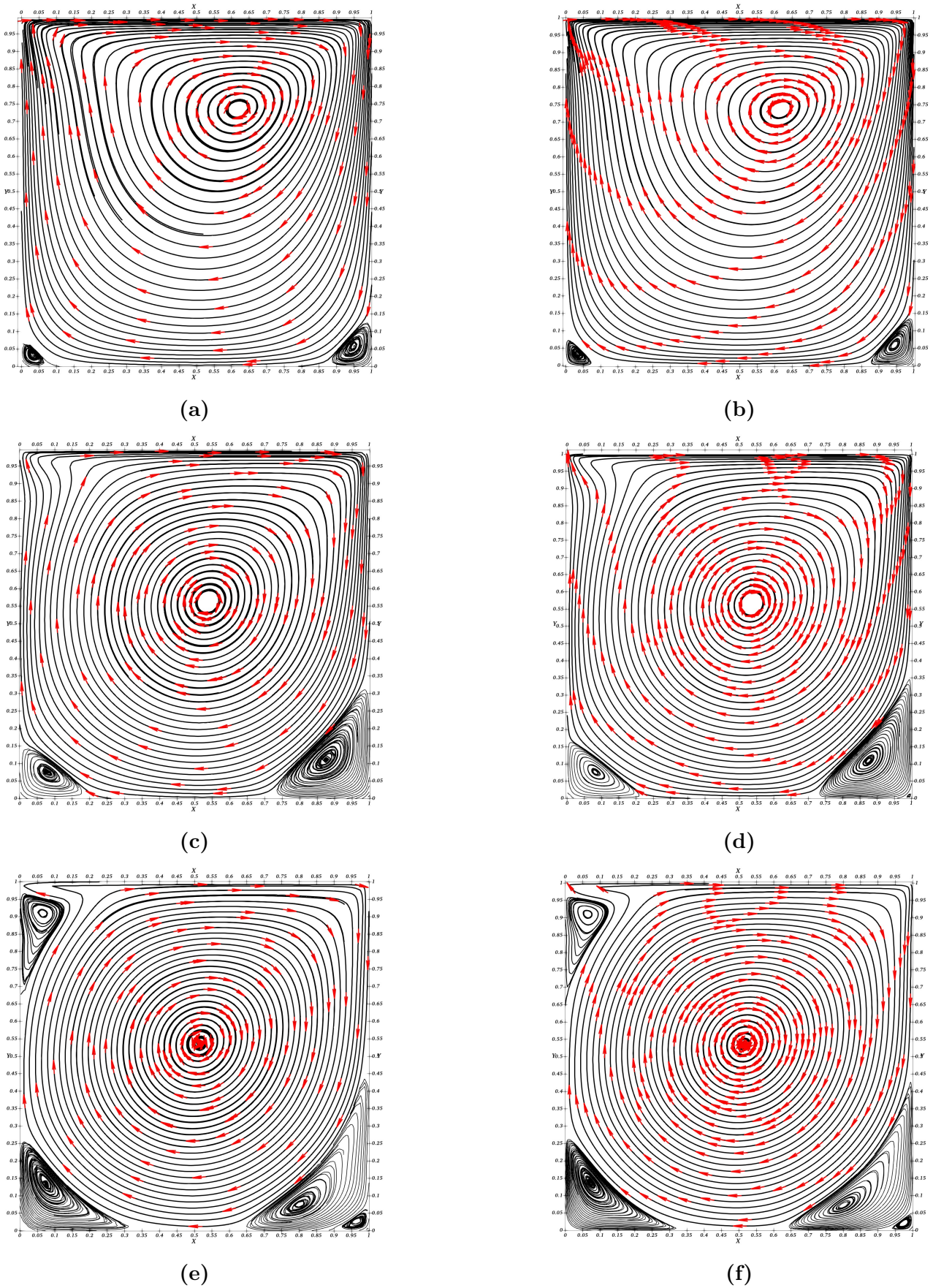


Figure 4.3: Arrangement method of streamline plots: organized by grid resolution (81 81 and 161 161) and Reynolds number (Re = 100, 1000, 5000).

4.2.2 Central Difference Scheme (CDs)



(a) **(b)** **(c)** **(d)** **(e)** **(f)**

Figure 4.4: Streamline patterns for lid-driven cavity flow computed using central difference scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81 × 81 grid; (b,d,f) 161 × 161 grid.

4.2.3 QUICK Scheme

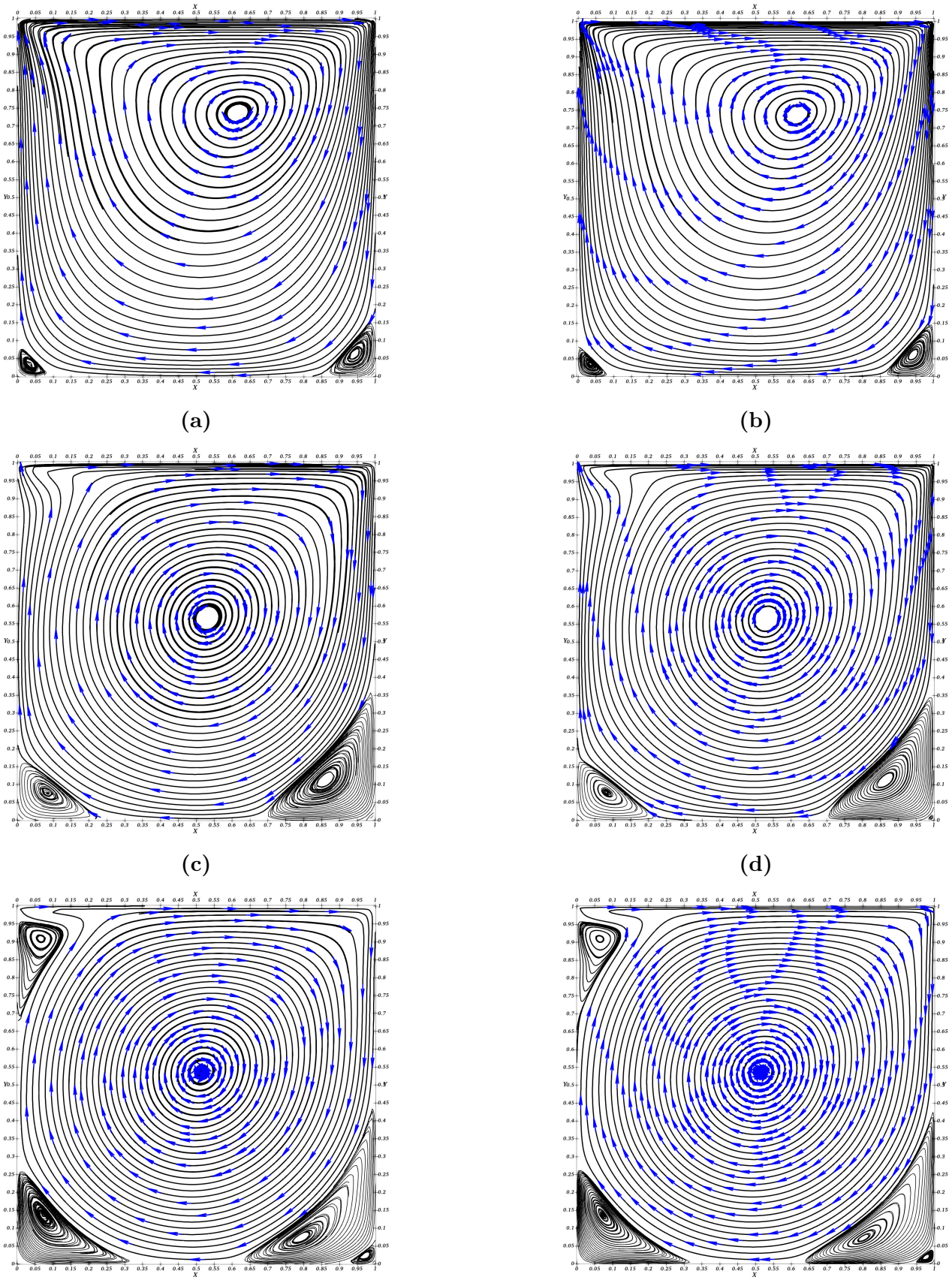


Figure 4.5: Streamline patterns for lid-driven cavity flow computed using QUICK scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81 × 81 grid; (b,d,f) 161 × 161 grid.

4.2.4 TVD-MUSCL Scheme

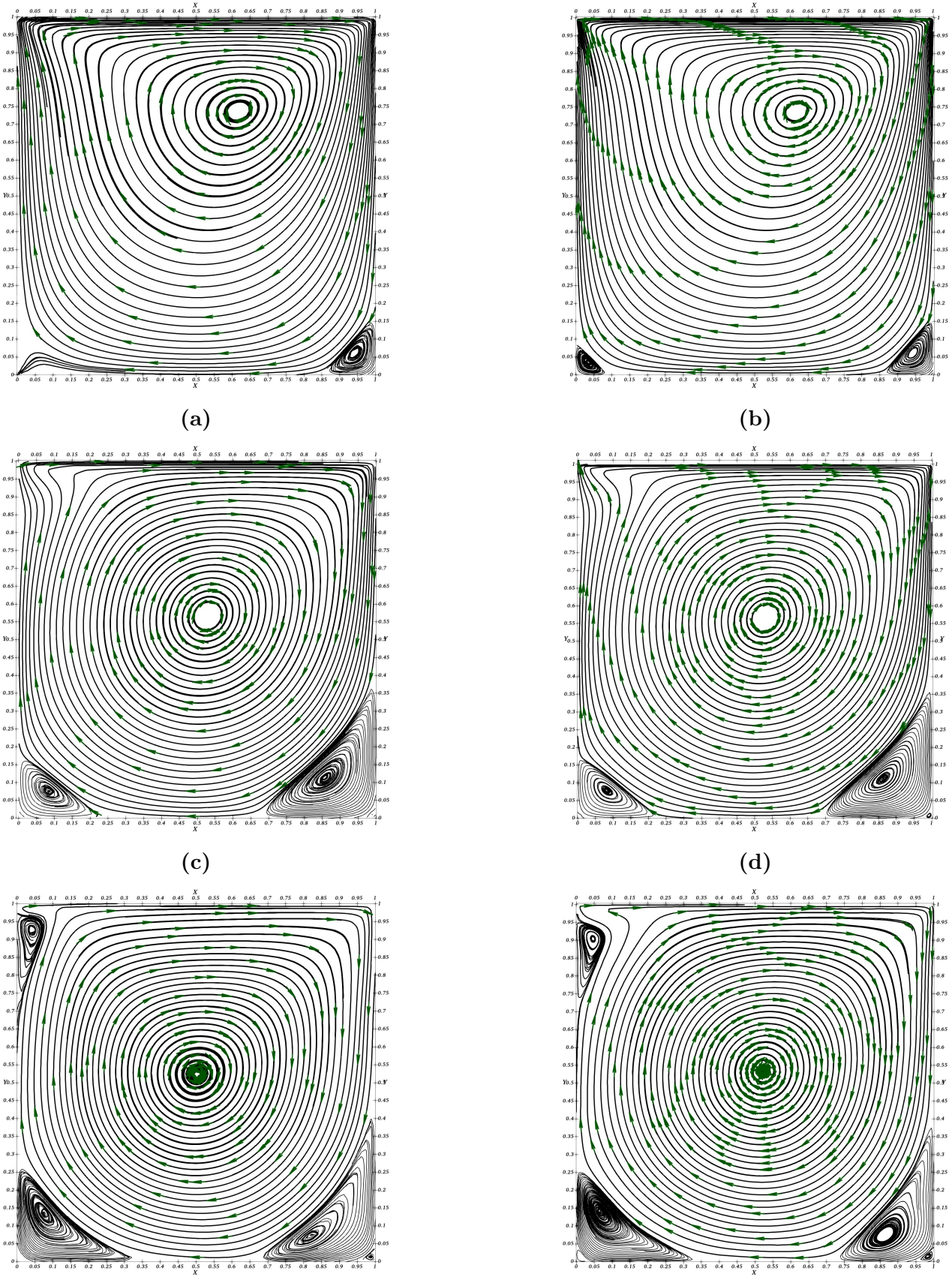


Figure 4.6: Streamline patterns for lid-driven cavity flow computed using TVD-MUSCL scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81×81 grid; (b,d,f) 161×161 grid.

4.3 Velocity Magnitude Contours

This section presents velocity magnitude contour plots for the same three numerical schemes across different Reynolds numbers and grid resolutions.

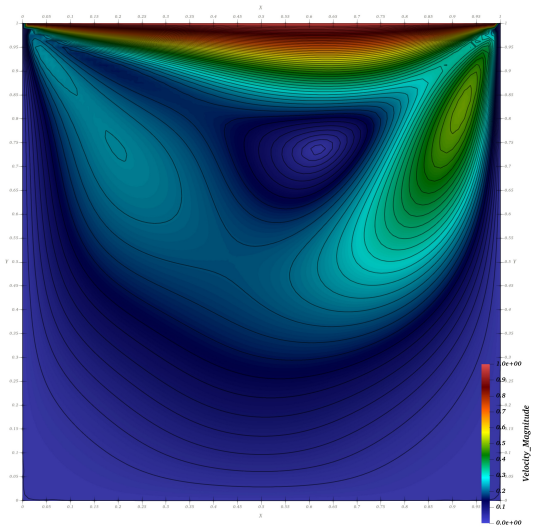
4.3.1 Arrangement of Velocity Magnitude Contours

The below figure shows the arrangement method of my velocity magnitude contour plots results, split them into 81 81 and 161 161 parts at first, and separate figures into three parts based on 3 types of Re numbers.

Re = 100 **CDs(QUICK,TVD)** **Grid 81x81** **Contour**	**Re = 100** **CDs(QUICK,TVD)** **Grid 161x161** **Contour**
Re = 1000 **CDs(QUICK,TVD)** **Grid 81x81** **Contour**	**Re = 1000** **CDs(QUICK,TVD)** **Grid 161x161** **Contour**
Re = 5000 **CDs(QUICK,TVD)** **Grid 81x81** **Contour**	**Re = 5000** **CDs(QUICK,TVD)** **Grid 161x161** **Contour**

Figure 4.7: Arrangement method of velocity magnitude contours: organized by grid resolution (81 81 and 161 161) and Reynolds number (Re = 100, 1000, 5000).

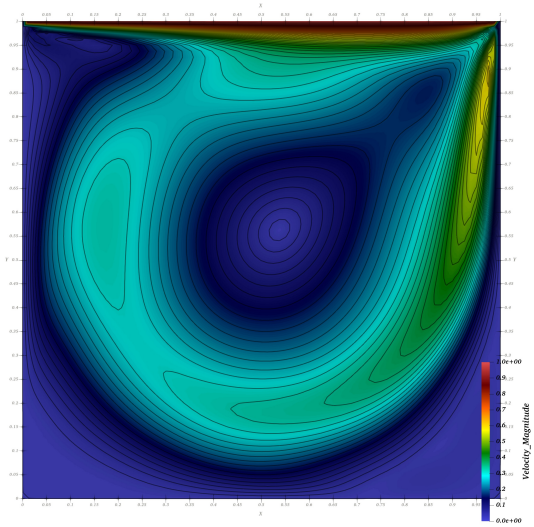
4.3.2 Central Difference Scheme (CDs)



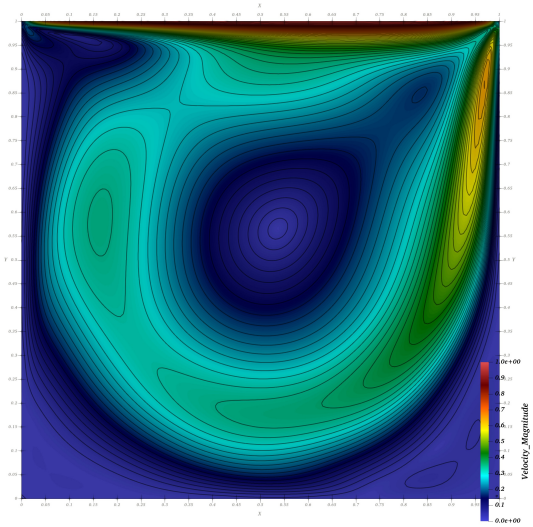
(a)

C_1_1_2.jpg

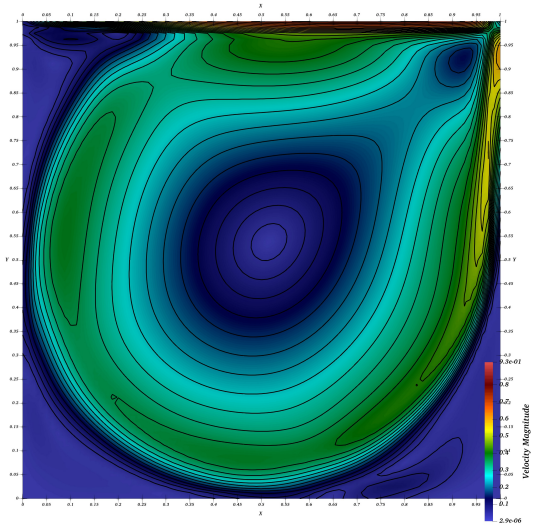
(b)



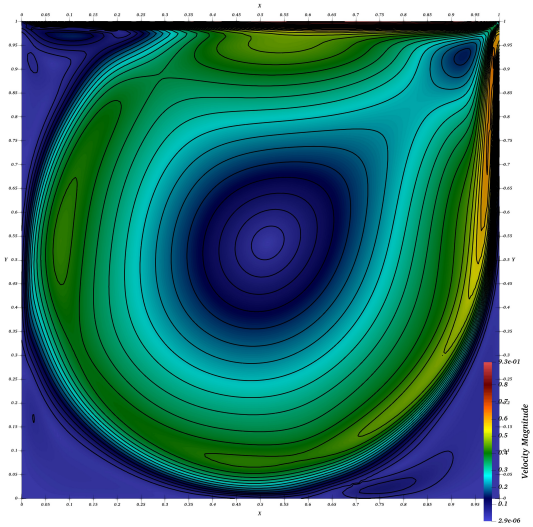
(c)



(d)



(e)



(f)

Figure 4.8: Velocity magnitude contours for lid-driven cavity flow computed using central difference scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81×81 grid; (b,d,f) 161×161 grid.

4.3.3 QUICK Scheme

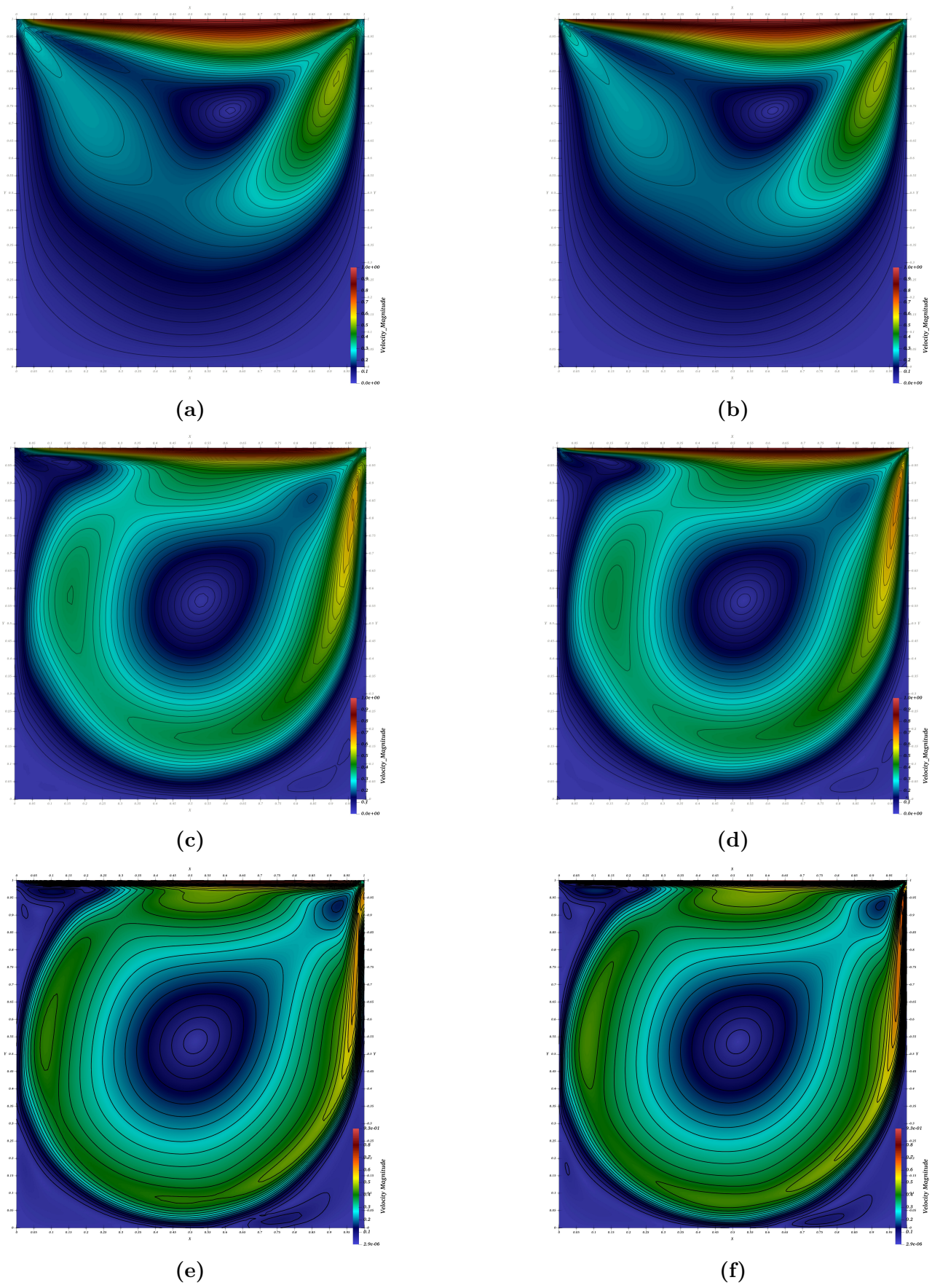
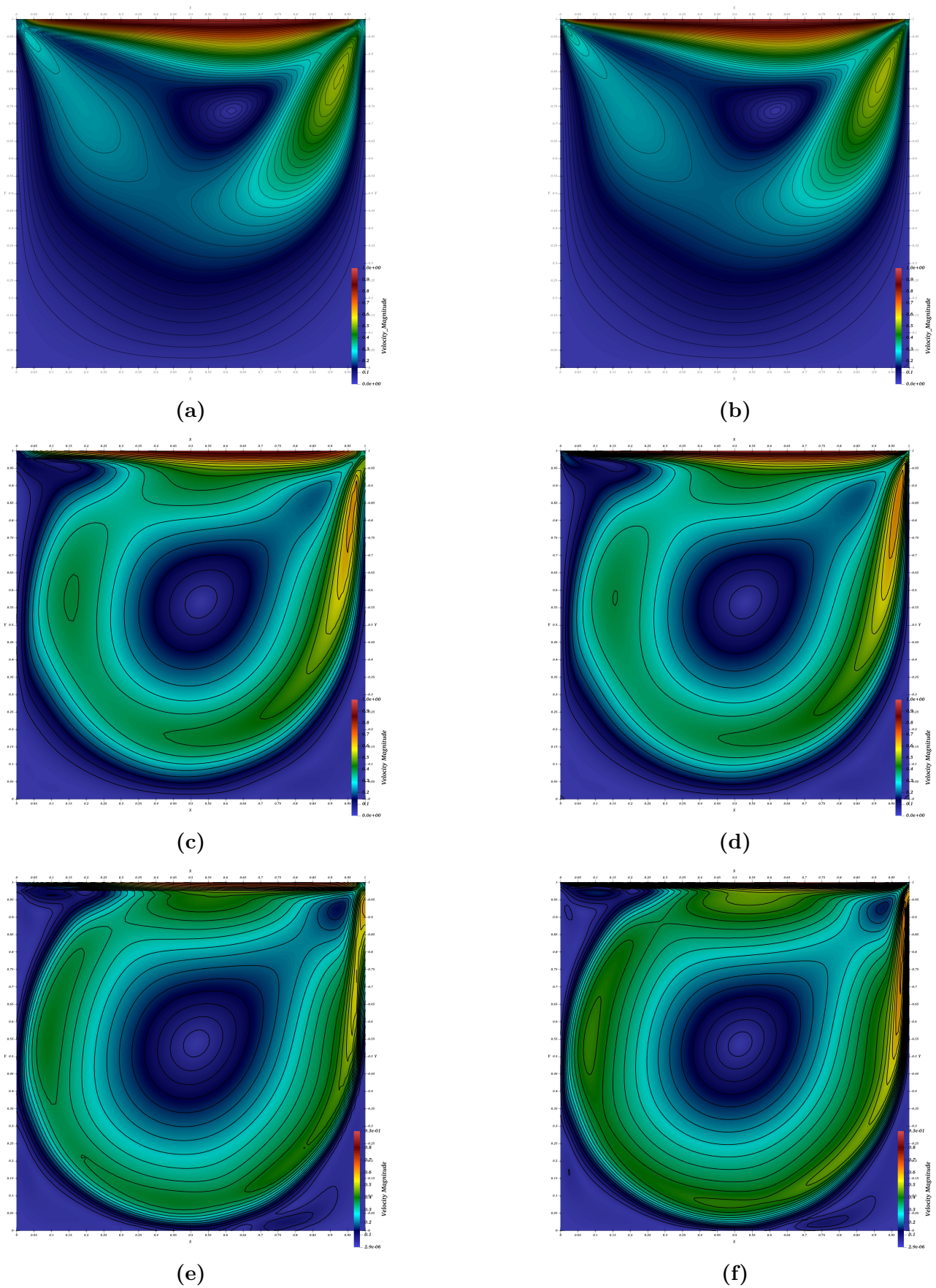


Figure 4.9: Velocity magnitude contours for lid-driven cavity flow computed using QUICK scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81 × 81 grid; (b,d,f) 161 × 161 grid.

4.3.4 TVD-MUSCL Scheme



(a) (b) (c) (d) (e) (f)

Figure 4.10: Velocity magnitude contours for lid-driven cavity flow computed using TVD-MUSCL scheme. (a,b) Re = 100; (c,d) Re = 1000; (e,f) Re = 5000. (a,c,e) 81×81 grid; (b,d,f) 161×161 grid.

4.4 Velocity Centerline Comparison for Different Re Numbers

This section presents the centerline velocity profiles comparing different Reynolds numbers for each numerical scheme.

4.4.1 Arrangement of u-Velocity and v-Velocity Profiles

Each subfigure presents results for three Reynolds numbers (Re = $100, 1000, 5000$). The arrangement method is summarized below: (1) u-velocity profiles on the right-hand side, and v-velocity profiles on the left-hand side; (2) Central Difference Scheme at top, QUICK in the middle, and TVD-MUSCL at bottom.

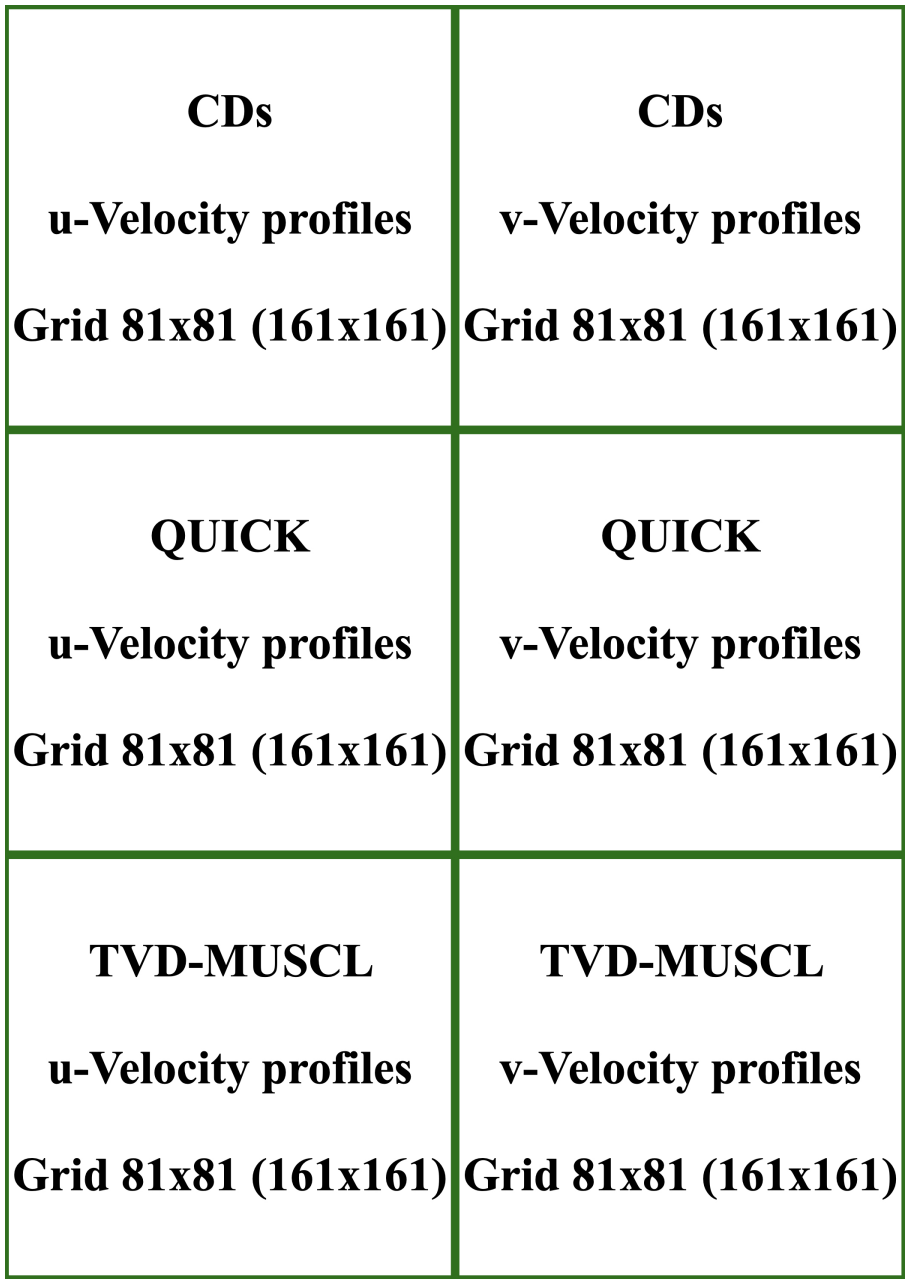
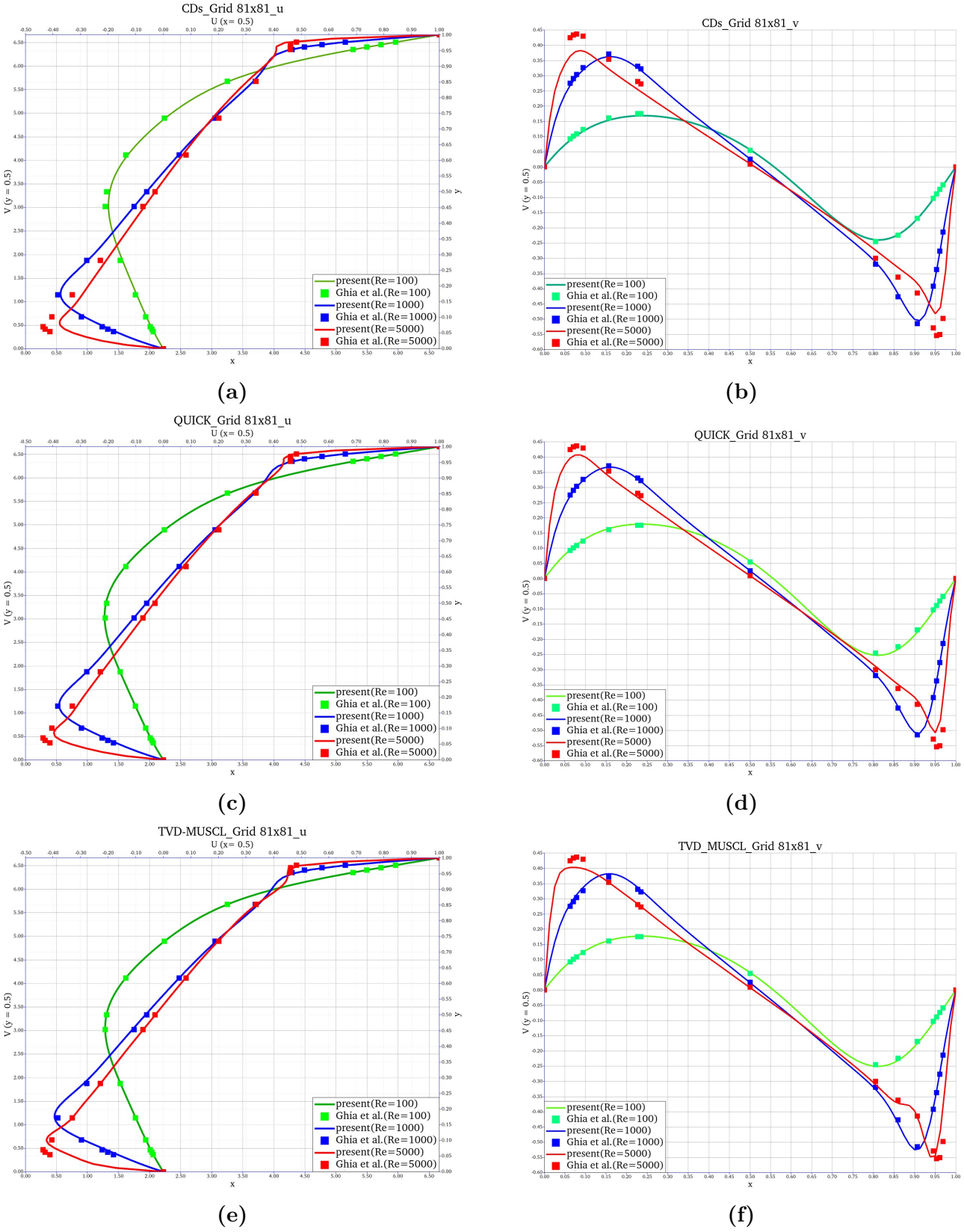


Figure 4.11: Arrangement method of velocity centerline profiles. Each row corresponds to a numerical scheme. Each column shows the u-velocity profiles on the left-hand side and the v-velocity profiles on the right-hand side. Results are presented for three Reynolds numbers (Re = $100, 1000, 5000$) and two grid resolutions (81×81 and 161×161).

4.4.2 Centerline Velocity Profiles - Grid 81x81



(a) **(b)** **(c)** **(d)** **(e)** **(f)**

Figure 4.12: Velocity profiles for lid-driven cavity flow at Re = 1000 on 81×81 grid. (a,c,e) u-velocity along vertical centerline; (b,d,f) v-velocity along horizontal centerline. (a,b) Central difference scheme; (c,d) QUICK scheme; (e,f) TVD-MUSCL scheme. Symbols: benchmark data from Ghia *et al.* (1982).

4.4.3 Centerline Velocity Profiles - Grid 161x161

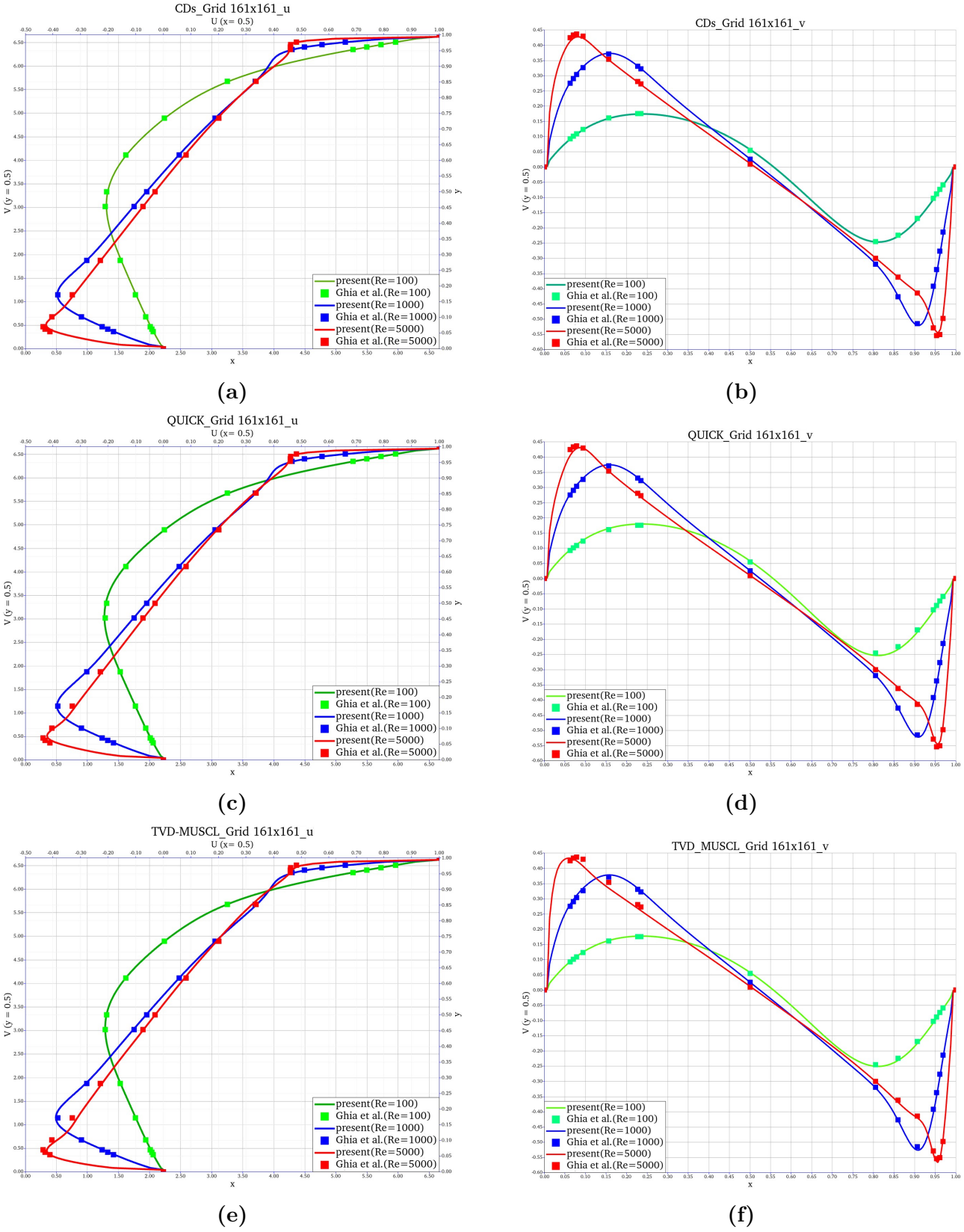


Figure 4.13: Velocity profiles for lid-driven cavity flow at Re = 1000 on 161 × 161 grid. (a,c,e) u-velocity along vertical centerline; (b,d,f) v-velocity along horizontal centerline. (a,b) Central difference scheme; (c,d) QUICK scheme; (e,f) TVD-MUSCL scheme. Symbols: benchmark data from Ghia *et al.* (1982).

4.5 Velocity Centerline Comparison for Different Schemes

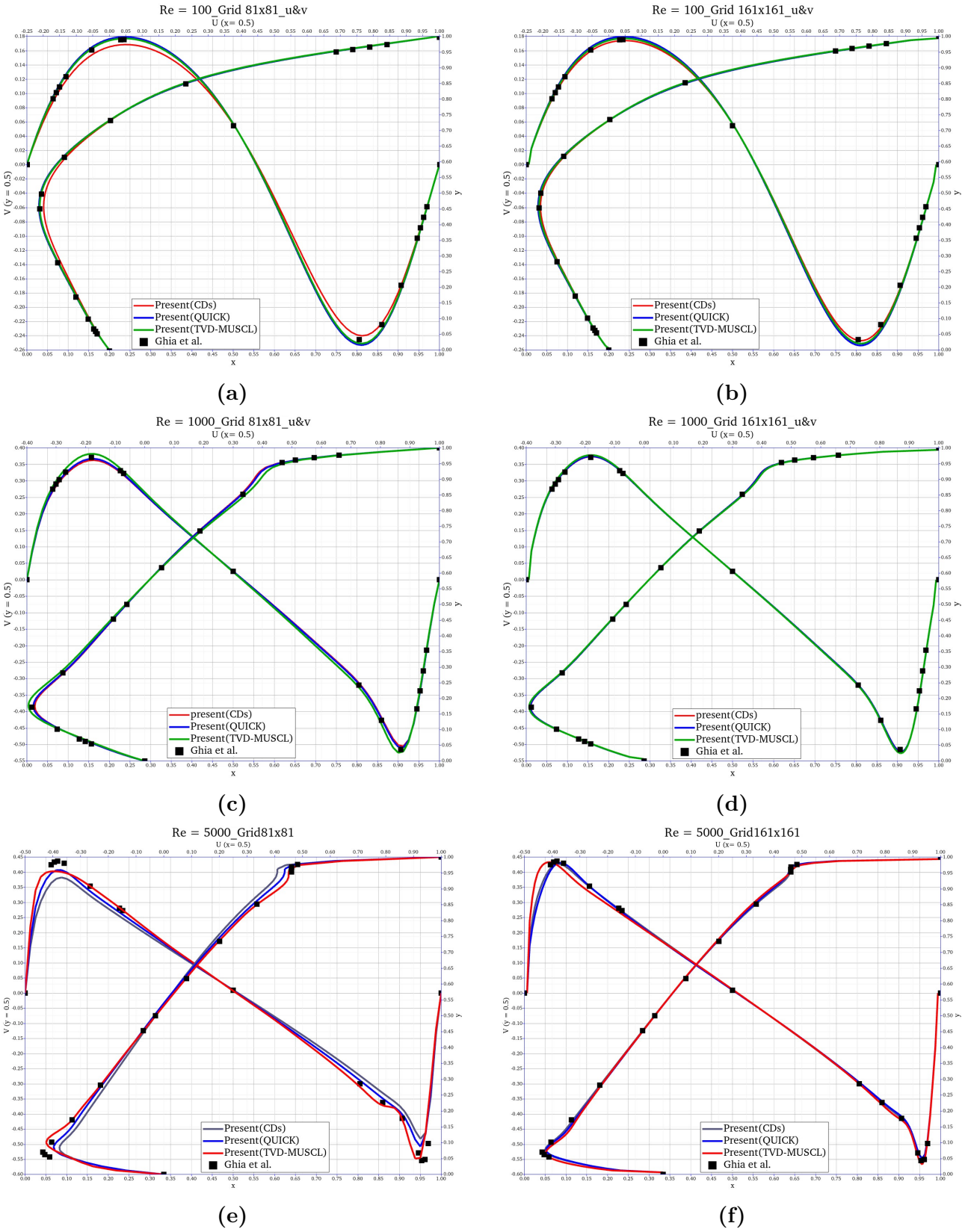


Figure 4.14: centerline velocity profiles for different numerical schemes at Re = 100 (a,b), 1000 (c,d), and 5000 (e,f). Left column: 81×81 grid; right column: 161×161 grid. Each panel shows u-velocity (left) and v-velocity (right) along centerlines. Symbols: Ghia *et al.* (1982).

4.6 Velocity Centerline Comparison for Different Grid Sizes at High Re Number

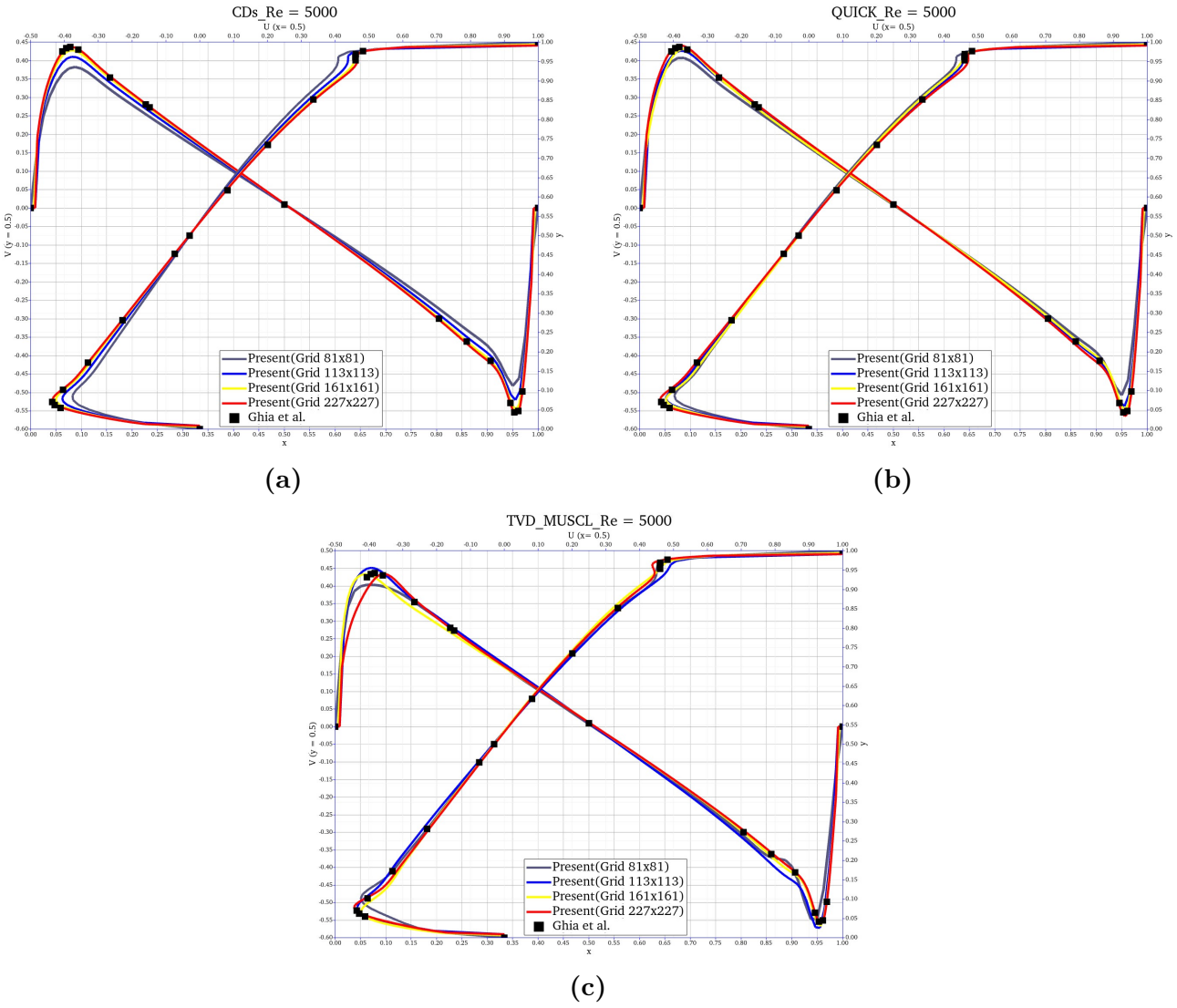


Figure 4.15: Grid convergence study for u-velocity profiles along vertical centerline at Re = 5000. (a) Central difference scheme; (b) QUICK scheme; (c) TVD-MUSCL scheme. Four grid resolutions: 81×81, 113×113, 161×161, and 227×227. Symbols: benchmark data from Ghia *et al.* (1982).

Figure 4.15 shows a clear grid-convergence trend at Re = 5000 for the vertical centerline u-velocity. The coarsest mesh (81×81) smooths the near-wall shear layer and weakens the extreme velocity levels, while refinement to 161×161 and 227×227 recovers the sharp gradient close to the lid and the stronger core recirculation. This behavior is expected at high Reynolds number because the boundary layer is thin and requires more points to resolve.
Scheme sensitivity is also visible. The CDs solution is more diffusive on coarse grids, so the deviation from the benchmark is larger in the high-gradient region. QUICK and TVD-MUSCL already track the benchmark reasonably well at 161×161, but the finest grid still improves the match near the velocity extrema. Overall, the comparison confirms that mesh refinement is essential for high-Re flows, and that higher-order schemes reduce numerical diffusion but still benefit from increased resolution. Whatever discrete scheme I used, from figure 4.15 we can see that when grid size is smaller (like 227×227), centerline velocity profiles for u-Velocity and v-Velocity are closer to the benchmark by Ghia et al.(1982).

4.7 Accuracy Comparison With Different Schemes

For grid size 81×81, 113×113, 161×161, and 227×227, I have completed the evaluation of the normalized L_1 errors for both the centerline u- and v-velocity profiles at a Reynolds number of 5000 across all grid sizes, using Ghia s benchmark data as the exact solution.

Table 4.1: L_1 error and grid convergence data for different numerical schemes. The L_1 norm is computed as L_1 norm $= \frac{1}{34}\sum_{i=1}^{17}\sum_{j=1}^{17}(|u - u_{\text{Ghia}}| + |v - v_{\text{Ghia}}|)$.

Grid size	$\ln(\Delta x)$	CDs		QUICK		TVD-MUSCL	
		L_1 Error	$\ln(L_1)$	L_1 Error	$\ln(L_1)$	L_1 Error	$\ln(L_1)$
81×81	-4.382	1.79073	-2.943	1.26386	-3.289	1.05693	-3.464
113×113	-4.718	0.924517	-3.686	0.593034	-4.149	0.744874	-4.129
161×161	-5.075	0.510966	-4.280	0.467266	-4.374	0.534575	-4.246
227×227	-5.421	0.279889	-4.882	0.305895	-4.794	0.370451	-4.615

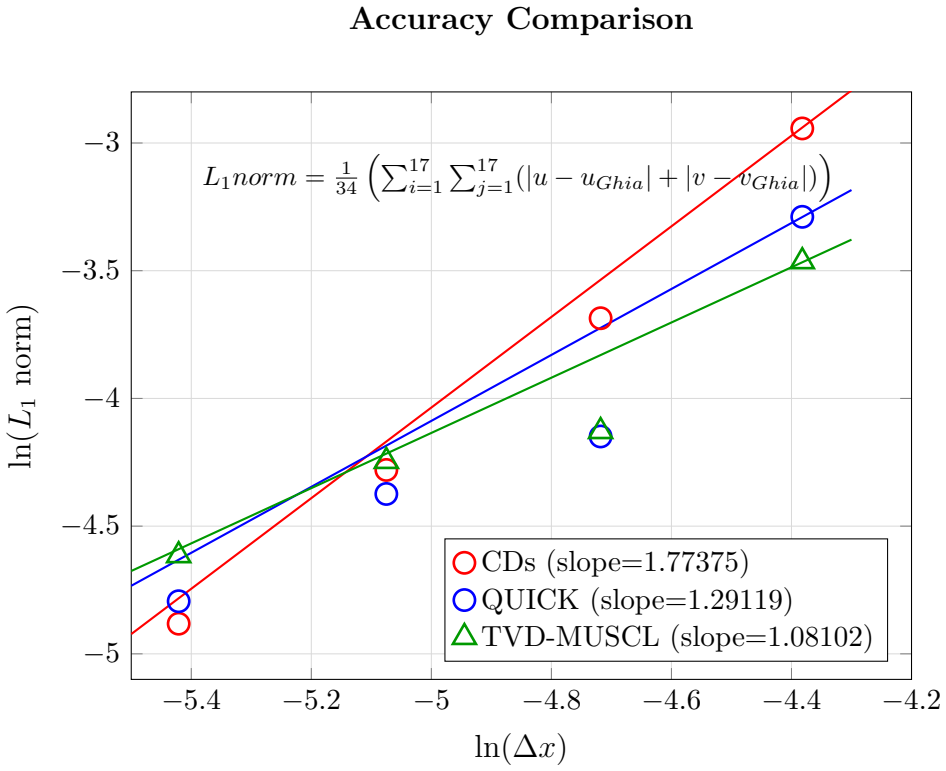


Figure 4.16: Accuracy comparison for difference schemes based on L_1 norm at Re = 5000. As the data shown in Table 4.1, the slope of the line decided by $ln(L_1 norm)$ and $ln(dx)$ is the accuracy of the scheme. The accuracy for CDs, QUICK, and TVD-MUSCL are approximately 1.77375, 1.29119. and 1.08102, respectively.

In the 227 227 grid, the error compared with the benchmark for CDS is less than that for QUICK.This may be due to the limited number of benchmark data points.The numerical solutions in the figure represent all my experience with various relaxation factor sets. The unusual finding is that inappropriate relaxation settings lead to either incorrect solutions or results that do not match the benchmark, so I maybe have a chance to find different solution with different set.

A Source Code

A.1 Header Files (.h files)

A.1.1 TDMA Solver

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <string>
#include <vector>
#pragma once
using namespace std ;
//writting TDMA solver
inline void TDMA(
    const vector<double>& D,       //primary delements
    const vector<double>& beta2,   //left hand side elemets
    const vector<double>& alpha2,  //right hand side elememts
    vector<double>& x,             // solutions
    const vector<double>& C,       // sourceterm
    int n,
    double U_left, double U_right) // left boundary and right boundary condition
{
    if (n <= 0) return;
    if ((int)D.size()!=n || (int)beta2.size()!=n || (int)alpha2.size()!=n ||
        (int)C.size()!=n || (int)x.size()!=n) {
        throw std::invalid_argument("All input vector sizes must equal n.");
    }

    vector<double> A(n, 0.0);
    vector<double> Cp(n, 0.0);

    // forward elimination
    double denom = D[0];
    if (denom == 0.0) throw std::runtime_error("Singular pivot at row 0.");
    A[0]  = (n > 1) ? (alpha2[0] / denom) : 0.0;
    Cp[0] = (C[0] - beta2[0]*U_left) / denom;


    for (int i = 1; i < n-1; ++i) {
        denom = D[i] - beta2[i]*A[i-1];
        if (denom == 0.0) throw std::runtime_error("Singular pivot at row " + std::to_string(i)
            );
        A[i]  = alpha2[i] / denom;
        Cp[i] = (C[i] - beta2[i]*Cp[i-1]) / denom;
    }

    //backward substituition
    if (n >= 2) {
        int i = n-1;
        denom = D[i] - beta2[i]*A[i-1];
        if (denom == 0.0) throw std::runtime_error("Singular pivot at last row.");
        Cp[i] = (C[i] - beta2[i]*Cp[i-1] - alpha2[i]*U_right) / denom;
    }


    x[n-1] = Cp[n-1];
    for (int i = n-2; i >= 0; --i) {
        x[i] = Cp[i] - A[i]*x[i+1];
    }
}


inline void TDMA_FVM(
    const vector<double>& D,
    const vector<double>& beta2,
    const vector<double>& alpha2,
    vector<double>& x,
    const vector<double>& C,
    int n)
{
    if (n <= 0) return;
```

```
    if ((int)D.size()!=n || (int)beta2.size()!=n || (int)alpha2.size()!=n ||
        (int)C.size()!=n || (int)x.size()!=n) {
        throw std::invalid_argument("All input vector sizes must equal n.");
    }

    vector<double> A(n, 0.0), Cp(n, 0.0);

    // forward i=0
    double denom = D[0];
    if (denom == 0.0) throw std::runtime_error("Singular pivot at row 0.");
    A[0]  = (n>1) ? (alpha2[0] / denom) : 0.0;
    Cp[0] = C[0] / denom;

    // forward i=1..n-1
    for (int i = 1; i < n; ++i) {
        denom = D[i] - beta2[i]*A[i-1];
        if (denom == 0.0) throw std::runtime_error("Singular pivot at row " + std::to_string(i)
            );
        A[i]  = (i < n-1) ? (alpha2[i] / denom) : 0.0;
        Cp[i] = (C[i] - beta2[i]*Cp[i-1]) / denom;
    }

    // backward
    x[n-1] = Cp[n-1];
    for (int i = n-2; i >= 0; --i) {
        x[i] = Cp[i] - A[i]*x[i+1];
    }
}

//Note: TDMA(D,beta,alpha,x,C,n,U_left,Uright)  , TDMA_FVM(D,beta,alpha,x,C,n)
//D  ; primary elements
//beta ; left hand side elements
//alpha ; right hand side elements
//x ; solutions
//C ; source term
//n ; number of the elements of the vector
```

Listing 5: 0.TDMA_solver.h

A.1.2 Pressure Correction Equation

```
// This code implements the inner iteration routine for the pressure correction equation.
// All four boundaries use Neumann conditions.
// The solver uses Jacobi iterations combined with under-relaxation.
#include <iostream>
#include <fstream>
#include <sstream>
#include <cmath>
#include <cstdlib>
#include <string>
#include <vector>
#include <iomanip>
#include <omp.h>
using namespace std ;
// Parameters used in the solver.
constexpr int Nx = 80;
constexpr int Ny = 80;
constexpr double dx = 1.0 / double(Nx);
constexpr double dy = 1.0 / double(Ny);
constexpr double rho = 1.0;
int n = Nx * Ny;
vector<double> a; // 1D array for left-hand side coefficients.
vector<double> b; // 1D array for right-hand side source terms.
vector<double> x;
vector<double> x_old;
vector<double> x_old2;
vector<double> x_nuse ;
extern double u_star[Nx+1][Ny] ; // Predicted x-velocity component.
extern double v_star[Nx][Ny+1] ; // Predicted y-velocity component.
extern double a_P[Nx+1][Ny] ; // Cell-centered coefficients for x-velocity.
extern double b_P[Nx][Ny+1] ; // Cell-centered coefficients for y-velocity.
extern double p_prime[Nx][Ny] ; // Pressure correction field.
extern int G;
const double lamda = 0.1 ;
int F, max_F = 4000;
```

```
35 double maxerror;
36 const float tolerance = 1e-4;
37 bool steadystate;
38
39
40
41 // Initialize matrices.
42 void initial(vector<double>& a, vector<double>& b , vector<double>& x) {
43     // Initialize all entries to zero.
44     #pragma omp parallel for
45     for(int i = 1; i <= n; i++) {
46         a[i] = 0.0;
47         b[i] = 0.0;
48         x[i] = 0.0;
49         x_old[i] = 0.0;
50     }
51 }
52
53 void GS(vector<double>& a, vector<double>& b , vector<double>& x , int n) {
54     // Copy current solution into x_old for error evaluation.
55      #pragma omp parallel for
56     for(int k = 1; k <= n; k++) {
57         x_old[k] = x[k];
58     }
59
60     // Note: Gauss-Seidel has data dependencies, so interior updates cannot be parallelized
          directly.
61     // Use Red-Black ordering to enable parallelization.
62
63     // Red nodes (I + J even).
64     #pragma omp parallel for collapse(2)    // Key section.
65     for(int I = 2; I <= Nx-1; I++) {
66         for(int J = 2; J <= Ny-1; J++) {
67             if((I+J) % 2 == 0) {  // Red node
68                 int idx = (J-1)*Nx + I ;
69                 double sum2 = 0.0 ;
70                  a[idx] = (((dy*dy)/(rho*(a_P[I][J-1])))
71                         +((dy*dy)/(rho*(a_P[I-1][J-1])))
72                         +((dx*dx)/(rho*(b_P[I-1][J])))
73                         +((dx*dx)/(rho*(b_P[I-1][J-1])))) ;// Diagonal coefficient
74                  a[idx+1] = -((dy*dy)/(rho*(a_P[I][J-1]))) ; // East coefficient
75                  a[idx-1] = -((dy*dy)/(rho*(a_P[I-1][J-1]))) ; // West coefficient
76                  a[idx+Nx] =-((dx*dx)/(rho*(b_P[I-1][J]))) ;// North coefficient
77                  a[idx-Nx] =-((dx*dx)/(rho*(b_P[I-1][J-1])));// South coefficient
78                 b[idx] = dy*u_star[I-1][J-1] - dy*u_star[I][J-1] + dx*v_star[I-1][J-1] - dx*
                      v_star[I-1][J] ;
79                 sum2 = a[idx+1]*x[idx+1] + a[idx-1]*x[idx-1]
80                      + a[idx+Nx]*x[idx+Nx] + a[idx-Nx]*x[idx-Nx];
81                 x[idx] = (b[idx] - sum2) / a[idx];
82                 x[idx] = x[idx]*lamda + x_old[idx]*(1-lamda);
83             }
84         }
85     }
86
87     // Black nodes (I + J odd).
88     #pragma omp parallel for collapse(2)
89     for(int I = 2; I <= Nx-1; I++) {
90         for(int J = 2; J <= Ny-1; J++) {
91             if((I+J) % 2 == 1) {  // Black node
92                 int idx = (J-1)*Nx + I ;
93                 double sum2 = 0.0 ;
94                 a[idx] = (((dy*dy)/(rho*(a_P[I][J-1])))
95                           +((dy*dy)/(rho*(a_P[I-1][J-1])))
96                           +((dx*dx)/(rho*(b_P[I-1][J])))
97                           +((dx*dx)/(rho*(b_P[I-1][J-1])))) ;// Diagonal coefficient
98                 a[idx+1] = -((dy*dy)/(rho*(a_P[I][J-1]))) ; // East coefficient
99                 a[idx-1] = -((dy*dy)/(rho*(a_P[I-1][J-1]))) ; // West coefficient
100                a[idx+Nx] =-((dx*dx)/(rho*(b_P[I-1][J]))) ;// North coefficient
101                a[idx-Nx] =-((dx*dx)/(rho*(b_P[I-1][J-1])));// South coefficient
102                b[idx] = dy*u_star[I-1][J-1] - dy*u_star[I][J-1] + dx*v_star[I-1][J-1] - dx*
                     v_star[I-1][J] ;
103                sum2 += a[idx+1] *x[idx+1] ;
104                sum2 += a[idx-1] *x[idx-1] ;
105                sum2 += a[idx+Nx] *x[idx+Nx] ;
106                sum2 += a[idx-Nx] *x[idx-Nx] ;
107                x[idx] = (b[idx] - sum2) / a[idx];
```

```
108                 x[idx] = x[idx]*lamda + x_old[idx]*(1-lamda);
109             }
110         }
111     }
112
113     // Right boundary (excluding corners).
114     #pragma omp parallel for
115     for(int J = 2; J <= Ny-1; J++) {
116         int idx = Nx*(J-1)+Nx;
117         a[idx] = (
118                         +((dy*dy)/(rho*(a_P[Nx-1][J-1])))
119                         +((dx*dx)/(rho*(b_P[Nx-1][J])))
120                        +((dx*dx)/(rho*(b_P[Nx-1][J-1])))) ;// Diagonal coefficient
121         a[idx-1] = -((dy*dy)/(rho*(a_P[Nx-1][J-1]))) ; // West coefficient
122         a[idx+Nx] = -((dx*dx)/(rho*(b_P[Nx-1][J]))) ;// North coefficient
123         a[idx-Nx] = -((dx*dx)/(rho*(b_P[Nx-1][J-1]))) ;// South coefficient
124         b[idx] = dy*u_star[Nx-1][J-1] - dy*u_star[Nx][J-1] + dx*v_star[Nx-1][J-1] - dx*v_star[
                Nx-1][J] ;
125         double sum2 = 0.0 ;
126         sum2 += a[idx-1] * x[idx-1] ;
127         sum2 += a[idx+Nx] * x[idx+Nx] ;
128         sum2 += a[idx-Nx] * x[idx-Nx] ;
129         x[idx] = (b[idx] - sum2) / a[idx];
130         x[idx] = x[idx]*lamda + x_old[idx]*(1-lamda);
131     }
132
133     // Left boundary (excluding corners).
134     #pragma omp parallel for
135     for(int J = 2; J <= Ny-1; J++) {
136         int idx = Nx*(J-1)+1;
137         a[idx] = (((dy*dy)/(rho*(a_P[1-1+1][J-1])))
138                         +((dx*dx)/(rho*(b_P[1-1][J-1+1])))
139                         +((dx*dx)/(rho*(b_P[1-1][J-1])))) ;// Diagonal coefficient
140         a[idx+1] = -((dy*dy)/(rho*(a_P[1][J-1]))) ; // East coefficient
141         a[idx+Nx] = -((dx*dx)/(rho*(b_P[1-1][J]))) ;// North coefficient
142         a[idx-Nx] = -((dx*dx)/(rho*(b_P[1-1][J-1]))) ;// South coefficient
143         b[idx] = dy*u_star[1-1][J-1] - dy*u_star[1][J-1] + dx*v_star[1-1][J-1] - dx*v_star
                [1-1][J]  ;
144         double sum2 = 0.0 ;
145         sum2 += a[idx+1] * x[idx+1] ;
146         sum2 += a[idx+Nx] * x[idx+Nx] ;
147         sum2 += a[idx-Nx] * x[idx-Nx] ;
148         x[idx] = (b[idx] - sum2) / a[idx];
149         x[idx] = x[idx]*lamda + x_old[idx]*(1-lamda);
150     }
151
152     // Top boundary (excluding corners).
153     #pragma omp parallel for
154     for(int I = 2; I <= Nx-1; I++) {
155         int idx = n - Nx + I ;
156         a[idx] = (((dy*dy)/(rho*(a_P[I][Ny-1])))
157                     +((dy*dy)/(rho*(a_P[I-1][Ny-1])))
158                     +((dx*dx)/(rho*(b_P[I-1][Ny-1])))) ;// Diagonal coefficient
159         a[idx+1] = -((dy*dy)/(rho*(a_P[I][Ny-1]))) ;
160         a[idx-1] = -((dy*dy)/(rho*(a_P[I-1][Ny-1]))) ;
161         a[idx-Nx] = -((dx*dx)/(rho*(b_P[I-1][Ny-1])))  ;
162         b[idx] =dy*u_star[I-1][Ny-1] - dy*u_star[I][Ny-1] + dx*v_star[I-1][Ny-1] - dx*v_star[I
                -1][Ny] ;
163         double sum2 = 0.0 ;
164         sum2 += a[idx+1] * x[idx+1] ;
165         sum2 += a[idx-1] * x[idx-1] ;
166         sum2 += a[idx-Nx] * x[idx-Nx] ;
167         x[idx] = (b[idx] - sum2) / a[idx];
168         x[idx] = x[idx]*lamda + x_old[idx]*(1-lamda) ;
169     }
170
171     // Bottom boundary (excluding corners).
172     #pragma omp parallel for
173     for(int I = 2; I <= Nx-1 ; I++) {
174         a[I] = (((dy*dy)/(rho*(a_P[I][1-1])))
175                     +((dy*dy)/(rho*(a_P[I-1][1-1])))
176                     +((dx*dx)/(rho*(b_P[I-1][1])))) ;// Diagonal coefficient
177         a[I+1] =-((dy*dy)/(rho*(a_P[I][1-1])));
178         a[I-1] =-((dy*dy)/(rho*(a_P[I-1][1-1]))) ;
179         a[I+Nx] =-((dx*dx)/(rho*(b_P[I-1][1])));
```

```
180         b[I] = dy*u_star[I-1][1-1] - dy*u_star[I][1-1] + dx*v_star[I-1][1-1] - dx*v_star[I
                -1][1] ;
181         double sum2 = 0.0 ;
182         sum2 += a[I+1] * x[I+1] ;
183         sum2 += a[I-1] * x[I-1] ;
184         sum2 += a[I+Nx] * x[I+Nx] ;
185         x[I] = (b[I] - sum2) / a[I];
186         x[I] = x[I]*lamda + x_old[I]*(1-lamda);
187     }
188
189     // Four corner points handled sequentially.
190     {
191         a[1] = ((dy*dy)/(rho*(a_P[1][1-1]))
192                          //+((dy*dy)/(rho*SAFE(a_P[I-1][J-1])))
193                        + ((dx*dx)/(rho*(b_P[1-1][1]))));
194                          //+((dx*dx)/(rho*SAFE(b_P[I-1][J-1])))) // Diagonal coefficient
195         a[1+1] = -((dy*dy)/(rho*(a_P[1][1-1]))) ; // East coefficient
196         //a[1][1-1] = -((dy*dy)/(rho*SAFE(a_P[I-1][J-1]))) ; // West coefficient
197         a[1+Nx] =-((dx*dx)/(rho*(b_P[1-1][1]))) ;// North coefficient
198         //a[1][1-Nx] =-((dx*dx)/(rho*SAFE(b_P[I-1][J-1])));// South coefficient
199         b[1] = dy*u_star[1-1][1-1] - dy*u_star[1][1-1] + dx*v_star[1-1][1-1] - dx*v_star
                [1-1][1] ;
200         double sum2 = 0.0 ;
201         sum2 += a[1+1] * x[1+1] ;
202         sum2 += a[1+Nx] * x[1+Nx] ;
203         x[1] = (b[1] - sum2) / a[1];
204         x[1] = x[1]*lamda + x_old[1]*(1-lamda);
205     }
206     {
207         a[Nx] = (
208                 +((dy*dy)/(rho*(a_P[Nx-1][1-1])))
209             +((dx*dx)/(rho*(b_P[Nx-1][1])))) ;// Diagonal coefficient
210         a[Nx-1] = -((dy*dy)/(rho*(a_P[Nx-1][1-1])));
211         a[2*Nx] = -((dx*dx)/(rho*(b_P[Nx-1][1])));
212         b[Nx] = dy*u_star[Nx-1][1-1] - dy*u_star[Nx][1-1] + dx*v_star[Nx-1][1-1] - dx*v_star[Nx
                -1][1] ;
213         double sum2 = 0.0 ;
214         sum2 += a[Nx-1] * x[Nx-1] ;
215         sum2 += a[2*Nx] * x[2*Nx] ;
216         x[Nx] = (b[Nx] - sum2) / a[Nx];
217         x[Nx] = x[Nx]*lamda + x_old[Nx]*(1-lamda);
218     }
219     {
220         a[n-Nx+1] = (((dy*dy)/(rho*(a_P[1][Ny-1])))
221             +((dx*dx)/(rho*(b_P[1-1][Ny-1])))) ;// Diagonal coefficient
222         a[n-Nx+2] =  -((dy*dy)/(rho*(a_P[1][Ny-1]))) ;
223         a[n-2*Nx+1] = -((dx*dx)/(rho*(b_P[1-1][Ny-1])));
224         b[n-Nx+1] =  dy*u_star[1-1][Ny-1] - dy*u_star[1][Ny-1] + dx*v_star[1-1][Ny-1] - dx*
                v_star[1-1][Ny] ;
225         double sum2 = 0.0 ;
226         sum2 += a[n-Nx+2] * x[n-Nx+2] ;
227         sum2 += a[n-2*Nx+1] * x[n-2*Nx+1] ;
228         x[n-Nx+1] = (b[n-Nx+1] - sum2) / a[n-Nx+1];
229         x[n-Nx+1] = x[n-Nx+1]*lamda + x_old[n-Nx+1]*(1-lamda);
230     }
231     {
232         a[n] =( +((dy*dy)/(rho*(a_P[Nx-1][Ny-1])))+((dx*dx)/(rho*(b_P[Nx-1][Ny-1])))) ;//
                Diagonal coefficient
233         a[n-1] = -((dy*dy)/(rho*(a_P[Nx-1][Ny-1]))) ;
234         a[n-Nx] =-((dx*dx)/(rho*(b_P[Nx-1][Ny-1])));
235         b[n] = dy*u_star[Nx-1][Ny-1] - dy*u_star[Nx][Ny-1] + dx*v_star[Nx-1][Ny-1] - dx*v_star[
                Nx-1][Ny] ;
236         double sum2 = 0.0 ;
237         sum2 += a[n-1] * x[n-1] ;
238         sum2 += a[n-Nx] * x[n-Nx] ;
239         x[n] = (b[n] - sum2) / a[n];
240         x[n] = x[n]*lamda + x_old[n]*(1-lamda);
241     }
242
243     // Compute maximum error via reduction.
244     maxerror = 0;
245     #pragma omp parallel for reduction(max:maxerror)
246     for(int k = 1; k <= n; k++) {
247         double error = fabs(x[k] - x_old[k]);
248         if(maxerror < error) {
249             maxerror = error;
```

```
250         }
251     }
252 }
253
254 void setreference(int m) {
255     // Compute the domain average.
256     double mean = 0.0;
257     #pragma omp parallel for reduction(+:mean)
258     for(int k=1; k<=n; ++k) {
259         mean += x[k];
260     }
261     mean /= n;
262
263     // Subtract the average.
264     #pragma omp parallel for
265     for(int k=1; k<=n; ++k) {
266         x[k] -= mean;
267     }
268 }
269
270 void step3() {
271     // Optional: set OpenMP thread count.
272     // omp_set_num_threads(8);  // Adjust to match available CPU cores.
273     // Resize working arrays.
274     a.assign(n+2,  0.0);
275     b.assign(n+2, 0.0);
276     x.assign(n+2, 0.0);
277     x_old.assign(n+2, 0.0);
278     x_old2.assign(n+2, 0.0);
279     x_nuse.assign(n+2, 0.0);
280
281
282     steadystate = false;
283     initial(a, b, x);
284
285     #pragma omp parallel for
286     for (int k=1; k<=n; ++k) {
287         x_old[k] = x[k];
288     }
289
290     for(F = 0; F < max_F ; F++) {
291         if(F > 0) {
292             GS(a, b, x, n);
293             setreference(F) ;
294         }
295         if(F > 100 && maxerror < tolerance) {
296             steadystate = true;
297             if(G%100 == 0){cout << "G = " << G << "_F = " << F << "_Maxerror = " << maxerror <<
                    "achieve the inner itteration converged !!! "<< endl;}
298             break;
299         }
300     }
301     if(!steadystate && G%100 == 0) {
302         cout << "\nWarning: Reached maximum iteration count, but not converged to steady state!
            " << endl;
303     }
304     // Map the 1D solution back to the 2D pressure field.
305     #pragma omp parallel for collapse(2)
306     for(int I = 1; I <= Nx; I++) {
307         for(int J = 1; J <= Ny; J++) {
308             p_prime[I-1][J-1] = x[(J-1)*Nx+ I];
309         }
310     }
311 }
```

Listing 6: 0.P_correction_eq.h

A.1.3 Pressure Correction Equation with TDMA

```
1 #include <iostream>
2 #include <fstream>
3 #include <sstream>
4 #include <cmath>
5 #include <cstdlib>
6 #include <string>
```

```
7  #include <vector>
8  #include <iomanip>
9  #include <omp.h>
10 #include <algorithm>
11 #include "0.TDMA_solver.h"
12 #pragma once
13 using namespace std ;
14 //I used parameters below.
15 constexpr int Nx = 80 ;
16 constexpr int Ny = 80 ;
17 constexpr double dx = 1.0 / double(Nx);
18 constexpr double dy = 1.0 / double(Ny);
19 constexpr double rho = 1.0;
20 const int n = Nx * Ny;
21 double Coe_P[Nx][Ny] , Coe_W[Nx][Ny] , Coe_E[Nx][Ny] , Coe_S[Nx][Ny] , Coe_N[Nx][Ny] , Source[
       Nx][Ny] ;
22 //infact we will define the Boundary Condition and the size od the vector
23 extern double u_star[Nx+1][Ny] ; //x-velocity predicted value
24 extern double v_star[Nx][Ny+1] ; //y-velocity predcted valure
25 extern double a_P[Nx+1][Ny] ; //xpart velocity pmain coefficient
26 extern double b_P[Nx][Ny+1] ; //y-part velocity main coefficient
27 extern double p_prime[Nx][Ny] ; //pressure correction vlaue
28 double p_prime_old[Nx][Ny] , p_prime_old2[Nx][Ny] ; //pressure correcrion old value
29
30
31 //initialize
32 void initialze() {
33     #pragma omp parallel
34     {
35         #pragma omp for collapse(2) schedule(static)
36         for (int I = 1; I <= Nx; ++I) {
37             for (int J = 1; J <= Ny; ++J) {
38                 Coe_P[I-1][J-1] = 0.0;
39                 Coe_W[I-1][J-1] = 0.0;
40                 Coe_E[I-1][J-1] = 0.0;
41                 Coe_S[I-1][J-1] = 0.0;
42                 Coe_N[I-1][J-1] = 0.0;
43                 Source[I-1][J-1] = 0.0;
44                 //First step : initialize the coefficient for solving the presure poisson
                       equation
45                 p_prime[I-1][J-1] = 0.0 ;
46             }
47         }
48     }
49 }
50
51 //setting coefficient
52 void coefficient(){
53     //       x_old
54     #pragma omp parallel default(shared)
55     {
56     #pragma omp  for collapse(2) schedule(static)
57     for(int I = 2; I <= Nx-1; I++) {
58         for(int J = 2; J <= Ny-1; J++) {
59             Coe_P[I-1][J-1] = (((dy*dy)/(rho*(a_P[I][J-1])))+((dy*dy)/(rho*(a_P[I-1][J-1])))+((
                   dx*dx)/(rho*(b_P[I-1][J])))+((dx*dx)/(rho*(b_P[I-1][J-1])))) ;
60             Coe_E[I-1][J-1] = -((dy*dy)/(rho*(a_P[I][J-1]))) ;
61             Coe_W[I-1][J-1] = -((dy*dy)/(rho*(a_P[I-1][J-1]))) ;
62             Coe_N[I-1][J-1] = -((dx*dx)/(rho*(b_P[I-1][J]))) ;
63             Coe_S[I-1][J-1] = -((dx*dx)/(rho*(b_P[I-1][J-1])));
64             Source[I-1][J-1] = dy*u_star[I-1][J-1] - dy*u_star[I][J-1] + dx*v_star[I-1][J-1] -
                   dx*v_star[I-1][J] ;
65
66         }
67     }
68     //right boundary nodes
69     #pragma omp  for schedule(static)
70     for(int J = 2; J <= Ny-1; J++) {
71         Coe_P[Nx-1][J-1] =(((dy*dy)/(rho*(a_P[Nx-1][J-1])))+((dx*dx)/(rho*(b_P[Nx-1][J])))+((dx
               *dx)/(rho*(b_P[Nx-1][J-1]))));
72         Coe_W[Nx-1][J-1] = -((dy*dy)/(rho*(a_P[Nx-1][J-1]))) ;
73         Coe_N[Nx-1][J-1] = -((dx*dx)/(rho*(b_P[Nx-1][J]))) ;
74         Coe_S[Nx-1][J-1] = -((dx*dx)/(rho*(b_P[Nx-1][J-1]))) ;
75         Source[Nx-1][J-1] = dy*u_star[Nx-1][J-1] - dy*u_star[Nx][J-1] + dx*v_star[Nx-1][J-1] -
               dx*v_star[Nx-1][J] ;
76     }
```

```
77      //left boundary nodes
78      #pragma omp  for schedule(static)
79      for(int J = 2; J <= Ny-1; J++) {
80          Coe_P[1-1][J-1] =(((dy*dy)/(rho*(a_P[1-1+1][J-1])))+((dx*dx)/(rho*(b_P[1-1][J-1+1])))
                +((dx*dx)/(rho*(b_P[1-1][J-1]))));
81          Coe_E[1-1][J-1] = -((dy*dy)/(rho*(a_P[1][J-1]))) ;
82          Coe_N[1-1][J-1] = -((dx*dx)/(rho*(b_P[1-1][J]))) ;
83          Coe_S[1-1][J-1] = -((dx*dx)/(rho*(b_P[1-1][J-1]))) ;
84          Source[1-1][J-1] = dy*u_star[1-1][J-1] - dy*u_star[1][J-1] + dx*v_star[1-1][J-1] - dx*
                v_star[1-1][J]  ;
85      }
86      //top boundary nodes
87      #pragma omp  for schedule(static)
88      for(int I = 2; I <= Nx-1; I++) {
89          Coe_P[I-1][Ny-1] =(((dy*dy)/(rho*(a_P[I][Ny-1])))+((dy*dy)/(rho*(a_P[I-1][Ny-1])))+((dx
                *dx)/(rho*(b_P[I-1][Ny-1])))) ;
90          Coe_E[I-1][Ny-1] = -((dy*dy)/(rho*(a_P[I][Ny-1]))) ;
91          Coe_W[I-1][Ny-1] = -((dy*dy)/(rho*(a_P[I-1][Ny-1])))  ;
92          Coe_S[I-1][Ny-1] = -((dx*dx)/(rho*(b_P[I-1][Ny-1])))  ;
93          Source[I-1][Ny-1] = dy*u_star[I-1][Ny-1] - dy*u_star[I][Ny-1] + dx*v_star[I-1][Ny-1] -
                dx*v_star[I-1][Ny] ;
94      }
95      //bottom boundary nodes
96      #pragma omp  for schedule(static)
97      for(int I = 2; I <= Nx-1 ; I++) {
98          Coe_P[I-1][1-1] = (((dy*dy)/(rho*(a_P[I][1-1])))+((dy*dy)/(rho*(a_P[I-1][1-1]))) +((dx*
                dx)/(rho*(b_P[I-1][1])))) ;
99          Coe_E[I-1][1-1] =-((dy*dy)/(rho*(a_P[I][1-1])));
100         Coe_W[I-1][1-1] =-((dy*dy)/(rho*(a_P[I-1][1-1]))) ;
101         Coe_N[I-1][1-1] =-((dx*dx)/(rho*(b_P[I-1][1])));
102         Source[I-1][1-1] = dy*u_star[I-1][1-1] - dy*u_star[I][1-1] + dx*v_star[I-1][1-1] - dx*
                v_star[I-1][1] ;
103     }
104     {
105         Coe_P[1-1][1-1] = 1.0;
106         Coe_E[1-1][1-1] = 0.0 ;
107         Coe_N[1-1][1-1] = 0.0;
108         Source[1-1][1-1] = 0.0 ;
109     }
110     {
111         Coe_P[Nx-1][1-1] =(((dy*dy)/(rho*(a_P[Nx-1][1-1])))+((dx*dx)/(rho*(b_P[Nx-1][1])))) ;
112         Coe_W[Nx-1][1-1] = -((dy*dy)/(rho*(a_P[Nx-1][1-1])));
113         Coe_N[Nx-1][1-1] = -((dx*dx)/(rho*(b_P[Nx-1][1])));
114         Source[Nx-1][1-1] = dy*u_star[Nx-1][1-1] - dy*u_star[Nx][1-1] + dx*v_star[Nx-1][1-1] -
                dx*v_star[Nx-1][1] ;
115     }
116     {
117         Coe_P[1-1][Ny-1] = (((dy*dy)/(rho*(a_P[1][Ny-1])))+((dx*dx)/(rho*(b_P[1-1][Ny-1])))) ;
118         Coe_E[1-1][Ny-1] =  -((dy*dy)/(rho*(a_P[1][Ny-1]))) ;
119         Coe_S[1-1][Ny-1] =  -((dx*dx)/(rho*(b_P[1-1][Ny-1])));
120         Source[1-1][Ny-1] =  dy*u_star[1-1][Ny-1] - dy*u_star[1][Ny-1] + dx*v_star[1-1][Ny-1] -
                dx*v_star[1-1][Ny] ;
121     }
122     {
123         Coe_P[Nx-1][Ny-1] =(((dy*dy)/(rho*(a_P[Nx-1][Ny-1])))+((dx*dx)/(rho*(b_P[Nx-1][Ny-1])))
                ); //coefficient for main elements
124         Coe_W[Nx-1][Ny-1] = -((dy*dy)/(rho*(a_P[Nx-1][Ny-1]))) ;
125         Coe_S[Nx-1][Ny-1] = -((dx*dx)/(rho*(b_P[Nx-1][Ny-1])));
126         Source[Nx-1][Ny-1] = dy*u_star[Nx-1][Ny-1] - dy*u_star[Nx][Ny-1] + dx*v_star[Nx-1][Ny
                -1] - dx*v_star[Nx-1][Ny] ;
127     }
128     }
129 }
130 void ADII_1(){
131     //Non-need to itteration and this is the directly solving process.
132     #pragma omp parallel default(shared)
133     {
134         //first : given value to the old value
135         #pragma omp for collapse(2) schedule(static) //average cutting workking for threads
136         for(int I = 1 ; I <= Nx ; I++){
137             for(int J = 1 ; J <= Ny ; J++){
138                 p_prime_old[I-1][J-1] = p_prime[I-1][J-1];
139             }
140         }
141         //find the coefficeints of the computing process D , beta , alpha , x , S , boundary
                condition
```

```
142         #pragma omp for schedule(static) //average cutting workking for threads
143         for(int J = 1 ; J <= Ny ; J++){//define the index from 0~Nx-2
144             vector <double> beta (Nx) ; vector <double> D (Nx) ; vector <double> alpha (Nx) ;
145             vector <double> x(Nx) ; vector <double> S (Nx) ; //for neumann condition , det the
                    boundary condition is zero through the conffi cients setting to get neumann
                    Condition
146             for(int I = 1 ; I <= Nx ; I++){
147             beta[I-1] = ((I == 1) ? 0.0 : Coe_W[I-1][J-1]) ;
148             D[I-1] =  ((I == 1 && J == 1) ? 1.0 :  Coe_P[I-1][J-1]) ;
149             alpha[I-1] =   (((I == 1 && J == 1) || I == Nx ) ? 0.0 :  Coe_E[I-1][J-1]) ;
150             x[I-1] = p_prime[I-1][J-1] ;
151             if(I == 1 && J == 1){ Source[I-1][J-1] = 0.0 ; }
152             else if(J == 1 || J == Ny){
153                 S[I-1] = Source[I-1][J-1] + ((J == 1) ?  - Coe_N[I-1][J-1]*p_prime_old[I-1][J]:
                        - Coe_S[I-1][J-1]*p_prime_old[I-1][J-2]);
154             }else{S[I-1] = Source[I-1][J-1] - Coe_S[I-1][J-1]*p_prime_old[I-1][J-2] - Coe_N[I
                    -1][J-1]*p_prime_old[I-1][J] ; }
155             }
156             TDMA_FVM(D , beta , alpha , x , S , Nx) ; //x[NX] = pressure correction value
157             for(int I = 1 ; I <= Nx ; I++){
158                 p_prime[I-1][J-1] = x[I-1] ;
159             }
160         }
161         //second : givee the value to the process value
162         #pragma omp for collapse(2) schedule(static) //average cutting workking for threads
163         for(int I = 1 ; I <= Nx ; I++){
164             for(int J = 1 ; J <= Ny ; J++){
165                 p_prime_old2[I-1][J-1] = p_prime[I-1][J-1];
166             }
167         }
168         #pragma omp for schedule(static) //average cutting workking for threads
169         for(int I = 1 ; I <=Nx ; I++){//define the index from 1 ~ Ny
170             vector <double> beta(Ny) ; vector <double> D(Ny) ; vector <double> alpha (Ny) ;
171             vector <double> x (Ny) ; vector <double> S (Ny) ; //For Neumann Condition , We
                    first set the B.C. is zero and through coefficient setting to schieve the
                    Neumann Condition
172             for(int J = 1 ; J <= Ny ; J++){
173             beta[J-1] = ((J == 1) ? 0.0 : Coe_S[I-1][J-1]) ;
174             D[J-1] = ((J == 1 && I == 1) ? 1.0 : Coe_P[I-1][J-1]) ;
175             alpha[J-1]= (((J == 1 && I == 1) || J == Ny ) ? 0.0 : Coe_N[I-1][J-1] ) ;
176             x[J-1] = p_prime[I-1][J-1] ;
177             if(I == 1 && J == 1){ Source[I-1][J-1] = 0.0 ; }
178             else if(I == 1 || I == Nx){S[J-1] = Source[I-1][J-1]+( (I == Nx) ? - Coe_W[I-1][J
                    -1]*p_prime_old2[I-2][J-1] : - Coe_E[I-1][J-1]*p_prime_old2[I][J-1] );}
179             else{S[J-1] = Source[I-1][J-1] - Coe_W[I-1][J-1]*p_prime_old2[I-2][J-1] - Coe_E[I
                    -1][J-1]*p_prime_old2[I][J-1] ; }
180             }
181             TDMA_FVM(D , beta , alpha , x , S , Ny) ;
182             for(int J = 1 ; J <= Ny ; J++){
183                 p_prime[I-1][J-1] =  x[J-1]  ;
184             }
185         }
186     }
187 }
188
189
190 void step3(){
191     initialze() ;
192     coefficient() ;
193     ADII_1() ;
194 }
```

Listing 7: 0.P_correction_eq_TDMA.h

A.2 Main Programs (.cpp files)

A.2.1 FDM Central Difference Scheme

```
1 // Finite difference SIMPLE solver for the 2D lid-driven cavity using a central difference
      scheme.
2 #include <iostream>
3 #include <fstream>
4 #include <sstream>
5 #include <cmath>
```

```cpp
#include <cstdlib>
#include <iomanip>
#include <string>
#include <vector>
#include <omp.h>
using namespace std ;

// Parameters used in the solver.
const int Nx = 161;
const int Ny = 161;
const int n = Nx * Ny ;
const int n1 = (Nx-2)*(Ny-2) ;
const double dx = 1.0/double(Nx-1) ;
const double dy = 1.0/double(Ny-1) ;
const double dt = 0.001 ;

int t , T ,G , K ;
const int  max_t = 10000000 ;
const int  max_G = 10000000 ;
double pu_star[Nx-1][Ny]; // Predicted x-momentum field.
double pv_star[Nx][Ny-1]; // Predicted y-momentum field.
double p_star[Nx][Ny]; // Predicted pressure field.
double p[Nx][Ny]; // Corrected pressure field.
double *x  ;
double *x_old ;
double p_prime[Nx][Ny]; // Pressure correction field.
double **a ;// Coefficient matrix.
double *b ;// Right-hand side vector.

const double  rho = 1.0 ;
const double  Re = 100 ;
const double  mu = (1.0/Re) ;

const double pu_bottom = 0.0 ;
const double pu_up = rho ;
const double pv_left = 0.0 ;
const double pv_right = 0.0 ;


double a1 = 2.0/(dx*dx) + 2.0/(dy*dy);
double b1 = -1.0/(dx*dx) ;
double c1 = -1.0/(dy*dy) ;

const double omega = 0.8 ;
double max_error ;
const float tolerance = 1e-6;
const float CE_tolerance = 1e-10;
bool CE_physical ;
bool steadystate ;

void allocate_memory(){
    x = new double[n1+2];
    x_old = new double[n1+2];
    b = new double[n1+2];

    a = new double*[n1+2];
    for(int i = 0; i < n1+2; i++){
        a[i] = new double[n1+2];
    }

    cout << "Memory allocated successfully for matrix size " << n1+2 << " x " << n1+2 << endl;
}

void deallocate_memory(){
    delete[] x;
    delete[] x_old;
    delete[] b;

    for(int i = 0; i < n1+2; i++){
        delete[] a[i];
    }
    delete[] a;

    cout << "Memory deallocated successfully" << endl;
}

```

```
82  double pressure(double x_coord , double y_coord){
83      int I = x_coord/dx ;
84      int J = y_coord/dy ;
85      return p[I][J] ;
86  }
87
88  double x_Velocity(double x_coord , double y_coord){
89      int i = (x_coord-(dx/2.0))/dx ;
90      int J = y_coord/dy ;
91      return pu_star[i][J]/rho;
92  }
93
94  double y_Velocity(double x_coord , double y_coord){
95      int I = x_coord/dx ;
96      int j = (y_coord-(dy/2.0))/dy ;
97      return pv_star[I][j] /rho;
98  }
99
100 double d(int I , int J , int m){
101     if(I < 2 || I > Nx-1 || J < 2 || J > Ny-1) return 0.0;
102
103     double dudx = (pu_star[I-1][J-1] - pu_star[I-2][J-1])/dx;
104     double dvdy = (pv_star[I-1][J-1] - pv_star[I-1][J-2])/dy;
105
106     return dudx + dvdy;
107 }
108
109 double get_max_residual(int k , int m) {
110     double max_residual = 0.0;
111
112     // Parallel residual computation.
113     #pragma omp parallel for collapse(2) reduction(max:max_residual)
114     for(int I = 2 ; I <= Nx-1; I++ ) {
115         for(int J = 2 ; J <= Ny-1; J++) {
116             double residual = abs(d(I,J,m));
117             if(residual > max_residual) {
118                 max_residual = residual;
119             }
120         }
121     }
122     return max_residual;
123 }
124
125 void time_init(int k){
126     // Parallel initialization.
127     #pragma omp parallel
128     {
129         // Initialize x-momentum field.
130         #pragma omp for collapse(2)
131         for(int i = 0; i < Nx-1; i++){
132             for(int J = 0; J < Ny; J++){
133                 pu_star[i][J] = 0.0;
134                 if(J == 0){
135                     pu_star[i][J] = pu_bottom ;
136                 }
137                 if(J == Ny-1){
138                     pu_star[i][J] = pu_up ;
139                 }
140             }
141         }
142
143         // Initialize y-momentum field.
144         #pragma omp for collapse(2)
145         for(int I = 0; I < Nx; I++){
146             for(int j = 0; j < Ny-1; j++){
147                 pv_star[I][j] = 0.0;
148                 if(I == 0){
149                     pv_star[I][j] = pv_left ;
150                 }
151                 if(I == Nx-1){
152                     pv_star[I][j] = pv_right ;
153                 }
154             }
155         }
156
157         // Initialize pressure fields.
```

```
158         #pragma omp for collapse(2)
159         for(int I = 0; I < Nx; I++){
160             for(int J = 0; J < Ny; J++){
161                 p[I][J] = 0.0;
162                 p_star[I][J] = 0.0;
163                 p_prime[I][J] = 0.0;
164             }
165         }
166     }
167 }
168
169 void comp_itter(double **a, double *b, int n1) {
170     // Parallel matrix initialization.
171     #pragma omp parallel for
172     for(int i = 1 ; i <= n1 ; i++){
173         for(int j = 1 ; j <= n1 ; j++){
174             a[i][j] = 0.0 ;
175         }
176         b[i] = 0.0 ;
177         x[i] = 0.0 ;
178         x_old[i] = 0.0 ;
179     }
180     a[n1+1][n1+1] = 1.0 ;
181     a[0][0] = 0.0 ;
182 }
183
184 void CD(double **a, double *b , int n , int m ){
185     // Set interior coefficients in parallel.
186     #pragma omp parallel for collapse(2)
187     for(int I = 3 ; I <= Nx-2 ; I++){
188         for(int J = 3 ; J <= Ny-2 ; J++){
189             int idx = (J-2)*(Nx-2) + (I-1) ;
190             b[idx] = -rho * d(I,J,m) ;
191             a[idx][idx] = a1 ;
192             a[idx][idx-1] = b1 ;
193             a[idx][idx+1] = b1 ;
194             a[idx][idx-(Nx-2)] = c1 ;
195             a[idx][idx+(Nx-2)] = c1 ;
196         }
197     }
198
199     // Boundary handling.
200     #pragma omp parallel sections
201     {
202         #pragma omp section
203         {
204             for(int J = 3 ; J <= Ny-2 ; J++){
205                 int idx = (J-2)*(Nx-2) + (2-1) ;
206                 b[idx] = -rho * d(2,J,m) ;
207                 a[idx][idx] = a1 ;
208                 a[idx][idx+1] = b1 ;
209                 a[idx][idx-(Nx-2)] = c1 ;
210                 a[idx][idx+(Nx-2)] = c1 ;
211             }
212         }
213
214         #pragma omp section
215         {
216             for(int J = 3 ; J <= Ny-2 ; J++){
217                 int idx = (J-2)*(Nx-2) + ((Nx-1)-1) ;
218                 b[idx] = -rho * d((Nx-1),J,m) ;
219                 a[idx][idx] = a1 ;
220                 a[idx][idx-1] = b1 ;
221                 a[idx][idx-(Nx-2)] = c1 ;
222                 a[idx][idx+(Nx-2)] = c1 ;
223             }
224         }
225
226         #pragma omp section
227         {
228             for(int I = 3 ; I <= Nx-2 ; I++){
229                 int idx = (2-2)*(Nx-2) + (I-1) ;
230                 b[idx] = -rho * d(I,2,m) ;
231                 a[idx][idx] = a1 ;
232                 a[idx][idx-1] = b1 ;
233                 a[idx][idx+1] = b1 ;
```

```
234                 a[idx][idx+(Nx-2)] = c1 ;
235             }
236         }
237
238         #pragma omp section
239         {
240             for(int I = 3 ; I <= Nx-2 ; I++){
241                 int idx = ((Ny-1)-2)*(Nx-2) + (I-1) ;
242                 b[idx] = -rho * d(I,(Ny-1),m) ;
243                 a[idx][idx] = a1 ;
244                 a[idx][idx-1] = b1 ;
245                 a[idx][idx+1] = b1 ;
246                 a[idx][idx-(Nx-2)] = c1 ;
247             }
248         }
249     }
250
251     // Corner handling.
252     int p1 = (2-2)*(Nx-2) + (2-1) ;
253     b[p1] = -rho * d(2,2,m) ;
254     a[p1][p1] = a1 ;
255     a[p1][p1+1] = b1 ;
256     a[p1][p1+(Nx-2)] = c1 ;
257
258     int p2 = (2-2)*(Nx-2) + ((Nx-1)-1) ;
259     b[p2] = -rho * d(Nx-1,2,m) ;
260     a[p2][p2] = a1 ;
261     a[p2][p2-1] = b1 ;
262     a[p2][p2+(Nx-2)] = c1 ;
263
264     int p3 = ((Ny-1)-2)*(Nx-2) + (2-1) ;
265     b[p3] = -rho * d(2,Ny-1,m) ;
266     a[p3][p3] = a1 ;
267     a[p3][p3+1] = b1 ;
268     a[p3][p3-(Nx-2)] = c1 ;
269
270     int p4 = ((Ny-1)-2)*(Nx-2) + ((Nx-1)-1) ;
271     b[p4] = -rho * d(Nx-1,Ny-1,m) ;
272     a[p4][p4] =  a1 ;
273     a[p4][p4-1] = b1 ;
274     a[p4][p4-(Nx-2)] = c1 ;
275 }
276
277 void momentum_time(int k , int m){
278     // Parallel update of the x-momentum field.
279     #pragma omp parallel for collapse(2)
280     for(int i = 1 ; i < Nx-2 ; i++){
281         for(int J = 1 ; J < Ny-1 ; J++){
282             double u_old = pu_star[i][J]/rho;
283             double u_e = pu_star[i+1][J]/rho;
284             double u_w = pu_star[i-1][J]/rho;
285             double u_n = (J < Ny-1) ? pu_star[i][J+1]/rho : u_old;
286             double u_s = (J > 0) ? pu_star[i][J-1]/rho : u_old;
287
288             double conv_x = u_old * (u_e - u_w)/(2*dx);
289
290             double v_avg = 0.0;
291             if(i > 0 && i < Nx-1 && J > 0 && J < Ny) {
292                 v_avg = 0.25*(pv_star[i][J] + pv_star[i+1][J] +
293                               pv_star[i][J-1] + pv_star[i+1][J-1])/rho;
294             }
295             double conv_y = v_avg * (u_n - u_s)/(2*dy);
296
297             double diff_x = mu/rho * (u_e - 2*u_old + u_w)/(dx*dx);
298             double diff_y = mu/rho * (u_n - 2*u_old + u_s)/(dy*dy);
299
300             double dp_dx = (p_star[i+1][J] - p_star[i][J])/dx;
301
302             pu_star[i][J] = pu_star[i][J] + dt*(-conv_x - conv_y + diff_x + diff_y - dp_dx/rho)
                  * rho;
303         }
304     }
305
306     // x-momentum boundary conditions.
307     #pragma omp parallel for
308     for(int i = 0; i < Nx-1; i++){
```

```
309         pu_star[i][0] = pu_bottom;
310         pu_star[i][Ny-1] = pu_up;
311     }
312 
313     // Parallel update of the y-momentum field.
314     #pragma omp parallel for collapse(2)
315     for(int I = 1 ; I < Nx-1 ; I++){
316         for(int j = 1 ; j < Ny-2 ; j++){
317             double v_old = pv_star[I][j]/rho;
318             double v_n = pv_star[I][j+1]/rho;
319             double v_s = pv_star[I][j-1]/rho;
320             double v_e = (I < Nx-1) ? pv_star[I+1][j]/rho : v_old;
321             double v_w = (I > 0) ? pv_star[I-1][j]/rho : v_old;
322 
323             double conv_y = v_old * (v_n - v_s)/(2*dy);
324 
325             double u_avg = 0.0;
326             if(I > 0 && I < Nx && j > 0 && j < Ny-1) {
327                 u_avg = 0.25*(pu_star[I][j] + pu_star[I][j+1] +
328                               pu_star[I-1][j] + pu_star[I-1][j+1])/rho;
329             }
330             double conv_x = u_avg * (v_e - v_w)/(2*dx);
331 
332             double diff_x = mu/rho * (v_e - 2*v_old + v_w)/(dx*dx);
333             double diff_y = mu/rho * (v_n - 2*v_old + v_s)/(dy*dy);
334 
335             double dp_dy = (p_star[I][j+1] - p_star[I][j])/dy;
336 
337             pv_star[I][j] = pv_star[I][j] + dt*(-conv_x - conv_y + diff_x + diff_y - dp_dy/rho)
                  * rho;
338         }
339     }
340 
341     // y-momentum boundary conditions.
342     #pragma omp parallel for
343     for(int j = 0; j < Ny-1; j++){
344         pv_star[0][j] = pv_left;
345         pv_star[Nx-1][j] = pv_right;
346     }
347 }
348 
349 void Jacobi(double **a, double *b, double *x, int n1 , int m) {
350     // Copy previous solution.
351     #pragma omp parallel for
352     for(int k = 1; k <= n1; k++) {
353         x_old[k] = x[k];
354     }
355 
356     // Set interior coefficients in parallel.
357    // For uneven workloads consider dynamic scheduling.
358    #pragma omp parallel for collapse(2) schedule(static)
359     for(int I = 3 ; I <= Nx-2 ; I++){
360         for(int J = 3 ; J <= Ny-2 ; J++){
361             int idx = (J-2)*(Nx-2) + (I-1) ;
362             double sum = 0.0 ;
363             // Five-point stencil.
364             sum += a[idx][idx+1] * x_old[idx+1] ;
365             sum += a[idx][idx-1] * x_old[idx-1] ;
366             sum += a[idx][idx+(Nx-2)] * x_old[idx+(Nx-2)] ;
367             sum += a[idx][idx-(Nx-2)] * x_old[idx-(Nx-2)] ;
368             x[idx] = (1-omega)*x_old[idx] + (omega/a[idx][idx]) * (b[idx] - sum) ;
369         }
370     }
371     // Handle all four boundaries in a single loop.
372 #pragma omp parallel
373 {
374     // Left and right boundaries.
375     #pragma omp for nowait
376     for(int J = 3; J <= Ny-2; J++) {
377         // Left boundary.
378         int idx = (J-2)*(Nx-2) + (2-1);
379         double sum = a[idx][idx+1] * x_old[idx+1] +
380                      a[idx][idx+(Nx-2)] * x_old[idx+(Nx-2)] +
381                      a[idx][idx-(Nx-2)] * x_old[idx-(Nx-2)];
382         x[idx] = (1-omega)*x_old[idx] + (omega/a[idx][idx]) * (b[idx] - sum);
383 
```

```
384         // Right boundary.
385         idx = (J-2)*(Nx-2) + ((Nx-1)-1);
386         sum = a[idx][idx-1] * x_old[idx-1] +
387               a[idx][idx+(Nx-2)] * x_old[idx+(Nx-2)] +
388               a[idx][idx-(Nx-2)] * x_old[idx-(Nx-2)];
389         x[idx] = (1-omega)*x_old[idx] + (omega/a[idx][idx]) * (b[idx] - sum);
390     }
391
392     // Top and bottom boundaries.
393     #pragma omp for nowait
394     for(int I = 3; I <= Nx-2; I++) {
395         // Bottom boundary.
396         int idx = (2-2)*(Nx-2) + (I-1);
397         double sum = a[idx][idx+1] * x_old[idx+1] +
398                      a[idx][idx-1] * x_old[idx-1] +
399                      a[idx][idx+(Nx-2)] * x_old[idx+(Nx-2)];
400         x[idx] = (1-omega)*x_old[idx] + (omega/a[idx][idx]) * (b[idx] - sum);
401
402         // Top boundary.
403         idx = ((Ny-1)-2)*(Nx-2) + (I-1);
404         sum = a[idx][idx+1] * x_old[idx+1] +
405               a[idx][idx-1] * x_old[idx-1] +
406               a[idx][idx-(Nx-2)] * x_old[idx-(Nx-2)];
407         x[idx] = (1-omega)*x_old[idx] + (omega/a[idx][idx]) * (b[idx] - sum);
408     }
409 }
410 #pragma omp parallel sections
411 {
412     #pragma omp section
413     {
414         int p1 = (2-2)*(Nx-2) + (2-1);
415         double sum = a[p1][p1+1] * x_old[p1+1] + a[p1][p1+(Nx-2)] * x_old[p1+(Nx-2)];
416         x[p1] = (1-omega)*x_old[p1] + (omega/a[p1][p1]) * (b[p1] - sum);
417     }
418
419     #pragma omp section
420     {
421         int p2 = (2-2)*(Nx-2) + ((Nx-1)-1);
422         double sum = a[p2][p2-1] * x_old[p2-1] + a[p2][p2+(Nx-2)] * x_old[p2+(Nx-2)];
423         x[p2] = (1-omega)*x_old[p2] + (omega/a[p2][p2]) * (b[p2] - sum);
424     }
425
426     #pragma omp section
427     {
428         int p3 = ((Ny-1)-2)*(Nx-2) + (2-1);
429         double sum = a[p3][p3+1] * x_old[p3+1] + a[p3][p3+(Nx-2)] * x_old[p3+(Nx-2)];
430         x[p3] = (1-omega)*x_old[p3] + (omega/a[p3][p3]) * (b[p3] - sum);
431     }
432
433     #pragma omp section
434     {
435         int p4 = ((Ny-1)-2)*(Nx-2) + ((Nx-1)-1);
436         double sum = a[p4][p4-1] * x_old[p4-1] + a[p4][p4+(Nx-2)] * x_old[p4+(Nx-2)];
437         x[p4] = (1-omega)*x_old[p4] + (omega/a[p4][p4]) * (b[p4] - sum);
438     }
439 }
440
441     // Compute maximum error (using reduction).
442     max_error = 0.0;
443     #pragma omp parallel for reduction(max:max_error)
444     for(int i = 1 ; i <= n1 ; i++){
445         double error = fabs(x[i] - x_old[i]);
446         if(error > max_error) max_error = error ;
447     }
448
449     // Update pressure correction field.
450     #pragma omp parallel for collapse(2)
451     for(int I = 2 ; I <= Nx-1 ; I++){
452         for(int J = 2 ; J <= Ny-1 ; J++){
453             p_prime[I-1][J-1] = x[(J-2)*(Nx-2) + (I-1)] ;
454         }
455     }
456
457     // Apply boundary conditions.
458     #pragma omp parallel
459     {
```

```
460         #pragma omp for
461         for(int I = 2 ; I <= Nx-1 ; I++){
462             p_prime[I-1][0] = p_prime[I-1][1] ;
463             p_prime[I-1][Ny-1] = p_prime[I-1][Ny-2] ;
464         }
465
466         #pragma omp for
467         for(int J = 2 ; J <= Ny-1 ; J++){
468             p_prime[0][J-1] = p_prime[1][J-1] ;
469             p_prime[Nx-1][J-1] = p_prime[Nx-2][J-1] ;
470         }
471     }
472
473     p_prime[0][0] = p_prime[1][1];
474     p_prime[Nx-1][0] = p_prime[Nx-2][1];
475     p_prime[0][Ny-1] = p_prime[1][Ny-2];
476     p_prime[Nx-1][Ny-1] = p_prime[Nx-2][Ny-2];
477 }
478
479 void pressure_time(int k , int m){
480     // Parallel pressure update.
481     #pragma omp parallel
482     {
483         #pragma omp for collapse(2)
484         for(int I = 2 ; I <= Nx-1 ; I++){
485             for(int J = 2 ; J <= Ny-1 ; J++){
486                 p[I-1][J-1] = p_prime[I-1][J-1] + p_star[I-1][J-1] ;
487             }
488         }
489
490         // Pressure boundary conditions.
491         #pragma omp for
492         for(int I = 2 ; I <= Nx-1 ; I++){
493             p[I-1][0] = p[I-1][1] ;
494             p[I-1][Ny-1] = p[I-1][Ny-2] ;
495         }
496
497         #pragma omp for
498         for(int J = 2 ; J <= Ny-1 ; J++){
499             p[0][J-1] = p[1][J-1] ;
500             p[Nx-1][J-1] = p[Nx-2][J-1] ;
501         }
502     }
503
504     p[0][0] = p[1][1];
505     p[Nx-1][0] = p[Nx-2][1];
506     p[0][Ny-1] = p[1][Ny-2];
507     p[Nx-1][Ny-1] = p[Nx-2][Ny-2];
508
509     // Parallel velocity correction.
510     #pragma omp parallel
511     {
512         // Correct x-velocity component.
513         #pragma omp for collapse(2)
514         for(int i = 1; i < Nx-2; i++){
515             for(int J = 1; J < Ny-1; J++){
516                 double dp_dx = (p_prime[i+1][J] - p_prime[i][J])/dx;
517                 pu_star[i][J] -= dt * dp_dx;
518             }
519         }
520
521         // Correct y-velocity component.
522         #pragma omp for collapse(2)
523         for(int I = 1; I < Nx-1; I++){
524             for(int j = 1; j < Ny-2; j++){
525                 double dp_dy = (p_prime[I][j+1] - p_prime[I][j])/dy;
526                 pv_star[I][j] -= dt * dp_dy;
527             }
528         }
529
530         // Reapply boundary conditions.
531         #pragma omp for
532         for(int i = 0; i < Nx-1; i++){
533             pu_star[i][0] = pu_bottom;
534             pu_star[i][Ny-1] = pu_up;
535         }
```

```
536
537         #pragma omp for
538         for(int j = 0; j < Ny-1; j++){
539             pv_star[0][j] = pv_left;
540             pv_star[Nx-1][j] = pv_right;
541         }
542     }
543 }
544
545 void refresh_pressure(int k , int m ){
546     #pragma omp parallel for collapse(2)
547     for(int I = 1 ; I <= Nx ; I++){
548         for(int J = 1 ; J <= Ny ; J++){
549             p_star[I-1][J-1] = p[I-1][J-1] ;
550         }
551     }
552 }
553
554 void output_combined_fields(int m) {
555     ostringstream name;
556     name << "1.Central Difference_Re = "<< Re << "_Grid"<< Nx << "x" << Ny << setfill('0') <<
            setw(6) << m << ".vtk";
557     ofstream out(name.str().c_str());
558
559     out << "# vtk DataFile Version 3.0\n";
560     out << "SIMPLE_CD_Combined_Fields\n";
561     out << "ASCII\n";
562     out << "DATASET STRUCTURED_POINTS\n";
563     out << "DIMENSIONS " << Nx << " " << Ny << " 1\n";
564     out << "ORIGIN 0 0 0\n";
565     out << "SPACING " << dx << " " << dy << " 1\n";
566
567     out << "POINT_DATA " << Nx * Ny << "\n";
568     out << "SCALARS Pressure double 1\n";
569     out << "LOOKUP_TABLE default\n";
570     for(int j = 0; j < Ny; j++) {
571         for(int i = 0; i < Nx; i++) {
572             out << scientific << p[i][j] << "\n";
573         }
574     }
575
576     out << "VECTORS Velocity double\n";
577     for(int j = 0; j < Ny; j++) {
578         for(int i = 0; i < Nx; i++) {
579             double u = 0.0, v = 0.0;
580
581             if(i > 0 && i < Nx-1) {
582                 u = 0.5 * (pu_star[i-1][j] + pu_star[i][j])/rho;
583             }
584             if(j > 0 && j < Ny-1) {
585                 v = 0.5 * (pv_star[i][j-1] + pv_star[i][j])/rho;
586             }
587
588             out << scientific << u << " " << v << " 0.0\n";
589         }
590     }
591
592     out.close();
593     cout << "VTK file has been output: " << name.str() << endl;
594 }
595
596 int main() {
597     // Configure OpenMP thread count.
598     int num_threads = omp_get_max_threads();
599     omp_set_num_threads(num_threads);
600
601     cout << "========================================" << endl;
602     cout << "SIMPLE_CD OpenMP Parallel Version" << endl;
603     cout << "Using " << num_threads << " threads" << endl;
604     cout << "========================================" << endl;
605     cout << "Mesh size : " << Nx << " x " << Ny << endl;
606     cout << "Grid space: dx=" << dx << ", dy=" << dy << endl;
607     cout << "Boundary conditions (u,v) : " << endl;
608     cout << "Left boundary = (0,0) ; Right boundary = (0,0) ; Bottom boundary = (0,0) ; Top
            boundary = (1,0) " << endl ;
609     cout << "Re = " << Re << ", viscosity = " << mu << endl ;
```

```
610
611     cout << "Allocating memory for pressure correction matrix..." << endl;
612     allocate_memory();
613
614     CE_physical = false;
615     t = 0 ;
616     time_init(n) ;
617
618     for(int t = 0 ; t <= max_t ; t++){
619         if(t%10000 == 0){
620             cout << "Time step :" << t << endl;
621             cout << "the next time step , mass imbalance = " << get_max_residual(n , t) << endl
                    ;
622             output_combined_fields(t) ;
623         }
624
625         momentum_time(n , t) ;
626
627         steadystate = false ;
628         G = 0 ;
629         comp_itter(a,b,n1) ;
630         CD(a,b,n1,t);
631
632         for(G = 0 ; G <= max_G ; G++){
633             if(t%10000 == 0 && G%5000 == 0){
634                 cout <<"       " << "Iteration:" << G << endl ;
635                 cout <<"       " << "Maximum iteration error = " << max_error << endl ;
636             }
637             Jacobi(a,b,x,n1,t) ;
638             if(G>0 && max_error < tolerance){
639                 steadystate = true ;
640                 K = G ;
641                 if(t%10000 == 0){
642                     cout << "       " << "Pressure correction convergence achieved!" << endl;
643                     cout << "       " << "Iteration:" << K << endl ;
644                     cout << "       " << "Maximum iteration error = " << max_error << endl ;
645                 }
646                 break ;
647             }
648         }
649
650         if(!steadystate){
651             K = G-1 ;
652             cout << "       " << "Maximum number of iterations reached, but pressure correction
                     convergence not achieved!" << endl;
653             cout << "       " << "Iteration:" << K << endl ;
654             cout << "       " << "Maximum iteration error = " << max_error << endl ;
655             T = t ;
656             break ;
657         }else{
658             pressure_time(n , t) ;
659             refresh_pressure(n , t) ;
660
661             if(t>0 && get_max_residual(n , t) < CE_tolerance){
662                 CE_physical = true ;
663                 T = t ;
664                 cout << "Continuous equation convergence achieved!" << endl ;
665                 cout << "Time step:" << T << endl ;
666                 cout << "the next time step , mass imbalance = " << get_max_residual(n , t) <<
                        endl ;
667                 output_combined_fields(T) ;
668                 break ;
669             }
670         }
671     }
672
673     if(!CE_physical){
674         if(!steadystate){
675             T = T-1 ;
676             cout << "Maximum number of iterations reached, but continuous equation convergence
                    not achieved!" << endl ;
677             cout << "Time step:" << T << endl ;
678             cout << "the next time step , mass imbalance = " << get_max_residual(n , T) << endl
                    ;
679         }else{
680             T = t-1 ;
```

```
681             cout << "Maximum time step reached, but continuous equation convergence not
                    achieved!" << endl ;
682             cout << "Time step:" << T << endl ;
683             cout << "the next time step , mass imbalance = " << get_max_residual(n , T) << endl
                    ;
684         }
685     }
686
687     cout << "Grid size " << Nx << "x" << Ny << " computation completed" << endl;
688
689     deallocate_memory();
690
691     return 0 ;
692 }
```

Listing 8: 1.FDM_CDs.cpp

A.2.2 FVM Non-Deferred Correction

```
1  // Finite volume SIMPLE solver for the Navier-Stokes equations.
2  #include <iostream>
3  #include <fstream>
4  #include <sstream>
5  #include <cmath>
6  #include <cstdlib>
7  #include <iomanip>
8  #include <string>
9  #include <vector>
10 #include <omp.h>
11 #include "0.P_correction_eq.h"
12 using namespace std ;
13 // Parameter definitions.
14 // Grid configuration used by the solver.
15 // Pressure grid has Nx cells along the x direction.
16 // u-velocity grid has Nx+1 cells along the x direction.
17 // v-velocity grid has Ny+1 cells along the y direction.
18 int G ;  // Current time step (starts from 0), different from inner iterations.
19 double min_step7 ;
20 const int  max_G = 10000000;// Maximum allowed time steps.
21 // u field indices: i in [0, Nx], j in [0, Ny-1].
22 // v field indices: i in [0, Nx-1], j in [0, Ny].
23 // Pressure indices: i in [0, Nx-1], j in [0, Ny-1].
24 double u_star[Nx+1][Ny] ; // Predicted u-velocity field.
25 double u_star_old[Nx+1][Ny] ;
26 double v_star[Nx][Ny+1] ; // Predicted v-velocity field.
27 double v_star_old[Nx][Ny+1] ;
28 double p_star[Nx][Ny] ; // Predicted pressure field.
29 double u_prime[Nx+1][Ny] ; // u-velocity correction.
30 double v_prime[Nx][Ny+1] ; // v-velocity correction.
31 double p_prime[Nx][Ny] ; // Pressure correction.
32 double u[Nx+1][Ny] ; // Corrected u-velocity field.
33 // Previous corrected u-velocity field.
34 double u_old[Nx+1][Ny] ;// Needs initialization before iterations.
35 double v[Nx][Ny+1] ; // Corrected v-velocity field.
36 // Previous corrected v-velocity field.
37 double v_old[Nx][Ny+1] ;// Needs initialization before iterations.
38 double p[Nx][Ny] ; // Corrected pressure field.
39
40 const double  Re = 5000.0 ;
41 // Boundary conditions for the u velocity.
42 const double u_left  = 0.0 ;// No penetration at the wall.
43 const double u_right = 0.0 ;// No penetration at the wall.
44 const double u_bottom = 0.0 ;// No-slip condition.
45 const double u_top = 1.0 ;// No-slip lid velocity.
46 // Boundary conditions for the v velocity.
47 const double v_left  = 0.0 ;// No-slip condition.
48 const double v_right = 0.0 ;// No-slip condition.
49 const double v_bottom = 0.0 ;// No penetration at the wall.
50 const double v_top = 0.0 ;// No penetration at the wall.
51 const double omega_P = 0.5 ; // Relaxation factor following Patankar recommendations.
52 const double omega_v  = 0.3 ;
53 const double omega_u  = 0.3 ;
54 double max_error ;
55 const float CE_tolerance = 1e-10; // Convergence check: predicted continuity residual.
56 bool CE_physical ;
```

```
57
58 // Convection coefficients shared by u and v discretizations.
59 const double D_e = (1.0/Re)*(dy/dx);
60 const double D_w = (1.0/Re)*(dy/dx);
61 const double D_n = (1.0/Re)*(dy/dx);
62 const double D_s = (1.0/Re)*(dy/dx);
63 double r_w_u[Nx+1][Ny] ; // Face selection coefficients for u.
64 double r_e_u[Nx+1][Ny] ;
65 double r_s_u[Nx+1][Ny] ;
66 double r_n_u[Nx+1][Ny] ;
67 double r_w_v[Nx][Ny+1] ; // Face selection coefficients for v.
68 double r_e_v[Nx][Ny+1] ;
69 double r_s_v[Nx][Ny+1] ;
70 double r_n_v[Nx][Ny+1] ;
71 // u-momentum convection coefficients; alpha factors determined by F.
72 double F_w_u[Nx+1][Ny] ;
73 double alpha_w_u[Nx+1][Ny] ;
74 double F_e_u[Nx+1][Ny] ;
75 double alpha_e_u[Nx+1][Ny] ;
76 double F_s_u[Nx+1][Ny] ;
77 double alpha_s_u[Nx+1][Ny] ;
78 double F_n_u[Nx+1][Ny] ;
79 double alpha_n_u[Nx+1][Ny] ;
80 // u-momentum linear combination coefficients.
81 double a_P[Nx+1][Ny] ;
82 double a_WW[Nx+1][Ny] ;
83 double a_W[Nx+1][Ny] ;
84 double a_E[Nx+1][Ny] ;
85 double a_EE[Nx+1][Ny] ;
86 double a_SS[Nx+1][Ny] ;
87 double a_S[Nx+1][Ny] ;
88 double a_N[Nx+1][Ny] ;
89 double a_NN[Nx+1][Ny] ;
90 double S1[Nx+1][Ny] ; // Source term for u-momentum equation.
91 // v-momentum convection coefficients; alpha factors determined by F.
92 double F_w_v[Nx][Ny+1] ;
93 double alpha_w_v[Nx][Ny+1] ;
94 double F_e_v[Nx][Ny+1] ;
95 double alpha_e_v[Nx][Ny+1] ;
96 double F_s_v[Nx][Ny+1] ;
97 double alpha_s_v[Nx][Ny+1] ;
98 double F_n_v[Nx][Ny+1] ;
99 double alpha_n_v[Nx][Ny+1] ;
100 // v-momentum linear combination coefficients.
101 double b_P[Nx][Ny+1] ;
102 double b_WW[Nx][Ny+1] ;
103 double b_W[Nx][Ny+1] ;
104 double b_E[Nx][Ny+1] ;
105 double b_EE[Nx][Ny+1] ;
106 double b_SS[Nx][Ny+1] ;
107 double b_S[Nx][Ny+1] ;
108 double b_N[Nx][Ny+1] ;
109 double b_NN[Nx][Ny+1] ;
110 double S2[Nx][Ny+1] ;// Source term for v-momentum equation.
111 // Update these matrices immediately after each predictor step.
112
113
114 //(1)
115 // CDs upstream and downstream iteration coefficients
116 double D(double x){
117   return 0.5  ;
118 }
119 double U(double x){
120     return 0.5 ;
121 }
122 double UU(double x){
123   return 0.0 ;
124
125 }
126
127 /*//(2)
128 // QUICK upstream and downstream iteration coefficients
129 double D(double x){
130   return 3.0/8.0     ;
131 }
132 double U(double x){
```

```
133     return 6.0/8.0 ;
134 }
135 double UU(double x){
136   return -1.0/8.0 ;
137
138 }*/
139
140
141 /*//(3)TVD-MUSCL upstream and downstream iteration coefficients
142 double D(double x) {
143     if (x <= 0) return 0.0;
144     else if (x > 0 && x < (1.0/3.0)) return 0.0;
145     else if (x >= (1.0/3.0) && x < 3.0) return 0.25;
146     else return 1.0;
147 }
148
149 double U(double x) {
150     if (x <= 0) return 1.0;
151     else if (x > 0 && x <= (1.0/3.0)) return 2.0;
152     else if (x > (1.0/3.0) && x <= 3.0) return 1.0;
153     else return 0.0;
154 }
155
156 double UU(double x) {
157     if (x <= 0) return 0.0;
158     else if (x > 0 && x < (1.0/3.0)) return -1.0;
159     else if (x >= (1.0/3.0) && x < 3.0) return -0.25;
160     else return 0.0;
161 }*/
162
163
164 // SIMPLE algorithm workflow.
165 // Step 1: initialize u, v, and pressure predictor fields.
166 void initialize(){
167     #pragma omp parallel  // Parallel initialization block.
168     {
169     // Initialize predicted u-velocity field.
170      #pragma omp parallel for collapse(2)
171     for(int i = 1 ; i <= Nx+1 ; i++){
172         // u-star defined with i in [0, Nx], j in [0, Ny-1].
173         for(int J= 1 ; J <= Ny ; J++){
174             u_star[i-1][J-1] = 0.0 ;
175             if(i == 1) u_star[i-1][J-1] = u_left ; // No-penetration wall.
176             if(i == Nx + 1) u_star[i-1][J-1] = u_right ; // No-penetration wall.
177         }
178     }
179     // Initialize previous corrected u field using the predictor.
180      #pragma omp parallel for collapse(2)
181     for(int i = 1 ; i <= Nx+1 ; i++){
182         for(int J = 1 ; J <= Ny ; J++){
183             u_old[i-1][J-1] = u_star[i-1][J-1] ;
184         }
185     }
186     // Initialize predicted v-velocity field.
187      #pragma omp parallel for collapse(2)
188     for(int I = 1 ; I <= Nx ; I++){
189         // v-star defined with i in [0, Nx-1], j in [0, Ny].
190         for(int j = 1 ; j <= Ny+1 ; j++){
191             v_star[I-1][j-1] = 0.0 ;
192             if(j == 1)v_star[I-1][j-1] = v_bottom ;// No-penetration wall.
193             if(j == Ny + 1)v_star[I-1][j-1] = v_top ;// No-penetration wall.
194         }
195     }
196     // Initialize previous corrected v field using the predictor.
197      #pragma omp parallel for collapse(2)
198     for(int I = 1 ; I <= Nx ; I++){
199         for(int j = 1 ; j <= Ny+1 ; j++){
200             v_old[I-1][j-1] = v_star[I-1][j-1] ;
201         }
202     }
203     // Initialize predicted pressure and correction.
204      #pragma omp parallel for collapse(2)
205     for(int I = 1 ; I <= Nx ; I++){
206         for(int J = 1 ; J<= Ny ; J++){
207             p_star[I-1][J-1] =  0.0  ; // Neumann boundary imposed via solver.
208             p_prime[I-1][J-1] = 0.0 ;
```

```
209         }
210     }
211     // Predictor uses: face-selection factors -> convection coefficients -> linear combination
            coefficients.
212     // Initialization order: face-selection, then convection, then linear coefficients.
213     // Face-selection factors for the u field (special treatment near north/south boundaries).
214      #pragma omp parallel for collapse(2)
215     for(int i = 1 ; i <= Nx+1 ; i++){
216         for(int J = 1 ; J <= Ny ; J++){
217             // Trivial cells get zero face factors.
218             if(i == 1 || i == Nx+1){
219                 r_w_u[i-1][J-1] = 0.0 ;
220                 r_e_u[i-1][J-1] = 0.0 ;
221                 r_n_u[i-1][J-1] = 0.0 ;
222                 r_s_u[i-1][J-1] = 0.0 ;
223             }else {
224                 // Handle each boundary separately (west/east/south/north cases).
225                 // West boundary.
226                 if(i == 2){
227                     r_w_u[i-1][J-1] = -1.0 ; // Upwind at west face.
228                 }else{
229                     r_w_u[i-1][J-1] = 1.0 ; // Interior initialization.
230                 }
231                 // East boundary.
232                 if(i == Nx){
233                     r_e_u[i-1][J-1] = -1.0 ; // Upwind at east face.
234                 }else{
235                     r_e_u[i-1][J-1] = 1.0 ;// Interior initialization.
236                 }
237                 // South boundary.
238                 if(J == 1){
239                     r_s_u[i-1][J-1] = 0.0 ; // Bottom cells do not need south factor.
240                 }else if(J == 2){
241                     r_s_u[i-1][J-1] = -1.0 ;// Upwind for stability.
242                 }else if(J == Ny){
243                     r_s_u[i-1][J-1] = -1.0 ;// Upwind near top boundary.
244                 }else{
245                     r_s_u[i-1][J-1] = 1.0 ;// Interior initialization.
246                 }
247                 // North boundary.
248                 if(J == 1){
249                     r_n_u[i-1][J-1] = -1.0 ; // Upwind near bottom.
250                 }else if (J == Ny-1){
251                     r_n_u[i-1][J-1] = -1.0 ;// Upwind near top.
252                 }else if(J == Ny){
253                     r_n_u[i-1][J-1] = 0.0 ; // Top cells do not need north factor.
254                 }else{
255                     r_n_u[i-1][J-1] = 1.0 ;// Interior initialization.
256                 }
257             }
258         }
259     }// After each predictor update, refresh these face-selection factors.
260     // Face-selection factors for the v field (special handling east/west).
261      #pragma omp parallel for collapse(2)
262     for(int I = 1 ; I <= Nx ; I++){
263         for(int j = 1 ; j <= Ny+1 ; j++){
264             // Trivial cells receive zero factors.
265             if(j == 1 || j == Ny+1){
266                 r_w_v[I-1][j-1] = 0.0 ;
267                 r_e_v[I-1][j-1] = 0.0 ;
268                 r_n_v[I-1][j-1] = 0.0 ;
269                 r_s_v[I-1][j-1] = 0.0 ;
270             }else {
271                 // Evaluate south/north/west/east boundaries separately.
272                 // South boundary.
273                 if(j == 2){
274                     r_s_v[I-1][j-1] = -1.0 ; // Upwind at bottom face.
275                 }else{
276                     r_s_v[I-1][j-1] = 1.0 ; // Interior initialization.
277                 }
278                 // North boundary.
279                 if(j == Ny){
280                     r_n_v[I-1][j-1] = -1.0 ; // Upwind at top face.
281                 }else{
282                     r_n_v[I-1][j-1] = 1.0 ; // Interior initialization.
283                 }
```

```
284                 // West boundary.
285                 if(I == 1){
286                     r_w_v[I-1][j-1] = 0.0 ; // Left cells do not need west factor.
287                 }else if (I == 2){
288                     r_w_v[I-1][j-1] = -1.0 ;// Upwind near inlet.
289                 }else if(I == Nx){
290                     r_w_v[I-1][j-1] = -1.0 ;// Upwind near outlet.
291                 }else{
292                     r_w_v[I-1][j-1] = 1.0 ;// Interior initialization.
293                 }
294                 // East boundary.
295                 if(I == 1){
296                     r_e_v[I-1][j-1] = -1.0 ; // Upwind near inlet.
297                 }else if (I == Nx-1){
298                     r_e_v[I-1][j-1] = -1.0 ; // Upwind near outlet.
299                 }else if(I == Nx){
300                     r_e_v[I-1][j-1] = 0.0 ; // Right cells do not need east factor.
301                 }else {
302                     r_e_v[I-1][j-1] = 1.0 ; // Interior initialization.
303                 }
304             }
305         }
306     }
307     // Initialize convection coefficient matrices.
308     // u-momentum convection coefficients.
309      #pragma omp parallel for collapse(2)
310     for(int i = 1 ; i <= Nx+1 ; i++){
311         for(int J = 1 ; J <= Ny ; J++){
312             if(i == 1 || i == Nx+1) {
313                 F_w_u[i-1][J-1] = 0.0 ;
314                 F_e_u[i-1][J-1] = 0.0 ;
315                 F_n_u[i-1][J-1] = 0.0 ;
316                 F_s_u[i-1][J-1] = 0.0 ;
317             }else{
318                 F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); // Cell center
                        aligns with storage.
319                 F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
320                 F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
321                 F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
322             }
323         }
324     }// Alpha factors for u, indexed by cell centers.
325      #pragma omp parallel for collapse(2)
326     for(int i = 1 ; i <= Nx+1 ; i++){
327         for(int J = 1 ; J <= Ny ; J++){
328             if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
329             else alpha_w_u[i-1][J-1] = 0.0 ;
330             if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
331             else alpha_e_u[i-1][J-1] = 0.0 ;
332             if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
333             else alpha_n_u[i-1][J-1] = 0.0 ;
334             if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
335             else alpha_s_u[i-1][J-1] = 0.0 ;
336         }
337     }
338     // v-momentum convection coefficients.
339      #pragma omp parallel for collapse(2)
340     for(int I = 1 ; I <= Nx ; I++){
341         for(int j = 1 ; j <= Ny+1 ; j++){
342             if(j == 1 || j == Ny+1) {
343                 F_w_v[I-1][j-1] = 0.0 ;
344                 F_e_v[I-1][j-1] = 0.0 ;
345                 F_n_v[I-1][j-1] = 0.0 ;
346                 F_s_v[I-1][j-1] = 0.0 ;
347             }else{
348                 F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
349                 F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
350                 F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);// Cell center
                        aligns with storage.
351                 F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
352             }
353         }
354     }// Alpha factors for v, indexed by cell centers.
355      #pragma omp parallel for collapse(2)
356     for(int I = 1 ; I <= Nx ; I++){
357         for(int j = 1 ; j <= Ny+1 ; j++){
```

```
358             if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
359             else alpha_w_v[I-1][j-1] = 0.0 ;
360             if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
361             else alpha_e_v[I-1][j-1] = 0.0 ;
362             if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
363             else alpha_n_v[I-1][j-1] = 0.0 ;
364             if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
365             else alpha_s_v[I-1][j-1] = 0.0 ;
366         }
367     }
368 }
369 }
370
371 // Step 2: update predicted u and v velocity fields.
372 void step2(int m){
373     // Update the u-momentum predictor coefficients.
374     #pragma omp parallel for collapse(2)
375     for(int i = 1 ; i <= Nx+1 ; i++){
376         for(int J = 1 ; J <= Ny ; J++){
377             // Reset all neighbor coefficients before accumulation.
378             a_P[i-1][J-1] = 0.0 ; // Reset pressure coefficients.
379             a_WW[i-1][J-1] = 0.0 ;
380             a_W[i-1][J-1] = 0.0 ;
381             a_E[i-1][J-1] = 0.0 ;
382             a_EE[i-1][J-1] = 0.0 ;
383             a_SS[i-1][J-1] = 0.0 ;
384             a_S[i-1][J-1] = 0.0 ;
385             a_N[i-1][J-1] = 0.0 ;
386             a_NN[i-1][J-1] = 0.0 ;
387             S1[i-1][J-1] = 0.0 ;
388             if(i == 1){
389                 a_P[i-1][J-1] += 1.0 ;
390                 S1[i-1][J-1] += u_left ;// No-penetration wall; no pressure gradient.
391                 u_star[i-1][J-1] = u_left ;
392             }else if(i == Nx+1){
393                 a_P[i-1][J-1] += 1.0 ;
394                 S1[i-1][J-1] += u_right ;// No-penetration wall; no pressure gradient.
395                 u_star[i-1][J-1] = u_right ;
396             }// Special treatments: near west/east boundaries and first/last rows.
397             // West boundary second column.
398             else{ // For remaining cells handle west/east/south/north faces separately.
399                 // West boundary.
400                 if(i == 2){// Second column uses upwind on the west face.
401                     a_W[i-1][J-1] += alpha_w_u[i-1][J-1]*(-F_w_u[i-1][J-1])-D_w;// Remove west
                            diffusive term explicitly.
402                     a_P[i-1][J-1] += (1-alpha_w_u[i-1][J-1])*(-F_w_u[i-1][J-1])+D_w;
403                     S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho ;
404                 }else{// Interior west faces use TVD-MUSCL.
405                     a_WW[i-1][J-1] += (alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J
                            -1]) ; // Upwind for west faces.
406                     a_W[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*D(r_w_u[i-1][J-1])+alpha_w_u[i
                            -1][J-1]*U(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1]) - D_w ; // Subtract
                            west diffusive term when processing this face.
407                     a_E[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i
                            -1][J-1]);
408                     a_P[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*U(r_w_u[i-1][J-1])+alpha_w_u[i
                            -1][J-1]*D(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1])+D_w ;
409                     S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho ;
410                 }
411                 // East boundary.
412                 if(i == Nx){// Second-to-last column uses upwind to the east.
413                     a_E[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*(F_e_u[i-1][J-1])-D_e ;// Subtract
                            east diffusive term explicitly.
414                     a_P[i-1][J-1] += alpha_e_u[i-1][J-1]*(F_e_u[i-1][J-1])+D_e;
415                     S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho ;
416                 }else{// Interior east faces use TVD-MUSCL.
417                     a_EE[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J
                            -1]) ; // Upwind for east faces.
418                     a_E[i-1][J-1] += (alpha_e_u[i-1][J-1]*D(r_e_u[i-1][J-1])+(1-alpha_e_u[i-1][
                            J-1])*U(r_e_u[i-1][J-1]))*(F_e_u[i-1][J-1]) - D_e ;// Remove east
                            diffusive contribution handled separately.
419                     a_W[i-1][J-1] += (alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J
                            -1]) ;
420                     a_P[i-1][J-1] += ((1-alpha_e_u[i-1][J-1])*D(r_e_u[i-1][J-1])+alpha_e_u[i
                            -1][J-1]*U(r_e_u[i-1][J-1]))*(F_e_u[i-1][J-1])+D_e;
421                     S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho ;
```

```
422                 }
423                 // South boundary (three cases).
424                 if(J == 1){// First row adjacent to bottom wall.
425                     a_P[i-1][J-1] += 2*D_s ;
426                     S1[i-1][J-1] += -(u_bottom)*(-F_s_u[i-1][J-1]) +2*D_s*(u_bottom) ;// No-
                            slip; apply Neumann pressure extrapolation at the south boundary.
427                 }else if(J == 2){// Second row uses extrapolation instead of the SS node.
428
429                     a_S[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(alpha_s_u[i
                            -1][J-1])*U(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])-D_s;
430                     a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
431                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*U(r_s_u[i-1][J-1])+(alpha_s_u[i
                            -1][J-1])*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1]) + D_s ;
432                     S1[i-1][J-1] +=  -2*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][
                            J-1])*u_bottom
433                     -u_star_old[i-1][J-2]*(-(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]))
434                     ;
435                 }else if(J == Ny){
436                     // No neighboring cell; use extrapolation.
437
438                     a_SS[i-1][J-1]+= (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J
                            -1]) ;
439                     a_S[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(alpha_s_u[i
                            -1][J-1])*U(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])-D_s ;
440                     a_P[i-1][J-1]+= ((1-alpha_s_u[i-1][J-1])*U(r_s_u[i-1][J-1])+alpha_s_u[i-1][
                            J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])+D_s;
441                     S1[i-1][J-1] += -2*u_top*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-
                            F_s_u[i-1][J-1])
442                     -u_star_old[i-1][J-1]*(-(1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1])*(-F_s_u
                            [i-1][J-1]))
443                     ;
444                 }else{
445                     a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J
                            -1]) ;
446                     a_S[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(alpha_s_u[i
                            -1][J-1])*U(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])-D_s ;// Remove south
                            diffusive term already accounted for.
447                     a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
448                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*U(r_s_u[i-1][J-1])+alpha_s_u[i
                            -1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])+D_s ;
449                 }
450                 // North boundary special cases: J==1, J==Ny, J==Ny-1.
451                 if(J == Ny){
452                     a_P[i-1][J-1] += 2*D_n ;
453                     S1[i-1][J-1] += -(u_top)*F_n_u[i-1][Ny-1] + 2*D_n*(u_top);// No-slip; apply
                             Neumann pressure extrapolation at the north boundary.
454                 }else if(J == 1){
455
456                     a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
457                     a_N[i-1][J-1] += (alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i-1][
                            J-1])*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1]) - D_n ;// Remove north
                            diffusive term when handling this face.
458                     a_P[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+(alpha_n_u[i
                            -1][J-1])*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1])+ D_n ;
459                     S1[i-1][J-1] += -2*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1])*u_bottom
460                     -u_star_old[i-1][J-1]*(-(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]))
461                     ;
462                 }else if(J == Ny-1){
463                     // No north cell; apply extrapolation.
464                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
465
466                     a_N[i-1][J-1] += (alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i-1][
                            J-1])*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1]) - D_n  ;// Remove north
                            diffusive contribution explicitly.
467                     a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
468                     a_P[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u[i
                            -1][J-1]*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1]) +D_n ;
```

```
469                     S1[i-1][J-1] += -2*u_top*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u
                            [i-1][J-1])
470                     -u_star_old[i-1][J]*(-(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]))
471                     ;
472                 }else{
473                     a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
474                     a_N[i-1][J-1] += (alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i-1][
                            J-1])*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1]) - D_n ;// Remove north
                            diffusive term when handling this face.
475                     a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
476                     a_P[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u[i
                            -1][J-1]*U(r_n_u[i-1][J-1]))*(F_n_u[i-1][J-1])+D_n ;
477                 }
478                 double WW = 0.0 , W = 0.0 , E = 0.0 , EE = 0.0 ;
479                 double SS = 0.0 , S = 0.0 , N = 0.0 , NN = 0.0 ;
480                 E = u_star[i][J-1] ;
481                 W = u_star[i-2][J-1] ;
482                 // West-west neighbor.
483                 if(i == 2){
484                     WW = 0.0 ;
485                 }else{
486                     WW = u_star[i-3][J-1] ;
487                 }
488                 // East-east neighbor.
489                 if(i == Nx){
490                     EE = 0.0 ;
491                 }else{
492                     EE = u_star[i+1][J-1] ;
493                 }
494                 // South-south neighbor.
495                 if(J == 1 || J == 2){
496                     SS = 0.0 ;
497                 }else{
498                     SS = u_star[i-1][J-3] ;
499                 }
500                 // South neighbor.
501                 if(J == 1){
502                     S = 0.0 ;
503                 }else{
504                     S = u_star[i-1][J-2] ;
505                 }
506                 // North neighbor.
507                 if(J == Ny){
508                     N = 0.0 ;
509                 }else{
510                     N = u_star[i-1][J] ;
511                 }
512                 // North-north neighbor.
513                 if(J == Ny || J == Ny-1){
514                     NN = 0.0 ;
515                 }else {
516                     NN = u_star[i-1][J+1] ;
517                 }
518                 u_star[i-1][J-1] = (-a_WW[i-1][J-1]*WW - a_W[i-1][J-1]*W - a_E[i-1][J-1]*E -
                        a_EE[i-1][J-1]*EE
519                     - a_SS[i-1][J-1]*SS - a_S[i-1][J-1]*S - a_N[i-1][J-1]*N - a_NN[i-1][J-1]*NN
520                     + S1[i-1][J-1]) / a_P[i-1][J-1];
521                 // Relaxation relation for the predicted velocity.
522                 u_star[i-1][J-1] = (1-omega_u)*u_star_old[i-1][J-1] + omega_u*u_star[i-1][J-1]
                        ;
523             }
524         }
525     }
526
527     // Update the v-momentum predictor coefficients.
528     #pragma omp parallel for collapse(2)
529     for(int I = 1 ; I <= Nx ; I++){
530         for(int j = 1 ; j <= Ny+1 ; j++){
531             // Reset stencil coefficients before accumulating contributions.
532             b_P[I-1][j-1] = 0.0 ;
533             b_WW[I-1][j-1] = 0.0 ;
534             b_W[I-1][j-1] = 0.0 ;
535             b_E[I-1][j-1] = 0.0 ;
```

```
536             b_EE[I-1][j-1] = 0.0 ;
537             b_SS[I-1][j-1] = 0.0 ;
538             b_S[I-1][j-1] = 0.0 ;
539             b_N[I-1][j-1] = 0.0 ;
540             b_NN[I-1][j-1] = 0.0 ;
541             S2[I-1][j-1] = 0.0 ;
542             if(j == 1){
543                 b_P[I-1][j-1] += 1.0 ;
544                 S2[I-1][j-1] += v_bottom ;// Enforce no-penetration; no pressure gradient on
                        the bottom boundary
545                 v_star[I-1][j-1] = v_bottom ;
546             }else if(j == Ny+1){
547                 b_P[I-1][j-1] += 1.0 ;
548                 S2[I-1][j-1] += v_top ;// Enforce no-penetration; no pressure gradient on the
                        top boundary
549                 v_star[I-1][j-1] = v_top ;
550             }// Special treatment for near-boundary rows (left/right first and second rows,
                    second rows adjacent to top/bottom)
551             // Bottom boundary second row
552             else{ // Remaining cells are categorized by west, east, south, and north boundary
                    cases
553                 // South boundary
554                 if(j == 2){// Second row above the bottom uses upwind on the south face
555                     b_S[I-1][j-1] += alpha_s_v[I-1][j-1]*(-F_s_v[I-1][j-1])-D_s;// Remove south
                            -face diffusive term when handling the boundary
556                     b_P[I-1][j-1] += (1-alpha_s_v[I-1][j-1])*(-F_s_v[I-1][j-1])+D_s;
557                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho ;
558                 }else{// Other interior points use TVD-MUSCL on the south face
559                     b_SS[I-1][j-1] += (alpha_s_v[I-1][j-1]*UU(r_s_v[I-1][j-1]))*(-F_s_v[I-1][j
                            -1]) ;
560                     b_S[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*D(r_s_v[I-1][j-1])+alpha_s_v[I
                            -1][j-1]*U(r_s_v[I-1][j-1]))*(-F_s_v[I-1][j-1]) - D_s ; // Remove south
                            -face diffusive term when handling the boundary
561                     b_N[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1]) ;
562                     b_P[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*U(r_s_v[I-1][j-1])+alpha_s_v[I
                            -1][j-1]*D(r_s_v[I-1][j-1]))*(-F_s_v[I-1][j-1])+D_s ;
563                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho ;
564                 }
565                 // North boundary
566                 if(j == Ny){// Second row below the top uses upwind on the north face
567                     b_N[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*(F_n_v[I-1][j-1])-D_n ;// Remove
                            north-face diffusive term when handling the boundary
568                     b_P[I-1][j-1] += alpha_n_v[I-1][j-1]*(F_n_v[I-1][j-1])+D_n;
569                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho ;
570                 }else{// Other interior points use TVD-MUSCL on the north face
571                     b_NN[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I-1][j
                            -1]) ;
572                     b_N[I-1][j-1] += (alpha_n_v[I-1][j-1]*D(r_n_v[I-1][j-1])+(1-alpha_n_v[I-1][
                            j-1])*U(r_n_v[I-1][j-1]))*(F_n_v[I-1][j-1]) - D_n ;// Remove north-face
                             diffusive term when handling the boundary
573                     b_S[I-1][j-1] += (alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I-1][j
                            -1]) ;
574                     b_P[I-1][j-1] += ((1-alpha_n_v[I-1][j-1])*D(r_n_v[I-1][j-1])+alpha_n_v[I
                            -1][j-1]*U(r_n_v[I-1][j-1]))*(F_n_v[I-1][j-1]) + D_n ;
575                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho ;
576                 }
577                 // West boundary (consider I = 1, 2, Nx separately)
578                 if(I == 2){// Second column from the west uses extrapolation instead of the WW
                        node
579
580                     b_W[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(alpha_w_v[I
                            -1][j-1])*U(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])-D_w;
581                     b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) ;
582                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*U(r_w_v[I-1][j-1])+(alpha_w_v[I
                            -1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])+D_w;
583                     S2[I-1][j-1] +=  -2*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1])*v_left
584                     -v_star_old[I-2][j-1]*(-(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]))
585                     ;
586                 }else if(I == 1){// Westernmost column adopts upwind on the west face
587                     b_P[I-1][j-1] +=  2*D_w;// Remove west-face diffusive term when handling
                            the boundary
588                     S2[I-1][j-1] += -(-F_w_v[I-1][j-1])*v_left+2*D_w*v_left;
```

```
589                 }else if(I == Nx){// E -> P
590                     // When I = Nx the east control volume does not exist; extrapolate from
                            interior values
591
592                     b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j
                            -1]);
593                     b_W[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+alpha_w_v[I
                            -1][j-1]*U(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) -D_w ;
594                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*U(r_w_v[I-1][j-1])+alpha_w_v[I
                            -1][j-1]*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) +D_w;
595                     S2[I-1][j-1] += -2*v_right*(((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))
                            *(-F_w_v[I-1][j-1]))
596                     -v_star_old[I-1][j-1]*(-(1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1])*(-F_w_v
                            [I-1][j-1]))
597                     ;
598                 }else{// Other interior points use TVD-MUSCL on the west face
599                     b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j
                            -1]) ;
600                     b_W[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(alpha_w_v[I
                            -1][j-1])*U(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) - D_w ; // Remove west
                            -face diffusive term when handling the boundary
601                     b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]);
602                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*U(r_w_v[I-1][j-1])+(alpha_w_v[I
                            -1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])+D_w ;
603                 }
604                 // East boundary (consider I = 1, Nx-1, Nx separately)
605                 if(I == 1){// Second column from the east uses extrapolation instead of the EE
                        node
606
607                     b_EE[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*F_e_v[I-1][
                            j-1] ;
608                     b_E[I-1][j-1] += ((alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                            -1][j-1])*U(r_e_v[I-1][j-1]))*(F_e_v[I-1][j-1])-D_e ;// Remove east-
                            face diffusive term when handling the boundary
609                     b_P[I-1][j-1] += ((alpha_e_v[I-1][j-1])*U(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                            -1][j-1])*D(r_e_v[I-1][j-1]))*(F_e_v[I-1][j-1])  +D_e ;
610                     S2[I-1][j-1] +=  -2*v_left*(alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*(F_e_v
                            [I-1][j-1])
611                     -v_star_old[I-1][j-1]*(-(alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*(F_e_v[I
                            -1][j-1]))
612                     ;
613                 }else if(I == Nx){// Easternmost column adopts upwind on the east face
614                     b_P[I-1][j-1] += 2*D_e ;// Remove east-face diffusive term when handling
                            the boundary
615                     S2[I-1][j-1] += -(F_e_v[I-1][j-1])*v_right +2*D_e*v_right;
616                 }else if(I == Nx-1){// EE node does not exist; extrapolate to E
617
618                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*U(r_e_v[I-1][j-1])+alpha_e_v[I
                            -1][j-1]*D(r_e_v[I-1][j-1]))*F_e_v[I-1][j-1]
619                     - D_e ;
620                     b_W[I-1][j-1] += (alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*F_e_v[I-1][j-1]
                            ;
621                     b_P[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v[I
                            -1][j-1]*U(r_e_v[I-1][j-1]))*F_e_v[I-1][j-1] + D_e ;
622                     S2[I-1][j-1] += -2*v_right*((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*
                            F_e_v[I-1][j-1]
623                     -v_star_old[I][j-1]*(-(1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*F_e_v[I
                            -1][j-1])
624                     ;
625                 }else{// Other interior points use TVD-MUSCL on the east face
626                     b_EE[I-1][j-1] += (1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                            -1]) ;
627                     b_E[I-1][j-1] += (alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I-1][
                            j-1])*U(r_e_v[I-1][j-1]))*(F_e_v[I-1][j-1]) - D_e ;// Remove east-face
                            diffusive term when handling the boundary
628                     b_W[I-1][j-1] += (alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                            -1]) ;
629                     b_P[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v[I
                            -1][j-1]*U(r_e_v[I-1][j-1]))*(F_e_v[I-1][j-1])+D_e;
630                 }
631                 double WW=0.0 , W=0.0 , E=0.0 , EE=0.0 ;
632                 double SS=0.0 , S=0.0 , N=0.0 , NN=0.0 ;
633                 S = v_star[I-1][j-2] ;
634                 N = v_star[I-1][j] ;
635                 //SS:
```

```
636                 if(j == 2){
637                     SS = 0.0 ;
638                 }else{
639                     SS = v_star[I-1][j-3] ;
640                 }
641                 //NN:
642                 if(j == Ny){
643                     NN = 0.0 ;
644                 }else{
645                     NN = v_star[I-1][j+1] ;
646                 }
647                 //WW:
648                 if(I == 1 || I == 2){
649                     WW = 0.0 ;
650                 }else{
651                     WW = v_star[I-3][j-1] ;
652                 }
653                 //W:
654                 if(I == 1){
655                     W = 0.0 ;
656                 }else{
657                     W = v_star[I-2][j-1] ;
658                 }
659                 //E:
660                 if(I == Nx){
661                     E = 0.0 ;
662                 }else{
663                     E = v_star[I][j-1] ;
664                 }
665                 //EE:
666                 if(I == Nx || I == Nx-1){
667                     EE = 0.0 ;
668                 }else{
669                     EE = v_star[I+1][j-1] ;
670                 }
671                 v_star[I-1][j-1] = (-b_WW[I-1][j-1]*WW - b_W[I-1][j-1]*W - b_E[I-1][j-1]*E -
                        b_EE[I-1][j-1]*EE
672                                 - b_SS[I-1][j-1]*SS - b_S[I-1][j-1]*S - b_N[I-1][j-1]*N - b_NN[I
                                    -1][j-1]*NN
673                                 + S2[I-1][j-1]) / b_P[I-1][j-1] ;
674                 // Apply under-relaxation to the predicted velocity.
675                 v_star[I-1][j-1] = (1-omega_u)*v_star_old[I-1][j-1] + omega_u*v_star[I-1][j-1]
                        ;
676             }
677         }
678     }
679 }
680 // Obtained u_star and v_star at the N + 1/2 stage.
681 // Function: mass flux after the first update of the velocity predictor.
682 double d(int I, int J , int m){
683     // I,J correspond to cell centers; m is the SIMPLE iteration counter, distinct from the
            velocity time level.
684     double u_e = u_star[I+1-1][J-1] ;// Velocity predictor evaluated at N+1/2
685     double u_w = u_star[I-1][J-1] ;// Velocity predictor evaluated at N+1/2
686     double v_n = v_star[I-1][J+1-1] ;// Velocity predictor evaluated at N+1/2
687     double v_s = v_star[I-1][J-1] ;// Velocity predictor evaluated at N+1/2
688     return rho*(u_e - u_w)*dy + rho*(v_n - v_s)*dx ;
689 } // Used to judge imbalance after the first velocity predictor update
690
691 // Step 4: update the corrected pressure field.
692 void step4(int m){
693     #pragma omp parallel for collapse(2)
694     for(int I = 1 ; I <= Nx ; I++){ // Pressure field (true, predicted, corrected) is stored at
            cell centers
695         for(int J = 1 ; J <= Ny ; J++){
696            p[I-1][J-1] = p_star[I-1][J-1] + omega_P*p_prime[I-1][J-1] ;
697         }// p_star[I-1][J-1] evaluated at step N; p_prime[I-1][J-1] at N+1
698     }// p_prime[I-1][J-1] evaluated at step N+1
699 }
700
701 // Step 5: update the velocity corrections. Boundary velocities are fixed by conditions, so
        corrections vanish on physical boundaries.
702 void step5(int m){
703     #pragma omp parallel for collapse(2)
704     // Update x-component velocity correction
705     for(int i = 1 ; i <= Nx+1 ; i++){
```

```
706         for(int J = 1 ; J <= Ny ; J++){
707             if(i == 1 || i == Nx+1){
708                 u_prime[i-1][J-1] = 0.0 ;// Boundary velocities already satisfy conditions and
                        are not iterated
709             }else{// Pressure correction gradient: current minus western neighbor
710                 u_prime[i-1][J-1] = -(p_prime[i-1][J-1] - p_prime[i-2][J-1])*dy/(rho*a_P[i-1][J
                        -1]) ;
711             }
712 
713         }
714     }
715     // Update y-component velocity correction
716     #pragma omp parallel for collapse(2)
717     for(int I = 1 ; I <= Nx ; I++){
718         for(int j = 1 ; j <= Ny+1 ; j++){
719             if(j == 1 || j == Ny+1){
720                 v_prime[I-1][j-1] = 0.0 ;// Boundary velocities already satisfy conditions and
                        are not iterated
721             }else{// Pressure correction gradient: current minus southern neighbor
722                 v_prime[I-1][j-1] = -(p_prime[I-1][j-1] - p_prime[I-1][j-2])*dx/(rho*b_P[I-1][j
                        -1]) ;
723             }
724         }
725     }
726 }
727 
728 // Used to compute the non-equilibrium mass flux.
729 double step5star(int m){
730     double sum = 0.0 ;
731     #pragma omp parallel for collapse(2) reduction(+:sum)  // *** Added for parallel reduction
            ***
732     for(int I = 1 ; I <= Nx ; I++){
733         for(int J = 1 ; J <= Ny ; J++){
734             sum = sum + fabs(d(I,J,m)) ;
735         }
736     }
737     return sum ;
738 }
739 
740 
741 // Step 6: update the corrected velocity field and refresh the previous field.
742 void step6(int m){// As noted earlier, physical boundary velocities are prescribed and not
        iterated
743     // Update the corrected velocity for the x component
744     #pragma omp parallel for collapse(2)
745     for(int i = 1 ; i <= Nx+1 ; i++){
746         for(int J = 1 ; J <= Ny ; J++){
747             if(i == 1 ){
748                 u[i-1][J-1] = 0.0 ;// Boundary velocity satisfied by conditions; no iteration
749             }else if (i == Nx+1){
750                 u[i-1][J-1] = 0.0 ;// Boundary velocity satisfied by conditions; no iteration
751             }else{
752                 u[i-1][J-1] = u_prime[i-1][J-1]+u_star[i-1][J-1];// u_star at N+1/2; u_prime at
                        N+1; u_old at N; u at N+1
753             }
754         }
755     }
756     // Refresh previous x-component velocity
757     #pragma omp parallel for collapse(2)
758     for(int i = 1 ; i <= Nx+1 ; i++){
759         for(int J = 1 ; J <= Ny ; J++){
760             u_old[i-1][J-1] = u[i-1][J-1] ; // u at N+1; u_old now stores the current correct
                    velocity
761         }
762     }
763     // Update the corrected velocity for the y component
764     #pragma omp parallel for collapse(2)
765     for(int I = 1 ; I <= Nx ; I++){
766         for(int j = 1 ; j <= Ny+1 ; j++){
767             if(j == 1 ){
768                 v[I-1][j-1] = 0.0 ;// Boundary velocity satisfied by conditions; no iteration (
                        no-penetration)
769             }else if (j == Ny+1){
770                 v[I-1][j-1] = 0.0 ;// Boundary velocity satisfied by conditions; no iteration (
                        no-penetration)
771             }else{
```

```
772                 v[I-1][j-1] = v_prime[I-1][j-1]+v_star[I-1][j-1];// v_star at N+1/2; v_prime at
                        N+1; v_old at N; v at N+1
773             }
774         }
775     }
776     // Refresh previous y-component velocity
777     #pragma omp parallel for collapse(2)
778     for(int I = 1 ; I <= Nx ; I++){
779         for(int j = 1 ; j <= Ny+1 ; j++){
780             v_old[I-1][j-1] = v[I-1][j-1] ; // v at N+1; v_old now stores the current correct
                    velocity
781         }
782     }
783 }
784 double step7(int m){
785     //81
786 
787     double sum = 0.0 ;
788     sum += fabs(u[40][0]-0) ;
789     sum += fabs(u[40][3]-(-0.41165)) ;
790     sum += fabs(u[40][4]-(-0.42901)) ;
791     sum += fabs(u[40][5]-(-0.43643)) ;
792     sum += fabs(u[40][7]-(-0.40435)) ;
793     sum += fabs(u[40][13]-(-0.3305)) ;
794     sum += fabs(u[40][22]-(-0.22855)) ;
795     sum += fabs(u[40][35]-(-0.07404)) ;
796     sum += fabs(u[40][39]-(-0.03039)) ;
797     sum += fabs(u[40][48]-(0.08183)) ;
798     sum += fabs(u[40][58]-(0.20087)) ;
799     sum += fabs(u[40][67]-(0.33556)) ;
800     sum += fabs(u[40][75]-(0.46036)) ;
801     sum += fabs(u[40][76]-(0.45992)) ;
802     sum += fabs(u[40][77]-(0.4612)) ;
803     sum += fabs(u[40][77]-(0.48223)) ;
804     sum += fabs(u[40][79]-(1)) ;
805     sum += fabs(v[0][40]-0) ;
806     sum += fabs(v[4][40]-(0.42447)) ;
807     sum += fabs(v[5][40]-(0.43329)) ;
808     sum += fabs(v[5][40]-(0.43648)) ;
809     sum += fabs(v[7][40]-(0.42951)) ;
810     sum += fabs(v[12][40]-(0.35368)) ;
811     sum += fabs(v[17][40]-(0.28066)) ;
812     sum += fabs(v[18][40]-(0.2728)) ;
813     sum += fabs(v[39][40]-(0.00945)) ;
814     sum += fabs(v[64][40]-(-0.30018)) ;
815     sum += fabs(v[69][40]-(-0.36214)) ;
816     sum += fabs(v[72][40]-(-0.41442)) ;
817     sum += fabs(v[76][40]-(-0.52876)) ;
818     sum += fabs(v[76][40]-(-0.55408)) ;
819     sum += fabs(v[77][40]-(-0.55069)) ;
820     sum += fabs(v[77][40]-(-0.49774)) ;
821     sum += fabs(v[79][40]-(0.0)) ;
822     /*
823     //113
824     double sum = 0.0 ;
825     sum += fabs(u[56][0]-0) ;
826     sum += fabs(u[56][6]-(-0.41165)) ;
827     sum += fabs(u[56][6]-(-0.42901)) ;
828     sum += fabs(u[56][7]-(-0.43643)) ;
829     sum += fabs(u[56][11]-(-0.40435)) ;
830     sum += fabs(u[56][19]-(-0.3305)) ;
831     sum += fabs(u[56][31]-(-0.22855)) ;
832     sum += fabs(u[56][50]-(-0.07404)) ;
833     sum += fabs(u[56][55]-(-0.03039)) ;
834     sum += fabs(u[56][69]-(0.08183)) ;
835     sum += fabs(u[56][82]-(0.20087)) ;
836     sum += fabs(u[56][95]-(0.33556)) ;
837     sum += fabs(u[56][106]-(0.46036)) ;
838     sum += fabs(u[56][107]-(0.45992)) ;
839     sum += fabs(u[56][108]-(0.4612)) ;
840     sum += fabs(u[56][109]-(0.48223)) ;
841     sum += fabs(u[56][111]-(1)) ;
842 
843     sum += fabs(v[0][56]-0) ;
844     sum += fabs(v[6][56]-(0.42447)) ;
845     sum += fabs(v[7][56]-(0.43329)) ;
```

```
846       sum += fabs(v[8][56]-(0.43648)) ;
847       sum += fabs(v[10][56]-(0.42951)) ;
848       sum += fabs(v[17][56]-(0.35368)) ;
849       sum += fabs(v[25][56]-(0.28066)) ;
850       sum += fabs(v[26][56]-(0.2728)) ;
851       sum += fabs(v[55][56]-(0.00945)) ;
852       sum += fabs(v[90][56]-(-0.30018)) ;
853       sum += fabs(v[96][56]-(-0.36214)) ;
854       sum += fabs(v[101][56]-(-0.41442)) ;
855       sum += fabs(v[105][56]-(-0.52876)) ;
856       sum += fabs(v[106][56]-(-0.55408)) ;
857       sum += fabs(v[107][56]-(-0.55069)) ;
858       sum += fabs(v[108][56]-(-0.49774)) ;
859       sum += fabs(v[111][56]-(0.0)) ;*/
860       /*
861       //161
862       double sum = 0.0 ;
863           sum += fabs(u[80][0]-0) ;
864       sum += fabs(u[80][8]-(-0.18109)) ;
865       sum += fabs(u[80][9]-(-0.20196)) ;
866       sum += fabs(u[80][10]-(-0.22220)) ;
867       sum += fabs(u[80][15]-(-0.29730)) ;
868       sum += fabs(u[80][27]-(-0.38289)) ;
869       sum += fabs(u[80][44]-(-0.27805)) ;
870       sum += fabs(u[80][71]-(-0.10648)) ;
871       sum += fabs(u[80][79]-(-0.06080)) ;
872       sum += fabs(u[80][98]-(0.05702)) ;
873       sum += fabs(u[80][117]-(0.18719)) ;
874       sum += fabs(u[80][135]-(0.33304)) ;
875       sum += fabs(u[80][151]-(0.46604)) ;
876       sum += fabs(u[80][153]-(0.51117)) ;
877       sum += fabs(u[80][154]-(0.57492)) ;
878       sum += fabs(u[80][155]-(0.65928)) ;
879       sum += fabs(u[80][159]-(1)) ;
880
881       sum += fabs(v[0][80]-0) ;
882       sum += fabs(v[9][80]-(0.27485)) ;
883       sum += fabs(v[10][80]-(0.29012)) ;
884       sum += fabs(v[12][80]-(0.30353)) ;
885       sum += fabs(v[14][80]-(0.32627)) ;
886       sum += fabs(v[24][80]-(0.37095)) ;
887       sum += fabs(v[35][80]-(0.33075)) ;
888       sum += fabs(v[37][80]-(0.32235)) ;
889       sum += fabs(v[80][80]-(0.02526)) ;
890       sum += fabs(v[129][80]-(-0.31966)) ;
891       sum += fabs(v[137][80]-(-0.42655)) ;
892       sum += fabs(v[145][80]-(-0.51550)) ;
893       sum += fabs(v[151][80]-(-0.39188)) ;
894       sum += fabs(v[152][80]-(-0.33714)) ;
895       sum += fabs(v[154][80]-(-0.27669)) ;
896       sum += fabs(v[155][80]-(-0.21388)) ;
897       sum += fabs(v[159][80]-(0.0)) ;*/
898       /*//227
899       double sum = 0.0 ;
900       sum += fabs(u[113][0]-0) ;
901       sum += fabs(u[113][12]-(-0.41165)) ;
902       sum += fabs(u[113][14]-(-0.42901)) ;
903       sum += fabs(u[113][15]-(-0.43643)) ;
904       sum += fabs(u[113][22]-(-0.40435)) ;
905       sum += fabs(u[113][38]-(-0.3305)) ;
906       sum += fabs(u[113][63]-(-0.22855)) ;
907       sum += fabs(u[113][102]-(-0.07404)) ;
908       sum += fabs(u[113][113]-(-0.03039)) ;
909       sum += fabs(u[113][139]-(0.08183)) ;
910       sum += fabs(u[113][165]-(0.20087)) ;
911       sum += fabs(u[113][192]-(0.33556)) ;
912       sum += fabs(u[113][215]-(0.46036)) ;
913       sum += fabs(u[113][217]-(0.45992)) ;
914       sum += fabs(u[113][218]-(0.4612)) ;
915       sum += fabs(u[113][220]-(0.48223)) ;
916       sum += fabs(u[113][225]-(1)) ;
917
918       sum += fabs(v[0][113]-0) ;
919       sum += fabs(v[14][113]-(0.42447)) ;
920       sum += fabs(v[15][113]-(0.43329)) ;
921       sum += fabs(v[17][113]-(0.43648)) ;
```

```
922         sum += fabs(v[21][113]-(0.42951)) ;
923         sum += fabs(v[35][113]-(0.35368)) ;
924         sum += fabs(v[51][113]-(0.28066)) ;
925         sum += fabs(v[52][113]-(0.2728)) ;
926         sum += fabs(v[113][113]-(0.00945)) ;
927         sum += fabs(v[181][113]-(-0.30018)) ;
928         sum += fabs(v[194][113]-(-0.36214)) ;
929         sum += fabs(v[204][113]-(-0.41442)) ;
930         sum += fabs(v[214][113]-(-0.52876)) ;
931         sum += fabs(v[215][113]-(-0.55408)) ;
932         sum += fabs(v[217][113]-(-0.55069)) ;
933         sum += fabs(v[218][113]-(-0.49774)) ;
934         sum += fabs(v[225][113]-(0.0)) ;*/
935         return sum ;
936     }
937
938     // Step 8: update the pressure predictor at cell centers.
939     void step8(int m){
940         #pragma omp parallel for collapse(2)  // *** Added for parallel execution ***
941         for(int I = 1 ; I <= Nx ; I++){
942             for(int J = 1 ; J <= Ny ; J++){
943                 p_star[I-1][J-1] = p[I-1][J-1] ; // p at N+1; p_star at N+1
944             }
945         }
946     }
947
948     // Step 9: refresh velocity predictors, interface limiters, and convective coefficients.
949     void step9(int m){
950         // Update u-component predictor; boundary velocities are imposed directly.
951         #pragma omp parallel for collapse(2)
952         for(int i = 1 ; i <= Nx+1 ; i++){
953             for(int J = 1 ; J <= Ny ; J++){
954                 if(i == 1){
955                     u_star[i-1][J-1] = u_left;
956                 }else if (i == Nx+1){
957                     u_star[i-1][J-1] = u_right ;
958                 }else{
959                     // Interior points, including top/bottom faces, take the corrected value.
960                     u_star[i-1][J-1] = u[i-1][J-1]; // u at N+1; u_star at N+1
961                     u_star_old[i-1][J-1] = u_star[i-1][J-1]; // Cache the predictor immediately
962                 }
963             }
964         }
965         // Update v-component predictor; boundary velocities are imposed directly.
966         #pragma omp parallel for collapse(2)
967         for(int I = 1 ; I <= Nx ; I++){
968             for(int j = 1 ; j <= Ny+1 ; j++){
969                 if(j == 1){
970                     v_star[I-1][j-1] = v_bottom ;
971                 }else if (j == Ny+1){
972                     v_star[I-1][j-1] = v_top ;
973                 }else{
974                     // Interior points, including left/right faces, take the corrected value.
975                     v_star[I-1][j-1] = v[I-1][j-1]; // v at N+1; v_star at N+1
976                     v_star_old[I-1][j-1] = v_star[I-1][j-1]; // Cache the predictor immediately
977                 }
978             }
979         }
980         // Update convective coefficients
981         // Convective coefficients for the u-momentum discrete equation
982         #pragma omp parallel for collapse(2)
983         for(int i = 1 ; i <= Nx+1 ; i++){
984             for(int J = 1 ; J <= Ny ; J++){
985                 if(i == 1 || i == Nx+1) {
986                     F_w_u[i-1][J-1] = 0.0 ;
987                     F_e_u[i-1][J-1] = 0.0 ;
988                     F_n_u[i-1][J-1] = 0.0 ;
989                     F_s_u[i-1][J-1] = 0.0 ;
990                 }else{// Recompute convective flux coefficients
991                     F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); // Cell centers
                            coincide with storage points
992                     F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
993                     F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
994                     F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
995                 }
996             }
```

```
997       }// alpha_f_u flags use control-volume center indices
998       #pragma omp parallel for collapse(2)
999       for(int i = 1 ; i <= Nx+1 ; i++){
1000          for(int J = 1 ; J <= Ny ; J++){
1001              if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
1002              else alpha_w_u[i-1][J-1] = 0.0 ;
1003              if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
1004              else alpha_e_u[i-1][J-1] = 0.0 ;
1005              if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
1006              else alpha_n_u[i-1][J-1] = 0.0 ;
1007              if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
1008              else alpha_s_u[i-1][J-1] = 0.0 ;
1009          }
1010      }
1011      // Convective coefficients for the v-momentum discrete equation
1012      #pragma omp parallel for collapse(2)
1013      for(int I = 1 ; I <= Nx ; I++){
1014          for(int j = 1 ; j <= Ny+1 ; j++){
1015              if(j == 1 || j == Ny+1) {
1016                  F_w_v[I-1][j-1] = 0.0 ;
1017                  F_e_v[I-1][j-1] = 0.0 ;
1018                  F_n_v[I-1][j-1] = 0.0 ;
1019                  F_s_v[I-1][j-1] = 0.0 ;
1020              }else{
1021                  F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
1022                  F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
1023                  F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);// Cell centers
                          coincide with storage points
1024                  F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
1025              }
1026          }
1027      }// alpha_f_v flags use control-volume center indices
1028      #pragma omp parallel for collapse(2)
1029      for(int I = 1 ; I <= Nx ; I++){
1030          for(int j = 1 ; j <= Ny+1 ; j++){
1031              if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
1032              else alpha_w_v[I-1][j-1] = 0.0 ;
1033              if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
1034              else alpha_e_v[I-1][j-1] = 0.0 ;
1035              if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
1036              else alpha_n_v[I-1][j-1] = 0.0 ;
1037              if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
1038              else alpha_s_v[I-1][j-1] = 0.0 ;
1039          }
1040      }
1041      // Update interface limiters
1042      #pragma omp parallel for collapse(2)
1043      for(int i = 1 ; i <= Nx+1 ; i++){
1044          for(int J = 1 ; J <= Ny ; J++){
1045              // Cells that do not require limiter evaluation are set to zero
1046              if(i == 1 || i == Nx+1){
1047                  r_w_u[i-1][J-1] = 0.0 ;
1048                  r_e_u[i-1][J-1] = 0.0 ;
1049                  r_n_u[i-1][J-1] = 0.0 ;
1050                  r_s_u[i-1][J-1] = 0.0 ;
1051              }else {
1052                  // Handle each boundary case separately (west, east, south, north)
1053                  // West boundary
1054                  if(i == 2){
1055                      r_w_u[i-1][J-1] = -1.0 ; // West boundary uses upwind scheme
1056                  }else{
1057                      double u_W = u_star[i-2][J-1];
1058                      double u_P = u_star[i-1][J-1];
1059                      double u_WW = u_star[i-3][J-1];
1060                      double u_E = u_star[i][J-1] ;
1061                      double D = u_W + alpha_w_u[i-1][J-1]*(u_P - u_W) ;
1062                      double U = u_P + alpha_w_u[i-1][J-1]*(u_W - u_P) ;
1063                      double UU = u_E + alpha_w_u[i-1][J-1]*(u_WW - u_E) ;
1064                      r_w_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; // Initial formulation
1065                  }
1066                  // East boundary
1067                  if(i == Nx){
1068                      r_e_u[i-1][J-1] = -1.0 ; // East boundary uses upwind scheme
1069                  }else{
1070                      double u_P = u_star[i-1][J-1] ;
1071                      double u_E = u_star[i][J-1] ;
```

```
1072                    double u_W = u_star[i-2][J-1] ;
1073                    double u_EE = u_star[i+1][J-1] ;
1074                    double D = u_P + alpha_e_u[i-1][J-1]*(u_E - u_P) ;
1075                    double U = u_E + alpha_e_u[i-1][J-1]*(u_P - u_E) ;
1076                    double UU = u_EE + alpha_e_u[i-1][J-1]*(u_W - u_EE) ;
1077                    r_e_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; // Initial formulation
1078                }
1079                // South boundary
1080                if(J == 1){
1081                    r_s_u[i-1][J-1] = 0.0 ; // Bottom boundary cells do not need the south-face
                            limiter
1082                }else if(J == 2){
1083                    double u_S = u_star[i-1][J-2] ;
1084                    double u_P = u_star[i-1][J-1] ;
1085                    double u_SS = 2*u_bottom-u_S ; // 2*boundary value minus the southern node
1086                    double u_N = u_star[i-1][J] ;
1087                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1088                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1089                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1090                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1091                    r_s_u[i-1][J-1] =  -1.0  ; // Use upwind scheme
1092                }else if(J == Ny){
1093                    double u_S = u_star[i-1][J-2] ;
1094                    double u_P = u_star[i-1][J-1] ;
1095                    double u_SS = u_star[i-1][J-3] ;
1096                    double u_N = 2*u_top - u_P ; // 2*boundary value minus the current node
1097                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1098                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1099                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1100                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1101                    r_s_u[i-1][J-1] =  -1.0  ; // Use upwind scheme
1102                }else{
1103                    double u_S = u_star[i-1][J-2] ;
1104                    double u_P = u_star[i-1][J-1] ;
1105                    double u_SS = u_star[i-1][J-3] ;
1106                    double u_N = u_star[i-1][J] ;
1107                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1108                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1109                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1110                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1111                }
1112                // North boundary
1113                if(J == 1){
1114                    double u_P = u_star[i-1][J-1] ;
1115                    double u_N = u_star[i-1][J] ;
1116                    double u_S = 2*u_bottom-u_P ; // 2*boundary value minus the current node
1117                    double u_NN = u_star[i-1][J+1] ;
1118                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1119                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1120                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1121                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1122                    r_n_u[i-1][J-1] =  -1.0  ; // Use upwind scheme
1123                }else if (J == Ny-1){
1124                    double u_P = u_star[i-1][J-1] ;
1125                    double u_N = u_star[i-1][J] ;
1126                    double u_S = u_star[i-1][J-2] ;
1127                    double u_NN = 2*u_top - u_N ; // 2*boundary value minus the north node
1128                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1129                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1130                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1131                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1132                    r_n_u[i-1][J-1] =  -1.0  ; // Use upwind scheme
1133                }else if(J == Ny){
1134                    r_n_u[i-1][J-1] = 0.0 ; // Top boundary cells do not need the north-face
                            limiter
1135                }else{
1136                    double u_P = u_star[i-1][J-1] ;
1137                    double u_N = u_star[i-1][J] ;
1138                    double u_S = u_star[i-1][J-2] ;
1139                    double u_NN = u_star[i-1][J+1] ;
1140                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1141                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1142                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1143                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initial formulation
1144                }
1145            }
```

```
1146            }
1147        }// Remember to refresh limiters immediately after updating predictors
1148        // Interface limiters for the v-component velocity field (special handling on east/west
                boundaries)
1149        #pragma omp parallel for collapse(2)
1150        for(int I = 1 ; I <= Nx ; I++){
1151            for(int j = 1 ; j <= Ny+1 ; j++){
1152                // Cells that do not require limiter evaluation are set to zero
1153                if(j == 1 || j == Ny+1){
1154                    r_w_v[I-1][j-1] = 0.0 ;
1155                    r_e_v[I-1][j-1] = 0.0 ;
1156                    r_n_v[I-1][j-1] = 0.0 ;
1157                    r_s_v[I-1][j-1] = 0.0 ;
1158                }else {
1159                    // Evaluate boundary cases sequentially (south, north, east, west)
1160                    // South boundary
1161                    if(j == 2){
1162                        r_s_v[I-1][j-1] = -1.0 ; // Bottom boundary uses an upwind scheme on the
                                south face
1163                    }else{
1164                        double v_S = v_star[I-1][j-2] ;
1165                        double v_P = v_star[I-1][j-1] ;
1166                        double v_SS = v_star[I-1][j-3] ;
1167                        double v_N = v_star[I-1][j] ;
1168                        double D = v_S + alpha_s_v[I-1][j-1]*(v_P - v_S) ;
1169                        double U = v_P + alpha_s_v[I-1][j-1]*(v_S - v_P) ;
1170                        double UU = v_N + alpha_s_v[I-1][j-1]*(v_SS - v_N) ;
1171                        r_s_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ; // Avoid zero denominators
1172                    }
1173                    // North boundary
1174                    if(j == Ny){
1175                        r_n_v[I-1][j-1] = -1.0 ; // Top boundary uses an upwind scheme on the north
                                face
1176                    }else{
1177                        double v_P = v_star[I-1][j-1] ;
1178                        double v_N = v_star[I-1][j] ;
1179                        double v_S = v_star[I-1][j-2] ;
1180                        double v_NN = v_star[I-1][j+1] ;
1181                        double D = v_P + alpha_n_v[I-1][j-1]*(v_N - v_P) ;
1182                        double U = v_N + alpha_n_v[I-1][j-1]*(v_P - v_N) ;
1183                        double UU = v_NN + alpha_n_v[I-1][j-1]*(v_S - v_NN) ;
1184                        r_n_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1185                    }
1186                    // West boundary
1187                    if(I == 1){
1188                        r_w_v[I-1][j-1] = 0.0 ; // Westernmost cells do not need the west-face
                                limiter
1189                    }else if (I == 2){
1190                        double v_W = v_star[I-2][j-1] ;
1191                        double v_P = v_star[I-1][j-1] ;
1192                        double v_WW = 2*v_left-v_W ; // 2*boundary value minus the west node
1193                        double v_E = v_star[I][j-1] ;
1194                        double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1195                        double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1196                        double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1197                        r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1198                        r_w_v[I-1][j-1] = -1.0 ; // Use upwind scheme
1199                    }else if(I == Nx){
1200                        double v_W = v_star[I-2][j-1] ;
1201                        double v_P = v_star[I-1][j-1] ;
1202                        double v_WW = v_star[I-3][j-1] ;
1203                        double v_E = 2*v_right - v_P ; // 2*boundary value minus the current node
1204                        double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1205                        double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1206                        double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1207                        r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1208                        r_w_v[I-1][j-1] = -1.0 ; // Use upwind scheme
1209                    }else{
1210                        double v_W = v_star[I-2][j-1] ;
1211                        double v_P = v_star[I-1][j-1] ;
1212                        double v_WW = v_star[I-3][j-1] ;
1213                        double v_E = v_star[I][j-1] ;
1214                        double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1215                        double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1216                        double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1217                        r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
```

```
                }
                // East boundary
                if(I == 1){
                    double v_P = v_star[I-1][j-1] ;
                    double v_E = v_star[I][j-1] ;
                    double v_W = 2*v_left - v_P ; // 2*boundary value minus the current node
                    double v_EE = v_star[I+1][j-1] ;
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                    r_e_v[I-1][j-1] = -1.0 ; // Use upwind scheme
                }else if (I == Nx-1){
                    double v_P = v_star[I-1][j-1] ;
                    double v_E = v_star[I][j-1] ;
                    double v_W = v_star[I-2][j-1] ;
                    double v_EE = 2*v_right - v_E ; // 2*boundary value minus the east node
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                    r_e_v[I-1][j-1] = -1.0 ; // Use upwind scheme
                }else if(I == Nx){
                    r_e_v[I-1][j-1] = 0.0 ; // Easternmost cells do not need the east-face
                        limiter
                }else {
                    double v_P = v_star[I-1][j-1] ;
                    double v_E = v_star[I][j-1] ;
                    double v_W = v_star[I-2][j-1] ;
                    double v_EE = v_star[I+1][j-1] ;
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                }
            }
        }
    }
}

// Output cell data for the finite-volume method.
void output_combined_fields(int m) {
    int nodes_i = Nx + 1;  // Number of nodes along x
    int nodes_j = Ny + 1;  // Number of nodes along y
    int cells_i = Nx;      // Number of cells along x
    int cells_j = Ny;      // Number of cells along y
    std::ostringstream name;
    name << "3.CDs_Re" << Re << "_Grid" << Nx+1 << "x" << Ny+1
         << "_nonDC_step" << std::setfill('0') << std::setw(6) << m
         << "_error" << min_step7 <<
          ".vtk";

    std::ofstream out(name.str());

    out << "# vtk DataFile Version 3.0\n";
    out << "3.CDs_Re = 5000_Grid" << Nx+1 << "x" << Ny+1 << "\n";
    out << "ASCII\n";
    out << "DATASET STRUCTURED_GRID\n";
    out << "DIMENSIONS " << nodes_i << " " << nodes_j << " 1\n";

    // Output node coordinates
    out << "POINTS " << nodes_i * nodes_j << " double\n";
    for(int j = 0; j < nodes_j; j++) {
        for(int i = 0; i < nodes_i; i++) {
            double x_coord = i * dx;
            double y_coord = j * dy;
            out << scientific << setprecision(6)
                << x_coord << " " << y_coord << " 0.0\n";
        }
    }

    out << "CELL_DATA " << cells_i * cells_j << "\n";

    // 1. Pressure field
    out << "SCALARS Pressure double 1\n";
    out << "LOOKUP_TABLE default\n";
```

```
1293        for(int J = 1; J <= cells_j; J++) {
1294            for(int I = 1; I <= cells_i; I++) {
1295                out << scientific << setprecision(6)
1296                    << p_star[I-1][J-1] << "\n";
1297            }
1298        }
1299
1300        // 2. Velocity field (no additional CELL_DATA declaration required)
1301        out << "VECTORS Velocity double\n";
1302        for(int J = 1; J <= cells_j; J++) {
1303            for(int I = 1; I <= cells_i; I++) {
1304                double u_val = 0.5 * (u_star[I-1][J-1] + u_star[I][J-1]);
1305                double v_val = 0.5 * (v_star[I-1][J-1] + v_star[I-1][J]);
1306
1307                out << scientific << setprecision(6)
1308                    << u_val << " " << v_val << " 0.0\n";
1309            }
1310        }
1311
1312        out.close();
1313        cout << "VTK document has been output: " << name.str() << endl;
1314    }
1315    /* //THis is for Tecplot output
1316    void output_combined_fields(int m) {
1317        const int cells_i = Nx; // cells in x
1318        const int cells_j = Ny; // cells in y
1319
1320        std::ostringstream name;
1321        name << "3.QUICK_Re" << Re << "_Grid" << Nx << "x" << Ny
1322                    << "_nonDC_step" << std::setfill('0') << std::setw(6) << m
1323                    << "_error" << min_step7 <<
1324                        ".dat";
1325
1326        std::ofstream out(name.str());
1327
1328        if (!out) {
1329            std::cerr << "Error: Cannot open file: " << name.str() << std::endl;
1330            return;
1331        }
1332
1333        // Tecplot header
1334        out << "TITLE = \"SIMPLE-QUICK CFD Simulation\"\n";
1335        out << "VARIABLES = \"X\", \"Y\", \"P\", \"U\", \"V\", \"Vel_Mag\", \"Vorticity\"\n";
1336        out << "ZONE T=\"Time_Step_" << m << "\", "
1337            << "I=" << cells_i << ", J=" << cells_j << ", "
1338            << "DATAPACKING=POINT\n"; // Remove trailing comma
1339
1340        // AUXDATA (quoting strings is more robust)
1341        out << "AUXDATA Re=\""    << Re         << "\"\n";
1342        out << "AUXDATA Grid=\"" << Nx << "x" << Ny << "\"\n";
1343        out << "AUXDATA dx=\""    << dx         << "\"\n";
1344        out << "AUXDATA dy=\""    << dy         << "\"\n";
1345
1346        out << std::scientific << std::setprecision(8);
1347
1348        for (int j = 0; j < cells_j; ++j) {
1349            for (int i = 0; i < cells_i; ++i) {
1350                // Cell center
1351                const double xC = (i + 0.5) * dx;
1352                const double yC = (j + 0.5) * dy;
1353
1354                // p at center
1355                const double pC = p[i][j]; // Assume p has size [Nx][Ny]
1356
1357                // Interpolate u,v from faces to the cell center (MAC: u(i+0.5,j), v(i,j+0.5))
1358                // Requires u sized [Nx+1][Ny] and v sized [Nx][Ny+1]
1359                const double uC = 0.5 * (u[i][j] + u[i+1][j]);
1360                const double vC = 0.5 * (v[i][j] + v[i][j+1]);
1361
1362                const double vel_mag = std::sqrt(uC*uC + vC*vC);
1363
1364                // Vorticity at the center using first differences consistent with the MAC grid
1365                double vort = 0.0;
1366                // Need i+1 <= Nx-1 => i <= Nx-2, j-1 >= 0 => j >= 1
1367                if (i <= cells_i - 2 && j >= 1) {
```

```
1368                const double dvdx = (v[i+1][j] - v[i][j]) / dx; // v: Nx × (Ny+1), use the same
                        j index
1369                const double dudy = (u[i][j]   - u[i][j-1]) / dy; // u: (Nx+1) × Ny, use the
                        same i index
1370                vort = dvdx - dudy;
1371            }
1372            // One-sided differences could be applied near boundaries to avoid zeros
1373
1374            out << xC << " " << yC << " "
1375                << pC << " " << uC << " " << vC << " "
1376                << vel_mag << " " << vort << "\n";
1377        }
1378    }
1379
1380    out.close();
1381    std::cout << "Tecplot file successfully written: " << name.str() << std::endl;
1382 }
1383 */
1384
1385 int main(){
1386     // *** Added initialization for thread count ***
1387     int num_threads = omp_get_max_threads();
1388     omp_set_num_threads(num_threads);
1389     cout << "Using " << num_threads << " threads" << endl;
1390     cout << "Using FVD with SIMPLE algorithm to solve 2D incompressible Navier-Stokes equations
         " << endl;
1391     cout << "Grid Size:" << Nx+1 << "x" << Ny+1 << endl;
1392     cout << "Grid spacing : " << dx << "x " << dy << endl;
1393     cout << "Boundary Conditions: " ;
1394     cout << "u: left = " << u_left << ", right = " << u_right << ", bottom = " << u_bottom << "
         , top = " << u_top << endl;
1395     cout << "v: left = " << v_left << ", right = " << v_right << ", bottom = " << v_bottom << "
         , top = " << v_top << endl;
1396     cout << "Re = " << Re << ",rho = " << rho << endl ;
1397     CE_physical = false;
1398     G = 0 ;
1399     initialize() ;
1400     min_step7 = 50.0 ;
1401     double min_G = 0.0  ;
1402     for(G = 0 ; G <= max_G ; G++){
1403
1404         step2(G) ;// Update velocity predictors (first update)
1405         step3() ;// Update pressure correction
1406         step4(G) ;// Update corrected pressure
1407         step5(G) ;// Update velocity corrections
1408         step6(G) ;// Update corrected velocity field
1409         // Step 7: convergence assessment
1410         if(min_step7 > step7(G)){ // Evaluate every iteration
1411                 min_step7 = step7(G) ;
1412                 min_G = G ;
1413                 //if(G > 1000 && min_step7 < 2.55   ){output_combined_fields(min_G) ;}
1414
1415             }
1416         if(G%1000 == 0){
1417             cout << "Itteration step = " << G << endl ;
1418             cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                     << endl ;
1419             cout << "currnt minerror(G =" << min_G << ")= "<< min_step7 << endl ;
1420         }
1421         if(G>0 && step5star(G) < 1e-10){
1422             CE_physical = true ;
1423             cout << "Haved converged at step G = " << G << endl ;
1424             cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                     <<endl ;
1425             cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
1426             output_combined_fields(G) ;
1427             break ;
1428         }
1429         step8(G) ;// Update pressure predictor
1430         step9(G) ;// Update velocity predictor, convective coefficients, and limiters
1431     }
1432     if(!CE_physical){
1433         cout << "Haved converged at step G = " << G << endl ;
1434         cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)  <<
             endl ;
1435         cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
```

```
1436            output_combined_fields(min_G) ;
1437        }
1438        cout << "Grid Size : " << Nx << "x" << Ny << "Completed." << endl ;
1439        return 0 ;
1440    }
```

Listing 9: 2.FVM_non-deferred correction.cpp

A.2.3 FVM Deferred Correction

```
1  // Solve the -NavierStokes equations with the SIMPLE algorithm using the finite-volume method.
2  // Deferred corrections are included with alternating iterations.
3  #include <iostream>
4  #include <fstream>
5  #include <sstream>
6  #include <cmath>
7  #include <cstdlib>
8  #include <iomanip>
9  #include <string>
10 #include <vector>
11 #include <omp.h>  // Keep OpenMP include after <vector>
12 #include "0.P_correction_eq_TDMA.h"
13 using namespace std ;
14 // Parameter definitions
15 // Pressure field has Nx cells in the x direction
16 // u-velocity has Nx+1 faces in the x direction
17 // v-velocity has Ny+1 faces in the y direction
18 int G ;  // Current iteration counter starting from 0 (distinct from linear solver iterations)
19 const int  max_G = 700000;// Maximum SIMPLE iterations
20 double min_step7 ;
21 // u-velocity domain: i   [0, Nx], j   [0, Ny-1]
22 // v-velocity domain: i   [0, Nx-1], j   [0, Ny]
23 // Pressure domain: i   [0, Nx-1], j   [0, Ny-1]
24 double u_star[Nx+1][Ny] ; // Predicted u-velocity
25 double u_star_old[Nx+1][Ny] ;
26 double v_star[Nx][Ny+1] ; // Predicted v-velocity
27 double v_star_old[Nx][Ny+1] ;
28 double p_star[Nx][Ny] ; // Predicted pressure
29 double u_prime[Nx+1][Ny] ; // u-velocity correction
30 double v_prime[Nx][Ny+1] ; // v-velocity correction
31 double p_prime[Nx][Ny] ; // Pressure correction
32 double u[Nx+1][Ny] ; // Corrected u-velocity
33 double u_old[Nx+1][Ny] ;// Previous corrected u-velocity (needs initialization)
34 double v[Nx][Ny+1] ; // Corrected v-velocity
35 double v_old[Nx][Ny+1] ;// Previous corrected v-velocity (needs initialization)
36 double p[Nx][Ny] ; // Corrected pressure
37
38 const double  Re = 5000.0 ;
39 // u-velocity boundary conditions
40 const double u_left  = 0.0 ;// Impermeable wall
41 const double u_right = 0.0 ;// Impermeable wall
42 const double u_bottom = 0.0 ;// No-slip
43 const double u_top = 1.0 ;// No-slip (lid velocity)
44 // v-velocity boundary conditions
45 const double v_left  = 0.0 ;// No-slip
46 const double v_right = 0.0 ;// No-slip
47 const double v_bottom = 0.0 ;// Impermeable wall
48 const double v_top = 0.0 ;// Impermeable wall
49 const double omega_P = 0.6 ; // Relaxation for pressure correction (per Pateanca: alpha_P  0.8)
50 double omega_u  = 0.2 ;
51 double omega_v  = 0.2 ;
52 double max_error ;
53 const float CE_tolerance = 1e-10; // Convergence tolerance for predicted continuity residual
54 bool CE_physical ;
55
56 // Diffusive coefficients for the discretized equations (shared by both velocity components)
57 const double D_e = (1.0/Re);
58 const double D_w = (1.0/Re);
59 const double D_n = (1.0/Re);
60 const double D_s = (1.0/Re);
61 double r_w_u[Nx+1][Ny] ; // Interface limiter factors for u
62 double r_e_u[Nx+1][Ny] ;
63 double r_s_u[Nx+1][Ny] ;
64 double r_n_u[Nx+1][Ny] ;
65 double r_w_v[Nx][Ny+1] ; // Interface limiter factors for v
```

```
66  double r_e_v[Nx][Ny+1] ;
67  double r_s_v[Nx][Ny+1] ;
68  double r_n_v[Nx][Ny+1] ;
69  // Convective coefficients for the u-momentum equation (alpha flags decided by F)
70  double F_w_u[Nx+1][Ny] ;
71  double alpha_w_u[Nx+1][Ny] ;
72  double F_e_u[Nx+1][Ny] ;
73  double alpha_e_u[Nx+1][Ny] ;
74  double F_s_u[Nx+1][Ny] ;
75  double alpha_s_u[Nx+1][Ny] ;
76  double F_n_u[Nx+1][Ny] ;
77  double alpha_n_u[Nx+1][Ny] ;
78  // Linear combination coefficients for the u-momentum equation
79  double a_P[Nx+1][Ny] ;
80  double a_WW[Nx+1][Ny] ;
81  double a_W[Nx+1][Ny] ;
82  double a_E[Nx+1][Ny] ;
83  double a_EE[Nx+1][Ny] ;
84  double a_SS[Nx+1][Ny] ;
85  double a_S[Nx+1][Ny] ;
86  double a_N[Nx+1][Ny] ;
87  double a_NN[Nx+1][Ny] ;
88  double S1[Nx+1][Ny] ; // Source term for the u-momentum equation
89  // Convective coefficients for the v-momentum equation (alpha flags decided by F)
90  double F_w_v[Nx][Ny+1] ;
91  double alpha_w_v[Nx][Ny+1] ;
92  double F_e_v[Nx][Ny+1] ;
93  double alpha_e_v[Nx][Ny+1] ;
94  double F_s_v[Nx][Ny+1] ;
95  double alpha_s_v[Nx][Ny+1] ;
96  double F_n_v[Nx][Ny+1] ;
97  double alpha_n_v[Nx][Ny+1] ;
98  // Linear combination coefficients for the v-momentum equation
99  double b_P[Nx][Ny+1] ;
100 double b_WW[Nx][Ny+1] ;
101 double b_W[Nx][Ny+1] ;
102 double b_E[Nx][Ny+1] ;
103 double b_EE[Nx][Ny+1] ;
104 double b_SS[Nx][Ny+1] ;
105 double b_S[Nx][Ny+1] ;
106 double b_N[Nx][Ny+1] ;
107 double b_NN[Nx][Ny+1] ;
108 double S2[Nx][Ny+1] ;// Source term for the v-momentum equation
109 // Remember to refresh these matrices immediately after updating the velocity predictor
110
111 // (1)
112 // CDs upstream and downstream iteration coefficients
113 double D(double x){
114   return 0.5  ;
115 }
116 double U(double x){
117     return 0.5 ;
118 }
119 double UU(double x){
120   return 0.0 ;
121
122 }
123
124 /*//(2)
125 // QUICK upstream and downstream iteration coefficients
126 double D(double x){
127   return 3.0/8.0     ;
128 }
129 double U(double x){
130     return 6.0/8.0 ;
131 }
132 double UU(double x){
133   return -1.0/8.0 ;
134
135 }*/
136
137
138 /*//(3) TVD-MUSCL upstream and downstream iteration coefficients
139 double D(double x) {
140     if (x <= 0) return 0.0;
141     else if (x > 0 && x < (1.0/3.0)) return 0.0;
```

```
142     else if (x >= (1.0/3.0) && x < 3.0) return 0.25;
143     else return 1.0;
144 }
145
146 double U(double x) {
147     if (x <= 0) return 1.0;
148     else if (x > 0 && x <= (1.0/3.0)) return 2.0;
149     else if (x > (1.0/3.0) && x <= 3.0) return 1.0;
150     else return 0.0;
151 }
152
153 double UU(double x) {
154     if (x <= 0) return 0.0;
155     else if (x > 0 && x < (1.0/3.0)) return -1.0;
156     else if (x >= (1.0/3.0) && x < 3.0) return -0.25;
157     else return 0.0;
158 }*/
159
160 // SIMPLE algorithm:
161 // Step 1: initialize velocity predictors and pressure predictor
162 void initialize(){
163     #pragma omp parallel  // *** Ensure OpenMP parallel region ***
164     {
165     // Initialize u-velocity predictor
166      #pragma omp parallel for collapse(2)
167     for(int i = 1 ; i <= Nx+1 ; i++){
168         // Domain of u_star: i   [0, Nx], j   [0, Ny-1]
169         for(int J= 1 ; J <= Ny ; J++){
170             u_star[i-1][J-1] = 0.0 ;
171             if(i == 1) u_star[i-1][J-1] = u_left ; // Impermeable wall
172             if(i == Nx + 1) u_star[i-1][J-1] = u_right ; // Impermeable wall
173         }
174     }
175     // Initialize previous u-velocity using the initial predictor; later iterations use
          corrected values
176      #pragma omp parallel for collapse(2)
177     for(int i = 1 ; i <= Nx+1 ; i++){
178         for(int J = 1 ; J <= Ny ; J++){
179             u_old[i-1][J-1] = u_star[i-1][J-1] ;
180         }
181     }
182     // Initialize v-velocity predictor
183      #pragma omp parallel for collapse(2)
184     for(int I = 1 ; I <= Nx ; I++){
185         // Domain of v_star: i   [0, Nx-1], j   [0, Ny]
186         for(int j = 1 ; j <= Ny+1 ; j++){
187             v_star[I-1][j-1] = 0.0 ;
188             if(j == 1)v_star[I-1][j-1] = v_bottom ;// Impermeable wall
189             if(j == Ny + 1)v_star[I-1][j-1] = v_top ;// Impermeable wall
190         }
191     }
192     // Initialize previous v-velocity similarly
193      #pragma omp parallel for collapse(2)
194     for(int I = 1 ; I <= Nx ; I++){
195         for(int j = 1 ; j <= Ny+1 ; j++){
196             v_old[I-1][j-1] = v_star[I-1][j-1] ;
197         }
198     }
199     // Initialize pressure predictor
200      #pragma omp parallel for collapse(2)
201     for(int I = 1 ; I <= Nx ; I++){
202         for(int J = 1 ; J<= Ny ; J++){
203             p_star[I-1][J-1] =  0.0  ; // Neumann boundary enforced through iterations
204             p_prime[I-1][J-1] = 0.0 ;
205         }
206     }
207     // Predictor uses limiter matrices → convective-coefficient matrices → linear-coefficient
          matrices.
208     // Update (or initialize) in that order.
209     // Initialize limiter matrices.
210     // u-component limiter factors (special handling for north/south boundaries)
211      #pragma omp parallel for collapse(2)
212     for(int i = 1 ; i <= Nx+1 ; i++){
213         for(int J = 1 ; J <= Ny ; J++){
214             // Cells that do not require evaluation get zero
215             if(i == 1 || i == Nx+1){
```

```
216                 r_w_u[i-1][J-1] = 0.0 ;
217                 r_e_u[i-1][J-1] = 0.0 ;
218                 r_n_u[i-1][J-1] = 0.0 ;
219                 r_s_u[i-1][J-1] = 0.0 ;
220             }else {
221                 // Handle each boundary case explicitly
222                 // West boundary
223                 if(i == 2){
224                     r_w_u[i-1][J-1] = -1.0 ; // West boundary uses upwind limiter
225                 }else{
226                     r_w_u[i-1][J-1] = 1.0 ; // Interior initialization
227                 }
228                 // East boundary
229                 if(i == Nx){
230                     r_e_u[i-1][J-1] = -1.0 ; // East boundary uses upwind limiter
231                 }else{
232                     r_e_u[i-1][J-1] = 1.0 ;// Interior initialization
233                 }
234                 // South boundary
235                 if(J == 1){
236                     r_s_u[i-1][J-1] = 0.0 ; // Bottom boundary does not need the south limiter
237                 }else if(J == 2){
238                     r_s_u[i-1][J-1] = -1.0 ;// Upwind limiter to avoid division by zero
239                 }else if(J == Ny){
240                     r_s_u[i-1][J-1] = -1.0 ;// Upwind limiter to avoid division by zero
241                 }else{
242                     r_s_u[i-1][J-1] = 1.0 ;// Interior initialization
243                 }
244                 // North boundary
245                 if(J == 1){
246                     r_n_u[i-1][J-1] = -1.0 ; // Upwind limiter to avoid division by zero
247                 }else if (J == Ny-1){
248                     r_n_u[i-1][J-1] = -1.0 ;// Upwind limiter to avoid division by zero
249                 }else if(J == Ny){
250                     r_n_u[i-1][J-1] = 0.0 ; // Top boundary does not need the north limiter
251                 }else{
252                     r_n_u[i-1][J-1] = 1.0 ;// Interior initialization
253                 }
254             }
255         }
256     }// Remember to refresh limiters whenever predictors change
257     // v-component limiter factors (special handling for east/west boundaries)
258      #pragma omp parallel for collapse(2)
259     for(int I = 1 ; I <= Nx ; I++){
260         for(int j = 1 ; j <= Ny+1 ; j++){
261             // Cells that do not require evaluation get zero
262             if(j == 1 || j == Ny+1){
263                 r_w_v[I-1][j-1] = 0.0 ;
264                 r_e_v[I-1][j-1] = 0.0 ;
265                 r_n_v[I-1][j-1] = 0.0 ;
266                 r_s_v[I-1][j-1] = 0.0 ;
267             }else {
268                 // Handle each boundary sequentially
269                 // South boundary
270                 if(j == 2){
271                     r_s_v[I-1][j-1] = -1.0 ; // Bottom boundary uses upwind limiter
272                 }else{
273                     r_s_v[I-1][j-1] = 1.0 ; // Interior initialization
274                 }
275                 // North boundary
276                 if(j == Ny){
277                     r_n_v[I-1][j-1] = -1.0 ; // Top boundary uses upwind limiter
278                 }else{
279                     r_n_v[I-1][j-1] = 1.0 ; // Interior initialization
280                 }
281                 // West boundary
282                 if(I == 1){
283                     r_w_v[I-1][j-1] = 0.0 ; // West boundary does not need the west limiter
284                 }else if (I == 2){
285                     r_w_v[I-1][j-1] = -1.0 ;// Upwind limiter to avoid division by zero
286                 }else if(I == Nx){
287                     r_w_v[I-1][j-1] = -1.0 ;// Upwind limiter to avoid division by zero
288                 }else{
289                     r_w_v[I-1][j-1] = 1.0 ;// Interior initialization
290                 }
291                 // East boundary
```

```
292                 if(I == 1){
293                     r_e_v[I-1][j-1] = -1.0 ; // Upwind limiter to avoid division by zero
294                 }else if (I == Nx-1){
295                     r_e_v[I-1][j-1] = -1.0 ; // Upwind limiter to avoid division by zero
296                 }else if(I == Nx){
297                     r_e_v[I-1][j-1] = 0.0 ; // East boundary does not need the east limiter
298                 }else {
299                     r_e_v[I-1][j-1] = 1.0 ; // Interior initialization
300                 }
301             }
302         }
303     }
304     // Initialize convective coefficient matrices
305     // u-momentum equation
306      #pragma omp parallel for collapse(2)
307     for(int i = 1 ; i <= Nx+1 ; i++){
308         for(int J = 1 ; J <= Ny ; J++){
309             if(i == 1 || i == Nx+1) {
310                 F_w_u[i-1][J-1] = 0.0 ;
311                 F_e_u[i-1][J-1] = 0.0 ;
312                 F_n_u[i-1][J-1] = 0.0 ;
313                 F_s_u[i-1][J-1] = 0.0 ;
314             }else{
315                 F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); // Control volume
                        centers align with storage points
316                 F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
317                 F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
318                 F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
319             }
320         }
321     }// alpha_f_u flags correspond to control-volume centers
322      #pragma omp parallel for collapse(2)
323     for(int i = 1 ; i <= Nx+1 ; i++){
324         for(int J = 1 ; J <= Ny ; J++){
325             if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
326             else alpha_w_u[i-1][J-1] = 0.0 ;
327             if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
328             else alpha_e_u[i-1][J-1] = 0.0 ;
329             if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
330             else alpha_n_u[i-1][J-1] = 0.0 ;
331             if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
332             else alpha_s_u[i-1][J-1] = 0.0 ;
333         }
334     }
335     // v-momentum equation
336      #pragma omp parallel for collapse(2)
337     for(int I = 1 ; I <= Nx ; I++){
338         for(int j = 1 ; j <= Ny+1 ; j++){
339             if(j == 1 || j == Ny+1) {
340                 F_w_v[I-1][j-1] = 0.0 ;
341                 F_e_v[I-1][j-1] = 0.0 ;
342                 F_n_v[I-1][j-1] = 0.0 ;
343                 F_s_v[I-1][j-1] = 0.0 ;
344             }else{
345                 F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
346                 F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
347                 F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);// Control volume
                        centers align with storage points
348                 F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
349             }
350         }
351     }// alpha_f_v flags correspond to control-volume centers
352      #pragma omp parallel for collapse(2)
353     for(int I = 1 ; I <= Nx ; I++){
354         for(int j = 1 ; j <= Ny+1 ; j++){
355             if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
356             else alpha_w_v[I-1][j-1] = 0.0 ;
357             if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
358             else alpha_e_v[I-1][j-1] = 0.0 ;
359             if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
360             else alpha_n_v[I-1][j-1] = 0.0 ;
361             if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
362             else alpha_s_v[I-1][j-1] = 0.0 ;
363         }
364     }
365 }
```

```
}

// Step 2: update u- and v-velocity predictors
void step2(int m){
    // Update the linear-combination coefficients of the x-momentum predictor
    #pragma omp parallel for collapse(2)
    for(int i = 1 ; i <= Nx+1 ; i++){
        for(int J = 1 ; J <= Ny ; J++){
            // Reset stencil coefficients before accumulation
            a_P[i-1][J-1] = 0.0 ;
            a_WW[i-1][J-1] = 0.0 ;
            a_W[i-1][J-1] = 0.0 ;
            a_E[i-1][J-1] = 0.0 ;
            a_EE[i-1][J-1] = 0.0 ;
            a_SS[i-1][J-1] = 0.0 ;
            a_S[i-1][J-1] = 0.0 ;
            a_N[i-1][J-1] = 0.0 ;
            a_NN[i-1][J-1] = 0.0 ;
            S1[i-1][J-1] = 0.0 ;
            if(i == 1){
                a_P[i-1][J-1] += 1.0 ;
                S1[i-1][J-1] += u_left ;// Impermeable wall; no pressure gradient on the west boundary
                u_star[i-1][J-1] = u_left ;
            }else if(i == Nx+1){
                a_P[i-1][J-1] += 1.0 ;
                S1[i-1][J-1] += u_right ;// Impermeable wall; no pressure gradient on the east boundary
                u_star[i-1][J-1] = u_right ;
            }// Special handling for near-boundary rows/columns (left/right second rows, top/bottom first and second rows)
            else{ // For remaining cells, address west/east/south/north boundaries case by case
                // West boundary
                if(i == 2){// Second column next to the west boundary uses upwind on the west face
                    a_W[i-1][J-1] += alpha_w_u[i-1][J-1]*(-F_w_u[i-1][J-1])-D_w;// Subtract west-face diffusion term when handling the boundary
                    a_P[i-1][J-1] += (1-alpha_w_u[i-1][J-1])*(-F_w_u[i-1][J-1])+D_w;
                    S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho ;
                }else{// West boundary upwind term uses W and P nodes
                    //a_WW[i-1][J-1] += (alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1]) ;
                    a_W[i-1][J-1] += (alpha_w_u[i-1][J-1]*1)*(-F_w_u[i-1][J-1]) - D_w ; // Subtract west-face diffusion term when handling the boundary
                    //a_E[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1]);
                    a_P[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*1)*(-F_w_u[i-1][J-1])+D_w ;
                    S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho
                    -u_star_old[i-3][J-1]*(alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1])
                    -u_star_old[i-2][J-1]*((1-alpha_w_u[i-1][J-1])*D(r_w_u[i-1][J-1])+alpha_w_u[i-1][J-1]*(U(r_w_u[i-1][J-1])-1))*(-F_w_u[i-1][J-1])
                    -u_star_old[i][J-1]*((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1])
                    -u_star_old[i-1][J-1]*((1-alpha_w_u[i-1][J-1])*(U(r_w_u[i-1][J-1])-1)+alpha_w_u[i-1][J-1]*D(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1])
                    ;
                }// Deferred correction complete.
                // East boundary
                if(i == Nx){// Second column next to the east boundary uses upwind on the east face
                    a_E[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*(F_e_u[i-1][J-1])-D_e ;// Subtract east-face diffusion term when handling the boundary
                    a_P[i-1][J-1] += alpha_e_u[i-1][J-1]*(F_e_u[i-1][J-1])+D_e;
                    S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho ;
                }else{// East boundary upwind term couples east node and current cell
                    //a_EE[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J-1]) ;
                    a_E[i-1][J-1] += ((1-alpha_e_u[i-1][J-1])*1)*(F_e_u[i-1][J-1]) - D_e ;// Subtract east-face diffusion term when handling the boundary
                    //a_W[i-1][J-1] += (alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J-1]) ;
                    a_P[i-1][J-1] += (alpha_e_u[i-1][J-1]*1)*(F_e_u[i-1][J-1])+D_e;
                    S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho
                    -u_star_old[i+1][J-1]*(1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J-1])
```

```
424                     -u_star_old[i][J-1]*(alpha_e_u[i-1][J-1]*D(r_e_u[i-1][J-1])+(1-alpha_e_u[i
                            -1][J-1])*(U(r_e_u[i-1][J-1])-1))*(F_e_u[i-1][J-1])
425                     -u_star_old[i-2][J-1]*(alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i
                            -1][J-1])
426                     -u_star_old[i-1][J-1]*((1-alpha_e_u[i-1][J-1])*D(r_e_u[i-1][J-1])+alpha_e_u
                            [i-1][J-1]*(U(r_e_u[i-1][J-1])-1))*(F_e_u[i-1][J-1])
427                     ;
428                 }// Deferred correction complete.
429                 // South boundary (three special cases)
430                 if(J == 1){// First row adjacent to the bottom boundary
431                     a_P[i-1][J-1] += 2*D_s ;
432                     S1[i-1][J-1] += -(u_bottom)*(-F_s_u[i-1][J-1]) +2*D_s*(u_bottom) ;// No-
                            slip wall; pressure on the south face uses current-cell value per
                            Neumann condition
433                 }else if(J == 2){// Second row above the bottom uses extrapolation instead of
                        an SS node; upwind uses south node and current cell
434                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s;
435                    //a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
436                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1]) + D_s ;
437                     S1[i-1][J-1] +=  -2*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][
                            J-1])*u_bottom
438                     +(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])*u_star_old[i
                            -1][J-2]
439                     -u_star_old[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(
                            alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])
440                     -u_star_old[i-1][J]*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1])
441                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+(
                            alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])
442                     ;// Deferred correction complete.
443                 }else if(J == Ny){// South boundary upwind term involves south node and current
                         cell
444                     // North neighbor does not exist; use extrapolation instead
445                     //a_SS[i-1][J-1]+= (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J
                            -1]) ;
446                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s ;
447                     a_P[i-1][J-1]+= ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])+D_s;
448                     S1[i-1][J-1] += -2*u_top*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-
                            F_s_u[i-1][J-1])
449                     +u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u
                            [i-1][J-1])
450                     -u_star_old[i-1][J-3]*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1])
451                     -u_star_old[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(
                            alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])
452                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+
                            alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1]) ; //
                            Deferred correction complete.
453                 }else{// South boundary upwind term involves south node and current cell
454                     //a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][
                            J-1]) ;
455                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s ;//
                            Subtract south-face diffusion term when handling the boundary
456                     //a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
457                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])+D_s ;
458                     S1[i-1][J-1] +=
459                     -u_star[i-1][J-3]*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J
                            -1])
460                     -u_star[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(alpha_s_u[i
                            -1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])
461                     -u_star_old[i-1][J]*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1])
462                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+
                            alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])
463                     ;// Deferred correction complete.
464                 }
465                 // North boundary splits into three special cases: J == 1, J == Ny, and J == Ny
                        -1
466                 if(J == Ny){
467                     a_P[i-1][J-1] += 2*D_n ;
468                     S1[i-1][J-1] += -(u_top)*F_n_u[i-1][Ny-1] + 2*D_n*(u_top);// No-slip wall;
                            pressure on the north face uses current-cell value per Neumann
                            condition
469                 }else if(J == 1){
```

```
470                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
471                     a_N[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) - D_n ;//
                            Subtract north-face diffusion term when handling the boundary
472                     a_P[i-1][J-1] += ((alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) + D_n ;
473                     S1[i-1][J-1] += -2*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1])*u_bottom
474                     +u_star_old[i-1][J-1]*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
475                     -u_star_old[i-1][J+1]*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
476                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
477                     -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+(
                            alpha_n_u[i-1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
478                     ;// Deferred correction complete.
479                 }else if(J == Ny-1){
480                     // No north neighbor; use extrapolation instead
481                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
482                     a_N[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*1*(F_n_u[i-1][J-1]) - D_n  ;//
                            Subtract north-face diffusion term when handling the boundary
483                     //a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
484                     a_P[i-1][J-1] += (alpha_n_u[i-1][J-1]*1)*(F_n_u[i-1][J-1]) +D_n ;
485                     S1[i-1][J-1] += -2*u_top*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u
                            [i-1][J-1])
486                     +u_star_old[i-1][J]*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
487                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
488                     -u_star_old[i-1][J-2]*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
489                     -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u
                            [i-1][J-1]*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
490                     ;// Deferred correction complete.
491                     }else{
492                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
493                     a_N[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) - D_n ;//
                            Subtract north-face diffusion term when handling the boundary
494                     //a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
495                     a_P[i-1][J-1] += (alpha_n_u[i-1][J-1]*1)*(F_n_u[i-1][J-1])+D_n ;
496                     S1[i-1][J-1] +=
497                     -u_star_old[i-1][J+1]*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
498                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
499                     -u_star_old[i-1][J-2]*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
500                     -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u
                            [i-1][J-1]*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
501                     ;
502                  }
503                 double WW = 0.0 , W = 0.0 , E = 0.0 , EE = 0.0 ;
504                 double SS = 0.0 , S = 0.0 , N = 0.0 , NN = 0.0 ;
505                 E = u_star[i][J-1] ;
506                 W = u_star[i-2][J-1] ;
507                 //WW:
508                 if(i == 2){
509                     WW = 0.0 ;
510                 }else{
511                     WW = u_star[i-3][J-1] ;
512                 }
513                 //EE:
514                 if(i == Nx){
515                     EE = 0.0 ;
516                 }else{
517                     EE = u_star[i+1][J-1] ;
518                 }
519                 //SS:
520                 if(J == 1 || J == 2){
521                     SS = 0.0 ;
522                 }else{
523                     SS = u_star[i-1][J-3] ;
```

```
524                 }
525                 //S :
526                 if(J == 1){
527                     S = 0.0 ;
528                 }else{
529                     S = u_star[i-1][J-2] ;
530                 }
531                 //N :
532                 if(J == Ny){
533                     N = 0.0 ;
534                 }else{
535                     N = u_star[i-1][J] ;
536                 }
537                 //NN:
538                 if(J == Ny || J == Ny-1){
539                     NN = 0.0 ;
540                 }else {
541                     NN = u_star[i-1][J+1] ;
542                 }
543                 u_star[i-1][J-1] = (-a_WW[i-1][J-1]*WW - a_W[i-1][J-1]*W - a_E[i-1][J-1]*E -
                        a_EE[i-1][J-1]*EE
544                     - a_SS[i-1][J-1]*SS - a_S[i-1][J-1]*S - a_N[i-1][J-1]*N - a_NN[i-1][J-1]*NN
545                     + S1[i-1][J-1]) / a_P[i-1][J-1];
546                 // Relaxation for the velocity predictor
547                 u_star[i-1][J-1] = (1-omega_u)*u_star_old[i-1][J-1] + omega_u*u_star[i-1][J-1]
                        ;
548             }
549         }
550     }
551
552     // Update the linear-combination coefficients of the y-momentum predictor
553     #pragma omp parallel for collapse(2)
554     for(int I = 1 ; I <= Nx ; I++){
555         for(int j = 1 ; j <= Ny+1 ; j++){
556             // Reset neighboring coefficients to zero before accumulation
557             b_P[I-1][j-1] = 0.0 ;
558             b_WW[I-1][j-1] = 0.0 ;
559             b_W[I-1][j-1] = 0.0 ;
560             b_E[I-1][j-1] = 0.0 ;
561             b_EE[I-1][j-1] = 0.0 ;
562             b_SS[I-1][j-1] = 0.0 ;
563             b_S[I-1][j-1] = 0.0 ;
564             b_N[I-1][j-1] = 0.0 ;
565             b_NN[I-1][j-1] = 0.0 ;
566             S2[I-1][j-1] = 0.0 ;
567             if(j == 1){
568                 b_P[I-1][j-1] += 1.0 ;
569                 S2[I-1][j-1] += v_bottom ;// Impermeable wall; zero pressure gradient at the
                        bottom boundary
570                 v_star[I-1][j-1] = v_bottom ;
571             }else if(j == Ny+1){
572                 b_P[I-1][j-1] += 1.0 ;
573                 S2[I-1][j-1] += v_top ;// Impermeable wall; zero pressure gradient at the top
                        boundary
574                 v_star[I-1][j-1] = v_top ;
575             }// Special cases: west boundary rows 1-2, east boundary rows 1-2, top row 2,
                    bottom row 2
576             // Bottom second row
577             else{ // For remaining cells, handle west, east, south, and north boundaries
                    separately
578                 // South boundary
579                 if(j == 2){// Second row above the bottom uses upwind on the south face
580                     b_S[I-1][j-1] += alpha_s_v[I-1][j-1]*(-F_s_v[I-1][j-1])-D_s;// Subtract
                            south-face diffusion term when handling the boundary
581                     b_P[I-1][j-1] += (1-alpha_s_v[I-1][j-1])*(-F_s_v[I-1][j-1])+D_s;
582                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho ;
583                 }else{// Other south faces use TVD-MUSCL; upwind involves the south node and
                        current cell
584                     //b_SS[I-1][j-1] += (alpha_s_v[I-1][j-1]*UU(r_s_v[I-1][j-1]))*(-F_s_v[I-1][
                            j-1]) ;
585                     b_S[I-1][j-1] += (alpha_s_v[I-1][j-1]*1)*(-F_s_v[I-1][j-1]) - D_s ; //
                            Subtract south-face diffusion term when handling the boundary
586                     //b_N[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1]) ;
587                     b_P[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*1)*(-F_s_v[I-1][j-1])+D_s ;
588                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho
```

```
589                     -v_star_old[I-1][j-3]*(alpha_s_v[I-1][j-1]*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1])
590                     -v_star_old[I-1][j-2]*((1-alpha_s_v[I-1][j-1])*D(r_s_v[I-1][j-1])+alpha_s_v
                            [I-1][j-1]*(U(r_s_v[I-1][j-1])-1))*(-F_s_v[I-1][j-1])
591                     -v_star_old[I-1][j]*((1-alpha_s_v[I-1][j-1])*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1])
592                     -v_star_old[I-1][j-1]*((1-alpha_s_v[I-1][j-1])*(U(r_s_v[I-1][j-1])-1)+
                            alpha_s_v[I-1][j-1]*D(r_s_v[I-1][j-1]))*(-F_s_v[I-1][j-1])
593                     ;// Deferred correction complete
594                 }
595                 // North boundary
596                 if(j == Ny){// Second row below the top uses upwind on the north face
597                     b_N[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*(F_n_v[I-1][j-1])-D_n ;// Subtract
                            north-face diffusion term when handling the boundary
598                     b_P[I-1][j-1] += alpha_n_v[I-1][j-1]*(F_n_v[I-1][j-1])+D_n;
599                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho ;
600                 }else{// Other north faces use TVD-MUSCL; upwind involves the north node and
                        current cell
601                     //b_NN[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1]) ;
602                     b_N[I-1][j-1] += ((1-alpha_n_v[I-1][j-1])*1)*(F_n_v[I-1][j-1]) - D_n ;//
                            Subtract north-face diffusion term when handling the boundary
603                     //b_S[I-1][j-1] += (alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I-1][j
                            -1]) ;
604                     b_P[I-1][j-1] += (alpha_n_v[I-1][j-1]*1)*(F_n_v[I-1][j-1]) + D_n ;
605                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho
606                     -v_star_old[I-1][j+1]*(1-alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1])
607                     -v_star_old[I-1][j]*(alpha_n_v[I-1][j-1]*D(r_n_v[I-1][j-1])+(1-alpha_n_v[I
                            -1][j-1])*(U(r_n_v[I-1][j-1])-1))*(F_n_v[I-1][j-1])
608                     -v_star_old[I-1][j-1]*(alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1])
609                     -v_star_old[I-1][j-1]*((1-alpha_n_v[I-1][j-1])*D(r_n_v[I-1][j-1])+alpha_n_v
                            [I-1][j-1]*(U(r_n_v[I-1][j-1])-1))*(F_n_v[I-1][j-1])
610                     ; // Deferred correction complete
611                 }
612                 // West boundary (three special cases: I == 1, 2, Nx)
613                 if(I == 2){// Second column next to the west boundary uses extrapolation (WW ->
                        W)
614                     b_W[I-1][j-1] += ((alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])-D_w;
615                     //b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) ;
616                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])+D_w;
617                     S2[I-1][j-1] +=  -2*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1])*v_left
618                     +v_star_old[I-2][j-1]*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1])
619                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(
                            alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1])
620                     -v_star_old[I][j-1]*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1])
621                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+(
                            alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])
622                     ; // Deferred correction complete
623                 }else if(I == 1){// First column on the west boundary uses upwind
624                     b_P[I-1][j-1] +=  2*D_w;// Subtract west-face diffusion term when handling
                            the boundary
625                     S2[I-1][j-1] += -(-F_w_v[I-1][j-1])*v_left+2*D_w*v_left;
626                 }else if(I == Nx){// E -> P
627                     // When I == Nx the east neighbor is absent; extrapolate back to the
                            current cell
628                     //b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1]);
629                     b_W[I-1][j-1] += (alpha_w_v[I-1][j-1]*1)*(-F_w_v[I-1][j-1]) -D_w ;
630                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1]) +D_w;
631                     S2[I-1][j-1] += -2*v_right*(((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))
                            *(-F_w_v[I-1][j-1]))
632                     +v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v
                            [I-1][j-1])
633                     -v_star_old[I-3][j-1]*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1])
634                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+alpha_w_v
                            [I-1][j-1]*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1])
635                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+
                            alpha_w_v[I-1][j-1]*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])
636                     ;// Deferred correction complete
```

```
637                 }else{// Other west faces use TVD-MUSCL
638                     //b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1]) ;
639                     b_W[I-1][j-1] += ((alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])-D_w ; //
                            Subtract west-face diffusion term when handling the boundary
640                     //b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]);
641                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])+D_w ;
642                     S2[I-1][j-1] +=
643                     -v_star_old[I-3][j-1]*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1])
644                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(
                            alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1])
645                     -v_star_old[I][j-1]*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1])
646                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+(
                            alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1])
647                     ;// Deferred correction complete
648                 }
649                 // East boundary (three special cases: I == 1, Nx-1, Nx)
650                 if(I == 1){// First column next to the east boundary uses extrapolation (W -> P
                        ); deferred correction couples east face and current cell
651                     //b_EE[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*F_e_v[I
                            -1][j-1] ;
652                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1])-D_e ;//
                            Subtract east-face diffusion term when handling the boundary
653                     b_P[I-1][j-1] += ((alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1])  +D_e ;
654                     S2[I-1][j-1] +=  -2*v_left*(alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*(F_e_v
                            [I-1][j-1])
655                     +v_star_old[I-1][j-1]*(alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*(F_e_v[I
                            -1][j-1])
656                     -v_star_old[I+1][j-1]*((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*F_e_v[I
                            -1][j-1]
657                     -v_star_old[I][j-1]*((alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+(1-alpha_e_v[
                            I-1][j-1])*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])
658                     -v_star_old[I-1][j-1]*((alpha_e_v[I-1][j-1])*(U(r_e_v[I-1][j-1])-1)+(1-
                            alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1]))*(F_e_v[I-1][j-1])
659                     ;// Deferred correction complete
660                 }else if(I == Nx){// Second column next to the east boundary uses upwind
661                     b_P[I-1][j-1] += 2*D_e ;// Subtract east-face diffusion term when handling
                            the boundary
662                     S2[I-1][j-1] += -(F_e_v[I-1][j-1])*v_right +2*D_e*v_right;
663                 }else if(I == Nx-1){// EE node absent; extrapolate to the east node
664                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*F_e_v[I-1][j-1]
665                     - D_e ;
666                     //b_W[I-1][j-1] += (alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*F_e_v[I-1][j
                            -1] ;
667                     b_P[I-1][j-1] += (alpha_e_v[I-1][j-1]*1)*F_e_v[I-1][j-1] + D_e ;
668                     S2[I-1][j-1] += -2*v_right*((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*
                            F_e_v[I-1][j-1]
669                     +v_star_old[I][j-1]*((1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1]))*F_e_v[I
                            -1][j-1]
670                     -v_star_old[I][j-1]*((1-alpha_e_v[I-1][j-1])*(U(r_e_v[I-1][j-1])-1)+
                            alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1]))*F_e_v[I-1][j-1]
671                     -v_star_old[I-2][j-1]*(alpha_e_v[I-1][j-1]*UU(r_e_v[I-1][j-1]))*F_e_v[I-1][
                            j-1]
672                     -v_star_old[I-1][j-1]*((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v
                            [I-1][j-1]*(U(r_e_v[I-1][j-1])-1))*F_e_v[I-1][j-1]
673                     ;
674                 }else{// Other east faces use TVD-MUSCL
675                     //b_EE[I-1][j-1] += (1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                            -1][j-1]) ;
676                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1]) - D_e ;//
                            Subtract east-face diffusion term when handling the boundary
677                     //b_W[I-1][j-1] += (alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                            -1]) ;
678                     b_P[I-1][j-1] += (alpha_e_v[I-1][j-1]*1)*(F_e_v[I-1][j-1]) + D_e ;
679                     S2[I-1][j-1] +=
680                     -v_star_old[I+1][j-1]*(1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                            -1][j-1])
681                     -v_star_old[I][j-1]*(alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                            -1][j-1])*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])
682                     -v_star_old[I-2][j-1]*(alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                            -1][j-1])
683                     -v_star_old[I-1][j-1]*((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v
                            [I-1][j-1]*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])
```

```
684                     ;// Deferred correction complete
685                 }
686                 double WW=0.0 , W=0.0 , E=0.0 , EE=0.0 ;
687                 double SS=0.0 , S=0.0 , N=0.0 , NN=0.0 ;
688                 S = v_star[I-1][j-2] ;
689                 N = v_star[I-1][j] ;
690                 //SS:
691                 if(j == 2){
692                     SS = 0.0 ;
693                 }else{
694                     SS = v_star[I-1][j-3] ;
695                 }
696                 //NN:
697                 if(j == Ny){
698                     NN = 0.0 ;
699                 }else{
700                     NN = v_star[I-1][j+1] ;
701                 }
702                 //WW:
703                 if(I == 1 || I == 2){
704                     WW = 0.0 ;
705                 }else{
706                     WW = v_star[I-3][j-1] ;
707                 }
708                 //W:
709                 if(I == 1){
710                     W = 0.0 ;
711                 }else{
712                     W = v_star[I-2][j-1] ;
713                 }
714                 //E:
715                 if(I == Nx){
716                     E = 0.0 ;
717                 }else{
718                     E = v_star[I][j-1] ;
719                 }
720                 //EE:
721                 if(I == Nx || I == Nx-1){
722                     EE = 0.0 ;
723                 }else{
724                     EE = v_star[I+1][j-1] ;
725                 }
726                 v_star[I-1][j-1] = (-b_WW[I-1][j-1]*WW - b_W[I-1][j-1]*W - b_E[I-1][j-1]*E -
                        b_EE[I-1][j-1]*EE
727                         - b_SS[I-1][j-1]*SS - b_S[I-1][j-1]*S - b_N[I-1][j-1]*N - b_NN[I
                            -1][j-1]*NN
728                         + S2[I-1][j-1]) / b_P[I-1][j-1] ;
729                 // Relaxation for the velocity predictor
730                 v_star[I-1][j-1] = (1-omega_u)*v_star_old[I-1][j-1] + omega_u*v_star[I-1][j-1]
                        ;
731             }
732         }
733     }
734 }
735 // u_star and v_star are now at the N+1/2 time level
736 // Function: mass flux after the first velocity predictor update
737 double d(int I, int J , int m){
738     // I,J are cell-center coordinates; m is the SIMPLE iteration index, distinct from the
            velocity time level
739     double u_e = u_star[I+1-1][J-1] ;// Velocity predictor currently at N+1/2
740     double u_w = u_star[I-1][J-1] ;// Velocity predictor currently at N+1/2
741     double v_n = v_star[I-1][J+1-1] ;// Velocity predictor currently at N+1/2
742     double v_s = v_star[I-1][J-1] ;// Velocity predictor currently at N+1/2
743     return rho*(u_e - u_w)*dy + rho*(v_n - v_s)*dx ;
744 } // Used after the first update of the velocity predictor
745
746 // Step 4: update the pressure field
747 void step4(int m){
748     #pragma omp parallel for collapse(2)
749     for(int I = 1 ; I <= Nx ; I++){ // Pressure values (true, predicted, corrected) are stored
            at cell centers
750         for(int J = 1 ; J <= Ny ; J++){
751            p[I-1][J-1] = p_star[I-1][J-1] + omega_P*p_prime[I-1][J-1] ;
752         }// p_star holds the N-level pressure; p_prime is evaluated at N+1
753     }// p_prime is at N+1
754 }
```

```
755
756 // Step 5: update velocity corrections; boundary nodes are fixed by boundary conditions so
        their corrections stay zero
757 void step5(int m){
758     #pragma omp parallel for collapse(2)
759     // Update x-velocity corrections
760     for(int i = 1 ; i <= Nx+1 ; i++){
761         for(int J = 1 ; J <= Ny ; J++){
762             if(i == 1 || i == Nx+1){
763                 u_prime[i-1][J-1] = 0.0 ;// Physical-boundary velocity already satisfies the
                        boundary condition
764             }else{// Pressure correction gradient between current and west neighbor
765                 u_prime[i-1][J-1] = -(p_prime[i-1][J-1] - p_prime[i-2][J-1])*dy/(rho*a_P[i-1][J
                        -1]) ; ;
766             }
767
768         }
769     }
770     // Update y-velocity corrections
771     #pragma omp parallel for collapse(2)
772     for(int I = 1 ; I <= Nx ; I++){
773         for(int j = 1 ; j <= Ny+1 ; j++){
774             if(j == 1 || j == Ny+1){
775                 v_prime[I-1][j-1] = 0.0 ;// Physical-boundary velocity already satisfies the
                        boundary condition
776             }else{// Pressure correction gradient between current and south neighbor
777                 v_prime[I-1][j-1] = -(p_prime[I-1][j-1] - p_prime[I-1][j-2])*dx/(rho*b_P[I-1][j
                        -1]) ;
778             }
779         }
780     }
781 }
782
783 // Step 6: update the velocity field and refresh the stored previous values
784 void step6(int m){// Physical-boundary velocities come directly from boundary conditions and
        are not iterated
785     // Update x-velocity field
786     #pragma omp parallel for collapse(2)
787     for(int i = 1 ; i <= Nx+1 ; i++){
788         for(int J = 1 ; J <= Ny ; J++){
789             if(i == 1 ){
790                 u[i-1][J-1] = 0.0 ;// Physical-boundary velocity already satisfies the boundary
                        condition
791             }else if (i == Nx+1){
792                 u[i-1][J-1] = 0.0 ;// Physical-boundary velocity already satisfies the boundary
                        condition
793             }else{
794                 u[i-1][J-1] = u_prime[i-1][J-1]+u_star[i-1][J-1];// u_star at N+1/2, u_prime at
                        N+1, u_old at N, u at N+1
795             }
796         }
797     }
798     // Store updated x-velocity as the old field for the next iteration
799     #pragma omp parallel for collapse(2)
800     for(int i = 1 ; i <= Nx+1 ; i++){
801         for(int J = 1 ; J <= Ny ; J++){
802             u_old[i-1][J-1] = u[i-1][J-1] ; // u stored at N+1, so u_old is refreshed to N+1
803         }
804     }
805     // Update y-velocity field
806     #pragma omp parallel for collapse(2)
807     for(int I = 1 ; I <= Nx ; I++){
808         for(int j = 1 ; j <= Ny+1 ; j++){
809             if(j == 1 ){
810                 v[I-1][j-1] = 0.0 ;// Physical-boundary velocity already satisfies the boundary
                        condition
811             }else if (j == Ny+1){
812                 v[I-1][j-1] = 0.0 ;// Physical-boundary velocity already satisfies the boundary
                        condition
813             }else{
814                 v[I-1][j-1] = v_prime[I-1][j-1]+v_star[I-1][j-1];// v_star at N+1/2, v_prime at
                        N+1, v_old at N, v at N+1
815             }
816         }
817     }
818     // Store updated y-velocity as the old field for the next iteration
```

```
819     #pragma omp parallel for collapse(2)
820     for(int I = 1 ; I <= Nx ; I++){
821         for(int j = 1 ; j <= Ny+1 ; j++){
822             v_old[I-1][j-1] = v[I-1][j-1] ; // v stored at N+1, so v_old is refreshed to N+1
823         }
824     }
825 }
826 double step5star(int m){
827     double sum = 0.0 ;
828     #pragma omp parallel for collapse(2) reduction(+:sum)  // Added reduction to accumulate the
          sum in parallel
829     for(int I = 1 ; I <= Nx ; I++){
830         for(int J = 1 ; J <= Ny ; J++){
831             sum = sum + fabs(d(I,J,m)) ;
832         }
833     }
834     return sum ;
835 }
836 // Step 7: accumulate the absolute non-equilibrium mass flux across control surfaces
837 double step7(int m){
838     //81
839     double sum = 0.0 ;
840     sum += fabs(u[40][0]-0) ;
841     sum += fabs(u[40][3]-(-0.41165)) ;
842     sum += fabs(u[40][4]-(-0.42901)) ;
843     sum += fabs(u[40][5]-(-0.43643)) ;
844     sum += fabs(u[40][7]-(-0.40435)) ;
845     sum += fabs(u[40][13]-(-0.3305)) ;
846     sum += fabs(u[40][22]-(-0.22855)) ;
847     sum += fabs(u[40][35]-(-0.07404)) ;
848     sum += fabs(u[40][39]-(-0.03039)) ;
849     sum += fabs(u[40][48]-(0.08183)) ;
850     sum += fabs(u[40][58]-(0.20087)) ;
851     sum += fabs(u[40][67]-(0.33556)) ;
852     sum += fabs(u[40][75]-(0.46036)) ;
853     sum += fabs(u[40][76]-(0.45992)) ;
854     sum += fabs(u[40][77]-(0.4612)) ;
855     sum += fabs(u[40][77]-(0.48223)) ;
856     sum += fabs(u[40][79]-(1)) ;
857     sum += fabs(v[0][40]-0) ;
858     sum += fabs(v[4][40]-(0.42447)) ;
859     sum += fabs(v[5][40]-(0.43329)) ;
860     sum += fabs(v[5][40]-(0.43648)) ;
861     sum += fabs(v[7][40]-(0.42951)) ;
862     sum += fabs(v[12][40]-(0.35368)) ;
863     sum += fabs(v[17][40]-(0.28066)) ;
864     sum += fabs(v[18][40]-(0.2728)) ;
865     sum += fabs(v[39][40]-(0.00945)) ;
866     sum += fabs(v[64][40]-(-0.30018)) ;
867     sum += fabs(v[69][40]-(-0.36214)) ;
868     sum += fabs(v[72][40]-(-0.41442)) ;
869     sum += fabs(v[76][40]-(-0.52876)) ;
870     sum += fabs(v[76][40]-(-0.55408)) ;
871     sum += fabs(v[77][40]-(-0.55069)) ;
872     sum += fabs(v[77][40]-(-0.49774)) ;
873     sum += fabs(v[79][40]-(0.0)) ;
874     /*
875     //113
876     double sum = 0.0 ;
877     sum += fabs(u[56][0]-0) ;
878     sum += fabs(u[56][6]-(-0.41165)) ;
879     sum += fabs(u[56][6]-(-0.42901)) ;
880     sum += fabs(u[56][7]-(-0.43643)) ;
881     sum += fabs(u[56][11]-(-0.40435)) ;
882     sum += fabs(u[56][19]-(-0.3305)) ;
883     sum += fabs(u[56][31]-(-0.22855)) ;
884     sum += fabs(u[56][50]-(-0.07404)) ;
885     sum += fabs(u[56][55]-(-0.03039)) ;
886     sum += fabs(u[56][69]-(0.08183)) ;
887     sum += fabs(u[56][82]-(0.20087)) ;
888     sum += fabs(u[56][95]-(0.33556)) ;
889     sum += fabs(u[56][106]-(0.46036)) ;
890     sum += fabs(u[56][107]-(0.45992)) ;
891     sum += fabs(u[56][108]-(0.4612)) ;
892     sum += fabs(u[56][109]-(0.48223)) ;
893     sum += fabs(u[56][111]-(1)) ;
```

```
894
895       sum += fabs(v[0][56]-0) ;
896       sum += fabs(v[6][56]-(0.42447)) ;
897       sum += fabs(v[7][56]-(0.43329)) ;
898       sum += fabs(v[8][56]-(0.43648)) ;
899       sum += fabs(v[10][56]-(0.42951)) ;
900       sum += fabs(v[17][56]-(0.35368)) ;
901       sum += fabs(v[25][56]-(0.28066)) ;
902       sum += fabs(v[26][56]-(0.2728)) ;
903       sum += fabs(v[55][56]-(0.00945)) ;
904       sum += fabs(v[90][56]-(-0.30018)) ;
905       sum += fabs(v[96][56]-(-0.36214)) ;
906       sum += fabs(v[101][56]-(-0.41442)) ;
907       sum += fabs(v[105][56]-(-0.52876)) ;
908       sum += fabs(v[106][56]-(-0.55408)) ;
909       sum += fabs(v[107][56]-(-0.55069)) ;
910       sum += fabs(v[108][56]-(-0.49774)) ;
911       sum += fabs(v[111][56]-(0.0)) ;*/
912       /*
913       //161
914       double sum = 0.0 ;
915           sum += fabs(u[80][0]-0) ;
916       sum += fabs(u[80][8]-(-0.18109)) ;
917       sum += fabs(u[80][9]-(-0.20196)) ;
918       sum += fabs(u[80][10]-(-0.22220)) ;
919       sum += fabs(u[80][15]-(-0.29730)) ;
920       sum += fabs(u[80][27]-(-0.38289)) ;
921       sum += fabs(u[80][44]-(-0.27805)) ;
922       sum += fabs(u[80][71]-(-0.10648)) ;
923       sum += fabs(u[80][79]-(-0.06080)) ;
924       sum += fabs(u[80][98]-(0.05702)) ;
925       sum += fabs(u[80][117]-(0.18719)) ;
926       sum += fabs(u[80][135]-(0.33304)) ;
927       sum += fabs(u[80][151]-(0.46604)) ;
928       sum += fabs(u[80][153]-(0.51117)) ;
929       sum += fabs(u[80][154]-(0.57492)) ;
930       sum += fabs(u[80][155]-(0.65928)) ;
931       sum += fabs(u[80][159]-(1)) ;
932
933       sum += fabs(v[0][80]-0) ;
934       sum += fabs(v[9][80]-(0.27485)) ;
935       sum += fabs(v[10][80]-(0.29012)) ;
936       sum += fabs(v[12][80]-(0.30353)) ;
937       sum += fabs(v[14][80]-(0.32627)) ;
938       sum += fabs(v[24][80]-(0.37095)) ;
939       sum += fabs(v[35][80]-(0.33075)) ;
940       sum += fabs(v[37][80]-(0.32235)) ;
941       sum += fabs(v[80][80]-(0.02526)) ;
942       sum += fabs(v[129][80]-(-0.31966)) ;
943       sum += fabs(v[137][80]-(-0.42655)) ;
944       sum += fabs(v[145][80]-(-0.51550)) ;
945       sum += fabs(v[151][80]-(-0.39188)) ;
946       sum += fabs(v[152][80]-(-0.33714)) ;
947       sum += fabs(v[154][80]-(-0.27669)) ;
948       sum += fabs(v[155][80]-(-0.21388)) ;
949       sum += fabs(v[159][80]-(0.0)) ;*/
950       /*//227
951       double sum = 0.0 ;
952       sum += fabs(u[113][0]-0) ;
953       sum += fabs(u[113][12]-(-0.41165)) ;
954       sum += fabs(u[113][14]-(-0.42901)) ;
955       sum += fabs(u[113][15]-(-0.43643)) ;
956       sum += fabs(u[113][22]-(-0.40435)) ;
957       sum += fabs(u[113][38]-(-0.3305)) ;
958       sum += fabs(u[113][63]-(-0.22855)) ;
959       sum += fabs(u[113][102]-(-0.07404)) ;
960       sum += fabs(u[113][113]-(-0.03039)) ;
961       sum += fabs(u[113][139]-(0.08183)) ;
962       sum += fabs(u[113][165]-(0.20087)) ;
963       sum += fabs(u[113][192]-(0.33556)) ;
964       sum += fabs(u[113][215]-(0.46036)) ;
965       sum += fabs(u[113][217]-(0.45992)) ;
966       sum += fabs(u[113][218]-(0.4612)) ;
967       sum += fabs(u[113][220]-(0.48223)) ;
968       sum += fabs(u[113][225]-(1)) ;
969
```

```
970       sum += fabs(v[0][113]-0) ;
971       sum += fabs(v[14][113]-(0.42447)) ;
972       sum += fabs(v[15][113]-(0.43329)) ;
973       sum += fabs(v[17][113]-(0.43648)) ;
974       sum += fabs(v[21][113]-(0.42951)) ;
975       sum += fabs(v[35][113]-(0.35368)) ;
976       sum += fabs(v[51][113]-(0.28066)) ;
977       sum += fabs(v[52][113]-(0.2728)) ;
978       sum += fabs(v[113][113]-(0.00945)) ;
979       sum += fabs(v[181][113]-(-0.30018)) ;
980       sum += fabs(v[194][113]-(-0.36214)) ;
981       sum += fabs(v[204][113]-(-0.41442)) ;
982       sum += fabs(v[214][113]-(-0.52876)) ;
983       sum += fabs(v[215][113]-(-0.55408)) ;
984       sum += fabs(v[217][113]-(-0.55069)) ;
985       sum += fabs(v[218][113]-(-0.49774)) ;
986       sum += fabs(v[225][113]-(0.0)) ;*/
987       return sum ;
988   }
989
990   // Step 8: refresh the pressure predictor at cell centers
991   void step8(int m){
992       #pragma omp parallel for collapse(2)  // Parallel refresh of the predictor field
993       for(int I = 1 ; I <= Nx ; I++){
994           for(int J = 1 ; J <= Ny ; J++){
995               p_star[I-1][J-1] = p[I-1][J-1] ; // Both p and p_star are stored at time level N+1
996           }
997       }
998   }
999
1000  // Step 9: refresh velocity predictors, limiter flags, and convective coefficients
1001  void step9(int m){
1002      // Update x-velocity predictor; boundary values come directly from boundary conditions
1003      #pragma omp parallel for collapse(2)
1004      for(int i = 1 ; i <= Nx+1 ; i++){
1005          for(int J = 1 ; J <= Ny ; J++){
1006              if(i == 1){
1007                  u_star[i-1][J-1] = u_left;
1008              }else if (i == Nx+1){
1009                  u_star[i-1][J-1] = u_right ;
1010              }else{
1011                  // Interior cells, including those adjacent to top/bottom boundaries, use the
                        corrected velocity
1012                  u_star[i-1][J-1] = u[i-1][J-1]; // u evaluated at N+1, so the predictor is also
                        at N+1
1013                  u_star_old[i-1][J-1] = u_star[i-1][J-1]; // Record predictor immediately
1014              }
1015          }
1016      }
1017      // Update y-velocity predictor; boundary values come directly from boundary conditions
1018      #pragma omp parallel for collapse(2)
1019      for(int I = 1 ; I <= Nx ; I++){
1020          for(int j = 1 ; j <= Ny+1 ; j++){
1021              if(j == 1){
1022                  v_star[I-1][j-1] = v_bottom ;
1023              }else if (j == Ny+1){
1024                  v_star[I-1][j-1] = v_top ;
1025              }else{
1026                  // Interior cells, including those adjacent to left/right boundaries, use the
                        corrected velocity
1027                  v_star[I-1][j-1] = v[I-1][j-1]; // v evaluated at N+1, so the predictor is also
                        at N+1
1028                  v_star_old[I-1][j-1] = v_star[I-1][j-1]; // Record predictor immediately
1029              }
1030          }
1031      }
1032      // Update convective coefficients
1033      // x-momentum convective coefficients in the discrete form
1034      #pragma omp parallel for collapse(2)
1035      for(int i = 1 ; i <= Nx+1 ; i++){
1036          for(int J = 1 ; J <= Ny ; J++){
1037              if(i == 1 || i == Nx+1) {
1038                  F_w_u[i-1][J-1] = 0.0 ;
1039                  F_e_u[i-1][J-1] = 0.0 ;
1040                  F_n_u[i-1][J-1] = 0.0 ;
1041                  F_s_u[i-1][J-1] = 0.0 ;
```

```
1042            }else{// Recompute convective coefficients
1043                F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); // Cell centers
                        align with storage points
1044                F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
1045                F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
1046                F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
1047            }
1048        }
1049    }// alpha_f_u flags are defined at cell centers
1050    #pragma omp parallel for collapse(2)
1051    for(int i = 1 ; i <= Nx+1 ; i++){
1052        for(int J = 1 ; J <= Ny ; J++){
1053            if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
1054            else alpha_w_u[i-1][J-1] = 0.0 ;
1055            if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
1056            else alpha_e_u[i-1][J-1] = 0.0 ;
1057            if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
1058            else alpha_n_u[i-1][J-1] = 0.0 ;
1059            if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
1060            else alpha_s_u[i-1][J-1] = 0.0 ;
1061        }
1062    }
1063    // y-momentum convective coefficients in the discrete form
1064    #pragma omp parallel for collapse(2)
1065    for(int I = 1 ; I <= Nx ; I++){
1066        for(int j = 1 ; j <= Ny+1 ; j++){
1067            if(j == 1 || j == Ny+1) {
1068                F_w_v[I-1][j-1] = 0.0 ;
1069                F_e_v[I-1][j-1] = 0.0 ;
1070                F_n_v[I-1][j-1] = 0.0 ;
1071                F_s_v[I-1][j-1] = 0.0 ;
1072            }else{
1073                F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
1074                F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
1075                F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);// Cell centers
                        align with storage points
1076                F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
1077            }
1078        }
1079    }// alpha_f_v flags are defined at cell centers
1080    #pragma omp parallel for collapse(2)
1081    for(int I = 1 ; I <= Nx ; I++){
1082        for(int j = 1 ; j <= Ny+1 ; j++){
1083            if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
1084            else alpha_w_v[I-1][j-1] = 0.0 ;
1085            if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
1086            else alpha_e_v[I-1][j-1] = 0.0 ;
1087            if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
1088            else alpha_n_v[I-1][j-1] = 0.0 ;
1089            if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
1090            else alpha_s_v[I-1][j-1] = 0.0 ;
1091        }
1092    }
1093    // Update interface ratio factors
1094    #pragma omp parallel for collapse(2)
1095    for(int i = 1 ; i <= Nx+1 ; i++){
1096        for(int J = 1 ; J <= Ny ; J++){
1097            // Cells outside the upwind stencil set the ratio to zero
1098            if(i == 1 || i == Nx+1){
1099                r_w_u[i-1][J-1] = 0.0 ;
1100                r_e_u[i-1][J-1] = 0.0 ;
1101                r_n_u[i-1][J-1] = 0.0 ;
1102                r_s_u[i-1][J-1] = 0.0 ;
1103            }else {
1104                // Handle each boundary case (west/east/south/north)
1105                // West boundary
1106                if(i == 2){
1107                    r_w_u[i-1][J-1] = -1.0 ; // West boundary uses upwind scheme
1108                }else{
1109                    double u_W = u_star[i-2][J-1];
1110                    double u_P = u_star[i-1][J-1];
1111                    double u_WW = u_star[i-3][J-1];
1112                    double u_E = u_star[i][J-1] ;
1113                    double D = u_W + alpha_w_u[i-1][J-1]*(u_P - u_W) ;
1114                    double U = u_P + alpha_w_u[i-1][J-1]*(u_W - u_P) ;
1115                    double UU = u_E + alpha_w_u[i-1][J-1]*(u_WW - u_E) ;
```

```
1116                    r_w_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; // Initialized ratio
1117                }
1118                // East boundary
1119                if(i == Nx){
1120                    r_e_u[i-1][J-1] = -1.0 ; // East boundary uses upwind scheme
1121                }else{
1122                    double u_P = u_star[i-1][J-1] ;
1123                    double u_E = u_star[i][J-1] ;
1124                    double u_W = u_star[i-2][J-1] ;
1125                    double u_EE = u_star[i+1][J-1] ;
1126                    double D = u_P + alpha_e_u[i-1][J-1]*(u_E - u_P) ;
1127                    double U = u_E + alpha_e_u[i-1][J-1]*(u_P - u_E) ;
1128                    double UU = u_EE + alpha_e_u[i-1][J-1]*(u_W - u_EE) ;
1129                    r_e_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; // Initialized ratio
1130                }
1131                // South boundary
1132                if(J == 1){
1133                    r_s_u[i-1][J-1] = 0.0 ; // Bottom boundary does not need a south-face ratio
1134                }else if(J == 2){
1135                    double u_S = u_star[i-1][J-2] ;
1136                    double u_P = u_star[i-1][J-1] ;
1137                    double u_SS = 2*u_bottom-u_S ; // 2*boundary value minus south node
1138                    double u_N = u_star[i-1][J] ;
1139                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1140                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1141                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1142                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
1143                }else if(J == Ny){
1144                    double u_S = u_star[i-1][J-2] ;
1145                    double u_P = u_star[i-1][J-1] ;
1146                    double u_SS = u_star[i-1][J-3] ;
1147                    double u_N = 2*u_top - u_P ; // 2*boundary value minus current cell
1148                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1149                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1150                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1151                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
1152                }else{
1153                    double u_S = u_star[i-1][J-2] ;
1154                    double u_P = u_star[i-1][J-1] ;
1155                    double u_SS = u_star[i-1][J-3] ;
1156                    double u_N = u_star[i-1][J] ;
1157                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1158                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1159                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1160                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
1161                }
1162                // North boundary
1163                if(J == 1){
1164                    double u_P = u_star[i-1][J-1] ;
1165                    double u_N = u_star[i-1][J] ;
1166                    double u_S = 2*u_bottom-u_P ; // 2*boundary value minus current cell
1167                    double u_NN = u_star[i-1][J+1] ;
1168                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1169                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1170                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1171                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
1172                }else if (J == Ny-1){
1173                    double u_P = u_star[i-1][J-1] ;
1174                    double u_N = u_star[i-1][J] ;
1175                    double u_S = u_star[i-1][J-2] ;
1176                    double u_NN = 2*u_top - u_N ; // 2*boundary value minus north node
1177                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1178                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1179                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1180                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
1181                }else if(J == Ny){
1182                    r_n_u[i-1][J-1] = 0.0 ; // Top boundary does not need a north-face ratio
1183                }else{
1184                    double u_P = u_star[i-1][J-1] ;
1185                    double u_N = u_star[i-1][J] ;
1186                    double u_S = u_star[i-1][J-2] ;
1187                    double u_NN = u_star[i-1][J+1] ;
1188                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1189                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1190                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1191                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; // Initialized ratio
```

```
1192                }
1193            }
1194        }
1195    }// Update interface ratios immediately after refreshing predictors
1196    // Interface ratios for the y-velocity field (east/west require special handling)
1197    #pragma omp parallel for collapse(2)
1198    for(int I = 1 ; I <= Nx ; I++){
1199        for(int j = 1 ; j <= Ny+1 ; j++){
1200            // Cells outside the upwind stencil set the ratio to zero
1201            if(j == 1 || j == Ny+1){
1202                r_w_v[I-1][j-1] = 0.0 ;
1203                r_e_v[I-1][j-1] = 0.0 ;
1204                r_n_v[I-1][j-1] = 0.0 ;
1205                r_s_v[I-1][j-1] = 0.0 ;
1206            }else {
1207                // Handle each boundary case (south, north, east, west)
1208                // South boundary
1209                if(j == 2){
1210                    r_s_v[I-1][j-1] = -1.0 ; // Bottom boundary uses upwind scheme
1211                }else{
1212                    double v_S = v_star[I-1][j-2] ;
1213                    double v_P = v_star[I-1][j-1] ;
1214                    double v_SS = v_star[I-1][j-3] ;
1215                    double v_N = v_star[I-1][j] ;
1216                    double D = v_S + alpha_s_v[I-1][j-1]*(v_P - v_S) ;
1217                    double U = v_P + alpha_s_v[I-1][j-1]*(v_S - v_P) ;
1218                    double UU = v_N + alpha_s_v[I-1][j-1]*(v_SS - v_N) ;
1219                    r_s_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ; // Avoid division by zero
1220                }
1221                // North boundary
1222                if(j == Ny){
1223                    r_n_v[I-1][j-1] = -1.0 ; // Top boundary uses upwind scheme
1224                }else{
1225                    double v_P = v_star[I-1][j-1] ;
1226                    double v_N = v_star[I-1][j] ;
1227                    double v_S = v_star[I-1][j-2] ;
1228                    double v_NN = v_star[I-1][j+1] ;
1229                    double D = v_P + alpha_n_v[I-1][j-1]*(v_N - v_P) ;
1230                    double U = v_N + alpha_n_v[I-1][j-1]*(v_P - v_N) ;
1231                    double UU = v_NN + alpha_n_v[I-1][j-1]*(v_S - v_NN) ;
1232                    r_n_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1233                }
1234                // West boundary
1235                if(I == 1){
1236                    r_w_v[I-1][j-1] = 0.0 ; // West boundary does not need a west-face ratio
1237                }else if (I == 2){
1238                    double v_W = v_star[I-2][j-1] ;
1239                    double v_P = v_star[I-1][j-1] ;
1240                    double v_WW = 2*v_left-v_W ; // 2*boundary value minus west node
1241                    double v_E = v_star[I][j-1] ;
1242                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1243                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1244                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1245                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1246                }else if(I == Nx){
1247                    double v_W = v_star[I-2][j-1] ;
1248                    double v_P = v_star[I-1][j-1] ;
1249                    double v_WW = v_star[I-3][j-1] ;
1250                    double v_E = 2*v_right - v_P ; // 2*boundary value minus current cell
1251                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1252                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1253                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1254                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1255                }else{
1256                    double v_W = v_star[I-2][j-1] ;
1257                    double v_P = v_star[I-1][j-1] ;
1258                    double v_WW = v_star[I-3][j-1] ;
1259                    double v_E = v_star[I][j-1] ;
1260                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1261                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1262                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1263                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1264                }
1265                // East boundary
1266                if(I == 1){
1267                    double v_P = v_star[I-1][j-1] ;
```

```
                    double v_E = v_star[I][j-1] ;
                    double v_W = 2*v_left - v_P ; // 2*boundary value minus current cell
                    double v_EE = v_star[I+1][j-1] ;
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                }else if (I == Nx-1){
                    double v_P = v_star[I-1][j-1] ;
                    double v_E = v_star[I][j-1] ;
                    double v_W = v_star[I-2][j-1] ;
                    double v_EE = 2*v_right - v_E ; // 2*boundary value minus east node
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                }else if(I == Nx){
                    r_e_v[I-1][j-1] = 0.0 ; // East boundary does not need an east-face ratio
                }else {
                    double v_P = v_star[I-1][j-1] ;
                    double v_E = v_star[I][j-1] ;
                    double v_W = v_star[I-2][j-1] ;
                    double v_EE = v_star[I+1][j-1] ;
                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
                }
            }
        }
    }
}
/*void output_combined_fields(int m) {
    const int cells_i = Nx; // cells in x
    const int cells_j = Ny; // cells in y

    std::ostringstream name;
    name << "3.QUICK_Re" << Re << "_Grid" << Nx << "x" << Ny
         << "_DC_step" << std::setfill('0') << std::setw(6) << m
         << "_error" << min_step7 <<
          ".dat";

    std::ofstream out(name.str());

    if (!out) {
        std::cerr << "Error: Cannot open file: " << name.str() << std::endl;
        return;
    }

    // Tecplot header
    out << "TITLE = \"SIMPLE-QUICK CFD Simulation\"\n";
    out << "VARIABLES = \"X\", \"Y\", \"P\", \"U\", \"V\", \"Vel_Mag\", \"Vorticity\"\n";
    out << "ZONE T=\"Time_Step_" << m << "\", "
        << "I=" << cells_i << ", J=" << cells_j << ", "
        << "DATAPACKING=POINT\n"; // Removed trailing comma

    // AUXDATA (quote strings for safety)
    out << "AUXDATA Re=\""   << Re        << "\"\n";
    out << "AUXDATA Grid=\"" << Nx << "x" << Ny << "\"\n";
    out << "AUXDATA dx=\""   << dx        << "\"\n";
    out << "AUXDATA dy=\""   << dy        << "\"\n";

    out << std::scientific << std::setprecision(8);

    for (int j = 0; j < cells_j; ++j) {
        for (int i = 0; i < cells_i; ++i) {
            // Cell center
            const double xC = (i + 0.5) * dx;
            const double yC = (j + 0.5) * dy;

            // p at center
            const double pC = p[i][j]; // Assume p has dimensions [Nx][Ny]

            // u, v interpolated from face values to cell center (MAC: u(i+0.5,j), v(i,j+0.5))
            // Requires u sized [Nx+1][Ny] and v sized [Nx][Ny+1]
            const double uC = 0.5 * (u[i][j] + u[i+1][j]);
```

```
            const double vC = 0.5 * (v[i][j] + v[i][j+1]);

            const double vel_mag = std::sqrt(uC*uC + vC*vC);

            // Vorticity at the center using MAC-consistent first-order differences
            double vort = 0.0;
            // Requires i+1 <= Nx-1 => i <= Nx-2 and j-1 >= 0 => j >= 1
            if (i <= cells_i - 2 && j >= 1) {
                const double dvdx = (v[i+1][j] - v[i][j]) / dx; // v: Nx × (Ny+1) using the same j
                const double dudy = (u[i][j]   - u[i][j-1]) / dy; // u: (Nx+1) × Ny using the same i
                vort = dvdx - dudy;
            }
            // One-sided differences at boundaries can avoid zero values

            out << xC << " " << yC << " "
                << pC << " " << uC << " " << vC << " "
                << vel_mag << " " << vort << "\n";
        }
    }

    out.close();
    std::cout << "Tecplot file successfully written: " << name.str() << std::endl;
}
*/

// Finite-volume cell data output
void output_combined_fields(int m) {
    int nodes_i = Nx + 1;  // Number of nodes in x
    int nodes_j = Ny + 1;  // Number of nodes in y
    int cells_i = Nx;      // Number of cells in x
    int cells_j = Ny;      // Number of cells in y

    std::ostringstream name;
    name << "3.CDs_Re" << Re << "_Grid" << Nx << "x" << Ny
         << "_DC_step" << std::setfill('0') << std::setw(6) << m
         << "_error" << min_step7 <<
          ".vtk";

    std::ofstream out(name.str());

    out << "# vtk DataFile Version 3.0\n";
    out << "3.CDs_Re = 5000_Grid 161x161\n";
    out << "ASCII\n";
    out << "DATASET STRUCTURED_GRID\n";
    out << "DIMENSIONS " << nodes_i << " " << nodes_j << " 1\n";

    // Write node coordinates
    out << "POINTS " << nodes_i * nodes_j << " double\n";
    for(int j = 0; j < nodes_j; j++) {
        for(int i = 0; i < nodes_i; i++) {
            double x_coord = i * dx;
            double y_coord = j * dy;
            out << scientific << setprecision(6)
                << x_coord << " " << y_coord << " 0.0\n";
        }
    }

    out << "CELL_DATA " << cells_i * cells_j << "\n";

    // 1. Write pressure field
    out << "SCALARS Pressure double 1\n";
    out << "LOOKUP_TABLE default\n";
    for(int J = 1; J <= cells_j; J++) {
        for(int I = 1; I <= cells_i; I++) {
            out << scientific << setprecision(6)
                << p_star[I-1][J-1] << "\n";
        }
    }

    // 2. Write velocity field (no additional CELL_DATA declaration needed)
    out << "VECTORS Velocity double\n";
    for(int J = 1; J <= cells_j; J++) {
        for(int I = 1; I <= cells_i; I++) {
            double u_val = 0.5 * (u_star[I-1][J-1] + u_star[I][J-1]);
```

```
1418                double v_val = 0.5 * (v_star[I-1][J-1] + v_star[I-1][J]);
1419
1420                out << scientific << setprecision(6)
1421                    << u_val << " " << v_val << " 0.0\n";
1422            }
1423        }
1424
1425        out.close();
1426        cout << "VTK document has been output: " << name.str() << endl;
1427    }
1428
1429
1430    int main(){
1431        // Enable OpenMP to use all available threads
1432        int num_threads = omp_get_max_threads();
1433        omp_set_num_threads(num_threads);
1434        cout << "Using " << num_threads << " threads" << endl;
1435        cout << "Using FVD with SIMPLE algorithm to solve 2D incompressible Navier-Stokes equations
               " << endl;
1436        cout << "Grid Size:" << Nx+1 << "x" << Ny+1 << endl;
1437        cout << "Grid spacing : " << dx << "x " << dy << endl;
1438        cout << "Boundary Conditions: " ;
1439        cout << "u: left = " << u_left << ", right = " << u_right << ", bottom = " << u_bottom << "
               , top = " << u_top << endl;
1440        cout << "v: left = " << v_left << ", right = " << v_right << ", bottom = " << v_bottom << "
               , top = " << v_top << endl;
1441        cout << "Re = " << Re << ",rho = " << rho << endl ;
1442        CE_physical = false;
1443        G = 0 ;
1444        initialize() ;
1445        min_step7 = 50.0 ;
1446        double min_G = 0.0  ;
1447        for(G = 0 ; G <= max_G ; G++){
1448
1449            step2(G) ;// Update velocity predictor (first pass)
1450            step3() ;// Update pressure correction
1451            step4(G) ;// Update pressure field
1452            step5(G) ;// Update velocity corrections
1453            step6(G) ;// Update velocity field
1454            // Step 7: convergence check
1455            if(min_step7 > step7(G)){ // Evaluate at every iteration
1456                    min_step7 = step7(G) ;
1457                    min_G = G ;
1458                    //if(G > 1000 && min_step7 < 2.55   ){output_combined_fields(min_G) ;}
1459
1460                }
1461            if(G%1000 == 0){
1462                cout << "Itteration step = " << G << endl ;
1463                cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                         << endl ;
1464                cout << "currnt minerror(G =" << min_G << ")= "<< min_step7 << endl ;
1465            }
1466            if(G>0 && step5star(G) < 1e-10){
1467                CE_physical = true ;
1468                cout << "Haved converged at step G = " << G << endl ;
1469                cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                         <<endl ;
1470                cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
1471                output_combined_fields(G) ;
1472                break ;
1473            }
1474            step8(G) ;// Refresh pressure predictor
1475            step9(G) ;// Refresh velocity predictor, convective coefficients, and interface ratios
1476        }
1477        if(!CE_physical){
1478            cout << "Haved converged at step G = " << G << endl ;
1479            cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)  <<
                 endl ;
1480            cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
1481            output_combined_fields(min_G) ;
1482        }
1483        cout << "Grid Size : " << Nx << "x" << Ny << "Completed." << endl ;
1484        return 0 ;
1485    }
```

Listing 10: 3.FVM_deferred correction.cpp

A.2.4 FVM Deferred Correction with ADI+TDMA

```cpp
// Solve the Navier-Stokes equations using the Finite Volume Method with the SIMPLE algorithm
// Add deferred correction and use alternating iteration
#include <iostream>
#include <fstream>
#include <sstream>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <string>
#include <vector>
#include <omp.h>  // Add after #include <vector>
#include "0.P_correction_eq_TDMA.h"
using namespace std ;
// Parameter settings
// Parameters to be used
// Pressure field has Nx cells in the x-direction
// x-velocity field has Nx+1 cells in the x-direction
// y-velocity field has Ny+1 cells in the y-direction
int G ;  // G is the current calculation step starting from 0, different from iteration step
const int  max_G = 500000 ; // Maximum calculation steps
double min_step7 ;
// x-velocity field: i in [0,Nx], j in [0,Ny-1]
// y-velocity field: i in [0,Nx-1], j in [0,Ny]
// Pressure field: i in [0,Nx-1], j in [0,Ny-1]
double u_star[Nx+1][Ny] ; // Predicted x-velocity field
double u_star_old[Nx+1][Ny] ;
double u_star_old2[Nx+1][Ny] ;
double v_star[Nx][Ny+1] ; // Predicted y-velocity field
double v_star_old[Nx][Ny+1] ;
double v_star_old2[Nx][Ny+1] ;
double p_star[Nx][Ny] ; // Predicted pressure field
double u_prime[Nx+1][Ny] ; // Corrected x-velocity field
double v_prime[Nx][Ny+1] ; // Corrected y-velocity field
double p_prime[Nx][Ny] ; // Corrected pressure field
double u[Nx+1][Ny] ; // Correct x-velocity field
// Create matrix for old correct x-velocity field
double v[Nx][Ny+1] ; // Correct y-velocity field
// Create matrix for old correct y-velocity field
double p[Nx][Ny] ; // Correct pressure field

const double  Re = 5000.0 ;
// Boundary conditions for x-velocity field
const double u_left  = 0.0 ; // No-penetration wall condition
const double u_right = 0.0 ; // No-penetration wall condition
const double u_bottom = 0.0 ; // No-slip condition
const double u_top = 1.0 ; // No-slip condition
// Boundary conditions for y-velocity field
const double v_left  = 0.0 ; // No-slip condition
const double v_right = 0.0 ; // No-slip condition
const double v_bottom = 0.0 ; // No-penetration wall condition
const double v_top = 0.0 ; // No-penetration wall condition
const double omega_P = 0.09 ; // According to Patankar's approach, the relaxation factor for pressure: alpha_P = 0.8
double omega_u  = 0.01 ;
double omega_v  = 0.01 ;
double max_error ;
const float CE_tolerance = 1e-10; // Convergence criterion: continuity equation for predicted values
bool CE_physical ;
//
// Diffusion coefficients of the discretized equations (shared by x and y components)
const double D_e = (1.0/Re)*(dy/dx);
const double D_w = (1.0/Re)*(dy/dx);
const double D_n = (1.0/Re)*(dy/dx);
const double D_s = (1.0/Re)*(dy/dx);
double r_w_u[Nx+1][Ny] ; // Interface decision coefficient for x-velocity field
double r_e_u[Nx+1][Ny] ;
double r_s_u[Nx+1][Ny] ;
double r_n_u[Nx+1][Ny] ;
double r_w_v[Nx][Ny+1] ; // Interface decision coefficient for y-velocity field
double r_e_v[Nx][Ny+1] ;
double r_s_v[Nx][Ny+1] ;
double r_n_v[Nx][Ny+1] ;
```

```
72 // Convection coefficients of the general discretized form of the x-momentum integral equation
       // The factor alpha is determined by F
73 double F_w_u[Nx+1][Ny] ;
74 double alpha_w_u[Nx+1][Ny] ;
75 double F_e_u[Nx+1][Ny] ;
76 double alpha_e_u[Nx+1][Ny] ;
77 double F_s_u[Nx+1][Ny] ;
78 double alpha_s_u[Nx+1][Ny] ;
79 double F_n_u[Nx+1][Ny] ;
80 double alpha_n_u[Nx+1][Ny] ;
81 // Linear combination coefficients of the general discretized form of the x-momentum integral
       equation
82 double a_P[Nx+1][Ny] ;
83 double a_WW[Nx+1][Ny] ;
84 double a_W[Nx+1][Ny] ;
85 double a_E[Nx+1][Ny] ;
86 double a_EE[Nx+1][Ny] ;
87 double a_SS[Nx+1][Ny] ;
88 double a_S[Nx+1][Ny] ;
89 double a_N[Nx+1][Ny] ;
90 double a_NN[Nx+1][Ny] ;
91 double S1[Nx+1][Ny] ; // Inhomogeneous term of the x-momentum equation
92 // Convection coefficients of the general discretized form of the y-momentum integral equation
       // The factor alpha is determined by F
93 double F_w_v[Nx][Ny+1] ;
94 double alpha_w_v[Nx][Ny+1] ;
95 double F_e_v[Nx][Ny+1] ;
96 double alpha_e_v[Nx][Ny+1] ;
97 double F_s_v[Nx][Ny+1] ;
98 double alpha_s_v[Nx][Ny+1] ;
99 double F_n_v[Nx][Ny+1] ;
100 double alpha_n_v[Nx][Ny+1] ;
101 // Linear combination coefficients of the general discretized form of the y-momentum integral
       equation
102 double b_P[Nx][Ny+1] ;
103 double b_WW[Nx][Ny+1] ;
104 double b_W[Nx][Ny+1] ;
105 double b_E[Nx][Ny+1] ;
106 double b_EE[Nx][Ny+1] ;
107 double b_SS[Nx][Ny+1] ;
108 double b_S[Nx][Ny+1] ;
109 double b_N[Nx][Ny+1] ;
110 double b_NN[Nx][Ny+1] ;
111 double S2[Nx][Ny+1] ; // Inhomogeneous term of the y-momentum equation
112 // Remember to update these matrices immediately after the predicted velocity field is updated
113
114
115 //(1)
116 // CDs upstream and downstream iteration coefficients
117 double D(double x){
118   return 0.5  ;
119 }
120 double U(double x){
121     return 0.5 ;
122 }
123 double UU(double x){
124   return 0.0 ;
125
126 }
127
128 /*//(2)
129 // QUICK upstream and downstream iteration coefficients
130 double D(double x){
131   return 3.0/8.0     ;
132 }
133 double U(double x){
134     return 6.0/8.0 ;
135 }
136 double UU(double x){
137   return -1.0/8.0 ;
138
139 }*/
140
141
142 /*//(3)TVD-MUSCL upstream and downstream iteration coefficients
143 double D(double x) {
```

```
144     if (x <= 0) return 0.0;
145     else if (x > 0 && x < (1.0/3.0)) return 0.0;
146     else if (x >= (1.0/3.0) && x < 3.0) return 0.25;
147     else return 1.0;
148 }
149
150 double U(double x) {
151     if (x <= 0) return 1.0;
152     else if (x > 0 && x <= (1.0/3.0)) return 2.0;
153     else if (x > (1.0/3.0) && x <= 3.0) return 1.0;
154     else return 0.0;
155 }
156
157 double UU(double x) {
158     if (x <= 0) return 0.0;
159     else if (x > 0 && x < (1.0/3.0)) return -1.0;
160     else if (x >= (1.0/3.0) && x < 3.0) return -0.25;
161     else return 0.0;
162 }*/
163
164
165 // SIMPLE algorithm:
166 // step1: Initialize predicted x-velocity field, predicted y-velocity field, and predicted
        pressure field
167 void initialize(){
168     #pragma omp parallel  // *** Need to add this line! ***
169     {
170     // Initialize predicted x-velocity field
171      #pragma omp parallel for collapse(2)
172     for(int i = 1 ; i <= Nx+1 ; i++){
173         // Domain of predicted x-velocity field: i in [0,Nx], j in [0,Ny-1]
174         for(int J= 1 ; J <= Ny ; J++){
175             u_star[i-1][J-1] = 0.0 ;
176             if(i == 1) u_star[i-1][J-1] = u_left ; // No-penetration wall condition
177             if(i == Nx + 1) u_star[i-1][J-1] = u_right ; // No-penetration wall condition
178         }
179     }
180     // Initialize old correct x-velocity field -> assigned from initial predicted values, later
            updated from true correct values
181      #pragma omp parallel for collapse(2)
182     for(int i = 1 ; i <= Nx+1 ; i++){
183         for(int J = 1 ; J <= Ny ; J++){
184             u_star_old[i-1][J-1] = u_star[i-1][J-1] ;
185             u_star_old2[i-1][J-1] = u_star_old[i-1][J-1];
186         }
187     }
188     // Initialize predicted y-velocity field
189      #pragma omp parallel for collapse(2)
190     for(int I = 1 ; I <= Nx ; I++){
191         // Domain of predicted y-velocity field: I in [0,Nx-1], j in [0,Ny]
192         for(int j = 1 ; j <= Ny+1 ; j++){
193             v_star[I-1][j-1] = 0.0 ;
194             if(j == 1)v_star[I-1][j-1] = v_bottom ; // No-penetration wall condition
195             if(j == Ny + 1)v_star[I-1][j-1] = v_top ; // No-penetration wall condition
196         }
197     }
198     // Initialize old correct y-velocity field -> assigned from initial predicted values, later
            updated from true correct values
199      #pragma omp parallel for collapse(2)
200     for(int I = 1 ; I <= Nx ; I++){
201         for(int j = 1 ; j <= Ny+1 ; j++){
202             v_star_old[I-1][j-1] = v_star[I-1][j-1] ;
203             v_star_old2[I-1][j-1] =v_star_old[I-1][j-1] ;
204         }
205     }
206     // Initialize predicted pressure field
207      #pragma omp parallel for collapse(2)
208     for(int I = 1 ; I <= Nx ; I++){
209         for(int J = 1 ; J<= Ny ; J++){
210             p_star[I-1][J-1] =  0.0  ; // Boundary values follow a Neumann condition enforced
                    by the iterative scheme
211             p_prime[I-1][J-1] = 0.0 ;
212         }
213     }
214     // Matrices required for the predictor equations: interface ratios -> convective fluxes ->
            linear-combination coefficients
```

```
215     // Initialization (and subsequent updates) must follow that order: interface ratios,
            convective terms, then linear coefficients
216     // Initialize interface ratio matrices
217     // x-velocity interface ratios (north and south faces require special handling)
218      #pragma omp parallel for collapse(2)
219     for(int i = 1 ; i <= Nx+1 ; i++){
220         for(int J = 1 ; J <= Ny ; J++){
221             // Cells that fall outside the upwind stencil simply set ratios to zero
222             if(i == 1 || i == Nx+1){
223                 r_w_u[i-1][J-1] = 0.0 ;
224                 r_e_u[i-1][J-1] = 0.0 ;
225                 r_n_u[i-1][J-1] = 0.0 ;
226                 r_s_u[i-1][J-1] = 0.0 ;
227             }else {
228                 // Handle each boundary separately (west, east, south, north)
229                 // West boundary
230                 if(i == 2){
231                     r_w_u[i-1][J-1] = -1.0 ; // West boundary uses upwind interpolation
232                 }else{
233                     r_w_u[i-1][J-1] = 0.0 ; // Default initialization
234                 }
235                 // East boundary
236                 if(i == Nx){
237                     r_e_u[i-1][J-1] = -1.0 ; // East boundary uses upwind interpolation
238                 }else{
239                     r_e_u[i-1][J-1] = 0.0 ;// Default initialization
240                 }
241                 // South boundary
242                 if(J == 1){
243                     r_s_u[i-1][J-1] = 0.0 ; // Bottom-most cells do not use a south-face ratio
244                 }else if(J == 2){
245                     r_s_u[i-1][J-1] = -1.0 ;// Upwind scheme with safeguards against division
                            by zero
246                 }else if(J == Ny){
247                     r_s_u[i-1][J-1] = -1.0 ;// Upwind scheme with safeguards against division
                            by zero
248                 }else{
249                     r_s_u[i-1][J-1] = 0.0 ;// Default initialization
250                 }
251                 // North boundary
252                 if(J == 1){
253                     r_n_u[i-1][J-1] = -1.0 ; // Upwind scheme with safeguards against division
                            by zero
254                 }else if (J == Ny-1){
255                     r_n_u[i-1][J-1] = -1.0 ;// Upwind scheme with safeguards against division
                            by zero
256                 }else if(J == Ny){
257                     r_n_u[i-1][J-1] = 0.0 ; // Top-most cells do not use a north-face ratio
258                 }else{
259                     r_n_u[i-1][J-1] = 0.0 ;// Default initialization
260                 }
261             }
262         }
263     }// Remember to refresh interface ratios immediately after updating predictors
264     // y-velocity interface ratios (east/west faces require special handling)
265      #pragma omp parallel for collapse(2)
266     for(int I = 1 ; I <= Nx ; I++){
267         for(int j = 1 ; j <= Ny+1 ; j++){
268             // Cells that fall outside the upwind stencil simply set ratios to zero
269             if(j == 1 || j == Ny+1){
270                 r_w_v[I-1][j-1] = 0.0 ;
271                 r_e_v[I-1][j-1] = 0.0 ;
272                 r_n_v[I-1][j-1] = 0.0 ;
273                 r_s_v[I-1][j-1] = 0.0 ;
274             }else {
275                 // Handle each boundary in turn (south, north, west, east)
276                 // South boundary
277                 if(j == 2){
278                     r_s_v[I-1][j-1] = -1.0 ; // Bottom boundary uses upwind interpolation
279                 }else{
280                     r_s_v[I-1][j-1] = 0.0 ; // Default initialization
281                 }
282                 // North boundary
283                 if(j == Ny){
284                     r_n_v[I-1][j-1] = -1.0 ; // Top boundary uses upwind interpolation
285                 }else{
```

```
286                     r_n_v[I-1][j-1] = 0.0 ; // Default initialization
287                 }
288                 // West boundary
289                 if(I == 1){
290                     r_w_v[I-1][j-1] = 0.0 ; // West-most cells do not require a west-face ratio
291                 }else if (I == 2){
292                     r_w_v[I-1][j-1] = -1.0 ;// Upwind scheme with safeguards against division
                            by zero
293                 }else if(I == Nx){
294                     r_w_v[I-1][j-1] = -1.0 ;// Upwind scheme with safeguards against division
                            by zero
295                 }else{
296                     r_w_v[I-1][j-1] = 0.0 ;// Default initialization
297                 }
298                 // East boundary
299                 if(I == 1){
300                     r_e_v[I-1][j-1] = -1.0 ; // Upwind scheme with safeguards against division
                            by zero
301                 }else if (I == Nx-1){
302                     r_e_v[I-1][j-1] = -1.0 ; // Upwind scheme with safeguards against division
                            by zero
303                 }else if(I == Nx){
304                     r_e_v[I-1][j-1] = 0.0 ; // East-most cells do not require an east-face
                            ratio
305                 }else {
306                     r_e_v[I-1][j-1] = 0.0 ; // Default initialization
307                 }
308             }
309         }
310     }
311     // Initialize the convective-flux coefficient matrices
312     // x-momentum convective coefficients in the discrete form
313      #pragma omp parallel for collapse(2)
314     for(int i = 1 ; i <= Nx+1 ; i++){
315         for(int J = 1 ; J <= Ny ; J++){
316             if(i == 1 || i == Nx+1) {
317                 F_w_u[i-1][J-1] = 0.0 ;
318                 F_e_u[i-1][J-1] = 0.0 ;
319                 F_n_u[i-1][J-1] = 0.0 ;
320                 F_s_u[i-1][J-1] = 0.0 ;
321             }else{
322                 F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); // Control-volume
                        centers align with storage locations
323                 F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
324                 F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
325                 F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
326             }
327         }
328     }// alpha_f_u flags are stored at the centers of control volumes
329      #pragma omp parallel for collapse(2)
330     for(int i = 1 ; i <= Nx+1 ; i++){
331         for(int J = 1 ; J <= Ny ; J++){
332             if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
333             else alpha_w_u[i-1][J-1] = 0.0 ;
334             if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
335             else alpha_e_u[i-1][J-1] = 0.0 ;
336             if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
337             else alpha_n_u[i-1][J-1] = 0.0 ;
338             if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
339             else alpha_s_u[i-1][J-1] = 0.0 ;
340         }
341     }
342     // y-momentum convective coefficients in the discrete form
343      #pragma omp parallel for collapse(2)
344     for(int I = 1 ; I <= Nx ; I++){
345         for(int j = 1 ; j <= Ny+1 ; j++){
346             if(j == 1 || j == Ny+1) {
347                 F_w_v[I-1][j-1] = 0.0 ;
348                 F_e_v[I-1][j-1] = 0.0 ;
349                 F_n_v[I-1][j-1] = 0.0 ;
350                 F_s_v[I-1][j-1] = 0.0 ;
351             }else{
352                 F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
353                 F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
354                 F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);// Control-volume
                        centers align with storage locations
```

```
355                 F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
356             }
357         }
358     }// alpha_f_v flags are stored at the centers of control volumes
359      #pragma omp parallel for collapse(2)
360     for(int I = 1 ; I <= Nx ; I++){
361         for(int j = 1 ; j <= Ny+1 ; j++){
362             if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
363             else alpha_w_v[I-1][j-1] = 0.0 ;
364             if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
365             else alpha_e_v[I-1][j-1] = 0.0 ;
366             if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
367             else alpha_n_v[I-1][j-1] = 0.0 ;
368             if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
369             else alpha_s_v[I-1][j-1] = 0.0 ;
370         }
371     }
372 }
373 }
374
375 //step2: update the predicted velocity fields (u and v components)
376 //step2: update the predicted velocity fields (u and v components)
377 void step2(int m){
378     //update the linear combination coefficients for the predicted u-momentum equation
379     #pragma omp parallel for collapse(2)
380     for(int i = 1 ; i <= Nx+1 ; i++){
381         for(int J = 1 ; J <= Ny ; J++){
382             //reset all neighbor coefficients to zero before accumulation
383             a_P[i-1][J-1] = 0.0 ;
384             a_WW[i-1][J-1] = 0.0 ;
385             a_W[i-1][J-1] = 0.0 ;
386             a_E[i-1][J-1] = 0.0 ;
387             a_EE[i-1][J-1] = 0.0 ;
388             a_SS[i-1][J-1] = 0.0 ;
389             a_S[i-1][J-1] = 0.0 ;
390             a_N[i-1][J-1] = 0.0 ;
391             a_NN[i-1][J-1] = 0.0 ;
392             S1[i-1][J-1] = 0.0 ;
393             if(i == 1){
394                 a_P[i-1][J-1] += 1.0 ;
395                 S1[i-1][J-1] += u_left ;//no-slip wall; west boundary has zero pressure
                        gradient
396                 u_star[i-1][J-1] = u_left ;
397             }else if(i == Nx+1){
398                 a_P[i-1][J-1] += 1.0 ;
399                 S1[i-1][J-1] += u_right ;//no-slip wall; east boundary has zero pressure
                        gradient
400                 u_star[i-1][J-1] = u_right ;
401             }// Special cases: west-column second row, east-column second row, and first/second
                    rows near bottom/top boundaries
402             // Second column adjacent to the west boundary
403             else{ // For all other interior cells, handle west/east/south/north faces
                    individually
404                 // West boundary
405                 if(i == 2){// Second column next to the west boundary uses an upwind scheme on
                        the west face
406                     a_W[i-1][J-1] += alpha_w_u[i-1][J-1]*(-F_w_u[i-1][J-1])-D_w;// Subtract
                            west-face diffusion when applying boundary treatment
407                     a_P[i-1][J-1] += (1-alpha_w_u[i-1][J-1])*(-F_w_u[i-1][J-1])+D_w;
408                     S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho ;
409                 }else{// General west-face upwind stencil involves W and P
410                     //a_WW[i-1][J-1] += (alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i-1][
                            J-1]) ;
411                     a_W[i-1][J-1] += (alpha_w_u[i-1][J-1]*1)*(-F_w_u[i-1][J-1])-D_w ; //
                            Subtract west-face diffusion when applying boundary treatment
412                     //a_E[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i
                            -1][J-1]);
413                     a_P[i-1][J-1] += ((1-alpha_w_u[i-1][J-1])*1)*(-F_w_u[i-1][J-1])+D_w ;
414                     S1[i-1][J-1] += -(-p_star[i-2][J-1])*dy/rho
415                     -u_star_old[i-3][J-1]*(alpha_w_u[i-1][J-1]*UU(r_w_u[i-1][J-1]))*(-F_w_u[i
                            -1][J-1])
416                     -u_star_old[i-2][J-1]*((1-alpha_w_u[i-1][J-1])*D(r_w_u[i-1][J-1])+alpha_w_u
                            [i-1][J-1]*(U(r_w_u[i-1][J-1])-1))*(-F_w_u[i-1][J-1])
417                     -u_star_old[i][J-1]*((1-alpha_w_u[i-1][J-1])*UU(r_w_u[i-1][J-1]))*(-F_w_u[i
                            -1][J-1])
```

```
418                     -u_star_old[i-1][J-1]*((1-alpha_w_u[i-1][J-1])*(U(r_w_u[i-1][J-1])-1)+
                            alpha_w_u[i-1][J-1]*D(r_w_u[i-1][J-1]))*(-F_w_u[i-1][J-1])
419                     ;
420                 }// Deferred correction complete
421                 // East boundary
422                 if(i == Nx){// Second column next to the east boundary uses an upwind scheme on
                        the east face
423                     a_E[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*(F_e_u[i-1][J-1])-D_e ;// Subtract
                            east-face diffusion when applying boundary treatment
424                     a_P[i-1][J-1] += alpha_e_u[i-1][J-1]*(F_e_u[i-1][J-1])+D_e;
425                     S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho ;
426                 }else{// General east-face upwind stencil involves E and P
427                     //a_EE[i-1][J-1] += (1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i
                            -1][J-1]) ;
428                     a_E[i-1][J-1] += ((1-alpha_e_u[i-1][J-1])*1)*(F_e_u[i-1][J-1])-D_e ;//
                            Subtract east-face diffusion when applying boundary treatment
429                     //a_W[i-1][J-1] += (alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i-1][J
                            -1]) ;
430                     a_P[i-1][J-1] += (alpha_e_u[i-1][J-1]*1)*(F_e_u[i-1][J-1])+D_e;
431                     S1[i-1][J-1] += -(p_star[i-1][J-1])*dy/rho
432                     -u_star_old[i+1][J-1]*(1-alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i
                            -1][J-1])//EE
433                     -u_star_old[i][J-1]*(alpha_e_u[i-1][J-1]*D(r_e_u[i-1][J-1])+(1-alpha_e_u[i
                            -1][J-1])*(U(r_e_u[i-1][J-1])-1))*(F_e_u[i-1][J-1])//E
434                     -u_star_old[i-2][J-1]*(alpha_e_u[i-1][J-1])*UU(r_e_u[i-1][J-1])*(F_e_u[i
                            -1][J-1])//W
435                     -u_star_old[i-1][J-1]*((1-alpha_e_u[i-1][J-1])*D(r_e_u[i-1][J-1])+alpha_e_u
                            [i-1][J-1]*(U(r_e_u[i-1][J-1])-1))*(F_e_u[i-1][J-1])//p
436                     ;
437                 }// Deferred correction complete
438                 // South boundary: treat three different rows
439                 if(J == 1){// First row adjacent to the bottom boundary
440                     a_P[i-1][J-1] += 2*D_s ;
441                     S1[i-1][J-1] += -(u_bottom)*(-F_s_u[i-1][J-1])+2*D_s*(u_bottom) ;// No-slip
                            condition; south boundary pressure equals the cell value via the
                            Neumann condition
442                 }else if(J == 2){// Second row just above the bottom boundary uses
                        extrapolation instead of the SS node; south-face upwind couples S and P
443                     //a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][
                            J-1]) ;
444                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s ;//
                            Subtract south-face diffusion when applying boundary treatment
445                     //a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
446                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])+D_s ;
447                     S1[i-1][J-1] +=
448                     -(2*u_bottom - u_star_old[i-1][J-2])*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J
                            -1]))*(-F_s_u[i-1][J-1]) //SS
449                     -u_star_old[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(
                            alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])//S
450                     -u_star_old[i-1][J]*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1])//N
451                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+
                            alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])//P
452                     ;// Deferred correction complete
453                 }else if(J == Ny){
454                     // Extrapolate the north node back to the current cell
455                      //a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
456                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s ;//
                            Subtract south-face diffusion when applying boundary treatment
457                     //a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
458                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])+D_s ;
459                     S1[i-1][J-1] +=
460                     -u_star_old[i-1][J-3]*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) //SS
461                     -u_star_old[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(
                            alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])//S
462                     -(2*u_top -u_star_old[i-1][J-1])*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J
                            -1]))*(-F_s_u[i-1][J-1])//N
463                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+
                            alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])//P
464                     ;// Deferred correction complete
465                 }else{// General south-face upwind stencil involves S and P
```

```
466                     //a_SS[i-1][J-1] += (alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i-1][
                            J-1]) ;
467                     a_S[i-1][J-1] += ((alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])-D_s ;//
                            Subtract south-face diffusion when applying boundary treatment
468                     //a_N[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) ;
469                     a_P[i-1][J-1] += ((1-alpha_s_u[i-1][J-1])*1)*(-F_s_u[i-1][J-1])+D_s ;
470                     S1[i-1][J-1] +=
471                     -u_star_old[i-1][J-3]*(alpha_s_u[i-1][J-1]*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1]) //SS
472                     -u_star_old[i-1][J-2]*((1-alpha_s_u[i-1][J-1])*D(r_s_u[i-1][J-1])+(
                            alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1))*(-F_s_u[i-1][J-1])//S
473                     -u_star_old[i-1][J]*((1-alpha_s_u[i-1][J-1])*UU(r_s_u[i-1][J-1]))*(-F_s_u[i
                            -1][J-1])//N
474                     -u_star_old[i-1][J-1]*((1-alpha_s_u[i-1][J-1])*(U(r_s_u[i-1][J-1])-1)+
                            alpha_s_u[i-1][J-1]*D(r_s_u[i-1][J-1]))*(-F_s_u[i-1][J-1])//P
475                     ;// Deferred correction complete
476                 }
477                 // North boundary requires three special cases: J == 1, J == Ny, and J == Ny-1
478                 if(J == Ny){
479                     a_P[i-1][J-1] += 2*D_n ;
480                     S1[i-1][J-1] += -(u_top)*F_n_u[i-1][Ny-1] + 2*D_n*(u_top);// No-slip
                            condition; north boundary pressure equals the cell value via the
                            Neumann condition
481                 }else if(J == 1){
482                      //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
483                     a_N[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) - D_n ;//
                            Subtract north-face diffusion when applying boundary treatment
484                     //a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
485                     a_P[i-1][J-1] += (alpha_n_u[i-1][J-1]*1)*(F_n_u[i-1][J-1])+D_n ;
486                     S1[i-1][J-1] +=
487                     -u_star_old[i-1][J+1]*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
488                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
489                     -(2*u_bottom - u_star_old[i-1][J-1])*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J
                            -1])*(F_n_u[i-1][J-1])
490                     -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u
                            [i-1][J-1]*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
491                     ;
492                 }else if(J == Ny-1){
493                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
494                     a_N[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) - D_n ;//
                            Subtract north-face diffusion when applying boundary treatment
495                     //a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
496                     a_P[i-1][J-1] += (alpha_n_u[i-1][J-1]*1)*(F_n_u[i-1][J-1])+D_n ;
497                     S1[i-1][J-1] +=
498                     -(2*u_top - u_star_old[i-1][J])*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])
                            *(F_n_u[i-1][J-1])
499                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
500                     -u_star_old[i-1][J-2]*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
501                     -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+alpha_n_u
                            [i-1][J-1]*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
502                     ;
503                 }else{
504                     //a_NN[i-1][J-1] += (1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1]) ;
505                     a_N[i-1][J-1] += ((1-alpha_n_u[i-1][J-1])*1)*(F_n_u[i-1][J-1]) - D_n ;//
                            Subtract north-face diffusion when applying boundary treatment
506                     //a_S[i-1][J-1] += (alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i-1][J
                            -1]) ;
507                     a_P[i-1][J-1] += (alpha_n_u[i-1][J-1]*1)*(F_n_u[i-1][J-1])+D_n ;
508                     S1[i-1][J-1] +=
509                     -u_star_old[i-1][J+1]*(1-alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
510                     -u_star_old[i-1][J]*(alpha_n_u[i-1][J-1]*D(r_n_u[i-1][J-1])+(1-alpha_n_u[i
                            -1][J-1])*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
511                     -u_star_old[i-1][J-2]*(alpha_n_u[i-1][J-1])*UU(r_n_u[i-1][J-1])*(F_n_u[i
                            -1][J-1])
```

```
512                             -u_star_old[i-1][J-1]*((1-alpha_n_u[i-1][J-1])*D(r_n_u[i-1][J-1])+
                                    alpha_n_u[i-1][J-1]*(U(r_n_u[i-1][J-1])-1))*(F_n_u[i-1][J-1])
513                     ;
514                  }
515             }
516         }
517     }
518     //update the linear combination coefficients for the predicted v-momentum equation
519     #pragma omp parallel default(shared)
520     {
521         #pragma omp for schedule(static) //average cutting workking for threads
522         for(int J = 1 ; J <= Ny ; J++){//define the index from 0~Nx-2
523         vector <double> beta (Nx-1) ;//left hand side elemets
524         vector <double> D (Nx-1) ; //primary elements
525         vector <double> alpha (Nx-1) ; //right and side elements
526         vector <double> x (Nx-1) ; //solutions
527         vector <double> S (Nx-1) ; //Source Vector
528         for(int k = 2 ; k <= Nx ; k++){
529             beta[k-2] = a_W[k-1][J-1] ;
530             D[k-2] = a_P[k-1][J-1] ;
531             alpha[k-2] = a_E[k-1][J-1] ;
532             x[k-2] = u_star[k-1][J-1] ;
533             if(J == 1){S[k-2] = S1[k-1][J-1] -u_star_old[k-1][J]*a_N[k-1][J-1];}
534             else if(J == Ny){ S[k-2] = S1[k-1][J-1]-u_star_old[k-1][J-2]*a_S[k-1][J-1] ; }
535             else {S[k-2] = S1[k-1][J-1]-u_star_old[k-1][J-2]*a_S[k-1][J-1]-u_star_old[k-1][J]*
                    a_N[k-1][J-1]; ; }
536         }
537         TDMA(D , beta , alpha , x , S , Nx-1 , u_star[1-1][J-1] , u_star[Nx+1-1][J-1]) ;
538             for(int i = 2 ; i <= Nx ; i++){
539                 u_star[i-1][J-1] = x[i-2] ;
540             }
541         }
542         #pragma omp for collapse(2) schedule(static) //average cutting workking for threads
543         for(int i = 1 ; i <= Nx+1 ; i++){
544             for(int J = 1 ; J <= Ny ; J++){
545                 u_star_old2[i-1][J-1] = u_star[i-1][J-1] ;
546             }
547         }//Global elements update to the old value2
548         //ADI-2 scaning direction from bottom to top and from left to right
549         //parameter setting : beta , D , alpha x , S , U_left , U_right , n
550         #pragma omp for schedule(static) //average cutting workking for threads
551         for(int i = 2 ; i <=Nx ; i++){//define the index from 1 ~ Ny
552             vector <double> beta (Ny) ; vector <double> D (Ny) ; vector <double> alpha (Ny) ;
553             vector <double> x (Ny) ; vector <double> S (Ny) ; //we would not worry about the
                    index > Ny because we haved set the extrapolation
554             for(int k = 1 ; k <= Ny ; k++){
555             beta[k-1] = a_S[i-1][k-1] ;
556             D[k-1] = a_P[i-1][k-1] ;
557             alpha[k-1] = a_N[i-1][k-1] ;
558             x[k-1] = u_star[i-1][k-1] ;
559             S[k-1] = S1[i-1][k-1]-u_star_old2[i-2][k-1]*a_W[i-1][k-1]-u_star_old2[i][k-1]*a_E[i
                    -1][k-1];
560             }
561             TDMA_FVM(D , beta , alpha , x , S , Ny) ;
562             for(int J = 1 ; J <= Ny ; J++){
563                 u_star[i-1][J-1] = x[J-1] ;
564             }
565         }
566         #pragma omp for collapse(2) schedule(static)
567         for(int i = 2 ; i <= Nx ; i++){
568             for(int J = 1 ; J <= Ny ; J++){
569                 u_star[i-1][J-1] = omega_u*u_star[i-1][J-1] + (1-omega_u)*u_star_old[i-1][J-1]
                        ;
570             }
571         }
572     }
573     #pragma omp parallel for collapse(2)
574     for(int I = 1 ; I <= Nx ; I++){
575         for(int j = 1 ; j <= Ny+1 ; j++){
576             //reset all neighbor coefficients to zero before accumulation
577             b_P[I-1][j-1] = 0.0 ;
578             b_WW[I-1][j-1] = 0.0 ;
579             b_W[I-1][j-1] = 0.0 ;
580             b_E[I-1][j-1] = 0.0 ;
581             b_EE[I-1][j-1] = 0.0 ;
582             b_SS[I-1][j-1] = 0.0 ;
```

```
583             b_S[I-1][j-1] = 0.0 ;
584             b_N[I-1][j-1] = 0.0 ;
585             b_NN[I-1][j-1] = 0.0 ;
586             S2[I-1][j-1] = 0.0 ;
587             if(j == 1){
588                 b_P[I-1][j-1] += 1.0 ;
589                 S2[I-1][j-1] += v_bottom ;//no-slip wall; bottom boundary has zero pressure
                        gradient
590                 v_star[I-1][j-1] = v_bottom ;
591             }else if(j == Ny+1){
592                 b_P[I-1][j-1] += 1.0 ;
593                 S2[I-1][j-1] += v_top ;//no-slip wall; top boundary has zero pressure gradient
594                 v_star[I-1][j-1] = v_top ;
595             }//special handling needed for first/second columns near west and east walls as
                    well as second rows near top and bottom
596             //second row next to the bottom boundary
597             else{ //all remaining cells are processed by checking west, east, south, and north
                    faces individually
598                 //south boundary
599                 if(j == 2){//second row above the bottom boundary uses upwind on the south face
600                     b_S[I-1][j-1] += alpha_s_v[I-1][j-1]*(-F_s_v[I-1][j-1])-D_s;//subtract
                            south-face diffusion when enforcing the boundary
601                     b_P[I-1][j-1] += (1-alpha_s_v[I-1][j-1])*(-F_s_v[I-1][j-1])+D_s;
602                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho ;
603                 }else{//interior south faces use the TVD-MUSCL stencil that involves the south
                        node and the current cell
604                     //b_SS[I-1][j-1] += (alpha_s_v[I-1][j-1]*UU(r_s_v[I-1][j-1]))*(-F_s_v[I-1][
                            j-1]) ;
605                     b_S[I-1][j-1] += (alpha_s_v[I-1][j-1]*1)*(-F_s_v[I-1][j-1])-D_s ; //
                            subtract south-face diffusion when enforcing the boundary
606                     //b_N[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1]) ;
607                     b_P[I-1][j-1] += ((1-alpha_s_v[I-1][j-1])*1)*(-F_s_v[I-1][j-1])+D_s ;
608                     S2[I-1][j-1] += -(-p_star[I-1][j-2])*dx/rho
609                     -v_star_old[I-1][j-3]*(alpha_s_v[I-1][j-1]*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1]) //SS
610                     -v_star_old[I-1][j-2]*((1-alpha_s_v[I-1][j-1])*D(r_s_v[I-1][j-1])+alpha_s_v
                            [I-1][j-1]*(U(r_s_v[I-1][j-1])-1))*(-F_s_v[I-1][j-1]) //S
611                     -v_star_old[I-1][j]*((1-alpha_s_v[I-1][j-1])*UU(r_s_v[I-1][j-1]))*(-F_s_v[I
                            -1][j-1]) //N
612                     -v_star_old[I-1][j-1]*((1-alpha_s_v[I-1][j-1])*(U(r_s_v[I-1][j-1])-1)+
                            alpha_s_v[I-1][j-1]*D(r_s_v[I-1][j-1]))*(-F_s_v[I-1][j-1]) //P
613                     ;//deferred correction completed
614                 }
615                 //north boundary
616                 if(j == Ny){//second row below the top wall uses upwind on the north face
617                     b_N[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*(F_n_v[I-1][j-1])-D_n ;//subtract
                            north-face diffusion when enforcing the boundary
618                     b_P[I-1][j-1] += alpha_n_v[I-1][j-1]*(F_n_v[I-1][j-1])+D_n;
619                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho ;
620                 }else{//other north faces employ the TVD-MUSCL stencil coupling the current and
                        north cells
621                     //b_NN[I-1][j-1] += (1-alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1]) ;
622                     b_N[I-1][j-1] += ((1-alpha_n_v[I-1][j-1])*1)*(F_n_v[I-1][j-1]) - D_n ;//
                            subtract north-face diffusion when enforcing the boundary
623                     //b_S[I-1][j-1] += (alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I-1][j
                            -1]) ;
624                     b_P[I-1][j-1] += (alpha_n_v[I-1][j-1]*1)*(F_n_v[I-1][j-1]) + D_n ;
625                     S2[I-1][j-1] += -(p_star[I-1][j-1])*dx/rho
626                     -v_star_old[I-1][j+1]*(1-alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1]) //NN
627                     -v_star_old[I-1][j]*(alpha_n_v[I-1][j-1]*D(r_n_v[I-1][j-1])+(1-alpha_n_v[I
                            -1][j-1])*(U(r_n_v[I-1][j-1])-1))*(F_n_v[I-1][j-1]) //N
628                     -v_star_old[I-1][j-2]*(alpha_n_v[I-1][j-1])*UU(r_n_v[I-1][j-1])*(F_n_v[I
                            -1][j-1]) //S
629                     -v_star_old[I-1][j-1]*((1-alpha_n_v[I-1][j-1])*D(r_n_v[I-1][j-1])+alpha_n_v
                            [I-1][j-1]*(U(r_n_v[I-1][j-1])-1))*(F_n_v[I-1][j-1]) //P
630                     ; //deferred correction completed
631                 }
632                 //west boundary (three cases: I = 1, 2, Nx)
633                 if(I == 2){//second column near the west wall uses extrapolation to replace the
                        WW value
634                     //b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1]) ;
```

```
635                     b_W[I-1][j-1] += ((alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])-D_w ; //
                            subtract west-face diffusion when enforcing the boundary
636                     //b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]);
637                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])+D_w ;
638                     S2[I-1][j-1] +=
639                     -(2*v_left - v_star_old[I-2][j-1])*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1])
                            )*(-F_w_v[I-1][j-1]) //WW
640                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(
                            alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1]) //W
641                     -v_star_old[I][j-1]*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) //E
642                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+(
                            alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) //P
643                     ;//deferred correction completed
644                 }else if(I == 1){//first column next to the west wall uses upwind on the west
                        face
645                     b_P[I-1][j-1] +=  2*D_w;//subtract west-face diffusion when enforcing the
                            boundary
646                     S2[I-1][j-1] += -(-F_w_v[I-1][j-1])*v_left+2*D_w*v_left;
647                  }else if(I == Nx){//E->P
648                      //b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) ;
649                     b_W[I-1][j-1] += ((alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])-D_w ; //
                            subtract west-face diffusion when enforcing the boundary
650                     //b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]);
651                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])+D_w ;
652                     S2[I-1][j-1] +=
653                     -v_star_old[I-3][j-1]*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) //WW
654                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(
                            alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1]) //W
655                     -(2*v_right - v_star_old[I-1][j-1])*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][
                            j-1]))*(-F_w_v[I-1][j-1]) //E
656                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+(
                            alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) //P
657                     ;//deferred correction completed
658                 }else{//remaining west faces use the TVD-MUSCL stencil
659                     //b_WW[I-1][j-1] += (alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I-1][
                            j-1]) ;
660                     b_W[I-1][j-1] += ((alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])-D_w ; //
                            subtract west-face diffusion when enforcing the boundary
661                     //b_E[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]);
662                     b_P[I-1][j-1] += ((1-alpha_w_v[I-1][j-1])*1)*(-F_w_v[I-1][j-1])+D_w ;
663                     S2[I-1][j-1] +=
664                     -v_star_old[I-3][j-1]*(alpha_w_v[I-1][j-1]*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) //WW
665                     -v_star_old[I-2][j-1]*((1-alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1])+(
                            alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1))*(-F_w_v[I-1][j-1]) //W
666                     -v_star_old[I][j-1]*((1-alpha_w_v[I-1][j-1])*UU(r_w_v[I-1][j-1]))*(-F_w_v[I
                            -1][j-1]) //E
667                     -v_star_old[I-1][j-1]*((1-alpha_w_v[I-1][j-1])*(U(r_w_v[I-1][j-1])-1)+(
                            alpha_w_v[I-1][j-1])*D(r_w_v[I-1][j-1]))*(-F_w_v[I-1][j-1]) //P
668                     ;//deferred correction completed
669                 }
670                 //east boundary (three cases: I = 1, Nx-1, Nx)
671                 if(I == 1){//second column near the east wall replaces the EE value by
                        extrapolation (W->P)
672                     //b_EE[I-1][j-1] += (1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                            -1][j-1]) ;
673                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1]) - D_e ;//
                            subtract east-face diffusion when enforcing the boundary
674                     //b_W[I-1][j-1] += (alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                            -1]) ;
675                     b_P[I-1][j-1] += (alpha_e_v[I-1][j-1]*1)*(F_e_v[I-1][j-1]) + D_e ;
676                     S2[I-1][j-1] +=
677                     -v_star_old[I+1][j-1]*(1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                            -1][j-1])//EE
678                     -v_star_old[I][j-1]*(alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                            -1][j-1])*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//E
679                     -(2*v_left - v_star_old[I-1][j-1])*(alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j
                            -1])*(F_e_v[I-1][j-1]) //W
680                     -v_star_old[I-1][j-1]*((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v
                            [I-1][j-1]*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//P
```

```
681                     ;//deferred correction completed
682                 }else if(I == Nx){//second column near the east wall uses upwind on the east
                        face
683                     b_P[I-1][j-1] += 2*D_e ;//subtract east-face diffusion when enforcing the
                        boundary
684                     S2[I-1][j-1] += -(F_e_v[I-1][j-1])*v_right +2*D_e*v_right;
685                 }else if(I == Nx-1){//EE node does not exist; extrapolate to the E location
686                     //b_EE[I-1][j-1] += (1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                        -1][j-1]) ;
687                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1]) - D_e ;//
                        subtract east-face diffusion when enforcing the boundary
688                     //b_W[I-1][j-1] += (alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                        -1]) ;
689                     b_P[I-1][j-1] += (alpha_e_v[I-1][j-1]*1)*(F_e_v[I-1][j-1]) + D_e ;
690                     S2[I-1][j-1] +=
691                     -(2*v_right-v_star_old[I][j-1])*(1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])
                        *(F_e_v[I-1][j-1])//EE
692                     -v_star_old[I][j-1]*(alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                        -1][j-1])*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//E
693                     -v_star_old[I-2][j-1]*(alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                        -1][j-1]) //W
694                     -v_star_old[I-1][j-1]*((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v
                        [I-1][j-1]*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//P
695                     ;//deferred correction completed
696                 }else{//remaining east faces use the TVD-MUSCL stencil
697                     //b_EE[I-1][j-1] += (1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                        -1][j-1]) ;
698                     b_E[I-1][j-1] += ((1-alpha_e_v[I-1][j-1])*1)*(F_e_v[I-1][j-1]) - D_e ;//
                        subtract east-face diffusion when enforcing the boundary
699                     //b_W[I-1][j-1] += (alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I-1][j
                        -1]) ;
700                     b_P[I-1][j-1] += (alpha_e_v[I-1][j-1]*1)*(F_e_v[I-1][j-1]) + D_e ;
701                     S2[I-1][j-1] +=
702                     -v_star_old[I+1][j-1]*(1-alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                        -1][j-1])//EE
703                     -v_star_old[I][j-1]*(alpha_e_v[I-1][j-1]*D(r_e_v[I-1][j-1])+(1-alpha_e_v[I
                        -1][j-1])*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//E
704                     -v_star_old[I-2][j-1]*(alpha_e_v[I-1][j-1])*UU(r_e_v[I-1][j-1])*(F_e_v[I
                        -1][j-1]) //W
705                     -v_star_old[I-1][j-1]*((1-alpha_e_v[I-1][j-1])*D(r_e_v[I-1][j-1])+alpha_e_v
                        [I-1][j-1]*(U(r_e_v[I-1][j-1])-1))*(F_e_v[I-1][j-1])//P
706                     ;//deferred correction completed
707                 }
708             }
709         }
710     }
711     //compute the y-velocity predicted value
712     #pragma omp parallel default(shared) // create the thream team and share the data memory
            for different threads
713     {   //computing range : I = [1,Nx] , j = [2,Ny]
714         //ADI-1 scanning direction is from left to right and from bottom to top
715         #pragma omp for schedule(static) //average cutting the working spaxc for my threads
716         for(int j = 2 ; j <= Ny ; j++){
717             vector <double> beta(Nx) ; vector<double> D(Nx) ; vector<double> alpha(Nx) ;
718             vector <double> x(Nx) ; vector<double> S(Nx) ; //Solving Range from 1 to Nx because
                    it is stiraged at the center of the Grid
719             for(int k = 1 ; k <= Nx ; k++){//For ADI-1 west and east using the new value and
                    north and south using old value
720             beta[k-1] = b_W[k-1][j-1] ;
721             D[k-1] = b_P[k-1][j-1] ;
722             alpha[k-1] = b_E[k-1][j-1] ;
723             x[k-1] = v_star[k-1][j-1] ;
724             S[k-1] = S2[k-1][j-1] -v_star_old[k-1][j-2]*b_S[k-1][j-1]-v_star_old[k-1][j]*b_N[k
                    -1][j-1] ;
725             }
726             TDMA_FVM(D , beta , alpha , x , S , Nx ) ;
727             for(int I = 1 ; I <= Nx ; I++){
728                 v_star[I-1][j-1] = x[I-1] ;
729             }
730         }
731         #pragma omp for collapse(2) schedule(static) //average cutting workking for threads
732         for(int I = 1 ; I <= Nx ; I++){
733             for(int j = 1 ; j <= Ny+1 ; j++){
734                 v_star_old2[I-1][j-1] = v_star[I-1][j-1] ;
735             }
736         }
```

```
737         #pragma omp for schedule(static)
738         //ADI-2 from bottom to top and from left to right scaning
739         for(int I = 1 ; I <= Nx ; I++){
740             vector <double> beta(Ny-1) ;
741             vector <double> D(Ny-1) ;
742             vector <double> alpha(Ny-1) ;
743             vector <double> x(Ny-1) ;
744             vector <double> S(Ny-1) ;
745             for(int k = 2 ; k <= Ny ; k++){
746                 beta[k-2] = b_S[I-1][k-1] ;
747                 D[k-2] = b_P[I-1][k-1] ;
748                 alpha[k-2] = b_N[I-1][k-1] ;
749                 x[k-2] = v_star[I-1][k-1] ;
750                 if(I == 1){ S[k-2] = S2[I-1][k-1] - v_star_old2[I][k-1]*b_E[I-1][k-1] ; }
751                 else if(I == Nx){ S[k-2] = S2[I-1][k-1] - v_star_old2[I-2][k-1]*b_W[I-1][k-1] ;
                        }
752                 else{ S[k-2] = S2[I-1][k-1] - v_star_old2[I-2][k-1]*b_W[I-1][k-1] - v_star_old2
                        [I][k-1]*b_E[I-1][k-1] ;}
753
754             }
755             TDMA(D , beta , alpha , x , S , Ny-1 , 0.0 , 0.0 ) ;
756             for(int j = 2 ; j <= Ny ; j++){
757                 v_star[I-1][j-1] = x[j-2]  ;
758             }
759         }
760         #pragma omp for collapse(2) schedule(static) //average cutting workking for threads
761         for(int I = 1 ; I <= Nx ; I++){
762             for(int j = 1 ; j <= Ny+1 ; j++){
763                 v_star[I-1][j-1] = omega_v*v_star[I-1][j-1]+(1-omega_v)*v_star_old[I-1][j-1] ;
764             }
765         }
766     }
767
768 }
769 //u_star and v_star are now available at step N+1/2
770 //Function: mass flux immediately after the first velocity predictor update
771 double d(int I, int J , int m){
772     //I,J reference the cell-center coordinates; m is the pressure iteration counter
773     double u_e = u_star[I+1-1][J-1] ;//velocity predictor is evaluated at N+1/2
774     double u_w = u_star[I-1][J-1] ;//velocity predictor is evaluated at N+1/2
775     double v_n = v_star[I-1][J+1-1] ;//velocity predictor is evaluated at N+1/2
776     double v_s = v_star[I-1][J-1] ;//velocity predictor is evaluated at N+1/2
777     return rho*(u_e - u_w)*dy + rho*(v_n - v_s)*dx ;
778 } //used after the first update of the velocity predictor
779
780
781 //step4: update the corrected pressure field
782 void step4(int m){
783     #pragma omp parallel for collapse(2)
784     for(int I = 1 ; I <= Nx ; I++){ //pressure values (corrected, predicted, and corrected)
            reside at cell centers
785         for(int J = 1 ; J <= Ny ; J++){
786            p[I-1][J-1] = p_star[I-1][J-1] + omega_P*p_prime[I-1][J-1] ;
787         }//p_star is evaluated at step N; p_prime at step N+1
788     }//p_prime is evaluated at step N+1
789 }
790
791 //step5: update velocity corrections; boundary velocities are fixed by the boundary conditions
        and should not iterate
792 void step5(int m){
793     #pragma omp parallel for collapse(2)
794     //update correction for the x-velocity component
795     for(int i = 1 ; i <= Nx+1 ; i++){
796         for(int J = 1 ; J <= Ny ; J++){
797             if(i == 1 || i == Nx+1){
798                 u_prime[i-1][J-1] = 0.0 ;//boundary velocities already satisfy the physical
                        constraints
799             }else{//pressure correction: current cell minus the west neighbor
800                 u_prime[i-1][J-1] = -(p_prime[i-1][J-1] - p_prime[i-2][J-1])*dy/(rho*a_P[i-1][J
                        -1]) ; ;
801             }
802
803         }
804     }
805     //update correction for the y-velocity component
806     #pragma omp parallel for collapse(2)
```

```
807     for(int I = 1 ; I <= Nx ; I++){
808         for(int j = 1 ; j <= Ny+1 ; j++){
809             if(j == 1 || j == Ny+1){
810                 v_prime[I-1][j-1] = 0.0 ;//boundary velocities already satisfy the physical
                        constraints
811             }else{//pressure correction: current cell minus the south neighbor
812                 v_prime[I-1][j-1] = -(p_prime[I-1][j-1] - p_prime[I-1][j-2])*dx/(rho*b_P[I-1][j
                        -1]) ;
813             }
814         }
815     }
816 }
817
818 double step5star(int m){
819     double sum = 0.0 ;
820     #pragma omp parallel for collapse(2) reduction(+:sum)  // accumulate the residuals in
            parallel
821     for(int I = 1 ; I <= Nx ; I++){
822         for(int J = 1 ; J <= Ny ; J++){
823             sum = sum + fabs(d(I,J,m)) ;
824         }
825     }
826     return sum ;
827 }
828
829 //step6: update the corrected velocity fields and refresh the stored old values
830 void step6(int m){//boundary velocities remain fixed by the boundary conditions
831     //update corrected x-velocity
832     #pragma omp parallel for collapse(2)
833     for(int i = 1 ; i <= Nx+1 ; i++){
834         for(int J = 1 ; J <= Ny ; J++){
835             if(i == 1 ){
836                 u[i-1][J-1] = 0.0 ;//boundary velocity imposed by the wall
837             }else if (i == Nx+1){
838                 u[i-1][J-1] = 0.0 ;//boundary velocity imposed by the wall
839             }else{
840                 u[i-1][J-1] = u_prime[i-1][J-1]+u_star[i-1][J-1];//u_star at N+1/2, u_prime at
                        N+1, u_old at N, final u at N+1
841             }
842         }
843     }
844     //update corrected y-velocity
845     #pragma omp parallel for collapse(2)
846     for(int I = 1 ; I <= Nx ; I++){
847         for(int j = 1 ; j <= Ny+1 ; j++){
848             if(j == 1 ){
849                 v[I-1][j-1] = 0.0 ;//impermeable wall
850             }else if (j == Ny+1){
851                 v[I-1][j-1] = 0.0 ;//impermeable wall
852             }else{
853                 v[I-1][j-1] = v_prime[I-1][j-1]+v_star[I-1][j-1];//v_star at N+1/2, v_prime at
                        N+1, v_old at N, final v at N+1
854             }
855         }
856     }
857 }
858 double step7(int m){
859     //81
860
861     double sum = 0.0 ;
862     sum += fabs(u[40][0]-0) ;
863     sum += fabs(u[40][3]-(-0.41165)) ;
864     sum += fabs(u[40][4]-(-0.42901)) ;
865     sum += fabs(u[40][5]-(-0.43643)) ;
866     sum += fabs(u[40][7]-(-0.40435)) ;
867     sum += fabs(u[40][13]-(-0.3305)) ;
868     sum += fabs(u[40][22]-(-0.22855)) ;
869     sum += fabs(u[40][35]-(-0.07404)) ;
870     sum += fabs(u[40][39]-(-0.03039)) ;
871     sum += fabs(u[40][48]-(0.08183)) ;
872     sum += fabs(u[40][58]-(0.20087)) ;
873     sum += fabs(u[40][67]-(0.33556)) ;
874     sum += fabs(u[40][75]-(0.46036)) ;
875     sum += fabs(u[40][76]-(0.45992)) ;
876     sum += fabs(u[40][77]-(0.4612)) ;
877     sum += fabs(u[40][77]-(0.48223)) ;
```

```
878       sum += fabs(u[40][79]-(1)) ;
879       sum += fabs(v[0][40]-0) ;
880       sum += fabs(v[4][40]-(0.42447)) ;
881       sum += fabs(v[5][40]-(0.43329)) ;
882       sum += fabs(v[5][40]-(0.43648)) ;
883       sum += fabs(v[7][40]-(0.42951)) ;
884       sum += fabs(v[12][40]-(0.35368)) ;
885       sum += fabs(v[17][40]-(0.28066)) ;
886       sum += fabs(v[18][40]-(0.2728)) ;
887       sum += fabs(v[39][40]-(0.00945)) ;
888       sum += fabs(v[64][40]-(-0.30018)) ;
889       sum += fabs(v[69][40]-(-0.36214)) ;
890       sum += fabs(v[72][40]-(-0.41442)) ;
891       sum += fabs(v[76][40]-(-0.52876)) ;
892       sum += fabs(v[76][40]-(-0.55408)) ;
893       sum += fabs(v[77][40]-(-0.55069)) ;
894       sum += fabs(v[77][40]-(-0.49774)) ;
895       sum += fabs(v[79][40]-(0.0)) ;
896       /*
897       //113
898       double sum = 0.0 ;
899       sum += fabs(u[56][0]-0) ;
900       sum += fabs(u[56][6]-(-0.41165)) ;
901       sum += fabs(u[56][6]-(-0.42901)) ;
902       sum += fabs(u[56][7]-(-0.43643)) ;
903       sum += fabs(u[56][11]-(-0.40435)) ;
904       sum += fabs(u[56][19]-(-0.3305)) ;
905       sum += fabs(u[56][31]-(-0.22855)) ;
906       sum += fabs(u[56][50]-(-0.07404)) ;
907       sum += fabs(u[56][55]-(-0.03039)) ;
908       sum += fabs(u[56][69]-(0.08183)) ;
909       sum += fabs(u[56][82]-(0.20087)) ;
910       sum += fabs(u[56][95]-(0.33556)) ;
911       sum += fabs(u[56][106]-(0.46036)) ;
912       sum += fabs(u[56][107]-(0.45992)) ;
913       sum += fabs(u[56][108]-(0.4612)) ;
914       sum += fabs(u[56][109]-(0.48223)) ;
915       sum += fabs(u[56][111]-(1)) ;
916
917       sum += fabs(v[0][56]-0) ;
918       sum += fabs(v[6][56]-(0.42447)) ;
919       sum += fabs(v[7][56]-(0.43329)) ;
920       sum += fabs(v[8][56]-(0.43648)) ;
921       sum += fabs(v[10][56]-(0.42951)) ;
922       sum += fabs(v[17][56]-(0.35368)) ;
923       sum += fabs(v[25][56]-(0.28066)) ;
924       sum += fabs(v[26][56]-(0.2728)) ;
925       sum += fabs(v[55][56]-(0.00945)) ;
926       sum += fabs(v[90][56]-(-0.30018)) ;
927       sum += fabs(v[96][56]-(-0.36214)) ;
928       sum += fabs(v[101][56]-(-0.41442)) ;
929       sum += fabs(v[105][56]-(-0.52876)) ;
930       sum += fabs(v[106][56]-(-0.55408)) ;
931       sum += fabs(v[107][56]-(-0.55069)) ;
932       sum += fabs(v[108][56]-(-0.49774)) ;
933       sum += fabs(v[111][56]-(0.0)) ;*/
934       /*
935       //161
936       double sum = 0.0 ;
937           sum += fabs(u[80][0]-0) ;
938       sum += fabs(u[80][8]-(-0.18109)) ;
939       sum += fabs(u[80][9]-(-0.20196)) ;
940       sum += fabs(u[80][10]-(-0.22220)) ;
941       sum += fabs(u[80][15]-(-0.29730)) ;
942       sum += fabs(u[80][27]-(-0.38289)) ;
943       sum += fabs(u[80][44]-(-0.27805)) ;
944       sum += fabs(u[80][71]-(-0.10648)) ;
945       sum += fabs(u[80][79]-(-0.06080)) ;
946       sum += fabs(u[80][98]-(0.05702)) ;
947       sum += fabs(u[80][117]-(0.18719)) ;
948       sum += fabs(u[80][135]-(0.33304)) ;
949       sum += fabs(u[80][151]-(0.46604)) ;
950       sum += fabs(u[80][153]-(0.51117)) ;
951       sum += fabs(u[80][154]-(0.57492)) ;
952       sum += fabs(u[80][155]-(0.65928)) ;
953       sum += fabs(u[80][159]-(1)) ;
```

```
954
955         sum += fabs(v[0][80]-0) ;
956         sum += fabs(v[9][80]-(0.27485)) ;
957         sum += fabs(v[10][80]-(0.29012)) ;
958         sum += fabs(v[12][80]-(0.30353)) ;
959         sum += fabs(v[14][80]-(0.32627)) ;
960         sum += fabs(v[24][80]-(0.37095)) ;
961         sum += fabs(v[35][80]-(0.33075)) ;
962         sum += fabs(v[37][80]-(0.32235)) ;
963         sum += fabs(v[80][80]-(0.02526)) ;
964         sum += fabs(v[129][80]-(-0.31966)) ;
965         sum += fabs(v[137][80]-(-0.42655)) ;
966         sum += fabs(v[145][80]-(-0.51550)) ;
967         sum += fabs(v[151][80]-(-0.39188)) ;
968         sum += fabs(v[152][80]-(-0.33714)) ;
969         sum += fabs(v[154][80]-(-0.27669)) ;
970         sum += fabs(v[155][80]-(-0.21388)) ;
971         sum += fabs(v[159][80]-(0.0)) ;*/
972         /*//227
973         double sum = 0.0 ;
974         sum += fabs(u[113][0]-0) ;
975         sum += fabs(u[113][12]-(-0.41165)) ;
976         sum += fabs(u[113][14]-(-0.42901)) ;
977         sum += fabs(u[113][15]-(-0.43643)) ;
978         sum += fabs(u[113][22]-(-0.40435)) ;
979         sum += fabs(u[113][38]-(-0.3305)) ;
980         sum += fabs(u[113][63]-(-0.22855)) ;
981         sum += fabs(u[113][102]-(-0.07404)) ;
982         sum += fabs(u[113][113]-(-0.03039)) ;
983         sum += fabs(u[113][139]-(0.08183)) ;
984         sum += fabs(u[113][165]-(0.20087)) ;
985         sum += fabs(u[113][192]-(0.33556)) ;
986         sum += fabs(u[113][215]-(0.46036)) ;
987         sum += fabs(u[113][217]-(0.45992)) ;
988         sum += fabs(u[113][218]-(0.4612)) ;
989         sum += fabs(u[113][220]-(0.48223)) ;
990         sum += fabs(u[113][225]-(1)) ;
991
992         sum += fabs(v[0][113]-0) ;
993         sum += fabs(v[14][113]-(0.42447)) ;
994         sum += fabs(v[15][113]-(0.43329)) ;
995         sum += fabs(v[17][113]-(0.43648)) ;
996         sum += fabs(v[21][113]-(0.42951)) ;
997         sum += fabs(v[35][113]-(0.35368)) ;
998         sum += fabs(v[51][113]-(0.28066)) ;
999         sum += fabs(v[52][113]-(0.2728)) ;
1000        sum += fabs(v[113][113]-(0.00945)) ;
1001        sum += fabs(v[181][113]-(-0.30018)) ;
1002        sum += fabs(v[194][113]-(-0.36214)) ;
1003        sum += fabs(v[204][113]-(-0.41442)) ;
1004        sum += fabs(v[214][113]-(-0.52876)) ;
1005        sum += fabs(v[215][113]-(-0.55408)) ;
1006        sum += fabs(v[217][113]-(-0.55069)) ;
1007        sum += fabs(v[218][113]-(-0.49774)) ;
1008        sum += fabs(v[225][113]-(0.0)) ;*/
1009        return sum ;
1010    }
1011
1012    //step8: refresh the pressure predictor at cell centers
1013    void step8(int m){
1014        #pragma omp parallel for collapse(2)  // use OpenMP to refresh predictions in parallel
1015        for(int I = 1 ; I <= Nx ; I++){
1016            for(int J = 1 ; J <= Ny ; J++){
1017                p_star[I-1][J-1] = p[I-1][J-1] ; //both p and p_star are now evaluated at step N+1
1018            }
1019        }
1020    }
1021
1022    //step9: update velocity predictors, interface ratios, and convective coefficients
1023    void step9(int m){
1024        //update x-velocity predictor; boundary values come directly from the imposed conditions
1025        #pragma omp parallel for collapse(2)
1026        for(int i = 1 ; i <= Nx+1 ; i++){
1027            for(int J = 1 ; J <= Ny ; J++){
1028                if(i == 1){
1029                    u_star[i-1][J-1] = u_left;
```

```
1030            }else if (i == Nx+1){
1031                u_star[i-1][J-1] = u_right ;
1032            }else{
1033                //internal cells, including top and bottom faces, inherit the corrected
                        velocity
1034                u_star[i-1][J-1] = u[i-1][J-1]; //u evaluated at N+1; new u_star also at N+1
1035                u_star_old[i-1][J-1] = u_star[i-1][J-1]; //store the predictor immediately
1036            }
1037        }
1038    }
1039    //update y-velocity predictor; boundary values come directly from the imposed conditions
1040    #pragma omp parallel for collapse(2)
1041    for(int I = 1 ; I <= Nx ; I++){
1042        for(int j = 1 ; j <= Ny+1 ; j++){
1043            if(j == 1){
1044                v_star[I-1][j-1] = v_bottom ;
1045            }else if (j == Ny+1){
1046                v_star[I-1][j-1] = v_top ;
1047            }else{
1048                //internal cells, including left and right faces, inherit the corrected
                        velocity
1049                v_star[I-1][j-1] = v[I-1][j-1]; //v evaluated at N+1; new v_star also at N+1
1050                v_star_old[I-1][j-1] = v_star[I-1][j-1]; //store the predictor immediately
1051            }
1052        }
1053    }
1054    //update convective coefficients
1055    //x-momentum convection terms in the general finite-volume discretization
1056    #pragma omp parallel for collapse(2)
1057    for(int i = 1 ; i <= Nx+1 ; i++){
1058        for(int J = 1 ; J <= Ny ; J++){
1059            if(i == 1 || i == Nx+1) {
1060                F_w_u[i-1][J-1] = 0.0 ;
1061                F_e_u[i-1][J-1] = 0.0 ;
1062                F_n_u[i-1][J-1] = 0.0 ;
1063                F_s_u[i-1][J-1] = 0.0 ;
1064            }else{//recompute convective fluxes
1065                F_e_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i][J-1]); //cell centers
                        align with the storage location
1066                F_w_u[i-1][J-1] = 0.5*dy*(u_star[i-1][J-1]+u_star[i-2][J-1]);
1067                F_n_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J]+v_star[i-2][J]);
1068                F_s_u[i-1][J-1] = 0.5*dx*(v_star[i-1][J-1]+v_star[i-2][J-1]);
1069            }
1070        }
1071    }//alpha_f_u switch; indices denote control-volume centers
1072    #pragma omp parallel for collapse(2)
1073    for(int i = 1 ; i <= Nx+1 ; i++){
1074        for(int J = 1 ; J <= Ny ; J++){
1075            if(F_w_u[i-1][J-1] > 0) alpha_w_u[i-1][J-1] = 1.0 ;
1076            else alpha_w_u[i-1][J-1] = 0.0 ;
1077            if(F_e_u[i-1][J-1] > 0) alpha_e_u[i-1][J-1] = 1.0 ;
1078            else alpha_e_u[i-1][J-1] = 0.0 ;
1079            if(F_n_u[i-1][J-1] > 0) alpha_n_u[i-1][J-1] = 1.0 ;
1080            else alpha_n_u[i-1][J-1] = 0.0 ;
1081            if(F_s_u[i-1][J-1] > 0) alpha_s_u[i-1][J-1] = 1.0 ;
1082            else alpha_s_u[i-1][J-1] = 0.0 ;
1083        }
1084    }
1085    //y-momentum convection terms in the general finite-volume discretization
1086    #pragma omp parallel for collapse(2)
1087    for(int I = 1 ; I <= Nx ; I++){
1088        for(int j = 1 ; j <= Ny+1 ; j++){
1089            if(j == 1 || j == Ny+1) {
1090                F_w_v[I-1][j-1] = 0.0 ;
1091                F_e_v[I-1][j-1] = 0.0 ;
1092                F_n_v[I-1][j-1] = 0.0 ;
1093                F_s_v[I-1][j-1] = 0.0 ;
1094            }else{
1095                F_w_v[I-1][j-1] = 0.5*dy*(u_star[I-1][j-1]+u_star[I-1][j-2]);
1096                F_e_v[I-1][j-1] = 0.5*dy*(u_star[I][j-1]+u_star[I][j-2]);
1097                F_n_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j]);//cell centers align
                        with the storage location
1098                F_s_v[I-1][j-1] = 0.5*dx*(v_star[I-1][j-1]+v_star[I-1][j-2]);
1099            }
1100        }
1101    }//alpha_f_v switch; indices denote control-volume centers
```

```
1102      #pragma omp parallel for collapse(2)
1103      for(int I = 1 ; I <= Nx ; I++){
1104          for(int j = 1 ; j <= Ny+1 ; j++){
1105              if(F_w_v[I-1][j-1] > 0) alpha_w_v[I-1][j-1] = 1.0 ;
1106              else alpha_w_v[I-1][j-1] = 0.0 ;
1107              if(F_e_v[I-1][j-1] > 0) alpha_e_v[I-1][j-1] = 1.0 ;
1108              else alpha_e_v[I-1][j-1] = 0.0 ;
1109              if(F_n_v[I-1][j-1] > 0) alpha_n_v[I-1][j-1] = 1.0 ;
1110              else alpha_n_v[I-1][j-1] = 0.0 ;
1111              if(F_s_v[I-1][j-1] > 0) alpha_s_v[I-1][j-1] = 1.0 ;
1112              else alpha_s_v[I-1][j-1] = 0.0 ;
1113          }
1114      }
1115      //update interface ratio indicators
1116      #pragma omp parallel for collapse(2)
1117      for(int i = 1 ; i <= Nx+1 ; i++){
1118          for(int J = 1 ; J <= Ny ; J++){
1119              //cells outside the stencil set the ratio to zero
1120              if(i == 1 || i == Nx+1){
1121                  r_w_u[i-1][J-1] = 0.0 ;
1122                  r_e_u[i-1][J-1] = 0.0 ;
1123                  r_n_u[i-1][J-1] = 0.0 ;
1124                  r_s_u[i-1][J-1] = 0.0 ;
1125              }else {
1126                  //treat each boundary case-by-case
1127                  //west boundary
1128                  if(i == 2){
1129                      r_w_u[i-1][J-1] = -1.0 ; //west boundary uses upwind weighting
1130                  }else{
1131                      double u_W = u_star[i-2][J-1];
1132                      double u_P = u_star[i-1][J-1];
1133                      double u_WW = u_star[i-3][J-1];
1134                      double u_E = u_star[i][J-1] ;
1135                      double D = u_W + alpha_w_u[i-1][J-1]*(u_P - u_W) ;
1136                      double U = u_P + alpha_w_u[i-1][J-1]*(u_W - u_P) ;
1137                      double UU = u_E + alpha_w_u[i-1][J-1]*(u_WW - u_E) ;
1138                      r_w_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1139                  }
1140                  //east boundary
1141                  if(i == Nx){
1142                      r_e_u[i-1][J-1] = -1.0 ; //east boundary uses upwind weighting
1143                  }else{
1144                      double u_P = u_star[i-1][J-1] ;
1145                      double u_E = u_star[i][J-1] ;
1146                      double u_W = u_star[i-2][J-1] ;
1147                      double u_EE = u_star[i+1][J-1] ;
1148                      double D = u_P + alpha_e_u[i-1][J-1]*(u_E - u_P) ;
1149                      double U = u_E + alpha_e_u[i-1][J-1]*(u_P - u_E) ;
1150                      double UU = u_EE + alpha_e_u[i-1][J-1]*(u_W - u_EE) ;
1151                      r_e_u[i-1][J-1] = (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1152                  }
1153                  //south boundary
1154                  if(J == 1){
1155                      r_s_u[i-1][J-1] = 0.0 ; //bottom-most control volumes do not need the south
                             ratio
1156                  }else if(J == 2){
1157                      double u_S = u_star[i-1][J-2] ;
1158                      double u_P = u_star[i-1][J-1] ;
1159                      double u_SS = 2*u_bottom-u_S ; //mirror south neighbor around the boundary
1160                      double u_N = u_star[i-1][J] ;
1161                      double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1162                      double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1163                      double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1164                      r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1165                  }else if(J == Ny){
1166                      double u_S = u_star[i-1][J-2] ;
1167                      double u_P = u_star[i-1][J-1] ;
1168                      double u_SS = u_star[i-1][J-3] ;
1169                      double u_N = 2*u_top - u_P ; //mirror the point across the top boundary
1170                      double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1171                      double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1172                      double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1173                      r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1174                  }else{
1175                      double u_S = u_star[i-1][J-2] ;
1176                      double u_P = u_star[i-1][J-1] ;
```

```
1177                    double u_SS = u_star[i-1][J-3] ;
1178                    double u_N = u_star[i-1][J] ;
1179                    double D = u_S + alpha_s_u[i-1][J-1]*(u_P - u_S) ;
1180                    double U = u_P + alpha_s_u[i-1][J-1]*(u_S - u_P) ;
1181                    double UU = u_N + alpha_s_u[i-1][J-1]*(u_SS - u_N) ;
1182                    r_s_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1183                }
1184                //north boundary
1185                if(J == 1){
1186                    double u_P = u_star[i-1][J-1] ;
1187                    double u_N = u_star[i-1][J] ;
1188                    double u_S = 2*u_bottom-u_P ; //mirror the point across the bottom boundary
1189                    double u_NN = u_star[i-1][J+1] ;
1190                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1191                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1192                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1193                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1194                }else if (J == Ny-1){
1195                    double u_P = u_star[i-1][J-1] ;
1196                    double u_N = u_star[i-1][J] ;
1197                    double u_S = u_star[i-1][J-2] ;
1198                    double u_NN = 2*u_top - u_N ; //mirror the north neighbor across the
                            boundary
1199                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1200                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1201                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1202                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1203                }else if(J == Ny){
1204                    r_n_u[i-1][J-1] = 0.0 ; //top-most control volumes do not need the north
                            ratio
1205                }else{
1206                    double u_P = u_star[i-1][J-1] ;
1207                    double u_N = u_star[i-1][J] ;
1208                    double u_S = u_star[i-1][J-2] ;
1209                    double u_NN = u_star[i-1][J+1] ;
1210                    double D = u_P + alpha_n_u[i-1][J-1]*(u_N - u_P) ;
1211                    double U = u_N + alpha_n_u[i-1][J-1]*(u_P - u_N) ;
1212                    double UU = u_NN + alpha_n_u[i-1][J-1]*(u_S - u_NN) ;
1213                    r_n_u[i-1][J-1] =  (U-UU)/(D-U+1e-30) ; //initial MUSCL ratio
1214                }
1215            }
1216        }
1217    }//after updating predictors, refresh the interface ratios immediately
1218    //interface ratios for the y-velocity (east and west faces require special handling)
1219    #pragma omp parallel for collapse(2)
1220    for(int I = 1 ; I <= Nx ; I++){
1221        for(int j = 1 ; j <= Ny+1 ; j++){
1222            //cells outside the stencil set the ratio to zero
1223            if(j == 1 || j == Ny+1){
1224                r_w_v[I-1][j-1] = 0.0 ;
1225                r_e_v[I-1][j-1] = 0.0 ;
1226                r_n_v[I-1][j-1] = 0.0 ;
1227                r_s_v[I-1][j-1] = 0.0 ;
1228            }else {
1229                //process each boundary configuration explicitly
1230                //south boundary
1231                if(j == 2){
1232                    r_s_v[I-1][j-1] = -1.0 ; //south boundary uses upwind weighting
1233                }else{
1234                    double v_S = v_star[I-1][j-2] ;
1235                    double v_P = v_star[I-1][j-1] ;
1236                    double v_SS = v_star[I-1][j-3] ;
1237                    double v_N = v_star[I-1][j] ;
1238                    double D = v_S + alpha_s_v[I-1][j-1]*(v_P - v_S) ;
1239                    double U = v_P + alpha_s_v[I-1][j-1]*(v_S - v_P) ;
1240                    double UU = v_N + alpha_s_v[I-1][j-1]*(v_SS - v_N) ;
1241                    r_s_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ; //avoid division by zero
1242                }
1243                //north boundary
1244                if(j == Ny){
1245                    r_n_v[I-1][j-1] = -1.0 ; //north boundary uses upwind weighting
1246                }else{
1247                    double v_P = v_star[I-1][j-1] ;
1248                    double v_N = v_star[I-1][j] ;
1249                    double v_S = v_star[I-1][j-2] ;
1250                    double v_NN = v_star[I-1][j+1] ;
```

```
1251                    double D = v_P + alpha_n_v[I-1][j-1]*(v_N - v_P) ;
1252                    double U = v_N + alpha_n_v[I-1][j-1]*(v_P - v_N) ;
1253                    double UU = v_NN + alpha_n_v[I-1][j-1]*(v_S - v_NN) ;
1254                    r_n_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1255                }
1256                //west boundary
1257                if(I == 1){
1258                    r_w_v[I-1][j-1] = 0.0 ; //left-most control volumes do not need the west
                            ratio
1259                }else if (I == 2){
1260                    double v_W = v_star[I-2][j-1] ;
1261                    double v_P = v_star[I-1][j-1] ;
1262                    double v_WW = 2*v_left-v_W ; //mirror the west neighbor across the boundary
1263                    double v_E = v_star[I][j-1] ;
1264                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1265                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1266                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1267                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1268                }else if(I == Nx){
1269                    double v_W = v_star[I-2][j-1] ;
1270                    double v_P = v_star[I-1][j-1] ;
1271                    double v_WW = v_star[I-3][j-1] ;
1272                    double v_E = 2*v_right - v_P ; //mirror the point across the east boundary
1273                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1274                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1275                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1276                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1277                }else{
1278                    double v_W = v_star[I-2][j-1] ;
1279                    double v_P = v_star[I-1][j-1] ;
1280                    double v_WW = v_star[I-3][j-1] ;
1281                    double v_E = v_star[I][j-1] ;
1282                    double D = v_W + alpha_w_v[I-1][j-1]*(v_P - v_W) ;
1283                    double U = v_P + alpha_w_v[I-1][j-1]*(v_W - v_P) ;
1284                    double UU = v_E + alpha_w_v[I-1][j-1]*(v_WW - v_E) ;
1285                    r_w_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1286                }
1287                //east boundary
1288                if(I == 1){
1289                    double v_P = v_star[I-1][j-1] ;
1290                    double v_E = v_star[I][j-1] ;
1291                    double v_W = 2*v_left - v_P ; //mirror the point across the west boundary
1292                    double v_EE = v_star[I+1][j-1] ;
1293                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
1294                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
1295                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
1296                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1297                }else if (I == Nx-1){
1298                    double v_P = v_star[I-1][j-1] ;
1299                    double v_E = v_star[I][j-1] ;
1300                    double v_W = v_star[I-2][j-1] ;
1301                    double v_EE = 2*v_right - v_E ; //mirror the east neighbor across the
                            boundary
1302                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
1303                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
1304                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
1305                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1306                }else if(I == Nx){
1307                    r_e_v[I-1][j-1] = 0.0 ; //right-most control volumes do not need the east
                            ratio
1308                }else {
1309                    double v_P = v_star[I-1][j-1] ;
1310                    double v_E = v_star[I][j-1] ;
1311                    double v_W = v_star[I-2][j-1] ;
1312                    double v_EE = v_star[I+1][j-1] ;
1313                    double D = v_P + alpha_e_v[I-1][j-1]*(v_E - v_P) ;
1314                    double U = v_E + alpha_e_v[I-1][j-1]*(v_W - v_E) ;
1315                    double UU = v_EE + alpha_e_v[I-1][j-1]*(v_W - v_EE) ;
1316                    r_e_v[I-1][j-1] = (U-UU)/(D-U+1e-30) ;
1317                }
1318            }
1319        }
1320    }
1321 }
1322
1323 //Export finite-volume cell data to VTK
```

```
1324 void output_combined_fields(int m) {
1325     int nodes_i = Nx + 1;  // number of nodes along x
1326     int nodes_j = Ny + 1;  // number of nodes along y
1327     int cells_i = Nx;      // number of cells along x
1328     int cells_j = Ny;      // number of cells along y
1329 
1330     std::ostringstream name;
1331     name << "3.TVD_Re" << Re << "_Grid" << Nx+1 << "x" << Ny+1
1332          << "_DC_step" << std::setfill('0') << std::setw(6) << m
1333          << "_error" << min_step7 <<
1334           ".vtk";
1335 
1336     std::ofstream out(name.str());
1337 
1338     out << "# vtk DataFile Version 3.0\n";
1339     out << "3.TVD_Re = 5000_Grid "<< Nx+1 << "x" << Ny+1 << "\n";
1340     out << "ASCII\n";
1341     out << "DATASET STRUCTURED_GRID\n";
1342     out << "DIMENSIONS " << nodes_i << " " << nodes_j << " 1\n";
1343 
1344     // write node coordinates
1345     out << "POINTS " << nodes_i * nodes_j << " double\n";
1346     for(int j = 0; j < nodes_j; j++) {
1347         for(int i = 0; i < nodes_i; i++) {
1348             double x_coord = i * dx;
1349             double y_coord = j * dy;
1350             out << scientific << setprecision(6)
1351                 << x_coord << " " << y_coord << " 0.0\n";
1352         }
1353     }
1354 
1355     out << "CELL_DATA " << cells_i * cells_j << "\n";
1356 
1357     // 1. pressure field
1358     out << "SCALARS Pressure double 1\n";
1359     out << "LOOKUP_TABLE default\n";
1360     for(int J = 1; J <= cells_j; J++) {
1361         for(int I = 1; I <= cells_i; I++) {
1362             out << scientific << setprecision(6)
1363                 << p_star[I-1][J-1] << "\n";
1364         }
1365     }
1366 
1367     // 2. velocity field (CELL_DATA already declared)
1368     out << "VECTORS Velocity double\n";
1369     for(int J = 1; J <= cells_j; J++) {
1370         for(int I = 1; I <= cells_i; I++) {
1371             double u_val = 0.5 * (u_star[I-1][J-1] + u_star[I][J-1]);
1372             double v_val = 0.5 * (v_star[I-1][J-1] + v_star[I-1][J]);
1373 
1374             out << scientific << setprecision(6)
1375                 << u_val << " " << v_val << " 0.0\n";
1376         }
1377     }
1378 
1379     out.close();
1380     cout << "VTK document has been output: " << name.str() << endl;
1381 }
1382 
1383 int main(){
1384     // initialize OpenMP with the available resources
1385     int num_threads = omp_get_max_threads();
1386     omp_set_num_threads(num_threads);
1387     cout << "Using " << num_threads << " threads" << endl;
1388     cout << "Using FVD with SIMPLE algorithm to solve 2D incompressible Navier-Stokes equations
         " << endl;
1389     cout << "Grid Size:" << Nx+1 << "x" << Ny+1 << endl;
1390     cout << "Grid spacing : " << dx << "x " << dy << endl;
1391     cout << "Boundary Conditions: " ;
1392     cout << "u: left = " << u_left << ", right = " << u_right << ", bottom = " << u_bottom << "
         , top = " << u_top << endl;
1393     cout << "v: left = " << v_left << ", right = " << v_right << ", bottom = " << v_bottom << "
         , top = " << v_top << endl;
1394     cout << "Re = " << Re << ",rho = " << rho << endl ;
1395     CE_physical = false;
1396     G = 0 ;
```

```
1397      initialize() ;
1398      min_step7 = 50.0 ;
1399      double min_G = 0.0  ;
1400      for(G = 0 ; G <= max_G ; G++){
1401
1402          step2(G) ;//update the velocity predictor (first update)
1403          step3() ;//update the pressure correction
1404          step4(G) ;//update the corrected pressure field
1405          step5(G) ;//update the velocity correction
1406          step6(G) ;//update the corrected velocity field
1407          //step7: convergence check
1408          if(min_step7 > step7(G)){ //evaluate each iteration
1409                  min_step7 = step7(G) ;
1410                  min_G = G ;
1411                  //if(G > 1000 && min_step7 < 2.55   ){output_combined_fields(min_G) ;}
1412
1413              }
1414          if(G%1000 == 0){
1415              cout << "Itteration step = " << G << endl ;
1416              cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                       << endl ;
1417              cout << "currnt minerror(G =" << min_G << ")= "<< min_step7 << endl ;
1418          }
1419          if(G>0 && step5star(G) < 1e-10){
1420              CE_physical = true ;
1421              cout << "Haved converged at step G = " << G << endl ;
1422              cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)
                       <<endl ;
1423              cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
1424              output_combined_fields(G) ;
1425              break ;
1426          }
1427          step8(G) ;//refresh the pressure predictor
1428          step9(G) ;//refresh velocity predictors, convection coefficients, and interface ratios
1429      }
1430      if(!CE_physical){
1431          cout << "Haved converged at step G = " << G << endl ;
1432          cout << "Non-equilibrium mass flow rate for (G = "<< G << " )step:" << step5star(G)  <<
                   endl ;
1433          cout << "currnt minerror(G =" << G << ")= "<< min_step7 << endl ;
1434          output_combined_fields(min_G) ;
1435      }
1436      cout << "Grid Size : " << Nx << "x" << Ny << "Completed." << endl ;
1437      return 0 ;
1438  }
```

Listing 11: 4.FVM_deferred correction_ADI+TDMA.cpp

B SIMPLE Algebraic System

Step	Equation
1	**predicted form of the discrete momentum integration equation**(B.1)
2	**pressure correction equation (pressure Poisson equation)**(B.2)
3	**pressure predicted-correction relationship**(B.3)
4	**Patankar's correction form of the discrete momentum integration equation**(B.4)
5	**velocity predicted-correction relationship**(B.5)
6	**predicted form of the continuity integration equation**(B.6)
7	**updating predicted pressure from correct pressure**(B.7)
8	**updating predicted velocity from correct velocity**(B.8)

Table B.1: SIMPLE algorithm discrete equation system

This appendix provides the detailed discrete equations corresponding to the steps summarised in Table B.1. The formulations follow the standard SIMPLE procedure for pressure Vvelocity coupling on a staggered grid.

B.1 Step 1: Predicted Momentum Equation

The discrete x Vmomentum equation for the predicted velocity $u_{i,J}^{*,N+1/2}$ is written with under-relaxation as

$$\begin{aligned}(a_P)_{i,J}^{*,N}\,\frac{u_{i,J}^{*,N+1/2}}{\alpha_u} \;&+\; (a_{WW})_{i,J}^{*,N}u_{i-2,J}^{*,N} + (a_W)_{i,J}^{*,N}u_{i-1,J}^{*,N} + (a_E)_{i,J}^{*,N}u_{i+1,J}^{*,N} + (a_{EE})_{i,J}^{*,N}u_{i+2,J}^{*,N}\\ &+\; (a_{SS})_{i,J}^{*,N}u_{i,J-2}^{*,N} + (a_S)_{i,J}^{*,N}u_{i,J-1}^{*,N} + (a_N)_{i,J}^{*,N}u_{i,J+1}^{*,N} + (a_{NN})_{i,J}^{*,N}u_{i,J+2}^{*,N}\\ &= -\frac{p_{I,J}^{*,N}-p_{I-1,J}^{*,N}}{\rho}\,\delta y + \frac{1-\alpha_u}{\alpha_u}(a_P)_{i,J}^{*,N}u_{i,J}^{N}.\end{aligned} \tag{B.1}$$

B.2 Step 2: Pressure-Correction Equation

Using the discrete continuity constraint, the pressure Vcorrection equation becomes

$$\begin{aligned}&\left(\frac{\delta y}{\rho(a_P)_{i+1,J}^{*,N+1/2}} + \frac{\delta y}{\rho(a_P)_{i,J}^{*,N+1/2}} + \frac{\delta x}{\rho(b_P)_{I,j+1}^{*,N+1/2}} + \frac{\delta x}{\rho(b_P)_{I,j}^{*,N+1/2}}\right)p_{I,J}^{\prime N+1}\\ &= \frac{\delta x}{\rho(b_P)_{I,j}^{*,N+1/2}}\,p_{I,J-1}^{\prime N} + \frac{\delta x}{\rho(b_P)_{I,j+1}^{*,N+1/2}}\,p_{I,J+1}^{\prime N}\\ &\quad + \frac{\delta y}{\rho(a_P)_{i,J}^{*,N+1/2}}\,p_{I-1,J}^{\prime N} + \frac{\delta y}{\rho(a_P)_{i+1,J}^{*,N+1/2}}\,p_{I+1,J}^{\prime N}\\ &\quad + u_{i,J}^{*,N+1/2} - u_{i+1,J}^{*,N+1/2} + v_{I,j}^{*,N+1/2} - v_{I,j+1}^{*,N+1/2}.\end{aligned} \tag{B.2}$$

B.3 Step 3: Pressure Update

The corrected pressure is updated using the relaxation factor α_p:

$$p_{I,J}^{N+1} = p_{I,J}^{*,N} + \alpha_p\,p_{I,J}^{\prime N+1}. \tag{B.3}$$

B.4 Step 4: Velocity Correction

The velocity correction follows directly from the linearised momentum operator:

$$u_{i,J}'^{N+1} = -\frac{p_{I,J}'^{N+1} - p_{I-1,J}'^{N+1}}{\rho\,(a_P)_{i,J}^{*,\,N+1/2}}\,\delta y. \tag{B.4}$$

B.5 Step 5: Velocity Update

$$u_{i,J}^{N+1} = u_{i,J}^{*,\,N+1/2} + u_{i,J}'^{N+1}. \tag{B.5}$$

B.6 Step 6: Continuity Check

A discrete conservation check is performed:

$$A_e\,u_{i+1,J}^{*,\,N+1/2} - A_w\,u_{i,J}^{*,\,N+1/2} + A_n\,v_{I,j+1}^{*,\,N+1/2} - A_s\,v_{I,j}^{*,\,N+1/2} = 0. \tag{B.6}$$

B.7 VB.8 Step 7 V8: Update of Predicted Fields

$$p_{I,J}^{*,\,N+1} = p_{I,J}^{N+1}. \tag{B.7}$$

$$u_{i,J}^{*,\,N+1} = u_{i,J}^{N+1} \tag{B.8}$$